Pramod J. Sadalage | Martin Fowler

NoSQL Essencial

Um Guia Conciso para o Mundo Emergente da Persistência Poliglota

novatec

NoSQL Essencial

Um Guia Conciso para o Mundo Emergente da Persistência Poliglota

Pramod J. Sadalage e Martin Fowler

Pearson
Novatec
São Paulo | 2019

Authorized translation from the English language edition, entitled NOSQL DISTILLED: A BRIEF GUIDE TO THE EMERGING WORLD OF POLYGLOT PERSISTENCE, 1st Edition, 0321826620 by SADALAGE, PRAMOD J.; FOWLER, MARTIN, published by Pearson Education, Inc, publishing as Addison-Wesley Professional, Copyright © 2013 Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc. PORTUGUESE language edition published by NOVATEC EDITORA LTDA., Copyright © 2013.

Tradução autorizada da edição em língua inglesa, intitulada NOSQL DISTILLED: A BRIEF GUIDE TO THE EMERGING WORLD OF POLYGLOT PERSISTENCE, 1st Edition, 0321826620 por SADALAGE, PRAMOD J.; FOWLER, MARTIN, publicada pela Pearson Education, Inc como Addison-Wesley Professional, Copyright © 2013 Pearson Education, Inc.

Todos os direitos reservados. Nenhuma parte deste livro deve ser reproduzida ou transmitida em qualquer formato ou por qualquer meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer sistema de recuperação de informações, sem permissão da Pearson Education, Inc. Edição em língua PORTUGUESA publicada pela Novatec Editora Ltda, Copyright © 2013.

© Novatec Editora Ltda. 2013.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates
Tradução: Acauan Fernandes
Revisão técnica: BrodTec.com / Aurelio Jargas
Revisão gramatical: Cristiane Bernardi
Editoração eletrônica: Carolina Kuwabata

ISBN: 978-85-7522-XXX

Histórico de edições impressas:

Outubro/2017 Terceira reimpressão
Novembro/2015 Segunda reimpressão
Abril/2014 Primeira reimpressão
Junho/2013 Primeira edição

Novatec Editora Ltda.
Rua Luís Antônio dos Santos 110
02460-000 – São Paulo, SP – Brasil
Tel.: +55 11 2959-6529
E-mail: novatec@novatec.com.br
Site: www.novatec.com.br
Twitter: twitter.com/novateceditora
Facebook: facebook.com/novatec
LinkedIn: linkedin.com/in/novatec

Para meus professores Gajanan Chinchwadkar, Dattatraya Mhaskar e Arvind Parchure. Vocês foram os que mais me inspiraram. Obrigado.

– Pramod

Para Cindy

– Martin

Sumário

Prefácio

Por que bancos de dados NoSQL são interessantes?
O que este livro contém
Quem deve ler este livro?
Quais são os bancos de dados?
Agradecimentos

Parte I ■ Compreender

Capítulo 1 ■ Por que NoSQL?

1.1 O valor dos bancos de dados relacionais
1.1.1 Chegando aos dados persistentes
1.1.2 Concorrência
1.1.3 Integração
1.1.4 Um modelo padrão (na sua maior Parte)
1.2 Incompatibilidade de impedância
1.3 Bancos de dados de integração e de aplicativos
1.4 Ataque dos clusters
1.5 Surgimento do NoSQL
1.6 Pontos chave

Capítulo 2 ■ Modelos de dados agregados

2.1 Agregados
2.1.1 Exemplo de relações e agregados
2.1.2 Consequências da orientação a agregados
2.2 Modelos de dados de chave-valor e de documentos
2.3 Armazenamentos de famílias de colunas
2.4 Resumindo os bancos de dados orientados a agregados
2.5 Leituras complementares
2.6 Pontos chave

Capítulo 3 ■ Mais detalhes sobre modelos de dados

3.1 Relacionamentos
3.2 Bancos de dados de grafos
3.3 Bancos de dados sem esquema
3.4 Visões materializadas (Materialized views)
3.5 Modelando para o acesso aos dados
3.6 Pontos chave

Capítulo 4 ■ Modelos de distribuição

4.1 Um único servidor
4.2 Fragmentação
4.3 Replicação mestre-escravo
4.4 Replicação ponto a ponto (p2p)

4.5 Combinando fragmentação e replicação
4.6 Pontos chave

Capítulo 5 ■ Consistência
5.1 Consistência de atualização
5.2 Consistência de leitura
5.3 Relaxando a consistência
5.3.1 Teorema CAP
5.4 Relaxando a durabilidade
5.5 Quóruns
5.6 Leituras complementares
5.7 Pontos chave

Capítulo 6 ■ Marcadores de versões
6.1 Transações comerciais e de sistema
6.2 Marcadores de versões em múltiplos nodos
6.3 Pontos chave

Capítulo 7 ■ Map-Reduce (Mapear-Reduzir)
7.1 Map-reduce básico
7.2 Particionando e combinando
7.3 Criando cálculos map-reduce
7.3.1 Um exemplo de map-reduce em duas etapas
7.3.2 Map-reduce incremental
7.4 Leituras complementares
7.5 Pontos chave

Parte II ■ Implementar

Capítulo 8 ■ Bancos de dados de chave-valor
8.1 Depósitos de chave-valor
8.2 Recursos dos depósitos de chave-valor
8.2.1 Consistência
8.2.2 Transações
8.2.3 Recursos de consultas
8.2.4 Estrutura de dados
8.2.5 Escalabilidade
8.3 Casos apropriados para uso
8.3.1 Armazenando informações de sessão
8.3.2 Perfis de usuários, preferências
8.3.3 Dados de carrinhos de compras
8.4 Quando não utilizar
8.4.1 Relacionamentos entre dados
8.4.2 Transações com múltiplas operações
8.4.3 Consulta por dados
8.4.4 Operações por conjuntos

Capítulo 9 ■ Bancos de dados de documentos
9.1 O que é um banco de dados de documentos?
9.2 Características

9.2.1 Consistência
9.2.2 Transações
9.2.3 Disponibilidade
9.2.4 Recursos de consulta
9.2.5 Escalabilidade
9.3 Casos de uso apropriados
9.3.1 Registro de eventos (log)
9.3.2 Sistema de Gerenciamento de Conteúdo (CMS), plataformas de blog
9.3.3 Análises web ou em tempo real (analytics)
9.3.4 Aplicativos de comércio eletrônico
9.4 Quando não utilizar
9.4.1 Transações complexas que abranjam diferentes operações
9.4.2 Consultas em estruturas agregadas variáveis

Capítulo 10 ■ Armazenamento em famílias de colunas

10.1 O que é um depósito de dados de família de colunas?
10.2 Características
10.2.1 Consistência
10.2.2 Transações
10.2.3 Disponibilidade
10.2.4 Recursos de consulta
10.2.5 Escalabilidade
10.3 Casos de uso apropriados
10.3.1 Registro de eventos (log)
10.3.2 Sistemas de gerenciamento de conteúdo (CMS), plataformas de blog
10.3.3 Contadores
10.3.4 Expirando o uso
10.4 Quando não utilizar

Capítulo 11 ■ Bancos de dados de grafos

11.1 O que é um banco de dados de grafos?
11.2 Recursos
11.2.1 Consistência
11.2.2 Transações
11.2.3 Disponibilidade
11.2.4 Recursos de consulta
11.2.5 Escalabilidade
11.3 Casos de uso apropriados
11.3.1 Dados conectados
11.3.2 Roteamento, envio e serviços baseados em localização
11.3.3 Mecanismos de recomendação
11.4 Quando não utilizar

Capítulo 12 ■ Migrações de esquema

12.1 Alterações no esquema
12.2 Alterações de esquema em SGBDRs
12.2.1 Migrações para projetos sem restrições anteriores
12.2.2 Migrações em projetos legados
12.3 Alterações no esquema em um armazenamento de dados NoSQL

12.3.1 Migração incremental
12.3.2 Migrações em bancos de dados de grafos
12.3.3 Alterando a estrutura do agregado
12.4 Leituras complementares
12.5 Pontos chave

Capítulo 13 ■ Persistência poliglota

13.1 Necessidades diferentes de armazenamento de dados
13.2 Uso de armazenamentos de dados poliglotas
13.3 Uso de serviços em vez do acesso direto ao depósitos de dados
13.4 Expandindo para melhorar a funcionalidade
13.5 Escolhendo a tecnologia certa
13.6 Preocupações empresariais com a persistência poliglota
13.7 Complexidade de instalação
13.8 Pontos chave

Capítulo 14 ■ Além do NoSQL

14.1 Sistemas de arquivos
14.2 Event sourcing
14.3 Imagem de memória
14.4 Controle de versões
14.5 Bancos de dados XML
14.6 Bancos de dados de objetos
14.7 Ponto chave

Capítulo 15 ■ Escolhendo o seu banco de dados

15.1 Produtividade do programador
15.2 Desempenho no acesso aos dados
15.3 Permanecendo com o padrão
15.4 Restringindo suas apostas
15.5 Pontos chave
15.6 Considerações finais

Bibliografia

Prefácio

Já estamos há aproximadamente vinte anos no mundo da computação empresarial. Temos visto muitas coisas mudarem nas linguagens, arquiteturas, plataformas e processos. Porém, durante todo esse tempo, algo tem sido constante – são bancos de dados relacionais que armazenam os dados. Existiram desafiantes; alguns tiveram sucesso em alguns nichos, mas no contexto global, a questão discutida pelos arquitetos, acerca do armazenamento de dados, tem sido sobre qual banco de dados relacional utilizar.

A estabilidade dessa área é de grande valor. Os dados de uma organização perduram muito mais tempo do que seus programas (pelo menos, é isso que as pessoas dizem, mas temos visto muitos programas bastante antigos por aí). É importante termos um armazenamento de dados estável, que seja bem compreendido e que possa ser acessado a partir de muitas plataformas de programação de aplicativos.

Atualmente, porém, há um novo desafiante na jogada, sob o confrontante nome de NoSQL. Ele foi criado a partir de uma necessidade de manipulação de volumes maiores de dados, o que impôs uma mudança fundamental no sentido do desenvolvimento de plataformas grandes de hardware por meio de clusters de servidores comerciais (commodity servers). Essa necessidade também trouxe à tona preocupações antigas quanto às dificuldades de se fazer com que o código de aplicativos seja bem executado no modelo de dados relacional.

O termo "NoSQL" não está definido de forma muito clara. Geralmente, é aplicado a alguns bancos de dados não relacionais recentes, como o Cassandra, o Mongo, o Neo4J e o Riak. Eles utilizam dados sem esquema, são executados em clusters e trocam a consistência tradicional por outras propriedades úteis. Os defensores dos bancos de dados NoSQL alegam que podem desenvolver sistemas com melhor desempenho, com melhor escalabilidade e que são mais simples de programar.

Esse é o primeiro sinal para o fim dos bancos de dados relacionais ou apenas mais um pretendente ao trono? Nossa resposta é "nenhum dos dois". Bancos de dados relacionais são uma ferramenta poderosa e a nossa expectativa é que ainda sejam utilizados por muitas décadas, mas certamente vemos uma mudança profunda no sentido de que eles não serão os únicos bancos de dados a serem utilizados. Estamos adentrando em um mundo de persistência poliglota, em que as empresas e até aplicativos individuais utilizarão múltiplas tecnologias para o gerenciamento de dados. Consequentemente, os arquitetos precisarão estar familiarizados com essas tecnologias e ser capazes de avaliar quais tipos podem utilizar para diferentes necessidades. Se não pensássemos assim, não teríamos gasto tempo e esforço escrevendo este livro, que busca apresentar informações suficientes para responder à questão sobre a possibilidade dos bancos de dados NoSQL merecerem ou não ser seriamente considerados em seus futuros projetos.

Cada projeto é diferente de outro e não há como escrever uma árvore de decisão simples para selecionar o armazenamento de dados correto. Em vez disso, o que estamos tentando fazer é fornecer uma base suficiente sobre como os bancos de dados NoSQL funcionam, de modo que seja possível fazer por si próprio esses julgamentos sem ter de pesquisar na

web inteira. Deixamos este livro deliberadamente pequeno para que você possa obter essa visão geral rapidamente. Ele não responderá a suas perguntas de modo definitivo, mas deve diminuir a gama de opções que você terá de considerar e ajudará a entender as perguntas que devem ser feitas.

Por que bancos de dados NoSQL são interessantes?

Percebemos dois motivos principais pelos quais as pessoas consideram utilizar um banco de dados NoSQL:

- **Produtividade no desenvolvimento de aplicativos:** muito trabalho de desenvolvimento de aplicativos é gasto no mapeamento de dados entre as estruturas de dados na memória e um banco de dados relacional. Um banco de dados NoSQL pode fornecer um modelo de dados que se adapte melhor às necessidades do aplicativo, simplificando, dessa forma, essa interação e resultando em menos código a ser escrito, depurado e melhorado;
- **Grandes quantidades de dados**: as organizações estão percebendo a importância de obter-se mais dados e processá-los mais rapidamente. Elas estão considerando caro e, às vezes, impossível, fazer isso com bancos de dados relacionais. O principal motivo é que um banco de dados relacional é projetado para execução em uma única máquina, mas geralmente é mais econômico executar grandes quantidades de dados e computações em clusters com muitas máquinas menores e mais baratas. Muitos bancos de dados NoSQL são projetados explicitamente para execução em clusters, de modo que são mais apropriados para cenários onde exista uma grande quantidade de dados.

O que este livro contém

Dividimos este livro em duas partes. A primeira parte concentra-se em conceitos básicos, necessários para julgar se bancos de dados NoSQL são relevantes e de que modo eles se diferenciam. Na segunda parte, concentramo-nos mais na implementação de sistemas com bancos de dados NoSQL.

O capítulo 1 explica o porquê do NoSQL ter tido uma ascensão tão rápida – a necessidade de processamento de volumes maiores de dados levou a uma mudança, em sistemas grandes, da escala vertical para a horizontal, em clusters. Isso explica uma característica importante do modelo de dados de muitos bancos de dados NoSQL – o armazenamento explícito de uma estrutura rica de dados intimamente relacionados que é acessada como uma unidade. Neste livro, chamamos esse tipo de estrutura de *agregado*.

O capítulo 2 descreve como os agregados manifestam-se em três dos principais modelos de dados de NoSQL: chave-valor (“Chave-valor e modelos de dados de documentos”, p. 50), documentos (“Chave-valor e modelos de dados de documentos”, p. 50) e famílias de colunas (“Armazenamentos de famílias de colunas”, p. 52). Os agregados fornecem uma unidade natural de interação para muitos tipos de aplicativos, o que melhora sua execução em um cluster e facilita a programação de acesso aos dados. O capítulo 3 enfoca as desvantagens dos agregados – a dificuldade de lidar com relacionamentos (“Relacionamentos”, p. 57) entre entidades em diferentes agregados. Isso nos leva, naturalmente, a bancos de dados de grafos (“Bancos de dados de grafos”, p. 59), um modelo de dados NoSQL que não se enquadra na orientação a agregados. Também

examinamos a característica comum de bancos de dados NoSQL de atuarem sem um esquema ("Bancos de dados sem esquema", p. 61) – um recurso que fornece maior flexibilidade, mas não tanta quanto você poderia imaginar a princípio.

Tendo visto o aspecto de modelagem de dados de NoSQL, passamos para a distribuição: o capítulo 4 descreve como os bancos de dados distribuem os dados para serem processados em clusters. Esse processo divide-se em fragmentação ("Fragmentação", p. 74) e em replicação, sendo que esta pode ser mestre-escravo ("Replicação mestre-escravo", p. 77) ou ponto a ponto ("Replicação ponto a ponto", p. 79). Com os modelos de distribuição definidos, podemos passar, então, para a questão da consistência. Bancos de dados NoSQL fornecem uma gama mais variada de opções de consistência do que os bancos de dados relacionais – o que é uma consequência de sua eficácia em clusters. Assim, o capítulo 5 discute como a consistência modifica-se para atualizações ("Consistência de atualização", p. 83) e leituras ("Consistência de leitura", p. 86), o papel dos quóruns ("Quóruns", p. 97) e como até um pouco de durabilidade ("Relaxando a durabilidade", p. 96) pode ser adaptada. Se você já tiver ouvido algo sobre NoSQL, certamente saberá a respeito do teorema CAP; a seção "Teorema CAP", na p. 91, explica o que ele é e como se aplica às bases de dados NoSQL.

Enquanto esses capítulos concentram-se, principalmente, nos princípios de como os dados são distribuídos e mantidos consistentes, os próximos dois discutem algumas ferramentas importantes que fazem tudo funcionar. O capítulo 6 descreve marcas de versões, que servem para registrar alterações e detectar inconsistências. O capítulo 7 traça um perfil do map-reduce, uma forma especial de organizar a computação paralela que se adapta bem a clusters e, portanto, a sistemas NoSQL.

Assim que definirmos os conceitos, passaremos para questões de implementação, examinando alguns bancos de dados que exemplificarão as quatro categorias: o capítulo 8 utiliza o Riak como exemplo de banco de dados de chave-valor, o capítulo 9 utiliza o MongoDB como exemplo de banco de dados de documentos, o capítulo 10 utiliza o Cassandra para explorar os bancos de dados de famílias de colunas e, finalmente, o capítulo 11 traz o Neo4J como exemplo de banco de dados de grafos. Devemos enfatizar que esse não é um estudo abrangente, pois há temas demais sobre os quais escrever, quanto mais experimentar. Esses exemplos não significam recomendações. Nosso objetivo é dar uma ideia da variedade de formas de armazenamento que existe e como as diferentes tecnologias de bancos de dados utilizam os conceitos esboçados anteriormente. Você verá que tipo de código precisa escrever para programar nesses sistemas e vislumbrará que raciocínio precisará fazer para utilizá-los.

Uma declaração comum sobre bancos de dados NoSQL é que, já que não possuem esquemas, não há dificuldade alguma em alterar a estrutura dos dados durante a vida de um aplicativo. Não concordamos com isso, pois um banco de dados sem esquema ainda assim possui um esquema implícito, que precisa de disciplina ao fazer mudanças, de forma que o capítulo 12 explica como executar a migração de dados tanto para esquemas fortes quanto para sistemas sem esquema.

Tudo isso deve deixar claro que NoSQL não é algo individual nem algo que substituirá os bancos de dados relacionais. O capítulo 13 examina esse futuro mundo da persistência poliglota, no qual diversos mundos de armazenamento de dados coexistirão, até mesmo dentro do mesmo aplicativo. O capítulo 14 expande, então, nossos horizontes para além

deste livro, analisando outras tecnologias que não vimos e que também poderão fazer parte desse mundo da persistência poliglota.

Ao obter todas essas informações, você finalmente terá condições de decidir quais tecnologias de armazenamento de dados utilizar, de modo que o capítulo final (capítulo 15, "Escolhendo o seu banco de dados", p. 209) apresenta alguns conselhos sobre como pensar nessas escolhas. Há dois fatores-chave – encontrar um modelo de programação produtivo onde o modelo de armazenamento de dados esteja bem alinhado com o seu aplicativo e assegurar-se de que possa obter o desempenho e a resiliência de que precisa. Já que esses são os primórdios da história de vida do NoSQL, sentimos muito não termos um procedimento bem definido a seguir, então você terá de testar suas opções no contexto de suas necessidades.

Essa é uma visão geral concisa – nós propositalmente limitamos o tamanho deste livro. Selecionamos as informações que consideramos mais importantes, de modo que você não tenha de fazê-lo. Se for investigar seriamente essas tecnologias, você precisará ir além do que examinamos aqui, mas esperamos que este livro forneça um bom contexto inicial a partir do qual possa começar.

Também precisamos enfatizar que essa é uma área da computação que está em constante mudança. Aspectos importantes acerca desses tipos de armazenamento modificam-se todos os anos, e surgem novos recursos, novos bancos de dados. Fizemos um esforço muito grande para enfocar os conceitos, o que achamos que será mais valioso entender mesmo quando a tecnologia correspondente modificar-se. Temos bastante confiança de que a maioria do que dissemos terá esta longevidade, mas estamos absolutamente seguros que nem tudo terá.

Quem deve ler este livro?

O público-alvo deste livro são as pessoas que estão pensando em utilizar algum tipo de banco de dados NoSQL, seja para um novo projeto ou porque estão encontrando barreiras que sugerem mudanças em um projeto existente.

Nosso objetivo é apresentar informações suficientes para você entender se a tecnologia NoSQL se aplica às suas necessidades e, se for o caso, qual ferramenta deve ser explorada com mais profundidade. Imaginamos que o nosso leitor principal seja um arquiteto ou líder técnico, mas acreditamos que este livro também é valioso para as pessoas envolvidas no gerenciamento de software que queiram ter uma visão geral dessa nova tecnologia. Também acreditamos que, se você for um desenvolvedor buscando uma visão geral da tecnologia, este livro será um bom ponto de partida.

Não entramos nos detalhes de programação e instalação de bancos de dados específicos – deixamos isso para livros mais especializados. Também mantivemo-nos muito firmes quanto ao limite de páginas, para que este livro seja uma introdução concisa. Este é o tipo de livro bom para ler durante uma viagem de avião: ele não responderá a todas as suas perguntas, mas lhe dará um bom conjunto de perguntas a serem feitas.

Se você já tiver mergulhado no mundo do NoSQL, este livro provavelmente não trará itens novos ao seu conhecimento. Entretanto, ainda assim pode ser útil, auxiliando-o a explicar para outras pessoas o que aprendeu. É importante conhecer as questões relacionadas ao NoSQL, especialmente se você estiver tentando convencer alguém a

considerar seu uso em um projeto.

Quais são os bancos de dados?

Neste livro, seguimos uma abordagem comum de categorizar os bancos de dados NoSQL de acordo com seu modelo de dados. Aqui está uma tabela dos quatro modelos de dados e alguns bancos de dados comuns que se enquadram em cada modelo. Essa não é uma lista abrangente, já que menciona apenas os bancos de dados mais comuns que encontramos. Quando este texto foi escrito, você podia encontrar listas mais abrangentes em `http://nosql-database.org` e `http://nosql.mypopescu.com/kb/nosql`. Para cada categoria, marcamos em itálico o banco de dados que utilizamos como exemplo no capítulo pertinente.

Nosso objetivo é escolher um representante de cada uma das categorias dos bancos de dados. Embora falemos sobre exemplos específicos, a maior parte da discussão deve se aplicar à categoria inteira, embora esses produtos sejam únicos e não possam ser generalizados como tal. Selecionaremos um banco de dados para cada um dos tipos: de chave-valor, de documentos, de famílias de colunas e de grafos. Mencionaremos, apropriadamente, outros produtos que possam satisfazer a uma necessidade específica de algum recurso.

Modelo de dados	Exemplos de bancos de dados
Chave-valor (“Bancos de dados de chave-valor”, p. 123)	BerkeleyDB
	LevelDB
	Memcached
	Project Voldemort
	Redis
	Riak
Documentos (“Bancos de dados de documentos”, p. 133)	CouchDB
	MongoDB
	OrientDB
	RavenDB
	Terrastore
Famílias de colunas (“Armazenamentos em famílias de colunas”, p. 147)	Amazon SimpleDB
	Cassandra
	HBase
	Hypertable
Grafos (“Bancos de dados de grafos”, p. 161)	FlockDB
	HyperGraphDB
	Infinite Graph

	Neo4J
	OrientDB

Essa classificação por modelo de dados é útil, porém, imperfeita. A distinção entre os diferentes tipos de modelos de dados, como a distinção entre bancos de dados de chave-valor e de documentos (“Modelos de dados de chave-valor e de documentos”, p. 50), muitas vezes é difusa. Muitos bancos de dados não se encaixam exatamente nas categorias; por exemplo, o OrientDB é considerado tanto um banco de dados de documentos quanto de grafos.

Agradecimentos

Nossos primeiros agradecimentos aos nossos colegas da ThoughtWorks, muitos dos quais têm aplicado NoSQL em nossos projetos de desenvolvimento nos últimos anos. Sua experiência foi a fonte principal tanto de nossa motivação para escrever este livro quanto de informações práticas sobre o valor dessa tecnologia. A experiência positiva que tivemos até agora com armazenamentos de dados NoSQL é a base do nosso ponto de vista, segundo o qual essa é uma tecnologia importante, que traz uma mudança significativa no armazenamento de dados.

Também gostaríamos de agradecer a vários grupos que fizeram palestras públicas e publicaram artigos e blogs sobre suas experiências com o NoSQL. Muito do progresso no desenvolvimento de software permanece desconhecido quando as pessoas não o compartilham. Agradecimentos especiais aqui para o Google e a Amazon, cujos artigos sobre o Bigtable e o Dynamo foram muito influentes, impulsionando o movimento do NoSQL. Também agradecemos às empresas que têm patrocinado e contribuído com o desenvolvimento open-source de bancos de dados NoSQL. Uma diferença interessante com relação às mudanças anteriores na área de armazenamento de dados é o quanto o movimento NoSQL apóia-se no trabalho open-source.

Agradecimentos especiais à ThoughtWorks, pelo tempo que pudemos trabalhar neste livro. Entramos para a ThoughtWorks mais ou menos ao mesmo tempo e estamos aqui há mais de uma década. A ThoughtWorks continua a ser um lar muito hospitaleiro para nós, uma fonte de conhecimento e prática, em um ambiente acolhedor de compartilhamento de aprendizagem – tão diferente de organizações tradicionais de desenvolvimento de sistemas.

A Bethany Anders-Beck, Ilias Bartolini, Tim Berglund, Duncan Craig, Paul Duvall, Oren Eini, Perryn Fowler, Michael Hunger, Eric Kascic, Joshua Kerievsky, Anand Krishnaswamy, Bobby Norton, Ade Oshineye, Thiyagu Palanisamy, Prasanna Pendse, Dan Pritchett, David Rice, Mike Roberts, Marko Rodriquez, Andrew Slocum, Toby Tripp, Steve Vinoski, Dean Wampler, Jim Webber e Wee Witthawaskul, que revisaram os primeiros esboços deste livro e nos auxiliaram a melhorá-lo com seus conselhos.

Além disso, eu, Pramod, gostaria de agradecer à Biblioteca Schaumburg por oferecer um ótimo serviço e disponibilizar um espaço silencioso para escrever; a Arhana e Arula, minhas filhas lindas, por entenderem que o papai ia para a biblioteca e não as levava com ele; a Rupali, minha esposa amada, por seu enorme apoio e ajuda para manter-me focado.

PARTE I

Compreender

CAPÍTULO 1

Por que NoSQL?

Praticamente, desde que iniciamos na área de software, os bancos de dados relacionais têm sido a escolha padrão para o armazenamento de grandes volumes de dados, especialmente no mundo dos aplicativos corporativos. Se você for um arquiteto iniciando um novo projeto, é provável que sua principal escolha esteja relacionada a qual banco de dados relacionais utilizar (e, muitas vezes, se a sua empresa já possuir um fornecedor fixo, nem isso). Houve situações em que uma tecnologia de banco de dados ameaçou tomar uma fatia do mercado, como aconteceu com os bancos de dados orientados a objetos na década de 1990, mas essas alternativas nunca foram bem-sucedidas.

Após um longo período de domínio, surge o atual interesse por bancos de dados NoSQL. Neste capítulo, exploraremos o porquê dos bancos de dados relacionais terem se tornado tão dominantes e o porquê de acharmos que a ascensão atual dos bancos de dados NoSQL não é passageira.

1.1 O valor dos bancos de dados relacionais

Os bancos de dados relacionais tornaram-se uma parte tão inerente à nossa cultura computacional que é comum não lhes dar sua devida importância. É útil, portanto, rever seus benefícios.

1.1.1 Chegando aos dados persistentes

Provavelmente, a maior importância de um banco de dados seja sua capacidade de armazenar grandes quantidades de dados persistentes. A maioria das arquiteturas de computadores possui duas áreas de memória: uma "memória principal", rápida e volátil, e uma "memória secundária", maior, porém mais lenta. A memória principal é limitada em espaço e perde todos os dados quando a energia é cortada ou algo inesperado acontece com o sistema operacional. Portanto, para que os dados sejam mantidos, devem ser gravados em um dispositivo de armazenamento secundário, comumente um disco, embora atualmente possa ser outro tipo de memória persistente.

O armazenamento secundário pode ser organizado de diversas formas. Para muitos aplicativos de produtividade (como processadores de textos), esse tipo de armazenamento consiste em um arquivo no sistema de arquivos do sistema operacional. Para a maioria dos aplicativos corporativos, entretanto, o armazenamento secundário é um banco de dados, o qual traz mais flexibilidade, se comparado a um sistema de arquivos, no armazenamento de grandes quantidades de dados. Isso permite que o aplicativo obtenha pequenas partes dessas informações de maneira rápida e fácil.

1.1.2 Concorrência

Aplicativos corporativos, geralmente, implicam muitas pessoas examinando o mesmo conjunto de dados ao mesmo tempo e, possivelmente, modificando-o. Na maior parte do

tempo, essas pessoas estão trabalhando em diferentes áreas desses dados, mas, ocasionalmente, operam na mesma parte. Por isso, é necessário coordenar essa interação para evitar diversas situações, como fazer a reserva do mesmo quarto de hotel para dois clientes simultaneamente.

A concorrência é notoriamente difícil de ser ajustada, com todos os tipos de erros que podem atrapalhar até mesmo o mais cuidadoso dos programadores. Já que aplicativos corporativos podem ter muitos usuários e outros sistemas, todos funcionando ao mesmo tempo, podem surgir problemas a qualquer momento. Bancos de dados relacionais ajudam a lidar com essas situações, controlando todo o acesso aos seus dados por meio de transações. Embora essa não seja uma solução para tudo (você ainda tem de lidar com erros de transações, por exemplo, quando tentar reservar um quarto que acabou de ser ocupado), o mecanismo transacional tem executado bem sua função de controlar a complexidade da concorrência.

As transações também desempenham um papel importante na hora de lidar com os erros, pois é possível fazer uma alteração e, caso ocorra um erro durante seu processamento, pode-se desfazê-la e voltar ao estado anterior.

1.1.3 Integração

Aplicativos corporativos fazem parte de um ecossistema rico, que requer múltiplos aplicativos, escritos por diferentes equipes, que colaboram umas com as outras na realização de suas tarefas. Esse tipo de colaboração entre aplicativos é inadequado, pois implica a extrapolação dos limites humanos organizacionais. Os aplicativos, muitas vezes, precisam utilizar os mesmos dados e, assim, as atualizações feitas por um devem ser acessíveis pelos demais.

Uma forma comum de fazer isso é por meio da **integração compartilhada de base de dados** [Hohpe e Woolf], em que múltiplos aplicativos armazenam seus dados em uma única base de dados. A utilização de uma única base de dados permite que todos os aplicativos utilizem facilmente os dados uns dos outros, enquanto o controle da concorrência da base de dados lida com aplicativos múltiplos da mesma forma com que lida com vários usuários em um único aplicativo.

1.1.4 Um modelo padrão (na sua maior parte)

Os bancos de dados relacionais foram bem-sucedidos porque trouxeram os benefícios principais, os quais esboçamos anteriormente, de uma forma padrão (na sua maior parte). Como consequência, desenvolvedores e profissionais da área de bancos de dados podem aprender o modelo relacional básico e aplicá-lo a muitos projetos.

Embora haja diferenças de um banco de dados relacional para outro, os mecanismos principais permanecem os mesmos: os dialetos SQL utilizados por diversos fornecedores são similares, e as transações realizam-se praticamente da mesma forma.

1.2 Incompatibilidade de impedância

Bancos de dados relacionais fornecem muitas vantagens, mas não são, de forma alguma, perfeitos. Desde que surgiram, há muita frustração em relação a seu uso.

Para os desenvolvedores de aplicativos, a maior frustração tem sido a diferença entre o

modelo relacional e as estruturas de dados na memória, comumente chamada de **incompatibilidade de impedância**. O modelo de dados relacional organiza os dados em uma estrutura de tabelas e linhas ou, mais apropriadamente, de relações e tuplas. No modelo relacional, uma **tupla** é um conjunto de pares nome-valor e uma **relação** é um conjunto de tuplas (a definição relacional de uma tupla é ligeiramente diferente daquela utilizada na matemática e em muitas linguagens de programação com um tipo de dados tupla, em que uma tupla é uma sequência de valores). Todas as operações em SQL consomem e retornam relações, o que leva à matematicamente elegante álgebra relacional.

Tal base, fundamentada nas relações, fornece certa elegância e simplicidade, mas também introduz limitações. Em especial, os valores de uma tupla relacional têm de ser simples – eles não podem conter nenhum tipo de estrutura, como um registro aninhado ou uma lista. Essa limitação não é válida para estruturas de dados na memória, que conseguem sustentar estruturas mais ricas do que as relações. Consequentemente, para utilizar uma estrutura de dados de memória mais rica, deve-se traduzi-la para uma representação relacional a fim de armazená-la em disco. Daí a incompatibilidade de impedância – duas representações diferentes que requerem tradução (Figura 1.1).

A incompatibilidade de impedância é causa de uma grande frustração para os desenvolvedores de aplicativos. Na década de 1990, muitas pessoas acreditavam que isso faria que os bancos de dados relacionais fossem substituídos por bancos de dados que repetissem, no disco, as estruturas de dados da memória. Aquela década foi marcada pelo crescimento das linguagens orientadas a objetos e, com elas, vieram os bancos de dados também orientados a objetos, ambos com a intenção de se tornarem o ambiente dominante para o desenvolvimento de software no novo milênio.

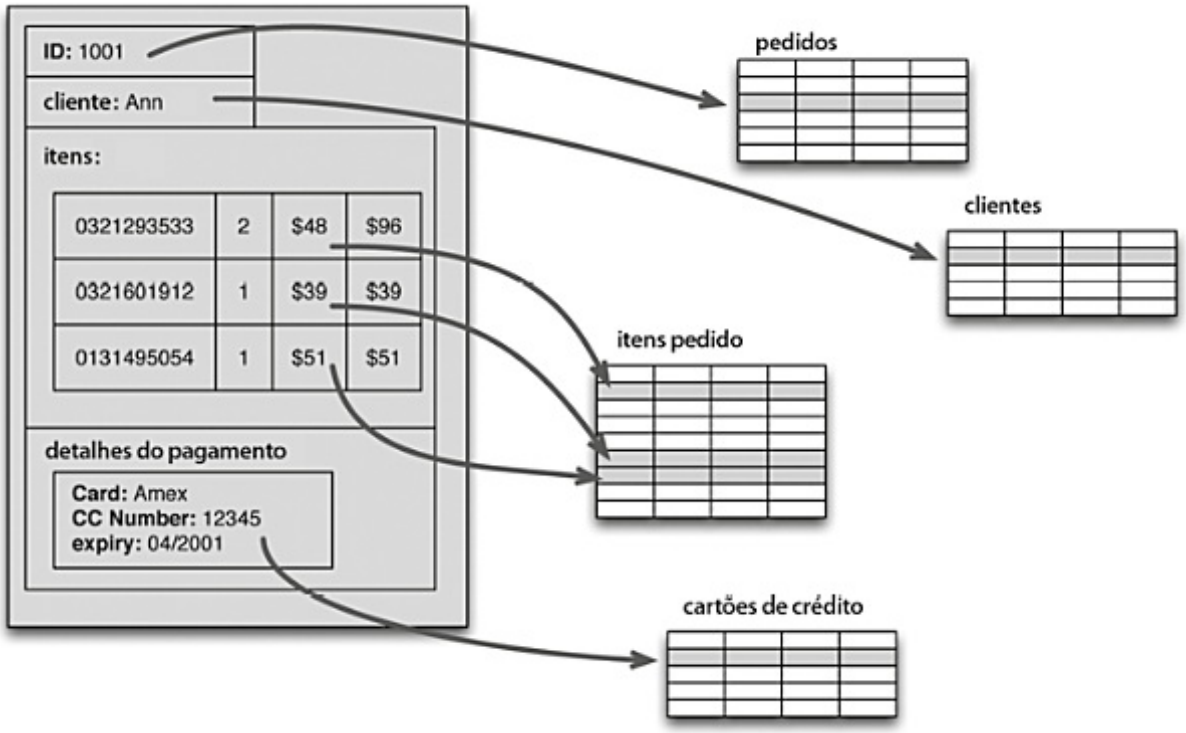

Figura 1.1 – Um pedido que aparenta ser uma estrutura única na interface de usuário é dividido em muitas linhas de muitas tabelas em um banco de dados relacional.

Entretanto, enquanto as linguagens de programação orientadas a objetos conseguiram se

tornar a força principal na área da programação, os bancos de dados orientados a objetos caíram na obscuridade. Os bancos de dados relacionais enfrentaram o desafio, salientando seu papel de mecanismo de integração, suportado pela linguagem padrão de manipulação de dados (SQL). Houve uma crescente divisão profissional entre desenvolvedores de aplicativos e administradores de bancos de dados.

Tornou-se muito mais fácil lidar com a incompatibilidade de impedância devido à ampla disponibilidade de frameworks de mapeamento objeto-relacional, como o Hibernate e o iBATIS, que implementam padrões de mapeamento bastante conhecidos [Fowler PoEAA]. Porém, a questão do mapeamento ainda é controversa. Frameworks de mapeamento objeto-relacional poupam muito trabalho pesado às pessoas, mas podem se tornar um problema quando estas exageram ao ignorar o banco de dados, comprometendo o desempenho da pesquisa.

Os bancos de dados relacionais dominaram o mundo da computação corporativa na década de 2000, mas, durante esse período, algumas brechas começaram a surgir em seu domínio.

1.3 Bancos de dados de integração e de aplicativos

O motivo exato pelo qual os bancos de dados relacionais triunfaram sobre aqueles orientados a objetos ainda é assunto de debates ocasionais entre os desenvolvedores de uma certa faixa etária. Mas, sob nosso ponto de vista, o principal fator foi a utilização da linguagem SQL como mecanismo de integração entre os aplicativos. Nesse cenário, o banco de dados atua como um **banco de dados de integração**, com múltiplos aplicativos, geralmente desenvolvidos por equipes distintas que armazenam seus dados em um banco de dados comum. Isso melhora a comunicação, porque todos os aplicativos atuam em um consistente conjunto de dados persistentes.

Há também desvantagens na integração compartilhada de bancos de dados. Uma estrutura projetada para integrar muitos aplicativos acaba tornando-se mais complexa – de fato, muitas vezes, incrivelmente mais complexa – do que qualquer necessidade de um único aplicativo. Além disso, se um aplicativo tiver de modificar seu armazenamento de dados, precisará coordenar essa modificação com todos os outros que utilizam o banco de dados. Aplicativos diferentes têm necessidades estruturais e de desempenho distintas, de modo que um índice requerido por um pode causar um problema nas inserções de outro. O fato de cada aplicativo geralmente pertencer a uma equipe diferente também significa que o banco de dados não poderá confiar que os aplicativos atualizem os dados e preservem a integridade deles, sendo necessário assumirem essa responsabilidade.

Uma abordagem diferente consiste em tratar seu banco de dados como um **banco de dados de aplicativo**, o qual só é acessado diretamente por um único aplicativo, que é gerenciado por apenas uma equipe. Apenas essa equipe precisará conhecer a estrutura dele, o que torna muito mais fácil manter e desenvolver o esquema. Uma vez que a equipe do aplicativo controla o banco de dados e o código do aplicativo, a responsabilidade pela integridade do banco de dados pode ser atribuída a esse código.

Questões relacionadas a interoperabilidade podem, agora, passar para a interface do aplicativo, permitindo melhores protocolos de interação e fornecendo suporte para alterá-los. Durante a década de 2000, assistimos a uma mudança distinta na direção dos serviços

web [Daigneau], em que os aplicativos comunicavam-se por meio de HTTP. Serviços web permitiram uma nova forma de mecanismo de comunicação amplamente utilizada, que concorre com o uso de SQL com bancos de dados compartilhados (muito desse trabalho foi realizado sob o título de "Arquitetura orientada a serviços", que chama a atenção devido à ausência de significado consistente).

Um aspecto interessante dessa mudança, na direção de serviços web como mecanismo de integração, foi que ela resultou em mais flexibilidade na estrutura dos dados que estavam sendo trocados. Se a comunicação for realizada em SQL, os dados devem ser estruturados na forma de relações. Entretanto, por meio de um serviço, podem ser utilizadas estruturas de dados mais ricas, com registros aninhados e listas. Estas, geralmente, são representadas na forma de documentos em XML ou, mais recentemente, JSON. De modo geral, é muito útil poder colocar uma estrutura rica de informações em uma única solicitação ou resposta para reduzir o número de idas e vindas envolvidas nas comunicações remotas.

Se os serviços forem utilizados para integração, os web – com texto via HTTP – são, na maioria das vezes, a melhor opção. Todavia, se as interações forem muito sensíveis quanto ao desempenho, talvez seja necessário um protocolo binário, mas, para isso, é preciso ter certeza dessa necessidade, já que o trabalho com os protocolos de texto é mais fácil. Consideremos, como exemplo, a Internet.

Decidida a utilização de um banco de dados de aplicativo, é necessário escolhê-lo. Uma vez que há um desacoplamento entre o banco de dados interno e os serviços por meio dos quais faz-se a comunicação com o mundo exterior, não deve importar a este o modo como os dados são armazenados, permitindo que sejam consideradas opções não relacionais. Além disso, há muitos recursos de bancos de dados relacionais, como a segurança, que são menos úteis para um banco de dados de aplicativo porque podem ser realizados pelo próprio aplicativo responsável pelo banco.

Entretanto, apesar dessa liberdade, não ficou evidente que bancos de dados de aplicativo incentivaram uma corrida para formas alternativas de armazenamento de dados. A maioria das equipes que se envolveu na abordagem de banco de dados de aplicativo permaneceu com bancos de dados relacionais. Afinal, utilizar um banco de dados de aplicativo traz muitas vantagens, mesmo ignorando a flexibilidade do banco de dados (motivo pelo qual a recomendamos). Bancos de dados relacionais são familiares e, geralmente, funcionam muito bem ou, pelo menos, suficientemente bem. Talvez, com o passar do tempo, possamos assistir a uma mudança, em que a escolha por bancos de dados de aplicativos ameace a hegemonia relacional, mas essa ameaça será de outra origem.

1.4 Ataque dos clusters

No início do novo milênio, o mundo da tecnologia foi assolado pela explosão da bolha das companhias .com da década de 1990. Enquanto muitas pessoas questionavam o futuro econômico da internet, a década de 2000 foi testemunha do aumento drástico em escala das diversas grandes propriedades web.

Esse aumento ocorria em muitas dimensões. Websites começaram a registrar atividades e estruturas de uma forma muito detalhada. Surgiram grandes conjuntos de dados: links, redes sociais, atividades em logs, dados de mapeamento. Devido ao crescimento na

quantidade de dados, houve um aumento também no número de usuários – os maiores websites desenvolveram-se até se tornarem enormes propriedades, servindo regularmente a um grande número de visitantes.

Lidar com o aumento de dados e com o tráfego exige mais recursos computacionais. Para comportar esse crescimento, há duas opções: ir para cima ou para fora. Ir para cima significa adquirir máquinas maiores, mais processadores, ter maior capacidade de armazenamento em disco e memória. Máquinas maiores, todavia, tornam-se cada vez mais caras, sem mencionar que há limites físicos quanto ao aumento de tamanho. A alternativa seria utilizar mais máquinas menores em um cluster. Um cluster de máquinas pequenas pode utilizar hardware mais acessível e acaba tornando-se mais barato para essa aplicação. Ele também pode ser mais resiliente – embora falhas em máquinas individuais sejam comuns, o cluster, como um todo, pode ser criado para continuar funcionando apesar dessas falhas, fornecendo alta confiabilidade.

À medida que grandes propriedades seguiram na direção dos clusters, surgiu um novo problema: bancos de dados relacionais não foram projetados para serem executados em clusters. Esses bancos, em clusters, como o Oracle RAC ou o Microsoft SQL Server, funcionam baseados no conceito de um subsistema de disco compartilhado. Eles utilizam um sistema de arquivos que reconhece clusters e grava em um subsistema de disco com alta disponibilidade. Isso significa, porém, que o cluster ainda tem o subsistema de disco como um único ponto de falhas. Bancos de dados relacionais também poderiam ser executados como servidores separados para conjuntos diferentes de dados, fragmentando, efetivamente, o banco de dados (“Fragmentação,” p. 74). Embora tal procedimento divida a carga, toda a fragmentação tem de ser controlada pelo aplicativo, que precisa rastrear quais servidores de banco de dados comunicam-se com quais partes dos dados. Além disso, perderemos quaisquer consultas, integridade referencial, transações ou controles de consistência que atravessarem os fragmentos. Uma expressão que ouvimos frequentemente, nesse contexto, é: “Não parece natural”.

Essas questões técnicas são exacerbadas pelos custos de licenças. Bancos de dados relacionais comerciais geralmente têm preços baseados na suposição de existir um único servidor, de modo que a execução em cluster aumenta os preços e leva a negociações frustrantes com os departamentos de compras.

Essa incompatibilidade entre bancos de dados relacionais e clusters levou algumas organizações a considerarem uma rota alternativa para o armazenamento de dados. Duas empresas em particular – Google e Amazon – têm sido muito influentes. Ambas estiveram à frente na execução de grandes clusters desse tipo; além disso, obtiveram grandes quantidades de dados. Isso deu a elas o motivo que faltava. Elas eram empresas bem-sucedidas e em crescimento, com fortes componentes técnicos, proporcionando-lhes os meios e a oportunidade. Não é surpresa o fato de que elas tinham em mente acabar com seus bancos de dados relacionais. Quando a década de 2000 chegou, ambas produziram artigos concisos, porém altamente influentes, a respeito de seus trabalhos: BigTable (Google) e Dynamo (Amazon).

Diz-se, frequentemente, que o Google e a Amazon operam em escalas muito maiores do que a maioria das organizações, de modo que as soluções de que precisam podem não ser relevantes para uma organização de tamanho médio. Embora seja verdade que a maioria dos projetos de software não necessite desse nível de escala, também é verdade que cada

vez mais organizações estão começando a explorar o que podem fazer, pois ao capturar e processar mais dados, acabam se deparando com os mesmos problemas. Assim, à medida que mais informações sobre o que o Google e a Amazon estavam fazendo tornaram-se públicas, mais pessoas começaram a explorar a criação de bancos de dados de forma semelhante – projetados explicitamente para atuar em um mundo de clusters. Embora as ameaças anteriores ao domínio dos bancos de dados relacionais tenham se tornado fantasmas, a ameaça dos clusters era séria.

1.5 Surgimento do NoSQL

É uma ironia incrível que o termo "NoSQL" tenha feito sua primeira aparição no final da década de 1990 com o nome de um banco de dados relacional de código aberto (open source) [Strozzi NoSQL]. Liderado por Carlo Strozzi, esse banco de dados armazena suas tabelas sob a forma de arquivos ASCII, e cada tupla é representada por uma linha com os campos separados por tabulações. O nome vem do fato de que o banco de dados não utiliza SQL como uma linguagem de consulta. Em vez disso, ele é manipulado por meio de shell scripts, que podem ser combinados em encadeamentos (pipelines) no Unix. Apesar da coincidência na terminologia, o NoSQL de Strozzi não teve influência sobre os bancos de dados que descrevemos neste livro.

O uso do termo "NoSQL" que conhecemos hoje é resultado de uma reunião realizada no dia 11 de junho de 2009, em São Francisco, nos Estados Unidos, organizada por Johan Oskarsson, um desenvolvedor de software de Londres. O exemplo do BigTable e do Dynamo inspirou a criação de vários projetos, que faziam experimentações com armazenamentos alternativos de dados, e discussões sobre o assunto haviam se tornado uma das partes essenciais das melhores conferências sobre software daquela época. Johan estava interessado em descobrir mais sobre esses novos bancos de dados enquanto estava em São Francisco para um evento sobre Hadoop. Já que dispunha de pouco tempo, achou que não seria viável visitá-los todos, de modo que decidiu organizar uma reunião em que todos pudessem estar presentes e apresentar seu trabalho para quem estivesse interessado em conhecê-lo.

Johan queria um nome para a reunião – algo que fosse um bom hashtag para o Twitter: curto, fácil de lembrar e sem muitos semelhantes no Google, de modo que uma pesquisa que utilizasse esse nome encontrasse rapidamente a reunião. Ele pediu sugestões no canal #cassandra do IRC e recebeu algumas, selecionando "NoSQL", de Eric Evans (um desenvolvedor na Rackspace, sem conexão com o Eric Evans do DDD – Domain Driven Design). Embora tivesse a desvantagem de ser negativo e não descrever realmente esses sistemas, tal opção satisfazia ao critério de hashtag. Naquela época, eles estavam pensando apenas em dar um nome para uma reunião e não esperavam que se tornaria o nome da tendência tecnológica como um todo [Oskarsson].

O termo "NoSQL" pegou como fogo em palha, mas nunca favoreceu uma definição precisa. A chamada original [NoSQL Meetup] para a reunião pedia por "bancos de dados não relacionais, distribuídos e de código aberto". As palestras [NoSQL Debrief] lá realizadas foram sobre Voldemort, Cassandra, Dynomite, HBase, Hypertable, CouchDB e MongoDB, mas o termo nunca ficou limitado a esse grupo original. Não há uma definição genericamente aceita nem uma autoridade para fornecer uma, de modo que tudo o que podemos fazer é discutir algumas características comuns em bancos de dados que tendem

a ser chamadas de "NoSQL".

Para começar, há o fato óbvio de que bancos de dados NoSQL não utilizam SQL. Alguns deles têm linguagens de consulta, e faz sentido que elas sejam semelhantes ao SQL para que sejam mais facilmente aprendidas. A CQL do Cassandra é assim, "exatamente como SQL (exceto onde não é)" [CQL]. Todavia, até hoje ninguém implementou algo que se ajustasse à noção bastante flexível do padrão SQL. Seria interessante ver o que aconteceria se um banco de dados NoSQL consagrado decidisse implementar um padrão SQL: o único resultado previsível para tal eventualidade geraria muita discussão.

Outra característica importante desses bancos de dados é que eles, geralmente, são projetos de código aberto. Embora o termo NoSQL seja frequentemente aplicado a sistemas de código fechado, existe uma noção de que o NoSQL seja um fenômeno de código aberto.

A maioria dos bancos de dados NoSQL é orientada pela necessidade de execução em clusters, o mesmo caso daqueles que foram abordados na reunião do dia 11 de junho de 2009. Isso tem uma influência sobre seu modelo de dados, assim como sobre sua abordagem quanto à consistência. Bancos de dados relacionais utilizam transações ACID (p. 50) para lidar com consistência em todo o banco de dados, o que é, inerentemente, conflitante com um ambiente de clusters, de modo que os bancos de dados NoSQL oferecem uma gama de opções para consistência e distribuição.

Entretanto, nem todos os bancos de dados NoSQL almejam a execução em clusters. Bancos de dados de grafos consistem em um estilo de banco de dados NoSQL que utiliza um modelo de distribuição semelhante aos bancos de dados relacionais, mas oferece um modelo de dados que os torna mais eficientes na manipulação de dados com relacionamentos complexos.

Os bancos de dados NoSQL, geralmente, baseiam-se nas necessidades da web no século XXI, de modo que, geralmente, apenas sistemas desenvolvidos nesse período são chamados NoSQL, excluindo, dessa forma, muitos dos bancos de dados criados antes do novo milênio.

Os bancos de dados NoSQL atuam sem um esquema, permitindo que sejam adicionados, livremente, campos aos registros do banco de dados, sem ter de definir primeiro quaisquer mudanças na estrutura. Isso é especialmente útil ao lidar com dados não uniformes e campos personalizados, os quais faziam que os bancos de dados relacionais utilizassem nomes como `customField6` (campoPersonalizado6) ou tabelas de campos personalizados, que são difíceis de processar e entender.

Tudo o que foi descrito anteriormente são características comuns de software que são descritos como bancos de dados NoSQL. Nenhumas delas é definidora e, de fato, é provável que nunca haja uma definição coerente de "NoSQL"... Entretanto, esse conjunto básico de características foi nosso guia ao escrever este livro. Nossa maior satisfação é que o surgimento de NoSQL abriu uma gama de opções para o armazenamento de dados. Consequentemente, tal abertura não deve ficar limitada ao que, geralmente, é classificado como um armazenamento NoSQL. Esperamos que outras opções de armazenamento de dados tornem-se mais aceitáveis, incluindo muitas que precederam o movimento NoSQL. Há um limite, contudo, para o que pode ser discutido de maneira útil neste livro, então decidimos nos concentrar nessa "não definição".

Quando ouvimos pela primeira vez o termo"NoSQL", a primeira pergunta é: o que significa um "não SQL"? A maioria das pessoas que fala sobre NoSQL diz que realmente significa "Not Only SQL"("Não Apenas SQL"), mas essa interpretação apresenta alguns problemas. Grande parte das pessoas escreve "NoSQL", enquanto "Not Only SQL" seria escrito como "NOSQL". Além disso, não faria muito sentido chamar algo de banco de dados NoSQL sob o significado "not only", porque Oracle e Postgres se enquadrariam nessa definição; por exemplo, nós provaríamos que as cores preta e branca são iguais e, então, seríamos atropelados nas faixas de segurança nos cruzamentos das ruas.

Para resolver essa questão, sugerimos que não nos preocupemos com o significado desse termo, mas com o que ele quer dizer (o que é recomendado na maioria dos acrônimos). Assim, quando "NoSQL" for aplicado a um banco de dados, ele refere-se a um conjunto mal definido de bancos de dados, na sua maioria em código aberto, desenvolvido no século XXI e não utilizando SQL.

A interpretação "não apenas" tem seu valor, já que descreve o ecossistema considerado por muitas pessoas como o futuro dos bancos de dados. Essa é, na verdade, o que consideramos a contribuição mais importante dessa forma de pensamento – é melhor pensar em NoSQL como um movimento em vez de uma tecnologia. Não achamos que os bancos de dados relacionais acabarão, pois eles ainda serão a forma mais comum de banco de dados em uso. Apesar de termos escrito este livro, ainda recomendamos os bancos de dados relacionais. Sua familiaridade, estabilidade, conjunto de recursos e suporte disponível são argumentos convincentes para a maioria dos projetos.

A diferença significativa é que agora vemos os bancos de dados relacionais como uma opção para o armazenamento de dados. Esse ponto de vista é, muitas vezes, chamado de **persistência poliglota** – utilizar diferentes armazenamentos de dados em diferentes circunstâncias. Em vez de escolher o banco de dados relacional mais utilizado por todos, precisamos entender a natureza dos dados que estamos armazenando e como queremos manipulá-los. O resultado é que a maioria das organizações terá uma mistura de tecnologias de armazenamento de dados para diferentes circunstâncias.

Para fazer esse mundo poliglota funcionar, nossa opinião é que as organizações também precisam mudar de bancos de dados de integração para bancos de dados de aplicativo. De fato supomos, neste livro, que você utilizará um banco de dados NoSQL como um banco de dados de aplicativo; geralmente não consideramos o NoSQL como uma boa escolha para bancos de dados de integração. Não vemos isso como uma desvantagem, já que achamos que, mesmo que você não utilize o NoSQL, mudar para o encapsulamento de dados em serviços é uma boa direção a seguir.

Em nossa história sobre o desenvolvimento do NoSQL, concentramo-nos em grandes volumes de dados sendo processados em clusters. Embora acreditemos que essa é a chave que levou à abertura do mundo dos bancos de dados, esse não é o único motivo pelo qual vemos equipes de projeto considerarem o uso de bancos de dados NoSQL. Um motivo, igualmente importante, é a antiga frustração diante do problema da incompatibilidade de impedância. A preocupação com grandes volumes de dados criou uma oportunidade para as pessoas pensarem novamente em suas necessidades de armazenamento de dados, e algumas equipes de desenvolvimento veem que utilizar um banco de dados NoSQL pode aumentar sua produtividade, simplificando o acesso ao banco de dados, mesmo que não tenham a necessidade de escalar para além de uma única máquina.

Assim, ao ler o restante deste livro, lembre-se de que há dois motivos principais para considerar o NoSQL. Um é lidar com o acesso a dados cujo tamanho e desempenho demandem um cluster; o outro é melhorar a produtividade de desenvolvimento de aplicativos utilizando um estilo de interação de dados mais conveniente.

1.6 Pontos chave

- Os bancos de dados relacionais são uma tecnologia bem-sucedida há vinte anos, fornecendo persistência, controle de concorrência e um mecanismo de integração.
- Desenvolvedores de aplicativos frustram-se com a incompatibilidade de impedância entre o modelo relacional e as estruturas de dados na memória.
- Há um movimento na direção contrária à utilização de bancos de dados como pontos de integração. Em vez disso, defende-se o encapsulamento de bancos de dados em aplicativos e a integração por meio de serviços.
- O fator vital para uma mudança no armazenamento de dados foi a necessidade de suporte a grandes volumes de dados por meio da execução em clusters. Bancos de dados relacionais não são projetados para serem executados eficientemente neles.
- O NoSQL é um neologismo acidental. Não há uma descrição oficial – tudo o que se pode fazer é uma observação das características comuns.
- As características comuns dos bancos de dados NoSQL são:
  - não utilizam o modelo relacional;
  - tem uma boa execução em clusters;
  - seu código é aberto (open source);
  - são criados para propriedades na web do século XXI;
  - não têm esquema.
- O resultado mais importante do surgimento do NoSQL é a persistência poliglota.

CAPÍTULO 2

Modelos de dados agregados

Um modelo de dados é o modo pelo qual percebemos e manipulamos nossos dados. Para as pessoas que utilizam um banco de dados, esse modelo descreve a forma pela qual interagimos com os dados desse banco. Ele difere de um modelo de armazenamento, que descreve como o banco de dados armazena e manipula os dados internamente. Em um mundo ideal, deveríamos ignorar o modelo de armazenamento, mas, na prática, precisamos ter, pelo menos, uma vaga ideia sobre ele, principalmente para atingir um desempenho razoável.

Coloquialmente, o termo "modelo de dados" significa, muitas vezes, o modelo dos dados específicos em um aplicativo. Um desenvolvedor pode apontar para um diagrama entidade-relacionamento (ER) de seu banco de dados e referir-se a ele como o seu modelo de dados contendo clientes, pedidos, produtos etc. Entretanto, neste livro utilizaremos o termo "modelo de dados", na maioria das vezes, para falar sobre o modelo pelo qual o gerenciador do banco de dados organiza seus dados – o que poderia ser mais formalmente chamado de metamodelo.

O modelo de dados dominante nas últimas duas décadas é o modelo de dados relacional, que é melhor visualizado como um conjunto de tabelas, em vez de uma página de uma planilha. Cada tabela possui linhas e cada linha representa uma entidade de interesse. Descrevemos essa entidade por meio de colunas, cada uma tendo um único valor. Uma coluna pode se referir a outra linha na mesma tabela ou em uma tabela diferente, o que constitui um relacionamento entre essas entidades (estamos utilizando terminologia informal, porém, comum, quando falamos em tabelas e linhas; os termos mais formais seriam relações e tuplas.)

Uma das mudanças mais óbvias trazidas pelo NoSQL é o afastamento do modelo relacional. Cada solução NoSQL possui um modelo diferente, os quais dividimos em quatro categorias amplamente utilizadas no ecossistema NoSQL: chave-valor, documento, famílias de colunas e grafos. Dessas, as três primeiras compartilham uma característica comum em seus modelos de dados – que chamaremos de "orientação agregada". Neste capítulo, explicaremos o que queremos dizer por orientação agregada e o que ela significa para os modelos de dados.

2.1 Agregados

O modelo relacional recebe as informações que queremos armazenar e divide-as em tuplas (linhas). Uma tupla é uma estrutura de dados limitada: ela captura um conjunto de valores, de modo que não é possível aninhar uma dentro da outra para obter registros aninhados nem é possível colocar uma lista de valores ou tuplas dentro de uma ou de outra. Essa simplicidade é a base do modelo relacional – ela nos permite pensar em todas as operações, como atuar em tuplas e retornar tuplas.

A orientação agregada utiliza uma abordagem diferente. Ela reconhece que você,

frequentemente, deseja trabalhar com dados na forma de unidades que tenham uma estrutura mais complexa do que um conjunto de tuplas. Pode ser útil pensar em termos de um registro complexo que permita que listas e outras estruturas de dados sejam aninhadas dentro dele. Conforme veremos, bancos de dados dos tipos de chave-valor, de documento e de família de colunas fazem uso desse registro mais complexo. Entretanto, não existe um termo comum que identifique esse registro; neste livro, utilizaremos o termo "agregado", que vem do Domain-Driven Design (Projeto Orientado a Domínios) [Evans]. Nele, um **agregado** é um conjunto de objetos relacionados que desejamos tratar como uma unidade. Em particular, é uma unidade de manipulação de dados e gerenciamento de consistência. Normalmente, preferimos atualizar agregados com operações atômicas e comunicarmo-nos com nosso armazenamento de dados em termos de agregados. Essa definição corresponde bem ao funcionamento dos bancos de dados de chave-valor, documento e família de colunas. Lidar com agregados facilita muito a execução desses bancos de dados em um cluster, uma vez que o agregado constitui uma unidade natural para replicação e fragmentação. Agregados também são, frequentemente, mais simples de ser manipulados pelos programadores de aplicativos, já que eles, muitas vezes, lidam com os dados por meio de estruturas agregadas.

2.1.1 Exemplo de relações e agregados

A esta altura, um exemplo pode ajudar a explicar o que estamos falando. Suponha que tenhamos de criar um website de comércio eletrônico: venderemos itens diretamente aos clientes pela web e teremos de armazenar informações sobre os usuários, o nosso catálogo de produtos, os pedidos, as remessas, os endereços de envio, os endereços de cobrança e os dados sobre o pagamento. Podemos utilizar esse cenário para modelar os dados por meio de um armazenamento de dados relacional, assim como armazenamentos de dados NoSQL, e falar sobre seus prós e contras. Para um banco de dados relacional, poderíamos começar com um modelo de dados mostrado na figura 2.1.

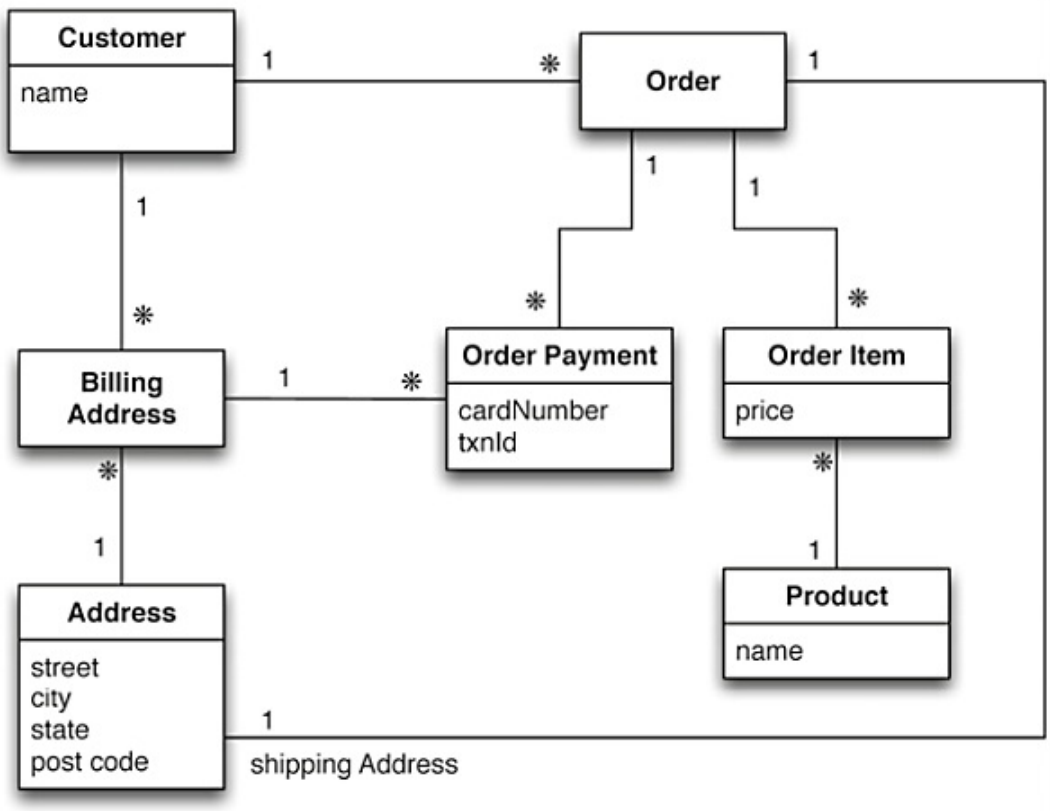

Figura 2.1 – Modelo de dados em um banco de dados relacional (utilizando a notação UML [Fowler UML]).

A figura 2.2 mostra alguns dados de exemplo para esse modelo.

Customer

Id	Name
1	Martin

Orders

Id	CustomerId	ShippingAddressId
99	1	77

Product

Id	Name
27	NoSQL Distilled

BillingAddress

Id	CustomerId	AddressId
55	1	77

OrderItem

Id	OrderId	ProductId	Price
100	99	27	32.45

Address

Id	City
77	Chicago

OrderPayment

Id	OrderId	CardNumber	BillingAddressId	txnId
33	99	1000-1000	55	abelif879rft

Figura 2.2 – Dados típicos utilizando o modelo de dados SGBDR.

Como somos bons soldados relacionais, tudo está devidamente normalizado, de modo que nenhum dado se repete em tabelas múltiplas. Também temos integridade referencial. Um sistema realista de pedidos, naturalmente, seria mais detalhado, mas esse é o benefício do espaço limitado de um livro.

Vejamos agora como esse modelo fica quando pensamos em termos mais voltados à orientação agregada (Figura 2.3).

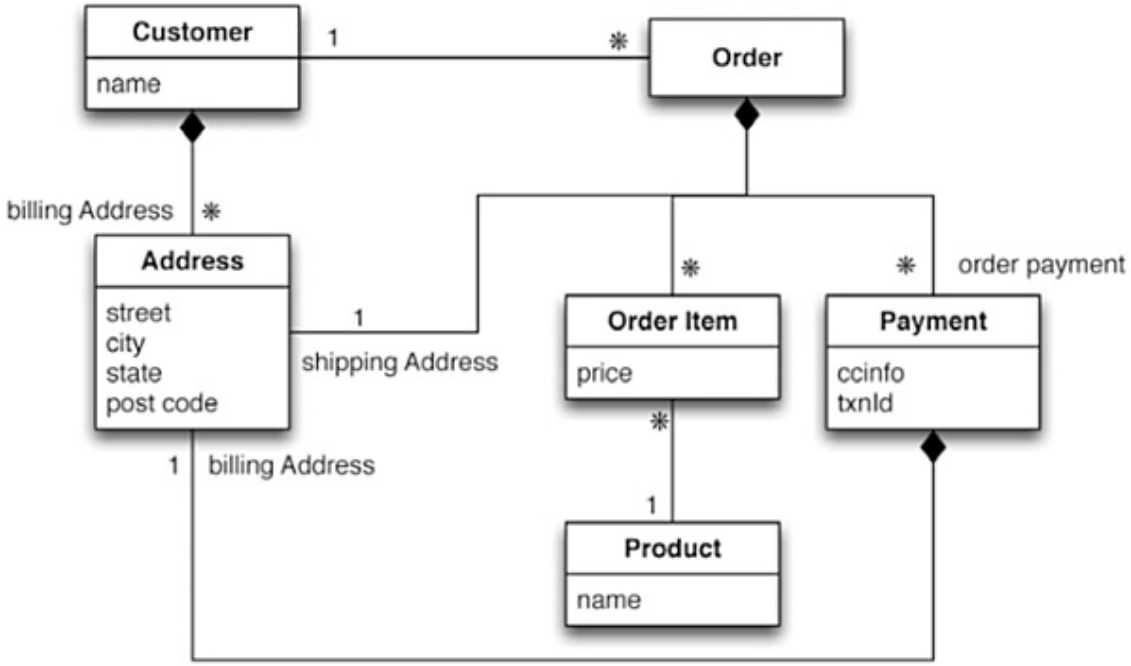

Figura 2.3 – Um modelo de dados agregado.

Mais uma vez, temos alguns dados de exemplo, os quais mostraremos em formato JSON, uma vez que essa é uma representação comum de dados na área de NoSQL.

```
// em clientes
{
   "id":1,
   "name":"Martin",
   "billingAddress":[{"city":"Chicago"}]
}
// em pedidos
{
   "id":99,
   "customerId":1,
   "orderItems":[
      {
         "productId":27,
         "price": 32.45,
         "productName": "NoSQL Essencial"
      }
   ],
   "shippingAddress":[{"city":"Chicago"}]
   "orderPayment":[
      {
```

```
            "ccinfo":"1000-1000-1000-1000",
            "txnId":"abelif879rft",
            "billingAddress": {"city": "Chicago"}
        }
    ],
}
```

Nesse modelo, temos dois agregados principais: cliente e pedido. Utilizamos o marcador de composição no UML, no formato de um losango preto, para mostrar como os dados são organizados na estrutura do agregado. O cliente contém uma lista de endereços de cobrança; o pedido contém uma lista de itens solicitados, um endereço de envio e pagamentos. O próprio pagamento contém um endereço de cobrança.

Um único registro lógico de endereço aparece três vezes nos dados de exemplo, mas, em vez de utilizar IDs, esse registro é tratado como um valor e copiado a cada vez. Isso se ajusta ao domínio no qual não gostaríamos que o endereço de envio nem o endereço de cobrança do pagamento fossem alterados. Em um banco de dados relacional, asseguraríamos que as linhas do endereço não seriam atualizadas, criando uma nova linha. Com agregados, podemos copiar toda a estrutura do endereço para o agregado, conforme a nossa necessidade.

A conexão entre o cliente e o pedido não está em nenhum agregado – é um relacionamento entre agregados. De forma semelhante, a conexão a partir de um item de pedido cruzaria em uma estrutura agregada separada para produtos, a qual ainda não vimos. Aqui, mostramos o nome do produto como parte do item do pedido – esse tipo de desnormalização é semelhante às permutas realizadas nos bancos de dados relacionais, mas é mais comum com agregados, pois queremos minimizar o número de agregados que acessamos durante uma interação de dados.

É importante que se perceba, aqui, não a forma específica pela qual definimos os limites dos agregados, mas o fato de que você tem de pensar sobre o acesso àqueles dados – e tornar isso parte do raciocínio ao desenvolver o modelo de dados do aplicativo. Certamente, poderíamos definir os limites dos agregados de modo diferente, colocando todos os pedidos de um cliente no agregado do cliente (Figura 2.4).

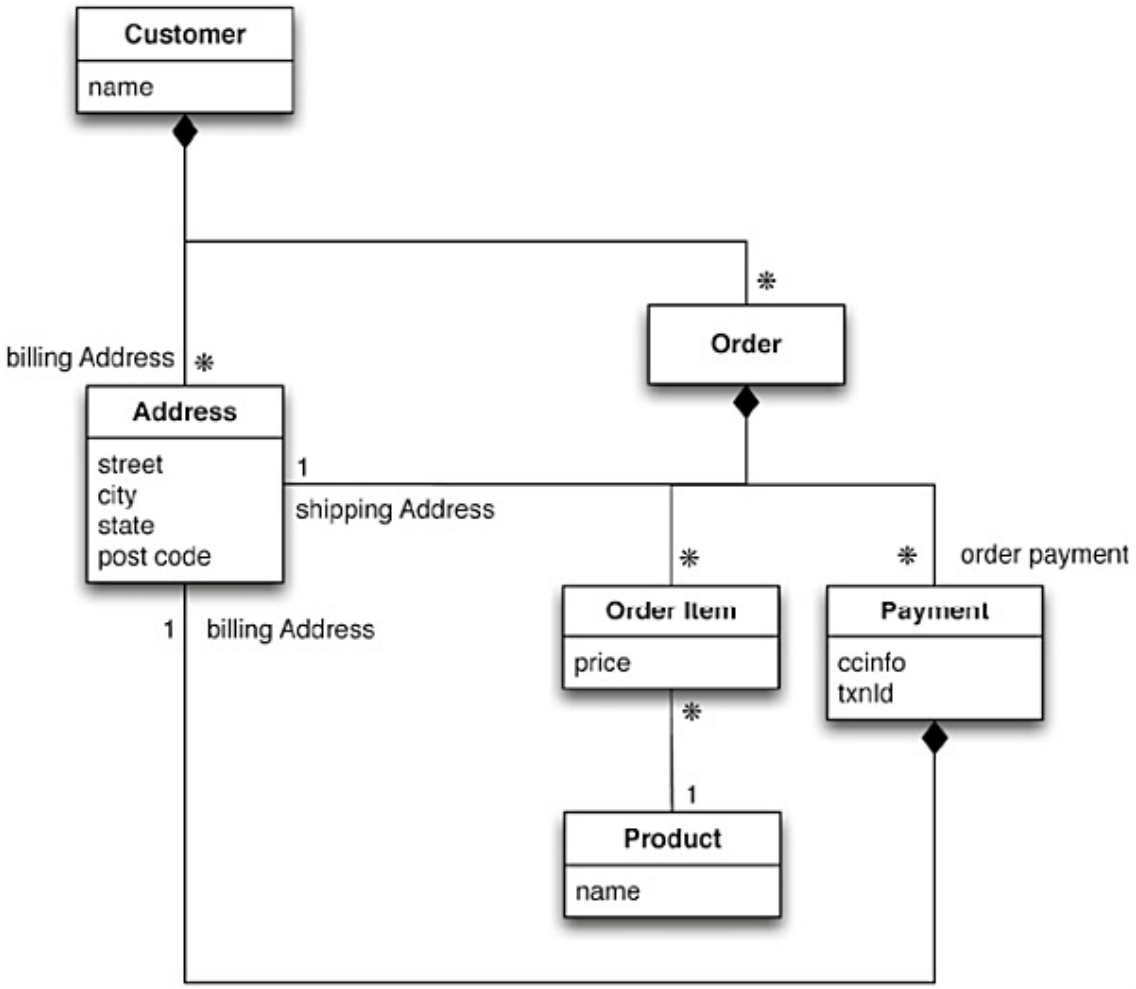

Figura 2.4 – Incorporar todos os objetos e os pedidos do cliente.

Utilizando o modelo de dados anterior, exemplos de `Customer` (Cliente) e `Order` (Pedido) teriam o seguinte formato:

```
// em clientes
{
   "customer": {
      "id": 1,
      "name": "Martin",
      "billingAddress": [{"city": "Chicago"}],
      "orders": [
         {
            "id":99,
            "customerId":1,
            "orderItems":[
               {
                  "productId":27,
                  "price": 32.45,
                  "productName": "NoSQL Essencial"
               }
            ],
            "shippingAddress":[{"city":"Chicago"}]
            "orderPayment":[
               {
```

```
                    "ccinfo":"1000-1000-1000-1000",
                    "txnId":"abelif879rft",
                    "billingAddress": {"city": "Chicago"}
                }],
            }]
        }
    }
```

Assim como na maioria das situações em modelagem, não há uma resposta universal acerca de como determinar os limites dos agregados. Isso depende inteiramente de como os dados serão manipulados. Se a intenção é acessar todos os pedidos de um cliente ao mesmo tempo, deve-se optar por um único agregado. Todavia, se o acesso será feito a um pedido de cada vez, então deve-se optar por ter agregados separados para cada pedido. Naturalmente, isso é muito específico para cada contexto; alguns aplicativos podem preferir tanto um quanto outro, mesmo dentro de um único sistema, e é exatamente por isso que muitas pessoas preferem ignorar os agregados.

2.1.2 Consequências da orientação a agregados

Embora o mapeamento relacional capture os diversos elementos de dados e seus relacionamentos de forma razoavelmente boa, ele o faz sem noção alguma de uma entidade agregada. Em nossa linguagem de domínio, poderíamos dizer que um pedido consiste em itens solicitados, em um endereço para envio e em um pagamento. Isso pode ser expresso, no modelo relacional, na forma de relacionamentos com chaves estrangeiras – mas não há o que diferencie relacionamentos que representam agregações daqueles que não o fazem. Como consequência, o banco de dados não pode utilizar um conhecimento da estrutura agregada para ajudar a armazenar e distribuir os dados.

Diversas técnicas de modelagem de dados forneceram formas de marcar agregados ou estruturas compostas. O problema, todavia, é que os modeladores raramente fornecem semânticas para tornar um relacionamento de agregados diferente do outro; onde há semântica, eles variam. Ao trabalhar com bancos de dados orientados a agregados, temos uma semântica mais clara a considerar, enfocando a unidade da interação com o armazenamento de dados. Não é, entretanto, uma propriedade de dados lógica: está relacionada com a forma pela qual os dados estão sendo utilizados pelos aplicativos. Essa é uma preocupação que muitas vezes está fora dos limites da modelagem de dados.

Bancos de dados relacionais não possuem o conceito de agregado em seu modelo de dados, de modo que os chamamos de "não agregados" (*aggregate-ignorant*). No mundo NoSQL, bancos de dados de grafos também são não agregados. Esse fato não é ruim. Muitas vezes, é difícil estabelecer bem os limites dos agregados, especialmente se os mesmos dados forem utilizados em muitos contextos diferentes. Um pedido constitui um bom agregado quando um cliente estiver fazendo e revisando pedidos, e quando o varejista estiver processando os pedidos. No entanto, se um varejista quiser analisar as vendas de seus produtos nos últimos meses, um agregado de pedidos tornar-se-á, então, um problema. Para obter o histórico das vendas do produto, será necessário pesquisar cada agregado do banco de dados separadamente. Assim, uma estrutura de agregado pode ser útil com algumas interações de dados, mas ser um obstáculo com outras. Um modelo não agregado permite que os dados de diferentes formas sejam facilmente examinados, tornando-se uma escolha melhor quando não houver uma estrutura primária para

manipular os dados.

O motivo mais importante para a orientação a agregados é que ela realmente auxilia a execução em um cluster, que é o argumento crucial para a ascensão do NoSQL. Se estivermos operando em um cluster, é necessário minimizar o número de nodos que precisamos pesquisar quando estivermos coletando os dados. Incluindo os agregados explicitamente, damos ao banco de dados informações importantes sobre quais partes dos dados serão manipuladas juntas e que, portanto, deverão ficar no mesmo nodo.

Os agregados têm uma consequência importante para transações. Bancos de dados relacionais permitem a manipulação de qualquer combinação de linhas de quaisquer tabelas em uma única transação. Tais transações são chamadas de **transações ACID**: Atômicas, Consistentes, Isoladas e Duráveis. ACID é um acrônimo bastante artificial; a questão real é a atomicidade: muitas linhas espalhadas por muitas tabelas são atualizadas como uma única operação. Ou essa operação tem sucesso total, ou falha integralmente e operações concorrentes ficam isoladas umas das outras, de modo que não possam ver uma atualização parcial.

Muitas vezes diz-se que bancos de dados NoSQL não suportam transações ACID e, por isso, sacrificam a consistência. Essa é uma simplificação muito genérica. De modo geral, é verdade que bancos de dados orientados a agregados não possuem transações ACID que se espalham por múltiplos agregados. Em vez disso, eles suportam manipulação atômica em um único agregado por vez, o que significa que, se precisarmos manipular múltiplos agregados de uma forma atômica, nós mesmos temos de gerenciar isso no código do aplicativo. Na prática, descobrimos que, na maior parte do tempo, podemos manter as nossas necessidades de atomicidade dentro de um único agregado; de fato, essa é a parte que se considera ao decidir-se sobre como dividir nossos dados em agregados. Também devemos lembrar que bancos de dados de grafos e outros bancos de dados não agregados suportam, geralmente, transações ACID de modo semelhante aos bancos de dados relacionais. Acima de tudo, o tema da consistência envolve muito mais do que apenas a questão de um banco de dados ser ACID ou não, conforme exploraremos no capítulo 5.

2.2 Modelos de dados de chave-valor e de documentos

Dissemos anteriormente que bancos de dados de chave-valor e de documentos eram fortemente orientados a agregados. O que queremos dizer é que pensamos nesses bancos de dados como criados, principalmente, por meio de agregados. Ambos os tipos de bancos de dados consistem em muitos agregados, tendo cada agregado uma chave ou ID utilizado para obter os dados.

Os dois modelos diferem no fato de que, em um banco de dados de chave-valor, o agregado é opaco para o banco de dados – apenas um grande amontoado, em sua maioria, de bits sem significado – e um banco de dados de documentos pode ver uma estrutura no agregado. A vantagem da opacidade é que podemos armazenar o que bem entendermos no agregado. O banco de dados pode impor algum limite de tamanho, mas, de forma geral, temos liberdade completa. Um banco de dados de documentos impõe limites sobre o que podemos inserir nele, definindo quais as estruturas e os tipos permitidos. Em compensação, porém, obtemos mais flexibilidade de acesso.

Com um armazenamento de chave-valor, somente podemos acessar um agregado

procurando por sua chave. Com um banco de dados de documentos, podemos submeter consultas ao banco de dados baseadas nos campos do agregado, podemos recuperar parte do agregado em vez de todo ele e, além disso, o banco de dados pode criar índices baseando-se no conteúdo do agregado.

Na prática, a linha entre chave-valor e documentos é um pouco difusa. As pessoas, muitas vezes, colocam um campo ID em um banco de dados de documentos para fazer uma pesquisa no estilo chave-valor. Bancos de dados classificados como chave-valor podem permitir estruturas de dados, além de um agregado opaco. Por exemplo, o Riak permite que você adicione metadados a agregados para indexação e conexões entre interagregados. Já o Redis permite que o agregado seja dividido em listas ou conjuntos. Você pode suportar consultas integrando ferramentas de pesquisa, como a Solr. Por exemplo, o Riak inclui um recurso de consulta que utiliza pesquisas do tipo Solr, buscando em quaisquer agregados que estejam armazenados como estruturas XML ou JSON.

Apesar dessa difusão, a distinção geral permanece. Com bancos de dados de chave-valor, esperamos, principalmente, procurar agregados utilizando uma chave. Com bancos de dados de documentos, esperamos, principalmente, submeter uma forma de consulta baseada na estrutura interna do documento; essa poderia ser uma chave, mas é mais provável que seja algo diferente.

2.3 Armazenamentos de famílias de colunas

Um dos primeiros bancos de dados NoSQL influentes foi o BigTable do Google [Chang etc.]. Seu nome evoca uma estrutura tabular que o banco de dados BigTable via como colunas esparsas e sem esquema. Conforme veremos em breve, não é bom pensar nessa estrutura como uma tabela, mas, sim, como um mapa em dois níveis. Todavia, seja qual for o modo pelo qual você pensa na estrutura, esse foi um modelo que influenciou bancos de dados posteriores, como o HBase e o Cassandra.

Esses bancos de dados no estilo do BigTable são, muitas vezes, chamados de armazenamento em colunas, mas esse nome já existe há algum tempo para descrever algo diferente. Armazenamentos do tipo de colunas anteriores a NoSQL, como C-Store [C-Store], estavam satisfeitos com SQL e o modelo relacional. O que os tornou diferentes foi o modo pelo qual armazenavam fisicamente os dados. A maioria dos bancos de dados utiliza a linha como unidade de armazenamento, o que, em particular, auxilia o desempenho de gravação. Entretanto, há muitos cenários em que gravações são raras, mas é necessário, frequentemente, ler algumas colunas de muitas linhas de uma só vez. Nessa situação, é melhor armazenar grupos de colunas para todas as linhas, como a unidade básica de armazenamento – motivo pelo qual esses bancos de dados são chamados de armazenamento em colunas.

O BigTable e seus descendentes seguem essa noção de armazenamento de grupos de colunas (famílias de colunas) juntas, mas diferem do C-Store e similares por abandonarem o modelo relacional e o SQL. Neste livro, referimo-nos a essa classe de bancos de dados como bancos de dados de famílias de colunas.

Talvez a melhor maneira de pensar no modelo de famílias de colunas seja como uma estrutura agregada em dois níveis. Assim como com armazenamentos de chave-valor, a

primeira chave é muitas vezes descrita como um identificador de linha, capturando o agregado de interesse. A diferença em relação a estruturas de famílias de colunas é que esse agregado de linha é formado, por si só, por um mapa com valores mais detalhados. Esses valores de segundo nível são chamados de colunas. Além de acessarem a linha como um todo, as operações também permitem a seleção de uma coluna em particular, de modo que, para obter o nome de um determinado cliente, conforme a figura 2.5, poderíamos fazer algo como `get('1234', 'name')`.

Bancos de dados de famílias de colunas organizam suas colunas em famílias de colunas. Cada coluna tem de fazer parte de uma única família de colunas e a coluna atua como uma unidade de acesso, com a suposição de que os dados de uma família de colunas em particular, geralmente, serão acessados juntos.

Isso também permite-nos pensar em como os dados são estruturados.

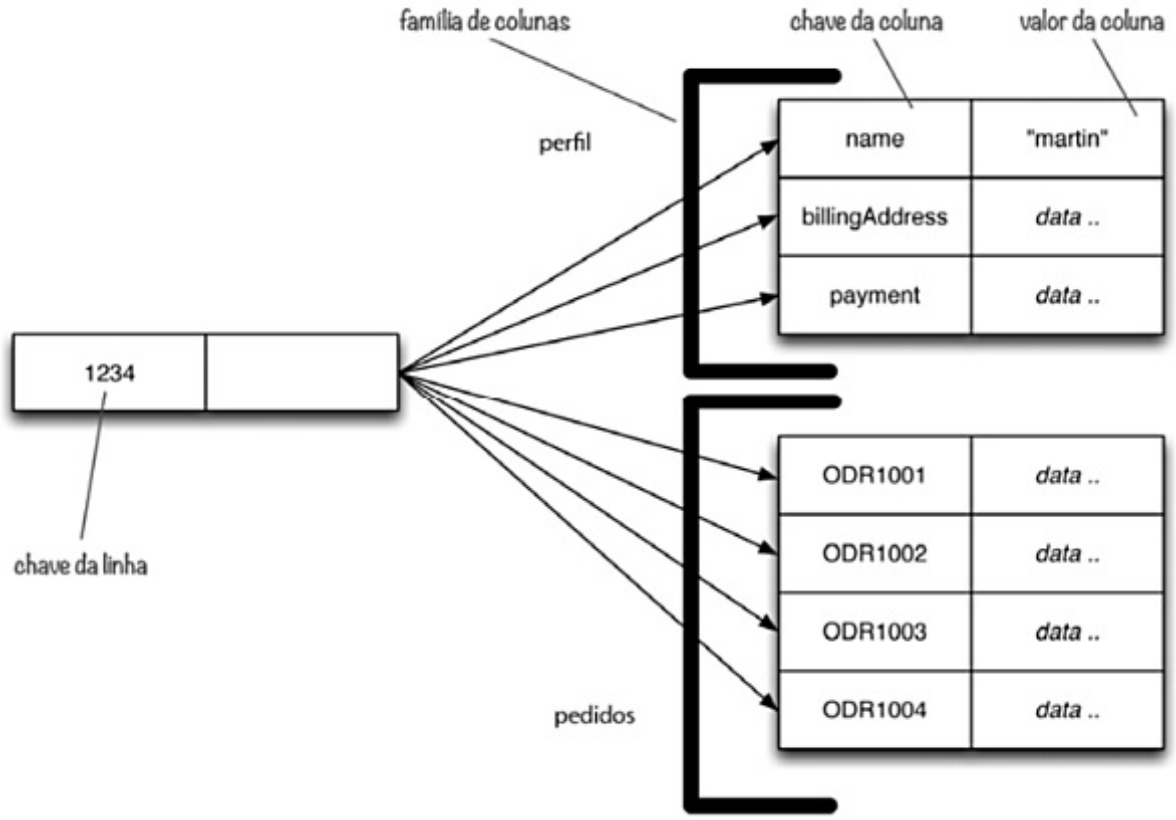

Figura 2.5 – Representando informações sobre o cliente em uma estrutura de família de colunas.

- **Orientado a linhas**: cada linha é um agregado (por exemplo, o cliente com o ID 1234) com famílias de colunas representando partes úteis de dados (perfil, histórico de pedidos) dentro desse agregado.
- **Orientado a colunas**: cada família de colunas define um tipo de registro (por exemplo, perfis de clientes) com linhas para cada um dos registros. Vamos pensar, então, em uma linha como a junção de registros em todas as famílias de colunas.

Esse último aspecto reflete a natureza de colunas dos bancos de dados de famílias de colunas. Uma vez que o banco de dados conhece esses agrupamentos comuns de dados, pode utilizar essa informação para o seu armazenamento e acesso. Ainda que um banco de

dados de documentos declare alguma estrutura ao banco de dados, cada documento ainda é visto como uma única unidade. Famílias de colunas dão uma característica bidimensional aos bancos de dados de famílias de colunas.

Essa terminologia está de acordo com o que foi estabelecido pelo BigTable, do Google, e o HBase, mas o Cassandra tem uma visão um pouco diferente. Uma linha no Cassandra ocorre apenas em uma família de colunas, mas esta pode conter supercolunas (colunas que contêm colunas aninhadas). As supercolunas no Cassandra são as que mais equivalem às clássicas famílias de colunas do BigTable.

Ainda pode ser confuso pensar em famílias de colunas como tabelas. Você pode adicionar qualquer coluna a qualquer linha e as linhas podem ter chaves de coluna muito diferentes. Embora novas colunas sejam adicionadas a linhas durante o acesso regular ao banco de dados, definir novas famílias de colunas é muito mais raro e pode envolver uma interrupção no banco de dados para que essa ação possa ser realizada.

O exemplo da figura 2.5 ilustra outro aspecto do banco de dados de famílias de colunas que pode não ser muito comum para pessoas acostumadas a tabelas relacionais: a família de colunas "de pedidos". Uma vez que as colunas podem ser adicionadas livremente, podemos modelar uma lista de itens tornando cada item uma coluna separada. Isso é muito peculiar se pensarmos em uma família de colunas como uma tabela, mas bastante natural se pensarmos em uma linha de família de colunas como um agregado. O Cassandra utiliza os termos "larga" e "estreita". **Linhas estreitas** têm poucas colunas, sendo as mesmas colunas utilizadas pelas diferentes linhas. Nesse caso, a família de colunas define um tipo de registro; cada linha é um registro e cada coluna, um campo. Uma **linha larga** possui muitas colunas (talvez milhares) com as linhas tendo colunas muito diferentes. Uma família de colunas largas modela uma lista, sendo cada coluna um elemento dessa lista.

Uma consequência das famílias de colunas largas é que uma família de colunas pode definir uma forma de ordenação para suas colunas. Dessa maneira, podemos acessar pedidos por sua chave e acessar faixas de pedidos por suas chaves. Embora isso possa não ser útil se as chaves dos pedidos forem suas IDs, seria, se criássemos a chave a partir da concatenação de data e ID (por exemplo, 20111027-1001).

Ainda que seja útil distinguir famílias de colunas por sua natureza, larga ou estreita, não há motivo técnico pelo qual uma família de colunas não possa conter colunas do tipo de campo e colunas do tipo de lista – embora isso possa confundir o tipo de ordenação.

2.4 Resumindo os bancos de dados orientados a agregados

Neste ponto, já examinamos material suficiente para lhe obter uma visão geral razoável dos três estilos diferentes de modelos de dados orientados a agregados e como eles diferem entre si.

O que todos compartilham é a noção de um agregado indexado por uma chave que pode ser utilizada para buscas. Esse agregado é central na execução em um cluster, uma vez que o banco de dados assegurará que todos os dados de um agregado estejam armazenados juntos em um nodo. O agregado também atua como a unidade atômica para atualizações, fornecendo, mesmo que limitado, um controle transacional útil.

Dentro dessa noção de agregado, há algumas diferenças. O modelo de dados de chave-

valor trata o agregado como um todo opaco, o que significa que somente será possível fazer uma pesquisa por chave para o agregado como um todo, não sendo possível executar uma consulta nem recuperar apenas uma parte do agregado.

O modelo de documentos torna o agregado transparente para o banco de dados, permitindo que sejam executadas consultas e recuperações parciais. Entretanto, pelo fato de o documento não possuir um esquema, o banco de dados não pode atuar muito na estrutura desse documento para otimizar o armazenamento e a recuperação de partes do agregado.

Modelos de famílias de colunas dividem o agregado em famílias de colunas, permitindo ao banco de dados tratá-las como unidades de dados dentro do agregado da linha. Isso impõe alguma estrutura ao agregado, mas também permite que o banco de dados aproveite a estrutura para melhorar sua acessibilidade.

2.5 Leituras complementares

Para saber mais sobre o conceito geral de agregados, que são muitas vezes utilizados também com bancos de dados relacionais, veja [Evans]. A comunidade do Domain-Driven Design é a melhor fonte para obter as informações mais recentes sobre agregados. Estas podem ser encontradas em `http://domaindrivendesign.org`.

2.6 Pontos chave

- Um agregado é um conjunto de dados com o qual interagimos como uma unidade. Agregados formam os limites de operações ACID com o banco de dados.
- Bancos de dados de chave-valor, de documentos e de famílias de colunas podem ser vistos como formas de bancos de dados orientados a agregados.
- Os agregados facilitam, para o banco de dados, o gerenciamento do armazenamento de dados em clusters.
- Bancos de dados orientados a agregados funcionam melhor quando a maioria da interação com os dados é realizada no mesmo agregado; bancos de dados não agregados são melhores quando as interações utilizam dados organizados em muitas formações diferentes.

CAPÍTULO 3

Mais detalhes sobre modelos de dados

Até aqui, examinamos a característica principal da maioria dos bancos de dados NoSQL: o uso de agregados e a forma como os bancos de dados orientados a agregados modelam-nos de diferentes formas. Embora os agregados sejam a parte central da história do NoSQL, há outros aspectos relacionados à modelagem de dados, os quais exploraremos neste capítulo.

3.1 Relacionamentos

Agregados são úteis, pois agrupam dados que são comumente acessados juntos. Mas ainda há muitos casos em que dados relacionados são acessados de formas diferentes. Considere a relação entre um cliente e todos os seus pedidos. Alguns aplicativos optarão por acessar o histórico de pedidos sempre que acessarem o cliente, o que se enquadra bem na combinação do cliente com seu histórico de pedidos em um único agregado.

Outros aplicativos, entretanto, processarão os pedidos individualmente e, assim, modelarão pedidos como agregados independentes. Nesse caso, os agregados de pedidos e de clientes serão separados, mas estarão relacionados de alguma maneira, de modo que qualquer ação executada sobre um pedido possa levar aos dados do cliente. A forma mais simples de obter essa conexão é inserindo o ID do cliente nos dados do agregado do pedido. Assim, para obter os dados do registro do cliente, deve-se ler o pedido, descobrir o ID do cliente e acessar novamente o banco de dados para ler os dados do cliente. Esse procedimento será eficiente e conveniente em muitos cenários, mas o banco de dados não terá ciência do relacionamento entre os dados. Esse é um detalhe importante, pois há vezes em que é útil que o banco de dados conheça essas conexões.

Consequentemente, muitos bancos de dados – até os de armazenamentos de chave-valor – fornecem meios de tornar esses relacionamentos visíveis para o banco de dados. O armazenamento de documentos disponibiliza o conteúdo do agregado para o banco de dados criar índices e consultas. O Riak, um armazenamento de chave-valor, permite que sejam inseridas informações de conexões em metadados, suportando a recuperação parcial e a capacidade de percorrer as conexões.

Um aspecto importante dos relacionamentos entre agregados é a maneira de eles lidarem com atualizações. Bancos de dados orientados a agregados tratam estes como a unidade de recuperação de dados. Consequentemente, a atomicidade somente é suportada dentro do conteúdo de um único agregado. Se atualizarmos múltiplos agregados de uma só vez, teremos de lidar com uma eventual falha durante esse processo. Bancos de dados relacionais podem ajudar, permitindo que sejam modificados múltiplos registros em uma única transação, fornecendo garantias ACID enquanto alteram muitas linhas.

Tudo isso significa que bancos de dados orientados a agregados tornam-se mais complexos se for necessário trabalhar em múltiplos agregados. Há diversas formas de lidar com isso, e elas serão exploradas posteriormente neste capítulo. Porém, a dificuldade

fundamental permanece.

Se houver dados baseados em muitos relacionamentos, deve-se preferir um banco de dados relacional a um armazenamento NoSQL. Embora isso seja verdade para bancos de dados orientados a agregados, vale a pena lembrar que bancos de dados relacionais também não são tão espetaculares no que diz respeito a relacionamentos complexos. Embora seja possível fazer consultas envolvendo junções (`JOIN`) em SQL, a situação pode, rapidamente, tornar-se complicada, tanto com a escrita do SQL, quanto com o desempenho resultante, quando o número de junções aumenta.

Isso torna o momento propício para apresentar outra categoria de bancos de dados que, muitas vezes, é colocada no mesmo grupo de NoSQL.

3.2 Bancos de dados de grafos

Bancos de dados de grafos não são habituais no contexto de NoSQL. A maioria dos bancos de dados NoSQL foi inspirada pela necessidade da execução em clusters, o que levou a modelos de dados orientados a agregados para grandes registros com conexões simples. Bancos de dados de grafos são motivados por uma frustração diferente com bancos de dados relacionais e, por isso, têm um modelo oposto – registros pequenos com interconexões complexas, similarmente à figura 3.1.

Nesse contexto, um grafo não é um gráfico ou um histograma; em vez disso, chamamos de grafo uma estrutura de dados de nodos conectados por arestas.

Na figura 3.1, temos uma rede de informações cujos nodos são muito pequenos (nada mais do que um nome), mas há uma estrutura rica de interconexões entre eles. Por meio dessa estrutura, é possível fazer solicitações, como “encontre os livros na categoria Bancos de Dados que tenham sido escritos por alguém de quem um amigo meu goste”.

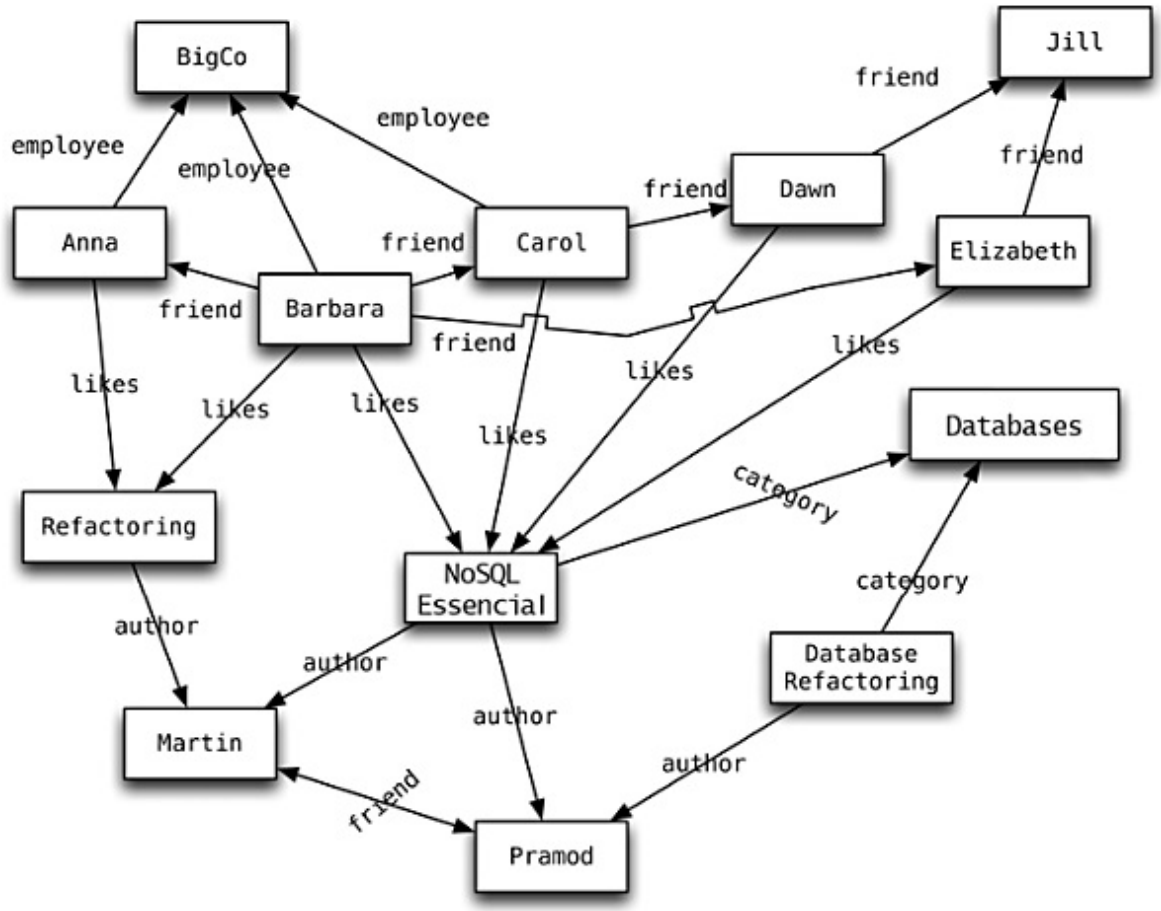

Figura 3.1 – Um exemplo de estrutura de grafo.

Bancos de dados de grafos são especializados na captura desse tipo de informação, mas em uma escala bem maior do que um diagrama legível poderia capturar. Isso é ótimo para representar quaisquer dados que consistam em relacionamentos complexos, como redes sociais, preferências de produtos ou regras de aceitabilidade.

O modelo de dado fundamental de um banco de dados de grafos é muito simples: nodos conectados por arestas (também chamadas de arcos). Além dessa característica essencial, há muita variação nos modelos de dados – em especial, nos mecanismos que você tem para armazenar dados em seus nodos e arestas. Estes são exemplos simples de algumas capacidades atuais que ilustram essa variedade de possibilidades: o FlockDB é, simplesmente, composto por nodos e arestas, sem um mecanismo de atributos adicionais; o Neo4J permite que sejam anexados objetos Java como propriedades em nodos e arestas, sem um esquema (“Recursos”, p. 163); o Infinite Graph armazena seus objetos Java, que são subclasses de seus tipos internos, como nodos e arestas.

Assim que é criado um grafo de nodos e arestas, um banco de dados de grafos permite consultas a essa rede por meio de operações de consulta projetadas com esse tipo de grafo em mente. É nesse momento que surgem as diferenças importantes entre os bancos de dados de grafos e os relacionais. Embora os bancos de dados relacionais possam implementar relacionamentos utilizando chaves estrangeiras, as junções necessárias para navegar por eles podem ser bastante custosas, o que significa que o desempenho é, muitas vezes, ruim para modelos de dados altamente conectados. Bancos de dados de grafos tornam menos custosas as travessias pelos relacionamentos. Em grande parte, isso ocorre

porque os bancos de dados de grafos modificam a maior parte do trabalho de navegação e de relacionamentos do momento da consulta para o momento da inserção. Isso compensa, naturalmente, situações em que o desempenho da consulta é mais importante do que a velocidade da inserção.

Na maior parte das vezes, encontraremos os dados navegando pela rede de arestas com consultas como "me informe sobre tudo o que a Anna e a Barbara gostam". Entretanto, será necessário um lugar para iniciar, de forma que, geralmente, alguns nodos podem ser indexados por um atributo, como o ID. Poderíamos buscar o ID (por exemplo, procurar as pessoas chamadas "Anna" e "Barbara") e, então, daríamos início ao uso das arestas. Ainda assim, bancos de dados de grafos esperam que a maior parte de seu trabalho de consulta seja navegar pelos relacionamentos.

A ênfase nos relacionamentos torna os bancos de dados de grafos muito diferentes dos orientados a agregados. Essa diferença no modelo de dados traz outras consequências, pois tais bancos de dados serão mais propensos a funcionar em um único servidor do que distribuídos em clusters. As transações ACID precisam cobrir múltiplos nodos e arestas para manter a consistência. Eles apenas têm em comum com os bancos de dados orientados a agregados sua rejeição ao modelo relacional e um aumento da atenção que receberam na mesma época do restante da área NoSQL.

3.3 Bancos de dados sem esquema

Um tema comumente discutido acerca de bancos de dados NoSQL é o fato de eles não utilizarem esquemas. Ao armazenar dados em um banco de dados relacional, primeiramente deve-se ter um esquema, ou seja, uma estrutura definida para o banco de dados, que diz quais tabelas existem, quais colunas existem e quais tipos de dados cada coluna pode armazenar. Antes de armazenar algum dado, deve-se ter o esquema definido para ele.

Com bancos de dados NoSQL, armazenar dados torna-se muito mais informal. Um armazenamento de chave-valor permite o armazenamento de quaisquer dados sob uma chave. Um banco de dados de documentos faz, efetivamente, o mesmo, uma vez que não tem restrições à estrutura dos documentos armazenados. Bancos de dados de famílias de colunas permitem que sejam armazenados quaisquer dados sob qualquer coluna escolhida. Bancos de dados de grafos permitem que sejam adicionadas, livremente, novas arestas e propriedades aos nodos e às arestas.

Defensores das estruturas livres de esquemas ficam felizes com essa liberdade e flexibilidade. Tendo um esquema, você precisa saber de antemão o que armazenar, mas isso pode ser difícil. Sem um esquema, você pode armazenar, facilmente, o que quiser. Isso permite alterar facilmente o armazenamento de dados à medida que você conhece mais sobre seu projeto. Você pode adicionar tranquilamente a ele novos elementos à medida que os descobre. Além disso, se concluir que não precisa mais de algo, pode simplesmente deixar de armazená-lo, sem se preocupar com a perda de dados antigos, como aconteceria se excluísse colunas em um esquema relacional.

Assim como na manipulação de alterações, o armazenamento sem esquema traz facilidade ao se lidar com **dados não uniformes** (dados nos quais cada registro possui um conjunto diferente de campos). Um esquema coloca todas as linhas de uma tabela em uma

camisa de força, o que se torna complexo se houver diferentes tipos de dados em diferentes linhas. O resultado são muitas colunas, geralmente nulas (uma tabela esparsa), ou colunas sem significado, como a `custom column 4` (coluna especial 4). A ausência de esquema evita isso, permitindo que cada registro contenha apenas o necessário – nada a mais, nada a menos.

Não utilizar esquemas é algo atrativo e, certamente, evita muitos problemas existentes em bancos de dados de esquemas fixos, mas novos problemas podem surgir. Se o que se deseja é armazenar alguns dados e exibi-los na forma de relatórios, como uma lista simples de linhas `fieldName: value` (*nomeDoCampo: valor*), então um esquema somente atrapalhará. Porém, geralmente, fazemos mais do que isso com nossos dados, fazemos com programas, que precisam saber que o endereço de cobrança é chamado de `billingAddress` (*endereço de cobrança*) e não `addressForBilling` (*endereço para cobrança*) e que o conteúdo do campo `quantify` (*quantificar*) será um inteiro 5 e não "cinco".

O fato crucial, e às vezes inconveniente, é que todas as vezes em que escrevemos um programa que acessa dados, este programa, quase sempre, baseia-se em alguma forma de esquema implícito. A menos que seja algo como:

```
// pseudocódigo
foreach (Record r in records) {
   foreach (Field f in r.fields) {
      print (f.name, f.value)
   }
}
```

Assume-se que determinados nomes de campos estão presentes e contêm dados com um significado, e também assume-se algo sobre o tipo de dados armazenados dentro desse campo. Programas não são humanos; eles não conseguem ler "qty" e deduzir que significa "quantity" (quantidade), pelo menos, não sem que o programe peça para fazê-lo. Assim, por mais livre de esquema que nosso banco de dados seja, geralmente, há a presença de um **esquema implícito**, que é um conjunto de suposições sobre a estrutura dos dados no código que os manipula.

Ter um esquema implícito no código do aplicativo resulta em alguns problemas, o que significa que, para poder entender qual o tipo dos dados, deve-se examinar o código do aplicativo. Se esse código estiver bem estruturado, é necessário encontrar um bom local a partir do qual se pode deduzir o esquema. Não há garantia, porém; tudo depende do quão claro o código do aplicativo é. Além disso, o banco de dados continua desconhecendo o esquema – ele não consegue utilizar o esquema para ajudar a decidir como armazenar e recuperar dados com eficiência. Ele não pode aplicar suas próprias validações sobre esses dados para garantir que aplicações diferentes não os manipulem de forma inconsistente.

Essas são as razões pelas quais os bancos de dados relacionais têm um esquema fixo e, de fato, são as razões pelas quais a maioria dos bancos de dados possuem esquemas fixos. Esquemas têm seu valor e a rejeição de esquemas, pelos bancos de dados NoSQL, é, realmente, bastante surpreendente.

Basicamente, um banco de dados sem esquema transfere o esquema para o código do aplicativo que o acessa, o que se torna problemático se múltiplos aplicativos, desenvolvidos por pessoas diferentes, acessarem o mesmo banco de dados. Essas situações podem ser resolvidas por meio de algumas abordagens. Uma delas é encapsular toda a

interação do banco de dados em um único aplicativo e integrá-lo com outros aplicativos utilizando webservices. Essa ação adapta-se bem à preferência atual de muitas pessoas pelo uso de webservices para integração. Outra abordagem seria delinear, com clareza, áreas diferentes de um agregado para acesso por diferentes aplicativos. Estas poderiam ser seções diferentes em um banco de dados de documentos ou diferentes famílias de colunas em um banco de dados de colunas de famílias.

Os fãs de NoSQL, muitas vezes, criticam os esquemas relacionais por estes terem de ser definidos de antemão e serem inflexíveis, mas isso não é verdade. Esquemas relacionais podem ser alterados a qualquer momento por meio de comandos com padrão SQL. Caso seja necessário, pode-se criar colunas de uma forma específica para armazenar dados não uniformes. Nós, raramente, vemos isso sendo feito, mas funcionou razoavelmente bem onde se fez. Na maioria das vezes, porém, a falta de uniformidade nos dados é um bom motivo para favorecer um banco de dados sem esquema.

Com o passar do tempo, a não utilização de esquemas causou um grande impacto nas alterações em uma estrutura de banco de dados, especialmente em dados mais uniformes. Embora não seja tão praticada quanto deveria, a alteração do esquema de um banco de dados relacional pode ser realizada de um modo controlado. De forma semelhante, deve-se exercitar o controle ao alterar o modo pelo qual os dados são armazenados em um banco de dados sem esquema, de forma que se possa acessar facilmente tanto os dados antigos quanto os novos. Além disso, a flexibilidade que a ausência de esquema confere somente se aplica dentro de um agregado – se for necessário alterar os limites de seu agregado, a migração é tão complexa quanto no caso relacional. Falaremos mais sobre a migração de bancos de dados posteriormente ("Migrações de esquema", p. 177).

3.4 Visões materializadas (Materialized views)

Quando falamos em modelos de dados orientados a agregados, enfatizamos suas vantagens. Se quisermos acessar pedidos, é útil ter todos os dados de um pedido em um único agregado, que possa ser armazenado e acessado como uma unidade. Todavia, a orientação a agregados possui uma desvantagem subjacente: o que acontece se um gerente de produto quiser saber quanto um determinado item vendeu nas últimas semanas? Agora, a orientação a agregados funciona contra você, forçando-o, possivelmente, a ler todos os pedidos no banco de dados para responder à questão. Você pode reduzir essa carga criando um índice no produto, mas você trabalhará contra a estrutura do agregado.

Bancos de dados relacionais têm uma vantagem aqui, porque sua falta de estrutura de agregados permite-lhes suportar o acesso aos dados de diferentes maneiras. Além disso, eles fornecem um mecanismo conveniente, que permite que os dados sejam examinados de forma diferente daquela como estão armazenados – visões (views). Uma visão é como uma tabela relacional (é uma relação), mas é definida pela computação sobre as tabelas básicas. Quando acessamos uma visão, o banco de dados computa os dados nela – uma forma útil de encapsulamento.

Visões fornecem um mecanismo para esconder do cliente a origem dos dados, derivados ou básicos, mas não se pode negar o fato de que algumas visões são custosas para computar. Para lidar com isso, foram inventadas as **visões materializadas** (materialized views), que são visões computadas de antemão e inseridas em cache no disco. Visões

materializadas são efetivas para dados muito lidos, mas podem se tornar um pouco desatualizadas.

Embora os bancos de dados NoSQL não tenham visões, eles podem ter consultas pré-computadas, postas em cache, reutilizando o termo "visão materializada" para descrevê-las. Também é um aspecto mais importante para os bancos de dados orientados a agregados do que para sistemas relacionais, uma vez que a maioria dos aplicativos terá de lidar com algumas consultas que não se adaptam bem à cultura de agregados (muitas vezes, os bancos de dados NoSQL criam visões materializadas utilizando uma computação *map-reduce* (mapeamento reduzido), sobre a qual falaremos no capítulo 7).

Há duas estratégias gerais para a criação de uma visão materializada. A primeira é a abordagem antecipada, em que você atualiza ao mesmo tempo a visão materializada e os dados básicos relacionados a ela. Nesse caso, adicionar um pedido também atualizaria o histórico de compras do agregado para cada produto. Essa é uma boa abordagem quando há mais leituras da visão materializada do que gravações e queremos que as visões materializadas sejam tão atualizadas quanto for possível. A abordagem do banco de dados de aplicativos (p. 30) é valiosa, nesse caso, uma vez que facilita a garantia de que quaisquer atualizações realizadas nos dados básicos também atualizam as visões materializadas.

Se você não quisermos arcar com a sobrecarga a cada atualização, é possível fazer as atualizações das visões materializadas em lotes e intervalos regulares. Para isso, será necessário entender seus requisitos de negócio para avaliar quão desatualizadas as visões materializadas podem ficar.

Pode-se criar visões materializadas fora do banco de dados fazendo a leitura dos dados, computando a visão e gravando de volta nesse banco. É mais frequente os próprios bancos de dados suportarem a criação de visões materializadas. Nesse caso, você fornece a computação que precisa ser feita e o banco de dados a executa quando necessário, de acordo com alguns parâmetros que você mesmo configura. Isso é especialmente útil para atualizações precoces de visões com *map-reduce incremental* ("Map-reduce incremental ", p. 118).

Visões materializadas podem ser utilizadas dentro do mesmo agregado. Um documento de pedido poderia incluir um elemento de resumo do pedido, o qual fornece informações resumidas sobre o pedido, de modo que uma consulta de um resumo de pedido não tenha de transferir o documento inteiro do pedido. Utilizar famílias de colunas diferentes para visões materializadas é um recurso comum de bancos de dados de famílias de colunas. Uma vantagem ao se fazer essa configuração é que ela permite a atualização da visão materializada na mesma operação atômica.

3.5 Modelando para o acesso aos dados

Conforme mencionado anteriormente, ao modelar agregados de dados, precisamos considerar como esses dados serão lidos, assim como quais são os efeitos colaterais sobre os dados relacionados àqueles agregados.

Começaremos com o modelo em que todos os dados do cliente estão em um armazenamento de chave-valor (Figura 3.2).

Nesse cenário, o aplicativo pode ler as informações do cliente e todos os dados relacionados utilizando a chave. Se os requisitos feitos forem de leitura dos pedidos ou dos

produtos vendidos em cada pedido, o objeto inteiro tem de ser lido e depois examinado no lado do cliente para obter resultados. Quando são necessárias referências, podemos alternar para armazenamentos de documentos e depois consultar dentro deles. Ainda se poderia, até mesmo, mudar os dados para o armazenamento de chave-valor de modo a separar o objeto do valor em objetos `Customer` (cliente) e `Order` (pedido) e, então, manter as referências desses objetos entre si.

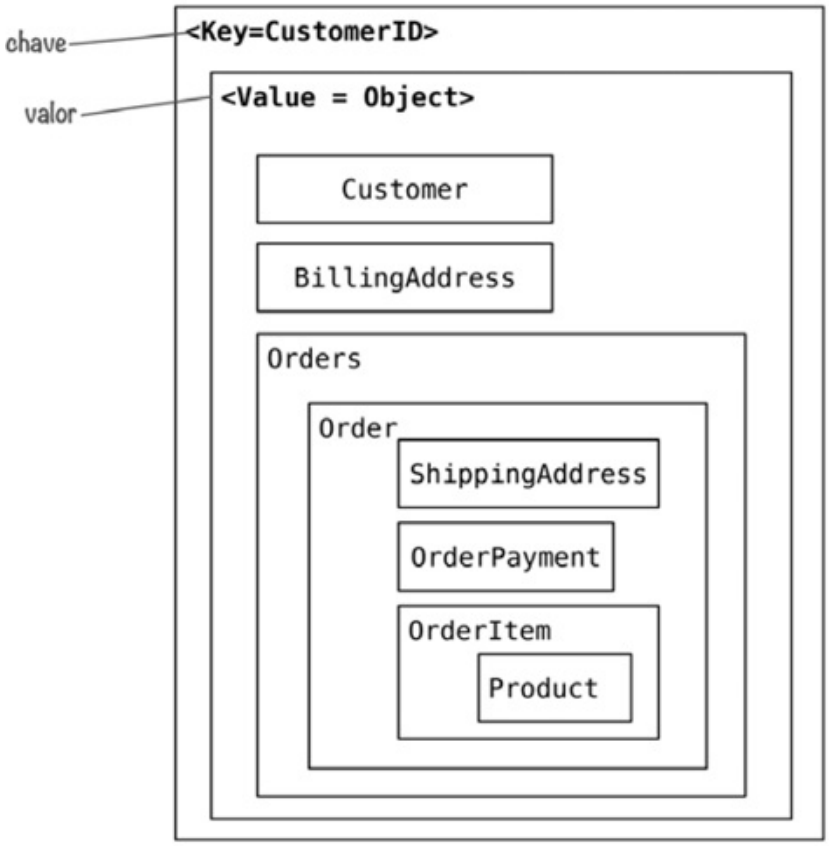

Figura 3.2 – Todos os objetos de clientes e seus pedidos estão embutidos.

Tendo as referências (Figura 3.3), podemos encontrar os pedidos independentemente do cliente (`Customer`) e, com a referência `orderId` em `Customer`, podemos encontrar todos os pedidos (`Orders`) do cliente (`Customer`). Utilizar agregados dessa maneira permite a otimização da leitura, mas temos de enviar a referência `orderId` para o `Customer` toda vez que houver um novo pedido (`Order`).

```
# Objeto Customer
{
   "customerId": 1,
   "customer": {
      "name": "Martin",
      "billingAddress": [{"city": "Chicago"}],
      "payment": [{"type": "debit","ccinfo": "1000-1000-1000-1000"}],
      "orders":[{"orderId":99}]
   }
}
# Objeto Order
{
   "customerId": 1,
```

```
    "orderId": 99,
    "order":{
       "orderDate":"Nov-20-2011",
       "orderItems":[{"productId":27, "price": 32.45}],
       "orderPayment":[{
          "ccinfo":"1000-1000-1000-1000",
          "txnId":"abelif879rft"
       }],
       "shippingAddress":{"city":"Chicago"}
    }
}
```

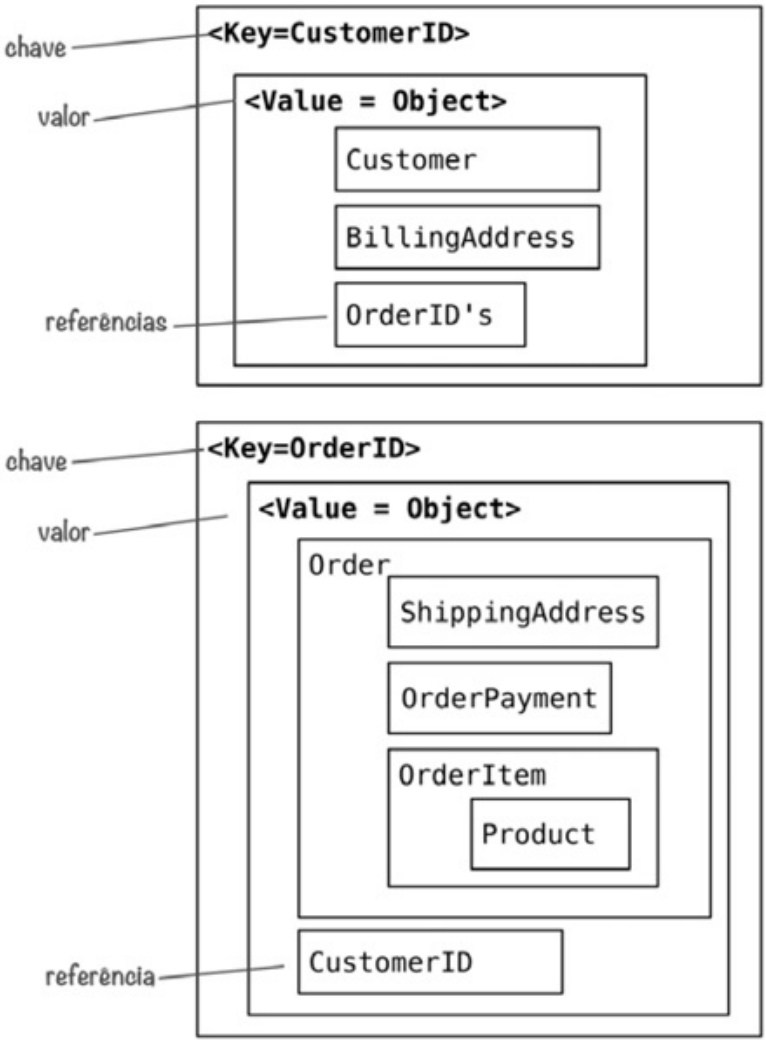

Figura 3.3 – Customer é armazenado separadamente de Order.

Agregados também podem ser utilizados para obter estatísticas: por exemplo, uma atualização no agregado pode preencher as informações sobre quais pedidos (`Orders`) contêm um determinado produto (`Product`). Essa desnormalização permite acesso rápido aos dados nos quais estamos interessados e é a base da **BI em Tempo Real** ou **Análise em Tempo Real,** na qual as empresas não dependem de lotes executados ao final do dia para povoar os dados em tabelas de *warehouses* e gerar análises; agora, elas podem preencher esse tipo de dado para vários tipos de requisitos sempre que o pedido for realizado pelo cliente.

```
{
```

```
    "itemid":27,
    "orders":{99,545,897,678}
}
{
    "itemid":29,
    "orders":{199,545,704,819}
}
```

Em armazenamentos de documentos, pelo fato de podermos fazer consultas dentro deles, é possível a remoção de referências a `Orders` no objeto `Customer`. Essa alteração elimina a necessidade de atualizar o objeto `Customer` quando novos pedidos forem realizados pelo cliente.

```
# Objeto Customer
{
    "customerId": 1,
    "name": "Martin",
    "billingAddress": [{"city": "Chicago"}],
    "payment": [{
        "type": "debit",
        "ccinfo": "1000-1000-1000-1000"
    }]
}
# Objeto Order
{
    "orderId": 99,
    "customerId": 1,
    "orderDate":"Nov-20-2011",
    "orderItems":[{"productId":27, "price": 32.45}],
    "orderPayment":[{
        "ccinfo":"1000-1000-1000-1000",
        "txnId":"abelif879rft"
    }],
    "shippingAddress":{"city":"Chicago"}
}
```

Uma vez que os armazenamentos de dados de documentos permitem que se faça a consulta pelos atributos de dentro do documento, pesquisas como "encontre todos os pedidos que incluam o produto *Refactoring Databases*" são possíveis. Entretanto, a decisão de criar um agregado para os itens e pedidos aos quais eles pertencem não é baseada na capacidade de consulta do banco de dados, mas, sim, na otimização de leitura desejada pelo aplicativo.

Ao modelar para armazenamentos de famílias de colunas, temos o benefício das colunas estarem ordenadas, permitindo-nos nomear colunas que sejam utilizadas com frequência, de modo a serem trazidas primeiramente. Ao utilizar as famílias de colunas para modelar os dados, é importante lembrar-se de fazê-lo pelas necessidades de sua consulta e não para o propósito de gravação; a regra é facilitar as consultas e desnormalizar os dados durante a gravação.

Como se pode imaginar, há várias formas de modelar os dados; uma é armazenar `Customer` e `Order` em famílias de colunas diferentes (Figura 3.4). Aqui, é importante observar que a referência a todos os pedidos realizados pelo cliente está na família de colunas de

`Customer`. Outras desnormalizações semelhantes, geralmente, são feitas de modo que o desempenho da consulta (leitura) melhore.

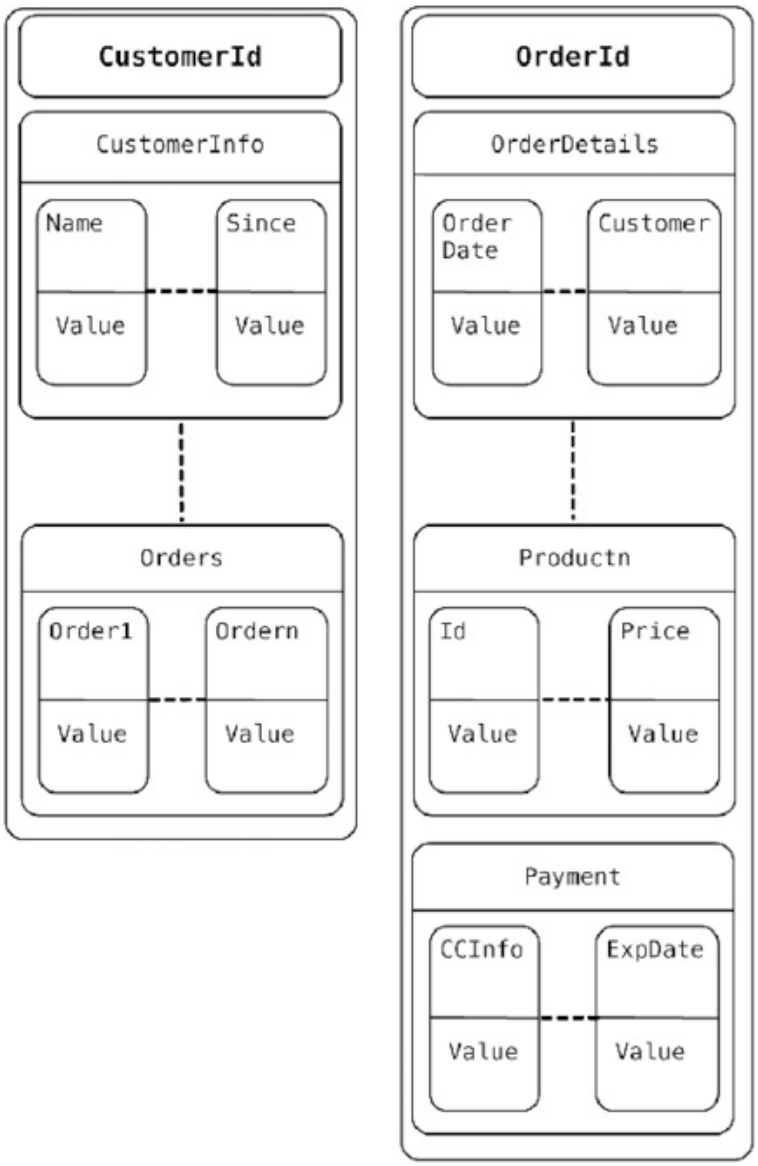

Figura 3.4 – Visão conceitual em um armazenamento de dados de coluna.

Ao utilizar bancos de dados de grafos para modelar os mesmos dados, modelamos todos os objetos como nodos, e as relações dentro deles, como relacionamentos, os quais têm tipos e significância direcional.

Cada nodo possui relacionamentos independentes com outros nodos. Esses relacionamentos têm nomes, como *COMPROU*, *PAGOU_COM* ou *PERTENCE_A* (Figura 3.5). Esses nomes de relacionamento permitem-nos percorrer o grafo. Digamos que você queira encontrar cada cliente que *COMPROU* um produto com o nome de *Refactoring Database*. Tudo o que precisamos fazer é encontrar o nodo do produto *Refactoring Database* e ver todos os `Customers` com o relacionamento de entrada *COMPROU*.

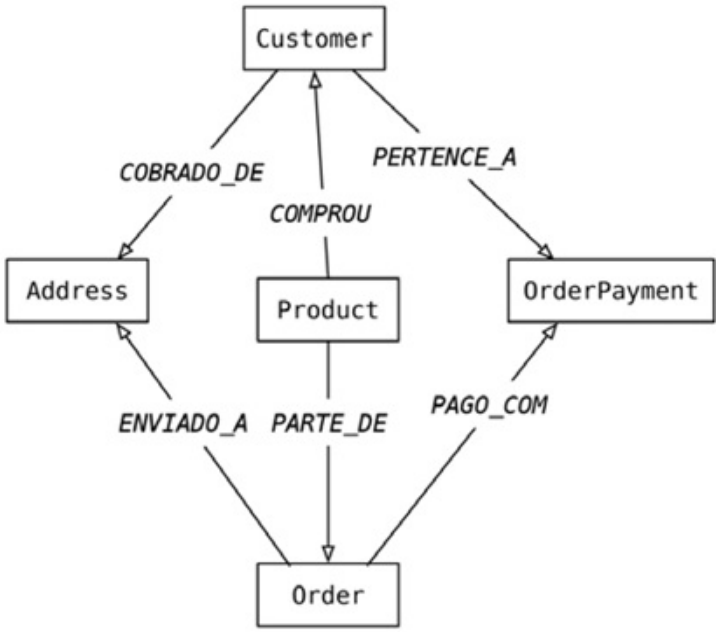

Figura 3.5 – Modelo de dados de comércio eletrônico com grafos.

Esse tipo de relacionamento transverso é muito fácil com bancos de dados de grafos. É especialmente conveniente quando precisamos utilizar os dados para recomendar produtos a usuários ou encontrar padrões em ações realizadas por eles.

3.6 Pontos chave

- Bancos de dados orientados a agregados tornam os relacionamentos entre agregados mais difíceis de lidar do que relacionamentos intra-agregados.
- Bancos de dados de grafos organizam os dados em grafos com nodos e arestas; eles funcionam melhor com dados que tenham estruturas de relacionamento complexas.
- Bancos de dados sem esquema permitem que campos sejam adicionados livremente aos registros, mas geralmente há um esquema implícito esperado pelos usuários dos dados.
- Bancos de dados orientados a agregados, muitas vezes, criam visões materializadas para fornecer dados organizados de um modo diferente de seus agregados primários. Isso, muitas vezes, é realizado com computações map-reduce.

CAPÍTULO 4

Modelos de distribuição

O que mais chama a atenção no NoSQL é sua capacidade de executar bancos de dados em um grande cluster. À medida que os volumes de dados aumentam, torna-se mais difícil e caro fazer uma expansão: comprar um servidor maior em que o banco de dados possa ser executado. Uma opção mais interessante seria a escalabilidade, executando-se o banco de dados em um cluster de servidores. A orientação a agregados adapta-se bem à escala porque o agregado é uma unidade natural para se utilizar de forma distribuída.

Dependendo do modelo de distribuição, pode-se obter um armazenamento de dados que permita lidar com quantidades maiores de dados, processar um tráfego maior de leitura ou gravação, ou mais disponibilidade quanto a atrasos e interrupções na rede. Esses são benefícios importantes, mas eles têm seu custo. A execução em um cluster introduz complexidade – portanto, não é algo a ser feito, a menos que os benefícios sejam muitos.

Em termos gerais, há dois caminhos a serem seguidos na distribuição de dados: a replicação e a fragmentação. A replicação obtém os mesmos dados e os copia em múltiplos nodos. A fragmentação coloca dados diferentes em nodos diferentes. Replicação e fragmentação são técnicas ortogonais: você pode utilizar uma ou ambas. A replicação tem duas formas: mestre-escravo e ponto a ponto. Discutiremos agora essas técnicas, começando pela mais simples até chegar à mais complexa: primeiro, a técnica com um único servidor, depois, a replicação mestre-escravo e a fragmentação para, finalmente, chegarmos à replicação ponto a ponto.

4.1 Um único servidor

A primeira e mais simples opção de distribuição é a que recomendamos com mais frequência: não utilizar distribuição alguma. Execute o banco de dados em uma única máquina que lide com todas as leituras e gravações no armazenamento de dados. Preferimos essa opção porque ela elimina todas as complexidades que as outras introduzem: é facilmente gerenciada pelos operadores e compreendida pelos desenvolvedores de aplicativos.

Embora muitos bancos de dados NoSQL sejam projetados em torno da ideia da execução em um cluster, pode fazer sentido utilizar o NoSQL com um modelo de distribuição com um único servidor, caso o modelo de dados do armazenamento NoSQL seja mais apropriado para o aplicativo. Bancos de dados de grafos são a categoria óbvia aqui – eles trabalham melhor em uma configuração com servidor único. Se a utilização dos dados for, na maior parte, para o processamento de agregados, então um armazenamento de documentos ou de chave-valor, com um único servidor, pode valer a pena, pois é mais fácil para os desenvolvedores de aplicativos.

Até o final deste capítulo, mostraremos as vantagens e complicações de esquemas de distribuição mais sofisticados. Não deixe que o volume de palavras faça-lhe pensar que preferiríamos essas opções. Se pudermos não distribuir nossos dados, sempre

escolheremos a abordagem de um único servidor.

4.2 Fragmentação

Frequentemente, um armazenamento de dados fica extremamente ocupado, pois várias pessoas estão acessando partes diferentes do conjunto dos dados. Nessas circunstâncias, podemos suportar a escalabilidade horizontal, colocando partes diferentes dos dados em servidores diferentes – uma técnica chamada de **fragmentação** (sharding) (Figura 4.1).

Em um cenário ideal, teremos usuários diferentes conversando com nodos de servidores diferentes. Cada usuário somente tem de conversar com um servidor, de modo que obtém respostas rápidas dele. A carga é bem balanceada entre os servidores: por exemplo, se tivermos dez servidores, cada um apenas tem de lidar com 10% da carga.

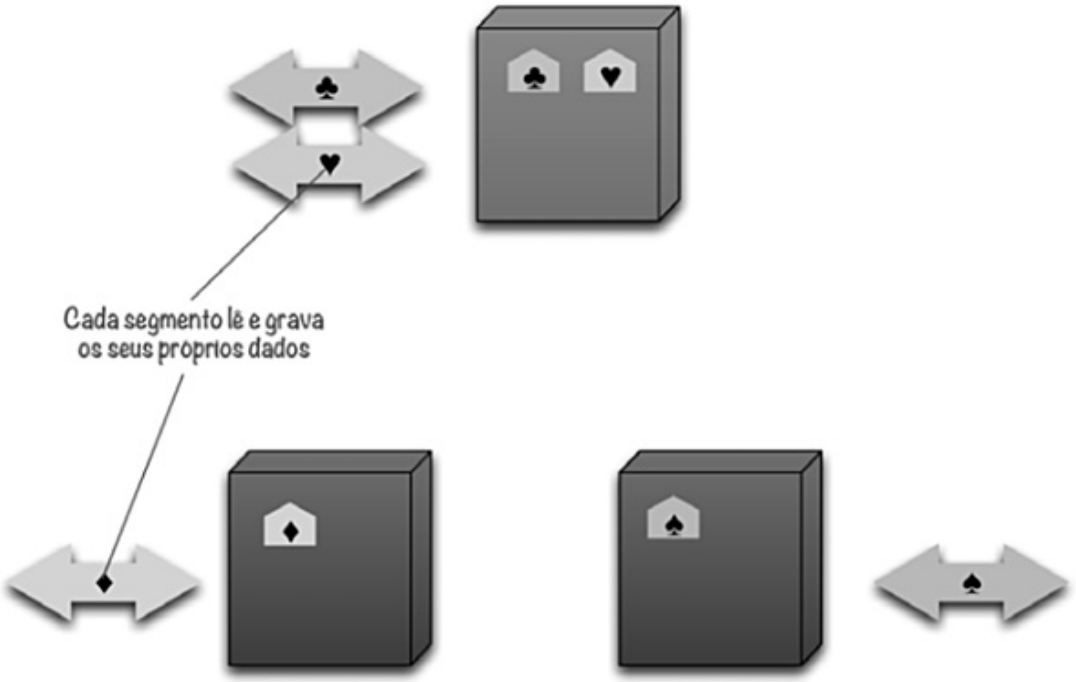

Figura 4.1 – A fragmentação coloca dados diferentes em nodos separados, cada um destes executando suas próprias leituras e gravações.

É claro que esse cenário é raro. Para chegarmos perto dele, temos de garantir que os dados acessados em conjunto sejam aglutinados no mesmo nodo e que esse aglutinado, por sua vez, esteja organizado dentro dos nodos para, dessa maneira, fornecer o melhor acesso de dados.

A primeira parte dessa questão é saber como aglutinar os dados de forma que um usuário obtenha a maior parte de seus dados de um único servidor. É aí que a orientação a agregados é realmente útil. A questão toda, a respeito dos agregados, é que os projetamos para que combinem os dados que são, normalmente, acessados em conjunto, de modo que os agregados se destaquem como uma unidade óbvia de distribuição.

Quando o assunto é a organização dos dados nos nodos, há diversos fatores que podem ajudar a melhorar o desempenho. Se você souber que a maioria dos acessos a determinados agregados é baseada em uma localização física, pode colocar os dados próximos ao lugar onde são acessados. Se tiver pedidos de alguém que mora em Boston,

pode colocar esses dados em seu centro de dados do leste dos EUA.

Outro fator é tentar manter a carga parelha. Isso significa que você deve tentar organizar os agregados para que sejam distribuídos de modo parelho pelos nodos para receber, dessa forma, quantidades iguais de carga, o que pode variar com o decorrer do tempo. Por exemplo, se algum dado tende a ser acessado em determinados dias da semana, então podem existir regras de domínio específico que você gostaria de utilizar.

Em alguns casos, é útil juntar agregados se você achar que eles podem ser lidos em sequência. O artigo BigTable [Chang etc.] descreve a manutenção de suas linhas em ordem léxica e a ordenação de endereços web está baseada nos nomes de domínio invertidos (por exemplo, `com.martinfowler`). Dessa maneira, os dados de múltiplas páginas poderiam ser acessados juntos para aumentar a eficiência do processamento.

Historicamente, a maioria das pessoas realiza a fragmentação como parte da lógica do aplicativo. Você poderia colocar todos os clientes cujo sobrenome começa com A a D em um fragmento e com E a G em outro. Isso complica o modelo de programação, já que o código do aplicativo precisa garantir que as consultas sejam distribuídas pelos diversos fragmentos. Além disso, rebalancear a fragmentação significa alterar o código do aplicativo e migrar os dados. Muitos bancos de dados NoSQL oferecem a **autofragmentação**, em que o banco de dados fica responsável por alocar os dados nos fragmentos e por assegurar-se de que o acesso aos dados vá para o fragmento correto. Isso pode facilitar muito o uso da fragmentação em um aplicativo.

A fragmentação é particularmente valiosa para a performance, pois pode melhorar o desempenho de leitura e gravação. Utilizar a replicação, especialmente com cache, pode melhorar muito o desempenho de leitura, mas faz pouco para aplicativos que tenham muitas gravações. A fragmentação fornece uma maneira de ampliar horizontalmente as gravações.

A fragmentação faz pouco para melhorar a resiliência quando utilizada isoladamente. Embora os dados estejam em nodos diferentes, uma falha de nodo torna indisponíveis os dados do fragmento, assim como acontece em uma solução de um único servidor. O benefício que a resiliência fornece é que apenas os usuários dos dados desse fragmento sofrerão; entretanto, não é bom ter um banco de dados com partes de seus dados faltando. Com um único servidor, é mais fácil compensar o esforço e o custo de manter o servidor funcionando; os clusters, geralmente, tentam utilizar máquinas menos confiáveis, aumentando as chances de falha no nodo. Provavelmente, na prática, a fragmentação, sozinha, diminuirá a resiliência.

Apesar de a fragmentação ser realizada mais facilmente com agregados, ainda não é um passo a ser dado sem preocupações. Alguns bancos de dados têm a intenção, desde o início, de utilizar a fragmentação. Nesse caso, é sábio executá-los em um cluster desde o início do desenvolvimento e, certamente, também na fase de produção. Outros bancos de dados utilizam a fragmentação como um segundo passo a partir de uma configuração de servidor único. Nesse caso, é melhor iniciar com um servidor único e apenas utilizar a fragmentação quando suas projeções de carga indicarem, claramente, que os recursos estão terminando.

De qualquer maneira, passar de um único nodo para a fragmentação será complicado. Ouvimos histórias de equipes que enfrentaram problemas, pois realizaram-na tarde

demais. Ao executá-la na produção, seu banco de dados ficou basicamente indisponível, pois o suporte à fragmentação consumiu todos os recursos do banco de dados para mover os dados para os novos fragmentos. A lição aqui é utilizar a fragmentação bem antes de precisar dela – quando os recursos forem suficientes para executá-la.

4.3 Replicação mestre-escravo

Com a distribuição mestre-escravo, você replica os dados em múltiplos nodos. Um nodo é designado como o mestre, ou primário, o qual é a fonte oficial dos dados e, geralmente, fica responsável por processar quaisquer atualizações nesses dados. Os outros nodos são escravos, ou secundários. Um processo de replicação sincroniza os escravos com o mestre (Figura 4.2).

A replicação mestre-escravo é mais útil para a escalabilidade quando há um conjunto de dados com muitas leituras. Você pode escalar horizontalmente para lidar com mais solicitações de leitura, adicionando mais nodos escravos e assegurando-se de que todas as solicitações de leitura sejam roteadas para os escravos. Entretanto, você ainda ficará limitado pela capacidade do mestre de processar as atualizações e de transmiti-las adiante. Consequentemente, não é um esquema bom para conjuntos de dados com muito tráfego de gravação, embora diminuir a carga do tráfego de leitura ajudará um pouco a lidar com a carga de gravação.

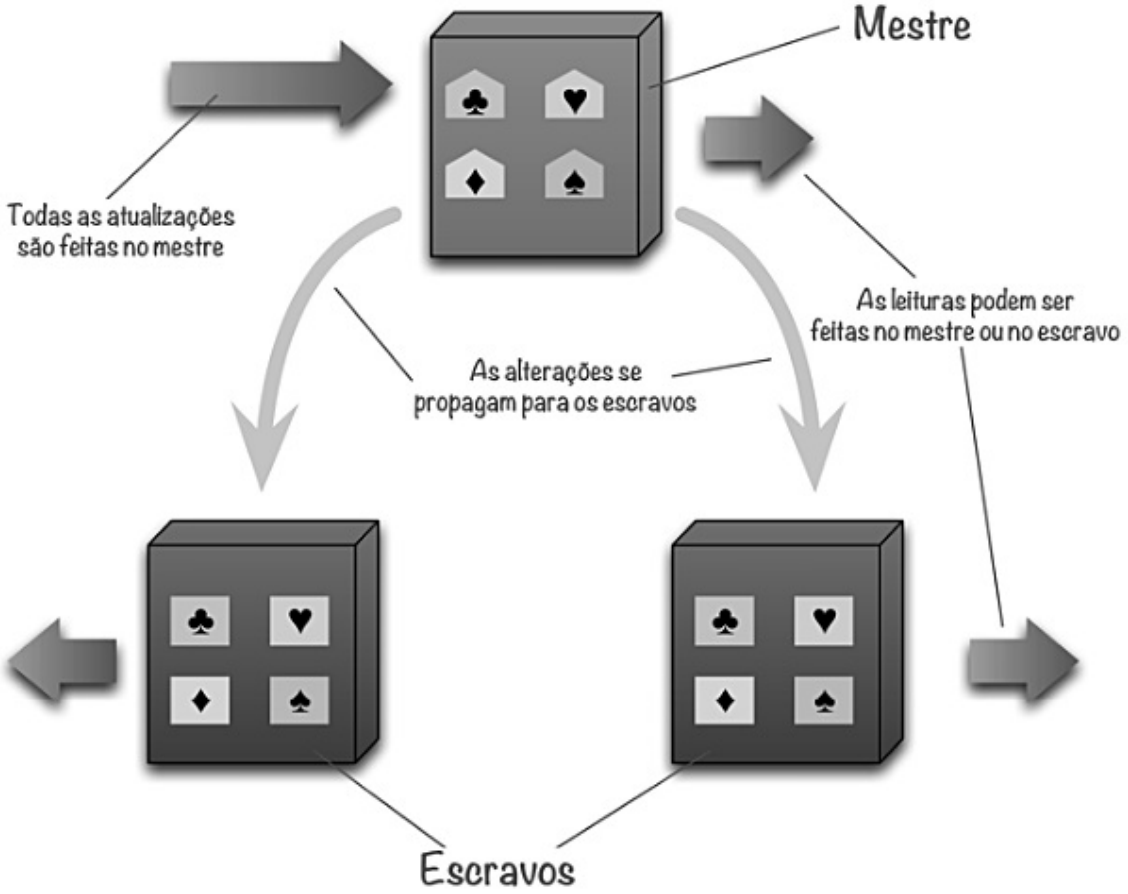

Figura 4.2 – Os dados são replicados do mestre para os escravos. O mestre serve a todas as gravações; as leituras podem vir do mestre ou dos escravos.

A segunda vantagem da replicação mestre-escravo é a **resiliência de leitura**: se o mestre

falhar, os escravos ainda podem lidar com as solicitações de leitura. Novamente, isso é útil se a maioria de seus acessos aos dados for para leitura. A falha no mestre elimina a capacidade de lidar com gravações até que ele seja restaurado ou que um novo mestre seja designado. Entretanto, ter escravos como replicações do mestre acelera a recuperação após uma falha, já que um escravo pode ser designado como um novo mestre muito rapidamente.

A capacidade de designar um escravo para substituir um mestre que tenha falhado significa que a replicação mestre-escravo é útil mesmo que não seja preciso escalabilidade. Todo o tráfego de leitura e gravação pode ir para o mestre, enquanto o escravo atua como uma cópia de segurança ativa. Nesse caso, é mais fácil pensar no sistema como um armazenamento de servidor único, com backup ativo. Você usufrui da conveniência de uma configuração de um único servidor, mas com maior resiliência, o que é especialmente útil se você quiser ser capaz de lidar bem com falhas no servidor.

Mestres podem ser designados manual ou automaticamente. A designação manual significa, geralmente, que, quando você configura seu cluster, configura um nodo como o mestre. Com a designação automática, você cria um cluster de nodos e eles elegem um como o mestre. Além da configuração mais simples, a designação automática significa que o cluster pode designar automaticamente um novo mestre quando outro falhar, reduzindo o tempo fora do ar.

Para obter resiliência de leitura, é necessário garantir que os caminhos de leitura e de gravação em seu aplicativo sejam diferentes, de modo que seja possível lidar com uma falha no caminho de gravação e ainda assim ler. Isso inclui ações, como colocar as leituras e gravações em conexões separadas do banco de dados – um recurso que não é suportado com frequência por bibliotecas de interação com bancos de dados. Assim como com qualquer recurso, você não tem certeza de que tem resiliência de leitura até realizar bons testes que desabilitem as gravações e verifiquem se as leituras ainda ocorrem.

A replicação traz alguns benefícios interessantes, mas também um lado negativo inevitável – a inconsistência. O risco é que clientes diferentes, lendo diferentes escravos, vejam valores diferentes, pois as alterações não foram todas propagadas para os escravos. No pior caso, isso significa que um cliente não conseguirá ler uma gravação que acabou de fazer. Mesmo que você utilize a replicação mestre-escravo apenas para a cópia de segurança ativa, isso pode ser um problema, pois, se o mestre falhar, quaisquer atualizações que não tenham sido transmitidas para a cópia de segurança estarão perdidas. Falaremos sobre como lidar com esses problemas posteriormente (“Consistência”, p. 83).

4.4 Replicação ponto a ponto (p2p)

A replicação mestre-escravo ajuda com a escalabilidade de leitura, mas não com a escalabilidade de gravação. Ela fornece resiliência contra a falha de um escravo, mas não de um mestre. Basicamente, o mestre ainda é um gargalo e um ponto único de falha. A replicação ponto a ponto (Figura 4.3) combate esses problemas, pois não tem um mestre. Todas as réplicas têm peso igual, todas podem receber gravações e a perda de alguma delas não impede o acesso ao armazenamento de dados.

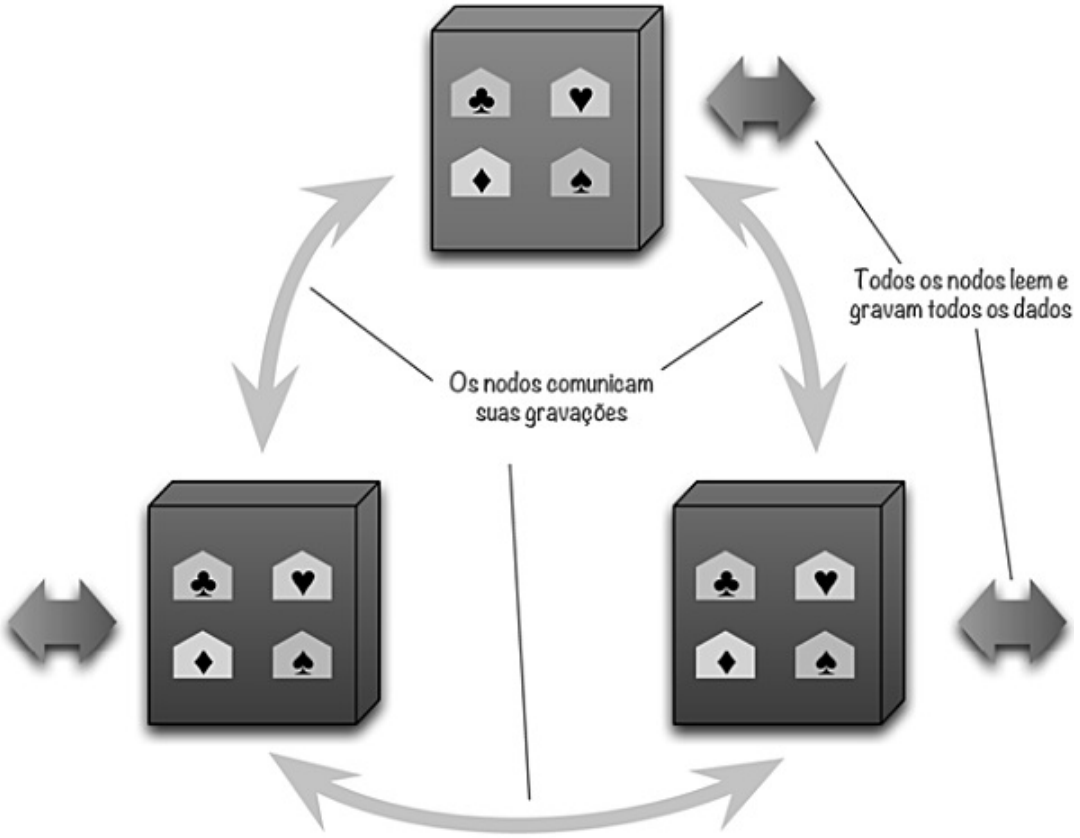

Figura 4.3 – A replicação ponto a ponto permite que todos os nodos façam leituras e gravações para todos os dados.

A perspectiva aqui parece muito boa. Com um cluster com replicação ponto a ponto, você pode contornar falhas nos nodos sem perder o acesso aos dados. Além disso, você pode adicionar facilmente nodos para melhorar o seu desempenho. Há muito a ser apreciado aqui, mas também há complicações. A maior delas é, novamente, a consistência. Quando você pode gravar em dois lugares diferentes, corre o risco de que duas pessoas tentem atualizar o mesmo registro ao mesmo tempo, o que gera um conflito de gravação. Inconsistências na leitura geram problemas, mas pelo menos são relativamente temporárias. Gravações inconsistentes são eternas.

Falaremos mais sobre como lidar com inconsistências de gravação posteriormente. Por enquanto, observaremos algumas opções gerais. Em um extremo, podemos garantir que, sempre que gravamos dados, as réplicas sejam coordenadas para evitar conflitos. Isso nos dá uma garantia tão forte quanto a do mestre, apesar do custo do tráfego de rede para coordenar as gravações. Não precisamos de que todas as réplicas concordem com a gravação, apenas uma maioria, de modo que ainda podemos sobreviver à perda de uma minoria dos nodos réplicas.

No outro extremo, podemos decidir lidar com uma gravação inconsistente. Há contextos nos quais podemos criar uma política que pode mesclar gravações inconsistentes. Nesse caso, podemos obter o benefício do desempenho integral de gravar em qualquer réplica.

Essas questões são os extremos de um espectro no qual trocamos consistência por disponibilidade.

4.5 Combinando fragmentação e replicação

Replicação e fragmentação são estratégias que podem ser combinadas. Se utilizarmos tanto a replicação mestre-escravo quanto a fragmentação (Figura 4.4), teremos múltiplos mestres, mas cada item de dado terá somente um mestre. Dependendo da configuração, podemos escolher um nodo para ser mestre para alguns dados e escravos para outros, ou podemos dedicar nodos para funções de mestre ou escravo.

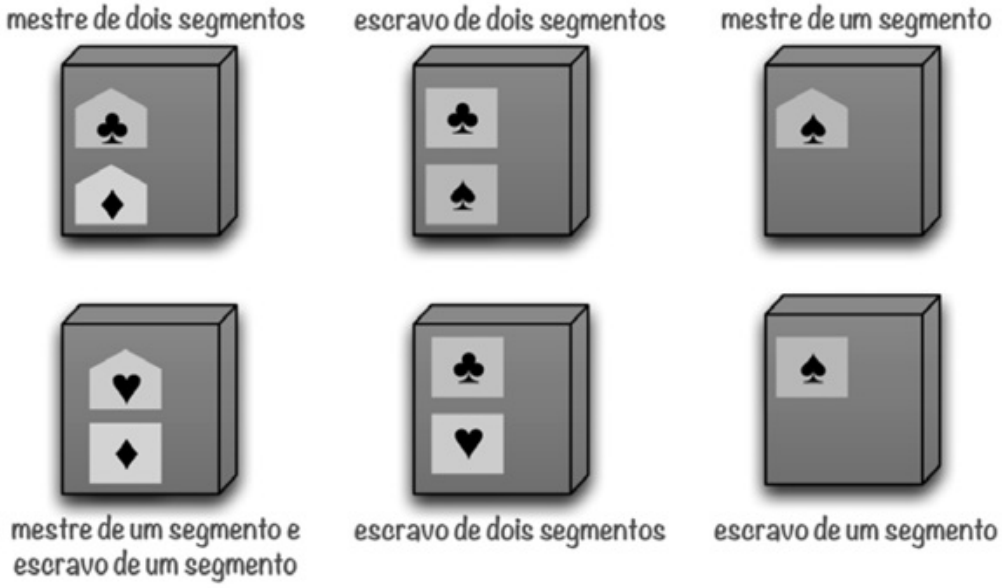

Figura 4.4 – Utilizando a replicação mestre-escravo junto com a fragmentação.

Utilizar a replicação ponto a ponto e a fragmentação é uma estratégia comum para bancos de dados de famílias de colunas. Em um cenário como esse, você poderia ter dezenas ou centenas de nodos em um cluster com dados fragmentados por eles. Um bom ponto de partida para a replicação ponto a ponto é ter uma replicação de fator 3, de modo que cada fragmento esteja presente em três nodos. Se um nodo falhar, então seus fragmentos estarão nos outros (Figura 4.5).

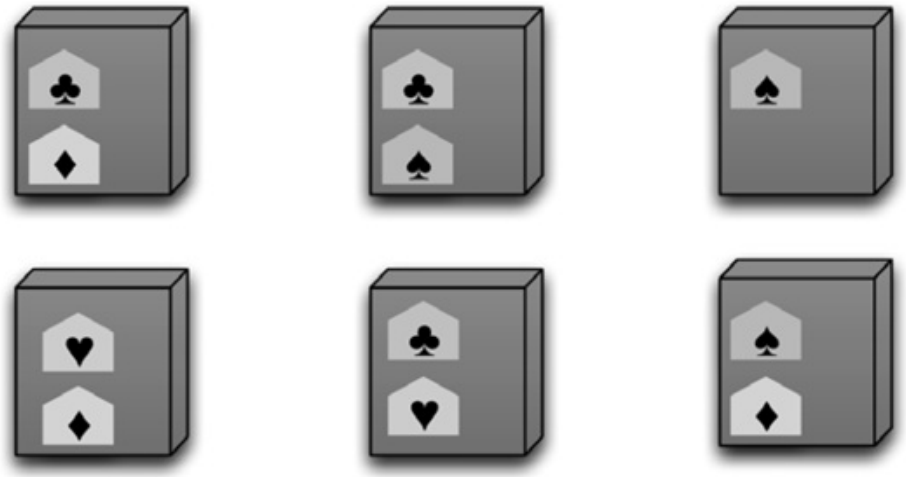

Figura 4.5 – Utilizando a replicação ponto a ponto junto com a fragmentação.

4.6 Pontos chave

- Há dois estilos de distribuição de dados: a fragmentação e a replicação.
  - A fragmentação distribui dados diferentes em múltiplos servidores, de modo que cada servidor atua como a única fonte de um subconjunto de dados.
  - A replicação copia os dados para múltiplos servidores, de modo que cada parte dos dados pode ser encontrada em múltiplos lugares.

  Um sistema pode utilizar uma ou ambas as técnicas.
- A replicação tem duas formas: a mestre-escravo e a ponto a ponto.
  - A replicação mestre-escravo torna um nodo a cópia oficial, a qual lida com gravações, enquanto os escravos sincronizam-se com o mestre e podem lidar com as leituras.
  - A replicação ponto a ponto permite gravações em qualquer nodo; os nodos são coordenados para sincronizar suas cópias dos dados.

  A replicação mestre-escravo reduz a chance de conflitos de atualização, mas a ponto a ponto evita carregar todas as gravações em um único ponto de falha.

CAPÍTULO 5

Consistência

Uma das maiores mudanças de um banco de dados relacional centralizado para um NoSQL orientado a clusters está em como você pensa na consistência. Bancos de dados relacionais tentam exibir uma forte consistência evitando todas as diversas inconsistências que discutiremos em breve. Assim que você começar a examinar o mundo NoSQL, surgirão sentenças, como "o teorema CAP" e "a consistência eventual", e assim que você começar a criar algo, terá de pensar em que tipo de consistência precisa para o seu sistema.

A consistência pode ter diversas formas e essa única palavra abrange uma diversidade de modos pelos quais os erros podem dificultar sua vida. Assim, começaremos conversando sobre os diversos formatos que a consistência pode assumir. Então, discutiremos o motivo pelo qual você pode querer relaxar a consistência (e sua irmã mais velha, a durabilidade).

5.1 Consistência de atualização

Iniciaremos considerando a atualização de um número de telefone. Coincidentemente, Martin e Pramod estão examinando o website da empresa e percebem que o número do telefone está desatualizado. Implausivelmente, ambos têm acesso para atualização, e atualizam o número ao mesmo tempo. Para tornar o exemplo interessante, supomos que eles atualizam o número diferentemente, porque cada um utiliza um formato ligeiramente distinto. Essa questão é chamada de **conflito escrita-escrita**: duas pessoas atualizando o mesmo item de dados ao mesmo tempo.

Quando as gravações chegam ao servidor, este as **serializará** – decidirá aplicar uma e depois a outra. Supomos que ele utilize a ordem alfabética e selecione primeiramente a atualização de Martin, e depois, a de Pramod. Sem qualquer controle de concorrência, a atualização de Martin seria aplicada e imediatamente sobrescrita pela de Pramod. Nesse caso, a de Martin é uma **atualização perdida**. Aqui, a atualização perdida não é um grande problema, mas muitas vezes é. Vemos isso como uma falha de consistência porque a atualização de Pramod foi baseada no estado anterior à de Martin, mas aplicada depois dela.

Abordagens para manter a consistência confrontando a concorrência são, muitas vezes, descritas como otimistas ou pessimistas. Uma abordagem **pessimista** evita que os conflitos ocorram; uma abordagem otimista, permite que eles ocorram, mas os detecta e atua para corrigi-los. Para conflitos de atualização, a abordagem pessimista mais comum é a que tem bloqueios de gravação, de modo que, para alterar um valor, é necessário obter um bloqueio, e o sistema assegura que apenas um cliente por vez pode obtê-lo. Assim, Martin e Pramod tentariam obter o bloqueio de gravação, mas apenas Martin (o primeiro) seria bem-sucedido. Pramod veria, então, o resultado da gravação de Martin antes de decidir se deseja realizar sua própria atualização.

Uma abordagem otimista comum é uma **atualização condicional**, em que qualquer cliente que execute uma atualização testa o valor antes de atualizá-lo para ver se ele foi

alterado desde sua última leitura. Nesse caso, a atualização de Martin seria bem-sucedida, mas a de Pramod falharia. O erro informaria Pramod de que ele deveria reexaminar o valor e decidir tentar ou não realizar uma nova atualização.

Tanto a abordagem pessimista quanto a otimista baseiam-se em uma serialização consistente das atualizações. Tendo um único servidor, é óbvio que ele tem de escolher um e depois o outro. Todavia, se houver mais de um servidor, como na replicação ponto a ponto, então dois nodos poderiam aplicar as atualizações em uma ordem diferente, resultando em um valor diferente para o número do telefone em cada ponto. Muitas vezes, quando as pessoas falam em concorrência em sistemas distribuídos, estão se referindo à consistência sequencial, assegurando que todos os nodos apliquem as operações na mesma ordem.

Há outra forma otimista de lidar com o conflito escrita-escrita: gravar ambas as atualizações e registrar aquelas que estão em conflito. Essa abordagem é familiar a muitos programadores que utilizam sistemas de controle de versões, especialmente os distribuídos, e, por sua natureza, muitas vezes incorrerá em gravações (*commits*) conflitantes. O próximo passo, novamente, vem do controle de versões: unir as duas atualizações de alguma forma. Você pode mostrar ambos os valores ao usuário e pedir que ele resolva o problema – isso é o que acontece se você atualizar o mesmo contato em seu telefone e em seu computador. De forma alternativa, o computador pode ser capaz de executar a união por si só; se for uma questão de formatação de número de telefone, ele consegue perceber e aplicar ao novo número o formato padrão. Qualquer resolução automática de conflitos de gravação é altamente específica ao domínio e precisa ser programada para cada caso especialmente.

Muitas vezes, quando as pessoas deparam-se com esses problemas pela primeira vez, sua reação é preferir a concorrência pessimista, pois estão determinadas a evitar conflitos. Embora, em alguns casos, essa seja a resposta correta, sempre há uma troca. A programação concorrente envolve um equilíbrio fundamental entre segurança (evitar erros, como conflitos de atualização) e responsividade (responder rapidamente ao cliente). Abordagens pessimistas, muitas vezes, degradam severamente a capacidade de resposta de um sistema em um nível que se torna inapropriado para o seu propósito. Esse problema é piorado pelo risco dos erros, pois a concorrência pessimista muitas vezes leva a conflitos de bloqueio (deadlocks) difíceis de evitar e depurar.

A replicação torna muito mais fácil encontrar conflitos de gravação. Se diferentes nodos tiverem cópias diferentes dos mesmos dados e estes puderem ser atualizados de forma independente, então você terá conflitos, a menos que adote medidas específicas para evitá-los. Utilizar um único nodo como destino de todas as gravações de alguns dados facilita muito a manutenção da consistência de atualização. Dos modelos de distribuição que discutimos anteriormente, todos, menos a replicação ponto a ponto, fazem isso.

5.2 Consistência de leitura

Ter um armazenamento de dados que mantenha a consistência da atualização já é alguma coisa, mas não garante que quem leia tais dados sempre obterá respostas consistentes para suas solicitações. Imaginemos um pedido que tenha itens e uma taxa de frete. Esta taxa é calculada com base nos itens do pedido. Se adicionarmos um item, precisaremos

recalcular e atualizar a taxa de frete. Em um banco de dados relacional, a taxa de frete e os itens do pedido estarão em tabelas separadas. O perigo da inconsistência é Martin adicionar um item ao seu pedido, Pramod ler os itens e a taxa e, então, Martin atualizar a taxa de frete. Essa é uma **leitura inconsistente** ou um **conflito de leitura-gravação**. Na figura 5.1, Pramod executou uma leitura durante a gravação de Martin.

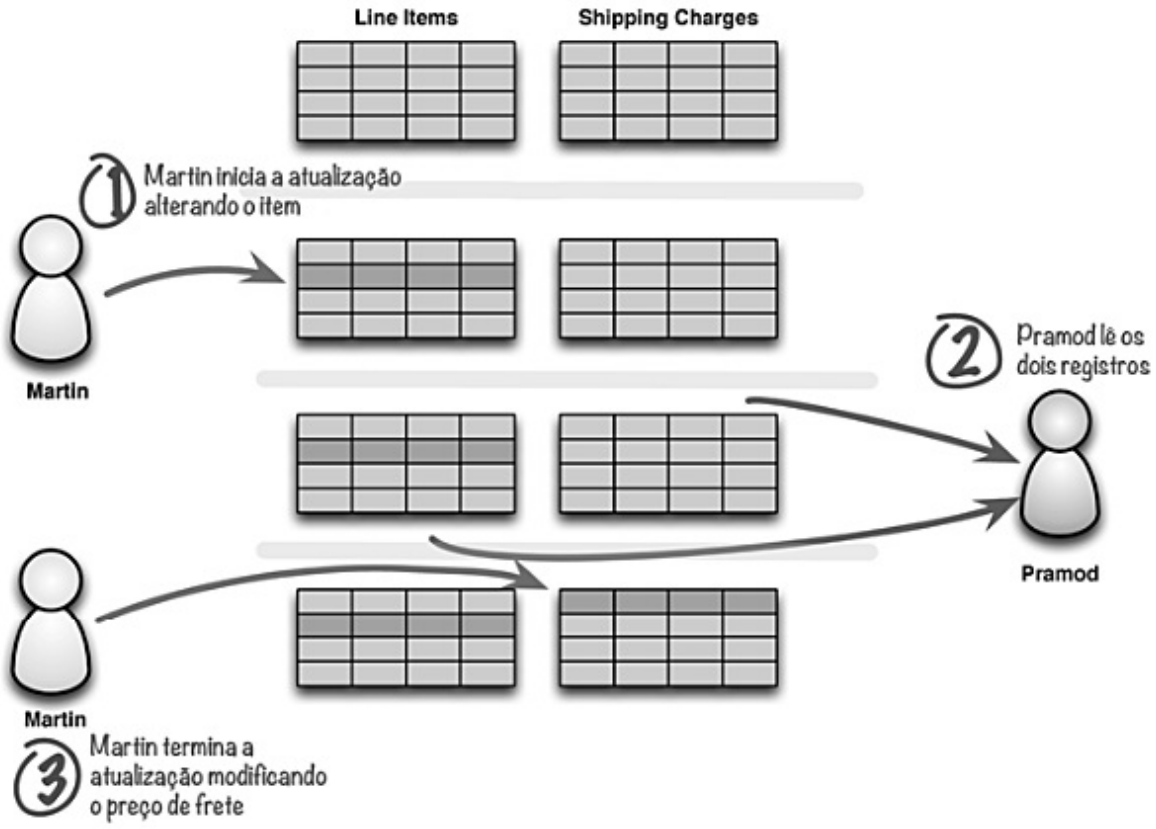

Figura 5.1 – Um conflito de leitura-gravação em consistência lógica.

Referimo-nos a esse tipo de consistência como **consistência lógica**, na qual se assegura de que diferentes itens de dados façam sentido juntos. Para evitar um conflito de leitura-gravação logicamente inconsistente, os bancos de dados relacionais suportam a noção de transações. Desde que Martin coloque suas duas gravações em uma transação, o sistema garante que Pramod lerá ambos os itens de dados antes ou depois da atualização.

Uma alegação comum é a seguinte: bancos de dados NoSQL não suportam transações e, assim, não conseguem ser consistentes. Tal alegação é, em grande parte, incorreta, pois desconsidera muitos detalhes importantes. Nossa primeira classificação é: qualquer declaração sobre a falta de transações geralmente se aplica somente a alguns bancos de dados NoSQL, especialmente aos orientados a agregados. Comparativamente, bancos de dados de grafos tendem a suportar transações ACID da mesma maneira que os bancos de dados relacionais.

Em segundo lugar, bancos de dados orientados a agregados suportam atualizações atômicas, mas apenas dentro de um único agregado. Isso significa que você terá consistência lógica dentro de um agregado, mas não entre agregados. Assim, no exemplo anterior, você poderia evitar cair em tal inconsistência se o pedido, a taxa de frete e os itens fizessem parte de um único agregado de pedido.

É claro que nem todos os dados podem ser colocados no mesmo agregado, de forma que

qualquer alteração que afete múltiplos agregados deixa aberto um período de tempo no qual os clientes poderiam executar uma leitura inconsistente. Esse período de tempo é chamado de **janela de inconsistência**. Um sistema NoSQL pode ter uma janela de inconsistência bastante curta: com relação a isso, a documentação da Amazon diz que a janela de inconsistência do seu serviço SimpleDB é geralmente menor do que um segundo.

Esse exemplo de leitura logicamente inconsistente é clássico e você o verá em qualquer livro que fale sobre a programação para bancos de dados. Assim que você introduz a replicação, entretanto, obtém um tipo completamente novo de inconsistência. Imaginemos que haja um último quarto de hotel disponível para um evento ao qual você queira ir. O sistema de reservas do hotel é executado em muitos nodos. O casal Martin e Cindy está pensando em reservar esse quarto, mas eles estão discutindo pelo telefone, porque Martin está em Londres e Cindy, em Boston. Enquanto isso, Pramod, que está em Bombaim, reserva esse último quarto. Isso atualiza a disponibilidade replicada do quarto, mas a atualização chega a Boston mais rapidamente do que a Londres. Quando Martin e Cindy utilizam seus navegadores para ver se o quarto ainda está disponível, Cindy o vê reservado e Martin, o vê disponível. Essa é outra leitura inconsistente, mas é um tipo diferente de falha na consistência, a qual chamamos de **consistência de replicação**: garantir que o mesmo item de dados tenha o mesmo valor quando lido a partir de diferentes réplicas (Figura 5.2).

No final, é claro, as atualizações serão totalmente propagadas e Martin verá que o quarto está reservado. Essa situação geralmente é chamada de **eventualmente consistente**, o que significa que, em algum momento, todos os nodos podem ter inconsistências de replicação, mas, se não houver mais atualizações, todos os nodos acabarão atualizados com o mesmo valor. Os dados que estão desatualizados são geralmente chamados de **obsoletos**, o que nos lembra que um cache é outra forma de replicação – basicamente seguindo o modelo de distribuição mestre-escravo.

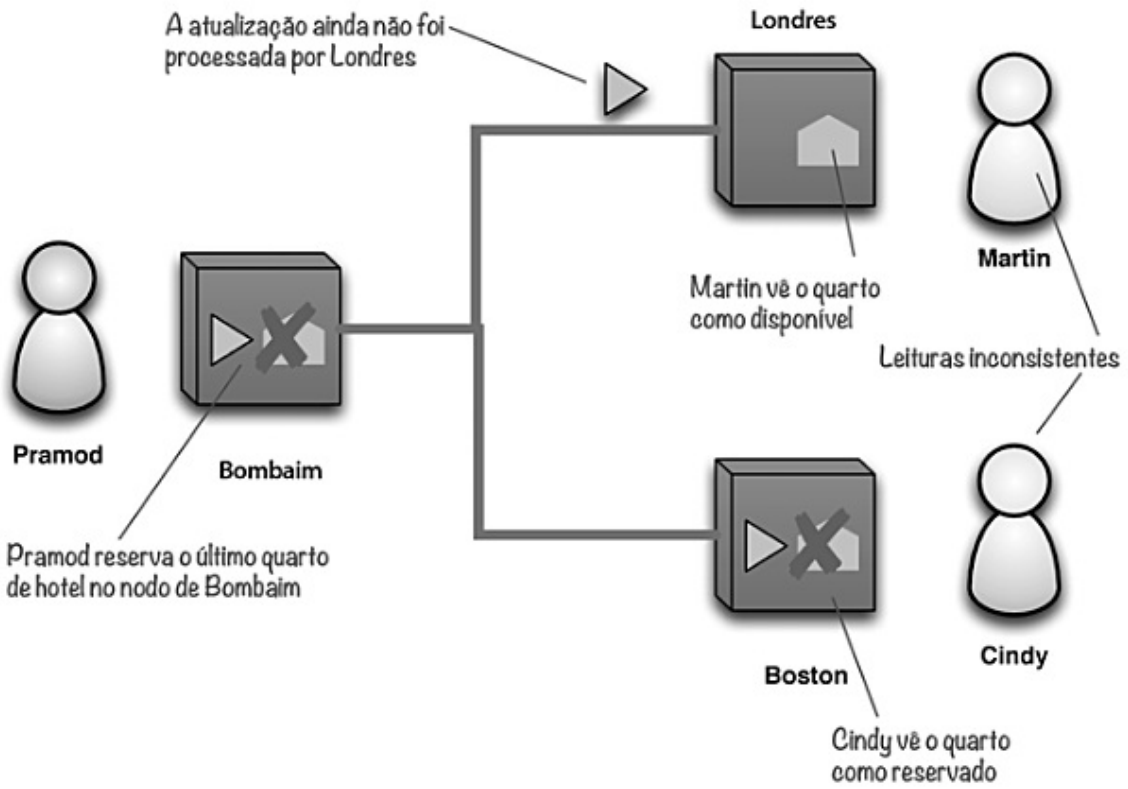

Figura 5.2 – Um exemplo de inconsistência de replicação.

Embora a consistência de replicação seja independente da consistência lógica, a replicação pode exacerbar uma inconsistência lógica, aumentando sua janela de inconsistência. Duas atualizações diferentes no mestre podem ser executadas em uma sucessão rápida, deixando uma janela de inconsistência de milissegundos. Todavia, atrasos na rede poderiam significar que a mesma janela de inconsistência duraria muito mais tempo em um escravo.

Garantias de consistência não são globais dentro de um aplicativo. Geralmente, você pode especificar o nível de consistência que quer com solicitações individuais. Isto permite que você utilize a consistência baixa na maior parte do tempo, quando isso não for um problema, mas solicitar consistência alta, quando for.

A presença de uma janela de inconsistência significa que pessoas diferentes verão algo diferente ao mesmo tempo. Se Martin e Cindy estivessem procurando quartos durante uma chamada telefônica transoceânica, poderia haver confusão. É mais comum que os usuários ajam independentemente, o que não é um problema. Todavia, janelas de inconsistência podem ser bastante problemáticas quando você tem inconsistência consigo mesmo. Considere como o exemplo a postagem de comentários em um blog. Poucas pessoas se preocuparão com janelas de inconsistência, mesmo de alguns minutos, enquanto as pessoas estiverem digitando suas ideias mais recentes. Frequentemente, os sistemas lidam com a carga desses sites executando-os em um cluster e balanceando a carga das solicitações que chegam aos diferentes nodos. Aí está um perigo: você pode postar uma mensagem utilizando um nodo e, então, atualizar seu navegador, mas a atualização vai para um nodo diferente, que ainda não recebeu sua postagem, parecendo, assim, que ela foi perdida.

Em situações como essa, você pode tolerar janelas de inconsistência razoavelmente

longas, mas precisa de consistência do tipo **leia-sua-escrita**, o que significa que, assim que tiver realizado uma atualização, certamente continuará a vê-la. Uma forma de obter isso é fornecendo **consistência de sessão**. Em uma sessão de usuário, existe consistência leia-sua-escrita. Isso significa que o usuário pode perder essa consistência se sua sessão terminar, por algum motivo, ou se ele acessar o mesmo sistema simultaneamente a partir de diferentes computadores. Porém, esses casos são relativamente raros.

Há algumas técnicas para fornecer consistência de sessão. Uma forma comum, e muitas vezes a mais simples, é criar uma **sessão persistente**: uma sessão associada a um nodo (é chamada também de **afinidade de sessão**). Uma sessão persistente permite que você garanta que, desde que mantenha a consistência de gravação de suas leituras em um nodo, a obterá também para sessões. A desvantagem é que sessões persistentes reduzem a capacidade do balanceador de carga de executar sua tarefa.

Outra abordagem para a consistência de sessão é utilizar marcadores de versão (“Marcadores de versão”, p. 101) e assegurar-se de que toda interação com o armazenamento de dados inclua o marcador de versão mais recente visto por uma sessão. O nodo servidor deve, então, assegurar-se de que possui as atualizações que incluem esse marcador de versão antes de responder a uma solicitação.

Manter a consistência da sessão com sessões persistentes e replicação mestre-escravo pode ser complexo se você quiser ler a partir dos escravos para melhorar o desempenho da leitura, mas, de qualquer forma, ainda precisará gravar no mestre. Uma forma de lidar com isso é fazer com que as gravações sejam enviadas para o escravo, que, então, assume a responsabilidade de encaminhá-las para o mestre, enquanto mantém a consistência da sessão para o seu cliente. Outra abordagem seria alternar a sessão para o mestre, temporariamente, quando uma gravação é executada, durante um tempo suficiente para que as leituras sejam executadas a partir do mestre, até que os escravos tenham efetuado as atualizações.

Estamos falando sobre consistência de replicação no contexto de um armazenamento de dados, mas esse também é um fator importante no projeto geral do aplicativo. Em diversas ocasiões, até mesmo um sistema de banco de dados simples terá seus dados apresentados para um usuário, que os analisará e depois os atualizará. Geralmente, não é uma boa ideia manter uma transação aberta durante a interação do usuário, pois há um perigo real de conflito quando o usuário tentar executar sua atualização. Isso leva a abordagens como bloqueios offline [Fowler PoEAA].

5.3 Relaxando a consistência

A consistência é algo bom, mas, infelizmente, às vezes temos de sacrificá-la. Sempre é possível projetar um sistema para evitar inconsistências, mas, muitas vezes, é impossível fazê-lo sem sacrificar outras características do sistema. Consequentemente, muitas vezes temos de balancear a consistência com algo diferente. Embora alguns arquitetos vejam isso como um desastre, nós vemos como parte das escolhas inevitáveis envolvidas no projeto do sistema. Além disso, diferentes domínios têm diferentes tolerâncias à inconsistência, logo, precisamos levar essa tolerância em consideração ao tomarmos nossas decisões.

O compromisso de consistência é um conceito familiar mesmo em sistemas de bancos de dados relacionais com um único servidor. Aqui, nossa principal ferramenta para impor

consistência é a transação, e transações podem fornecer garantia de alta consistência. Entretanto, sistemas de transações geralmente têm a capacidade de relaxar os níveis de isolamento, permitindo que as consultas leiam dados que ainda não tenham sido confirmados. Na prática, vimos que a maioria dos aplicativos diminui a consistência do maior nível de isolamento (serializado) para obter um desempenho efetivo. Vemos mais comumente pessoas utilizando o nível de isolamento de leitura confirmada, o qual elimina alguns conflitos de leitura-gravação, mas permite outros.

Muitos sistemas não utilizam transações porque o impacto das transações no desempenho é alto demais. Vimos isso acontecer de formas diferentes. Em pequena escala, vimos a popularidade do MySQL durante a época em que ele não suportava transações. Muitos websites gostavam da alta velocidade do MySQL e eram preparados para trabalhar sem transações. Por outro lado, alguns websites muito grandes, como o eBay [Pritchett], tiveram de deixar transações de lado para obter um desempenho aceitável, isto é, especialmente verdadeiro quando é necessário introduzir fragmentação. Mesmo sem essas restrições, muitos desenvolvedores de aplicativos precisam interagir com sistemas remotos, que não podem ser incluídos apropriadamente nos limites de uma transação, de modo que atualizações fora de transações ocorrem comumente em aplicativos empresariais.

5.3.1 Teorema CAP

No mundo NoSQL, é comum referirmo-nos ao teorema CAP como o motivo pelo qual pode-se precisar relaxar a consistência. Ele foi proposto originalmente por Eric Brewer, em 2000 [Brewer], e recebeu uma prova formal de Seth Gilbert e Nancy Lynch [Lynch e Gilbert], alguns anos depois (talvez você já tenha ouvido alguma referência à Conjectura de Brewer).

A declaração básica do teorema CAP é que, dadas as três propriedades, de Consistência, de Disponibilidade (Availability) e de Tolerância a partições, somente é possível obter duas delas. Obviamente, isso depende muito de como são definidas essas propriedades, e diferentes opiniões levam a diversos debates sobre quais são as reais consequências do teorema CAP.

A **Consistência** é muito semelhante àquela que já definimos. A **Disponibilidade** tem um significado especial no contexto do CAP, segundo o qual se você puder conversar com um nodo no cluster, pode ler e gravar dados. Isso é sutilmente diferente do significado habitual, que exploraremos posteriormente. **Tolerância a partições**, no entanto, significa que o cluster pode suportar falhas na comunicação que o dividam em múltiplas partições incapazes de se comunicar entre si, situação conhecida como **divisão cerebral** (split brain) (Figura 5.3).

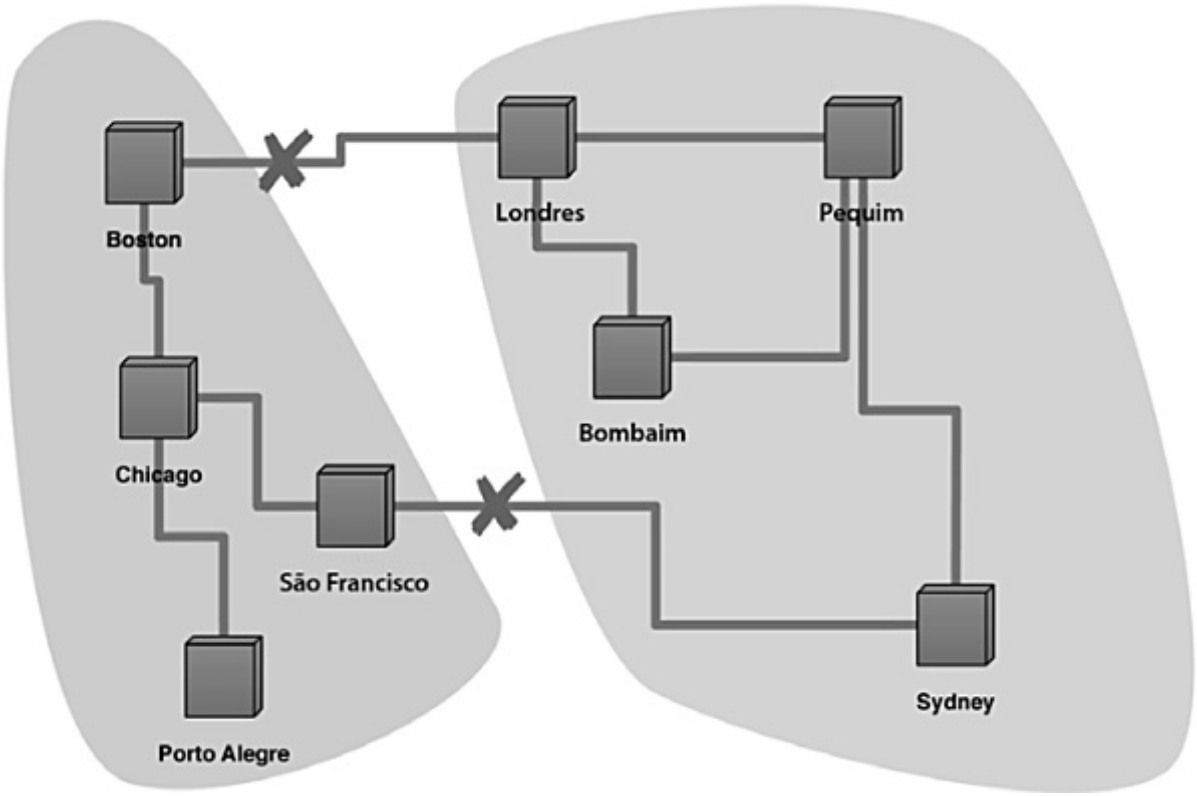

Figura 5.3 – Com duas interrupções nas linhas de comunicação, a rede divide-se em dois grupos.

Um sistema com um único servidor é o exemplo óbvio de um sistema CA (Consistency e Availability), o qual possui Consistência e Disponibilidade, mas não Tolerância a partições. Uma única máquina não pode ser particionada, de modo que não é necessário preocupar-se com a Tolerância a partições. Há apenas um nodo; se ele estiver funcionando, estará disponível. Estar funcionando e manter consistência é algo razoável. Esse é o mundo no qual a maioria dos sistemas de bancos de dados relacionais funciona.

É teoricamente possível ter um cluster CA. Entretanto, isso significaria que, se houvesse uma partição no cluster, todos os seus nodos sairiam do ar, de modo que nenhum cliente poderia se comunicar com um nodo. Pela definição habitual de "disponível", isso significaria uma falta de disponibilidade, mas é aí que o uso especial de "disponibilidade" por parte do CAP fica confuso. CAP define disponibilidade assim: "toda solicitação recebida por um nodo que não esteja falhando no sistema deve resultar em uma resposta" [Lynch and Gilbert]. Portanto, um nodo com falhas e não respondendo não infere em uma falta de disponibilidade CAP.

Isso significa que você pode criar um cluster CA, mas tem de assegurar-se de que ele somente se particionará rara e completamente, o que pode ser feito, pelo menos, dentro de um datacenter, mas, geralmente, é proibitivamente caro. Lembre-se de que, para tirar do ar todos os nodos de um cluster em uma partição, você também tem de detectar a partição em tempo hábil, o que não é algo fácil.

Assim, os clusters têm de ser tolerantes a partições de rede. E aqui está a questão real do teorema CAP. Embora o teorema CAP seja, muitas vezes, declarado como "você só pode ter dois de três", na prática ele diz que, em um sistema que possa sofrer partições, como os sistemas distribuídos, você tem de balancear a consistência com a disponibilidade. Essa

não é uma decisão binária; muitas vezes, você pode abrir mão de um pouco de consistência para obter um pouco de disponibilidade. O sistema resultante não seria nem perfeitamente consistente nem perfeitamente disponível, mas teria uma combinação razoável para suas necessidades específicas.

Um exemplo deve ilustrar isso. Martin e Pramod estão tentando reservar o último quarto do hotel por meio de um sistema que utiliza distribuição ponto a ponto com dois nodos (Londres para Martin e Bombaim para Pramod). Se quisermos assegurar a consistência, então, quando Martin tentar reservar seu quarto no nodo de Londres, este nodo deve se comunicar com o nodo de Bombaim antes de confirmar a reserva. Basicamente, ambos os nodos devem concordar com a serialização de suas solicitações. Isso nos dá consistência, mas se a conexão da rede falhar, nenhum dos sistemas poderá reservar qualquer quarto do hotel, sacrificando a disponibilidade.

Uma forma de melhorar a disponibilidade é designar um nodo como o mestre para um determinado hotel e assegurar-se de que todas as reservas sejam processadas por ele. Se esse mestre fosse Bombaim, então ele ainda poderia processar as reservas de quartos para esse hotel e Pramod obteria a reserva do último quarto. Se utilizarmos a replicação mestre-escravo, os usuários de Londres poderão ver as informações inconsistentes sobre os quartos, mas não poderão fazer uma reserva e gerar uma inconsistência de atualização. Entretanto, os usuários esperam que isso possa acontecer nessa situação e, assim, novamente, o ajuste funciona para esse caso de uso específico.

Isso melhora a situação, mas ainda não podemos reservar um quarto no nodo de Londres para o hotel cujo mestre está em Bombaim, no caso de a conexão falhar. Na terminologia CAP, essa é uma falha de disponibilidade, já que Martin pode se comunicar com o nodo de Londres, mas o nodo de Londres não consegue atualizar os dados. Para obter mais disponibilidade, poderíamos permitir que ambos os sistemas continuassem aceitando reservas no hotel, mesmo se a conexão da rede caísse. O perigo é Martin e Pramod reservarem, ambos, o último quarto do hotel. Entretanto, dependendo de como o hotel trabalha, isso pode não ser um problema. Muitas vezes, empresas de viagem toleram um pouco de overbooking, para lidar com viajantes que não comparecem. De modo oposto, alguns hotéis sempre mantêm alguns quartos vazios mesmo quando estão lotados, para poderem trocar o hóspede de um quarto com problemas ou para acomodar alguma reserva especial realizada tardiamente. Alguns podem até cancelar a reserva com um pedido de desculpas assim que detectam o conflito, argumentando que o custo desse cancelamento é menor do que o custo de perder reservas por falhas na rede.

O exemplo clássico de permitir gravações inconsistentes é o carrinho de compras, conforme discutido no Dynamo [Dynamo, da Amazon]. Você sempre pode gravar em seu carrinho de compras, mesmo se falhas na rede significarem que você pode acabar com múltiplos carrinhos de compras. O processo de finalização das compras pode juntar os dois carrinhos por meio de uma união dos itens dos carrinhos, transferindo todos para um único carrinho. Quase sempre essa é a solução correta, mas, se não for, o usuário tem a oportunidade de examinar o carrinho antes de completar o pedido.

A lição aqui é que, embora a maioria dos desenvolvedores de software tratem a consistência de atualização como "A forma como as coisas devem ser" (The Way Things Must Be), há casos em que você pode lidar elegantemente com respostas inconsistentes a solicitações. Essas situações estão intimamente associadas ao domínio e requerem

conhecimento sobre ele para serem resolvidas. Geralmente, não podem ser resolvidas junto com a equipe de desenvolvimento apenas; é necessário conversar com especialistas na área. Se você conseguir encontrar uma forma de lidar com atualizações inconsistentes, terá mais opções para aumentar a disponibilidade e o desempenho. Para um carrinho de compras, isso significa que os compradores devem sempre poder comprar, e fazer isso rapidamente. E, como Americanos Patriotas, sabemos como é importante apoiar o Nosso Destino Varejista.

Uma lógica semelhante aplica-se à consistência de leitura. Se você estiver fazendo transações financeiras utilizando computadores, talvez não tolere quaisquer dados que não estejam atualizados. Entretanto, se você estiver postando notícias em um website de mídia, talvez possa tolerar páginas antigas por alguns minutos.

Nesses casos, é necessário saber o quão tolerante você será em relação a leituras desatualizadas e por quanto tempo a janela de inconsistência pode existir – muitas vezes, em termos de duração média, no pior caso, e alguma medida de distribuição para as durações. Itens de dados diferentes podem ter tolerâncias diferentes à obsolescência e, assim, podem precisar de diferentes valores em sua configuração de replicação.

Defensores do NoSQL dizem, frequentemente, que em vez de seguirem as propriedades ACID das transações relacionais, sistemas NoSQL seguem as propriedades BASE – basicamente disponível, estado soft, consistência eventual (dos termos originais em inglês Basically Available, Soft state, Eventual consistency) [Brewer]. Embora sintamos que deveríamos mencionar o acrônimo BASE aqui, não achamos que seja útil. O acrônimo é ainda mais artificial do que o ACID, e nem "basicamente disponível" nem "estado soft" foram bem definidos. Também devemos enfatizar que, quando Brewer apresentou a noção de BASE, vimos o balanceamento entre ACID e BASE como um espectro, não como uma escolha binária.

Incluímos essa discussão sobre o teorema CAP porque ele é utilizado (e abusado) frequentemente quando falamos de compromissos envolvendo consistência em bancos de dados distribuídos. Entretanto, geralmente é melhor não pensar no equilíbrio entre consistência e disponibilidade, mas, sim, entre consistência e latência. Podemos resumir muito da discussão sobre consistência na distribuição, dizendo que é possível melhorar a consistência envolvendo mais nodos na interação. Porém, cada nodo que adicionamos aumenta o tempo de resposta a essa interação. Podemos pensar, então, em disponibilidade como o limite de latência, o qual estamos preparados para tolerar; assim que a latência ficar muito alta, desistimos e tratamos os dados como indisponíveis – o que adapta bem sua definição ao contexto do CAP.

5.4 Relaxando a durabilidade

Até aqui, falamos sobre consistência, que é o que a maioria das pessoas quer dizer quando fala nas propriedades ACID de transações de bancos de dados. A chave da Consistência é serializar as solicitações formando unidades de trabalho Atômicas e Isoladas. Contudo, a maioria das pessoas desdenha o relaxamento da durabilidade, afinal, para que serve um depósito de dados se há o risco de perder atualizações?

Há casos em que você talvez queira trocar um pouco da durabilidade por um desempenho melhor. Se um banco de dados puder ser executado, em sua maior parte, na

memória, aplicar as atualizações em sua representação de memória e descarregar periodicamente as alterações no disco, talvez tenha mais responsividade. O custo é que, se o servidor falhar, quaisquer atualizações realizadas desde a última descarga serão perdidas.

Esse equilíbrio pode valer a pena, por exemplo, armazenando o estado da sessão do usuário. Um website com muito tráfego pode ter muitos usuários e manter, temporariamente, informações sobre o que cada um está fazendo em algum tipo de estado de sessão. Há muita atividade nesse estado, o que gera muita demanda e afeta a responsividade do website. A questão vital é que perder os dados da sessão não seria um desastre, embora pudesse gerar um pouco de aborrecimento, mas este talvez fosse menor do que o causado por um website mais lento. Isso o torna um bom candidato para gravações não duráveis. Com frequência, as necessidades de durabilidade podem ser especificadas caso a caso, de modo que as atualizações mais importantes possam forçar uma descarga para o disco.

Outro exemplo de relaxamento na durabilidade é a captura de dados de telemetria a partir de dispositivos físicos. Pode ser preferível capturar dados mais rapidamente, mesmo que as últimas atualizações sejam perdidas, caso o servidor falhe.

Outra classe de equilíbrio da durabilidade surge com os dados replicados. Uma falha na **durabilidade da replicação** ocorre quando um nodo processa uma atualização, mas falha antes que essa atualização seja replicada para os outros nodos. Isso pode acontecer, por exemplo, se você tiver um modelo de distribuição mestre-escravo, no qual os escravos apontem para um novo mestre automaticamente se o existente falhar. Se o mestre falhar, todas as gravações que não foram repassadas às réplicas serão perdidas. Quando o mestre voltar a ficar online, essas atualizações entrarão em conflito com as que ocorreram desde então. Pensamos neste como um problema de durabilidade, pois achamos que a atualização foi bem-sucedida, uma vez que o mestre a reconheceu, mas uma falha no nodo mestre fez com que fosse perdida.

Se você estiver suficientemente confiante em trazer o mestre de volta online rapidamente, esse é um motivo para não executar um redirecionamento automático após uma falha. Caso contrário, você pode melhorar a durabilidade da replicação assegurando-se de que o mestre espere que algumas réplicas reconheçam a atualização antes dele confirmá-la para o cliente. É óbvio, entretanto, que isso atrasará as atualizações e deixará o cluster indisponível se os escravos falharem – assim, mais uma vez, temos um equilíbrio que dependerá do quão vital é a durabilidade. Além da durabilidade básica, é útil que chamadas individuais indiquem de qual nível de durabilidade precisam.

5.5 Quóruns

Balancear consistência e durabilidade não é uma situação de tudo ou nada. Quanto mais nodos forem envolvidos em uma solicitação, maior a chance de evitar uma inconsistência. Isso leva, naturalmente, à questão: quantos nodos precisam ser envolvidos para que seja obtida uma alta consistência?

Imagine alguns dados replicados em três nodos. Não é necessário que todos eles reconheçam uma gravação para assegurar alta consistência, apenas dois deles, a maioria, já é o suficiente. Se as gravações forem conflitantes, apenas uma pode obter a maioria. Isso é chamado de **quórum de gravação** e é expresso em uma desigualdade um pouco

pretensiosa, `W > N/2`, significando que o número de nodos participantes da gravação (`W`) deve ser maior do que a metade do número de nodos envolvidos na replicação (`N`). O número de réplicas, muitas vezes, é chamado de **fator de replicação**.

De modo semelhante ao quórum de gravação, existe a noção de quórum de leitura: quantos nodos devem ser contatados para garantir que tenha a alteração mais atualizada? O quórum de leitura é um pouco mais complicado, pois depende de quantos nodos precisam confirmar uma gravação.

Vamos considerar um fator de replicação igual a 3. Se todas as gravações precisarem que dois nodos confirmem (`W = 2`), então precisamos contatar pelo menos dois nodos para assegurar-nos de que temos os dados mais recentes. No entanto, se as gravações são confirmadas por um único nodo (`W = 1`), precisaremos conversar com todos os três nodos para ter certeza de que estamos com os dados atualizados. Nesse caso, já que não temos um quórum de gravação, podemos ter um conflito de atualização. Porém, contatando leitores suficientes, podemos nos assegurar de detectá-lo. Assim, obteremos leituras fortemente consistentes, mesmo sem ter alta consistência nas nossas gravações.

Esse relacionamento entre o número de nodos que devem ser contatados para uma leitura (`R`), aqueles que confirmam uma gravação (`W`) e o fator de replicação (`N`) pode ser considerado uma desigualdade: pode-se ter uma leitura altamente consistente se `R + W > N`.

Essas desigualdades são escritas com um modelo de distribuição ponto a ponto em mente. Se tivermos uma distribuição mestre-escravo, devemos apenas gravar no mestre para evitar conflitos de gravação e, de modo semelhante, apenas ler do mestre para evitar conflito de leitura/gravação. Com essa notação, é comum confundir o número de nodos no cluster com o fator de replicação, mas estes, geralmente, são diferentes. Podemos ter 100 nodos no cluster, mas apenas um fator de replicação três, com a maior parte da distribuição ocorrendo devido à fragmentação.

De fato, a maioria dos especialistas sugere que um fator de replicação igual a três é suficiente para se ter uma boa resiliência. Isso permite que um único nodo falhe, mas ainda assim resta um quórum de leitura e gravação. Se você tiver rebalanceamento automático, não demorará muito para que o cluster crie uma terceira réplica, de modo que a chance de perder uma segunda réplica antes de uma substituição torna-se pequena.

O número de nodos que participam de uma operação pode variar de acordo com o tipo de operação. Na gravação, poderíamos necessitar de um quórum para alguns tipos de atualização, mas não para outros, dependendo da importância que dermos à consistência e disponibilidade. De forma semelhante, uma leitura que necessita de velocidade, mas que pode tolerar dados desatualizados, deve contatar menos nodos.

Muitas vezes, torna-se necessário considerar ambas. Se você precisar de leituras altamente consistentes e rápidas, pode solicitar que as gravações sejam reconhecidas por todos os nodos, permitindo, assim, que as leituras contatem apenas um (`N = 3`, `W = 3`, `R = 1`). Isso significa que suas gravações serão lentas, já que terão de contatar todos os três nodos e você não pode tolerar a perda de um deles. Mas, em algumas circunstâncias, esse pode ser o compromisso a assumir.

A questão é que você tem uma gama de opções com as quais trabalhar e pode escolher qual combinação de problemas e vantagens prefere. Alguns autores, que escrevem sobre NoSQL, falam sobre um balanceamento simples entre consistência e disponibilidade, mas

esperamos que você tenha percebido que é muito mais flexível – e mais complicado – do que parece.

5.6 Leituras complementares

Há todo tipo de postagens em blogs e artigos interessantes na Internet sobre consistência em sistemas distribuídos, mas a fonte mais útil para nós foi [Tanenbaum e Van Steen], Pois eles fazem um excelente trabalho ao organizar muitos dos fundamentos de sistemas distribuídos. Além disso, é o melhor lugar para ir se você quiser aprofundar seus conhecimentos sobre o assunto abordado neste capítulo.

Quando estávamos terminando este livro, a IEEE Computer lançou uma edição especial [IEEE Computer Fev 2012] sobre a crescente influência do teorema CAP, sendo uma fonte útil de mais esclarecimentos sobre esse tópico.

5.7 Pontos chave

- Conflitos de gravação ocorrem quando dois clientes tentam gravar os mesmos dados ao mesmo tempo. Conflitos de leitura-gravação ocorrem quando um cliente lê dados inconsistentes durante a gravação de outro cliente.
- Abordagens pessimistas bloqueiam os registros de dados para evitar conflitos. Abordagens otimistas detectam conflitos e os resolvem.
- Sistemas distribuídos veem conflitos de leitura-gravação devido a alguns nodos receberem atualizações e outros não. Consistência eventual significa que, em algum momento, o sistema se tornará consistente, assim que as gravações tiverem sido propagadas para todos os nodos.
- Os clientes geralmente querem a consistência leia-sua-escrita, o que significa que um cliente pode gravar e imediatamente depois ler o novo valor. Isso pode ser difícil se a leitura e a gravação ocorrerem em nodos diferentes.
- Para obter boa consistência, muitos nodos devem ser envolvidos em operações de dados, mas isso aumenta a latência. Assim, muitas vezes você tem de balancear a consistência com a latência.
- O teorema CAP declara que se você obtiver uma partição de rede, deverá balancear a disponibilidade dos dados com a consistência.
- A durabilidade também pode ser balanceada com a latência, especialmente se você quiser continuar operando no caso de haver falhas nos dados replicados.
- Você não precisa contatar todas as réplicas para preservar uma alta consistência com replicação; precisa de apenas um quórum suficientemente grande.

CAPÍTULO 6

Marcadores de versões

Muitos críticos dos bancos de dados NoSQL enfatizam sua falta de suporte a transações, que consistem em uma ferramenta útil para ajudar os programadores a garantir consistência. Um motivo pelo qual muitos proponentes do NoSQL não se preocupam tanto com a falta de transações é que bancos de dados NoSQL orientados a agregados suportam atualizações atômicas dentro de um agregado – e agregados são projetados de modo que seus formatos de dados sejam uma unidade natural de atualização. Dito isso, é verdade que as necessidades transacionais devem ser levadas em consideração quando for preciso decidir qual banco de dados utilizar.

Como parte desse processo, é importante lembrar que transações têm limitações. Mesmo dentro de um sistema transacional, ainda temos de lidar com atualizações que requerem intervenção humana e, geralmente, não podem ser executadas dentro de transações, pois isso significaria deixar uma transação aberta tempo demais. Podemos lidar com isso utilizando **marcadores de versões**, os quais acabam sendo úteis em outras situações também, especialmente quando nos afastamos do modelo de distribuição com um único servidor.

6.1 Transações comerciais e de sistema

A necessidade de suportar a consistência de atualização sem transações é, geralmente, um recurso comum de sistemas, mesmo quando eles são criados com base em bancos de dados transacionais. Quando os usuários pensam em transações, geralmente se referem a **transações comerciais**. Uma transação comercial pode ser algo como navegar por um catálogo de produtos, selecionar uma garrafa de Talisker a um bom preço, preencher informações sobre seu cartão de crédito e confirmar o pedido. Ainda assim, tudo isso não ocorrerá dentro da transação de sistema fornecida pelo banco de dados, porque implicaria bloquear o banco de dados enquanto o usuário estivesse tentando encontrar seu cartão de crédito e seus colegas o chamassem para almoçar.

Geralmente, os aplicativos apenas iniciam uma transação de sistema no final da interação com o usuário, de modo que os bloqueios somente são mantidos por um período de tempo curto. O problema, contudo, é que cálculos e decisões podem ter sido baseados em dados já alterados. A lista de preços pode ter atualizado o preço do Talisker ou alguém pode ter atualizado o endereço do cliente, alterando os custos de envio.

As técnicas gerais para lidar com isso são a concorrência offline [Fowler PoEAA], úteis em situações NoSQL também. Uma abordagem especialmente útil é o Optimistic Offline Lock (bloqueio offline otimista) [Fowler PoEAA], uma forma de atualização condicional, em que uma operação do cliente relê quaisquer informações nas quais a transação de negócio se baseie e, ainda, verifica se elas não mudaram desde que foram lidas anteriormente e exibidas para o usuário. Uma boa forma de fazer isso é assegurando-se de que os registros do banco de dados contenham alguma forma de marcador de versão: um

campo que mude a cada vez que os dados subjacentes no registro sejam alterados. Quando você ler os dados, a versão deve ser anotada, de modo que, quando for gravar dados, possa verificar se a versão sofreu alterações.

Você pode ter se deparado com essa técnica ao atualizar recursos com HTTP [HTTP]. Uma forma de fazer isso é utilizando e-tags. Sempre que você obtém um recurso, o servidor responde com uma e-tag no cabeçalho. Essa e-tag é uma string opaca que indica a versão do recurso. Se você atualizar esse recurso, pode utilizar uma atualização condicional, fornecendo a e-tag que obteve do seu último GET. Se o recurso tiver sido alterado no servidor, as e-tags não corresponderão e o servidor recusará a atualização, retornando uma resposta 412 (Pré-condição Falhou).

Alguns bancos de dados fornecem um mecanismo semelhante de atualização condicional que permite que você assegure-se de que as atualizações não sejam baseadas em dados desatualizados. Você mesmo pode fazer isso, embora, nesse caso, tenha de garantir que nenhuma outra thread possa ser executada no recurso entre sua leitura e sua atualização (às vezes, isso é chamado de operação CAS – compare-and-set, "comparar e configurar" –, cujo nome vem das operações CAS executadas em processadores. A diferença é que uma CAS de processador compara um valor antes de configurá-lo, enquanto uma atualização condicional de um banco de dados compara um marcador de versão do valor).

Há diversas formas de criar marcadores de versões. Pode-se utilizar um contador, incrementando-o sempre que atualizar o recurso. Contadores são úteis, uma vez que tornam mais fácil descobrir se uma versão é mais recente que outra. Por outro lado, necessitam de que o servidor gere o valor do contador e, também, precisam de um único mestre para garantir que os contadores não sejam duplicados.

Outra abordagem seria criar um GUID, um grande número aleatório com a garantia de que seja único. Ele utiliza algumas combinações de datas, informações de hardware e quaisquer outras fontes de aleatoriedade que se possa obter. O interessante a respeito de GUIDs é que eles podem ser gerados por qualquer um, e nunca haverá uma duplicata; a desvantagem é que são grandes e não podem ser comparados diretamente para saber qual é o mais recente.

Uma terceira abordagem seria criar uma hash dos conteúdos da origem. Com uma chave de hash suficientemente grande, uma hash de conteúdo pode ser globalmente única, como um GUID, e também pode ser gerada por qualquer um; a vantagem é que são deterministas – qualquer nodo gerará a mesma hash de conteúdo para os mesmos dados de origem. Entretanto, como os GUIDs, elas não podem ser comparadas diretamente para saber qual é a mais recente e, além disso, podem ser bem extensas.

Uma quarta abordagem seria utilizar o timestamp da última atualização. Assim como os contadores, eles são razoavelmente curtos e podem ser comparados diretamente para saber qual é o mais recente, mas têm a vantagem de não necessitar de um único mestre. Múltiplas máquinas podem gerar timestamps, mas, para funcionarem apropriadamente, seus relógios precisam ser mantidos em sincronia. Um nodo com um relógio dessincronizado pode causar todos os tipos de corrupção de dados. Se o timestamp não for suficientemente granular, também há o perigo de ter duplicatas, pois não adianta utilizar timestamps de precisão de milissegundos se houver muitas atualizações por milissegundos.

Você pode juntar as vantagens desses diferentes esquemas de marcadores de versões e utilizar mais de um para criar um marcador composto. Por exemplo, o CouchDB utiliza uma combinação de contador e hash de conteúdo. Isso permite, geralmente, que marcadores de versões sejam comparados para saber qual é o mais recente, mesmo ao utilizar a replicação ponto a ponto. Se dois pontos forem atualizados ao mesmo tempo, a combinação do mesmo contador e de diferentes hashes de conteúdo facilitarão a identificação do conflito.

Além de ajudar a evitar conflitos de atualização, marcadores de versões também são úteis para fornecer consistência de sessão (p. 90).

6.2 Marcadores de versões em múltiplos nodos

O marcador de versão básico funciona bem quando temos uma única fonte oficial de dados, como na replicação com um único servidor ou mestre-escravo. Nesse caso, o marcador de versão é controlado pelo mestre. Todos os escravos seguem os marcadores do mestre. Esse sistema, porém, tem de ser melhorado em um modelo de distribuição ponto a ponto, pois não há mais um único local criando os marcadores de versões.

Ao solicitar os mesmos dados a dois nodos, corre-se o risco de eles darem respostas diferentes. Se isso acontecer, sua reação pode variar dependendo da causa dessa diferença. Pode acontecer que uma atualização tenha alcançado apenas um nodo. Nesse caso, você pode aceitar a mais recente (supondo que consiga saber qual é). De forma alternativa, você pode ter se deparado com uma atualização inconsistente e, então, precisa decidir como lidar com ela. Nessa situação, uma e-tag ou um GUID não serão suficientes, uma vez que eles não lhe dão informações suficientes sobre os relacionamentos.

A forma mais simples de marcador de versão é um contador. Cada vez que um nodo atualiza os dados, incrementa o contador e coloca o valor dele no marcador de versão. Se você tiver réplicas de escravos azuis e verdes de um único mestre e o nodo azul responder com um marcador de versão igual a 4, enquanto o nodo verde, com um igual a 6, saberá que a resposta do verde é a mais recente.

Em situações com múltiplos mestres, precisamos de algo mais completo. Uma abordagem utilizada por sistemas distribuídos de controle de versão é assegurar-se de que todos os nodos contenham um histórico dos marcadores de versões. Dessa forma, você poderá ver se a resposta do azul é anterior à do verde. Para isso, seria necessário que os clientes guardassem históricos de marcadores de versões ou que os nodos servidores armazenassem históricos de marcadores de versões e os incluíssem ao enviar os dados. Isso também detecta uma inconsistência, a qual veríamos se obtivéssemos dois marcadores de versões e nenhum deles tivesse o outro em seu histórico. Embora sistemas de controle de versões tenham esses tipos de históricos, eles não são encontrados em bancos de dados NoSQL.

Uma abordagem simples, porém problemática, seria utilizar timestamps. O problema principal aqui é que, geralmente, é difícil assegurar-se de que os nodos tenham uma noção consistente de tempo, especialmente se as atualizações acontecerem rapidamente. Se o relógio de um dos nodos sair de sincronia, isso pode causar problemas de todos os tipos. Além disso, você não consegue detectar conflitos de gravação com timestamps, de modo que isso somente funciona bem quando temos um único mestre. Nesse caso, geralmente, é

melhor utilizar um contador.

A abordagem mais comum utilizada por sistemas NoSQL ponto a ponto é uma forma especial de marcador de versão que chamamos de **marcador vetorial**. Basicamente, um marcador vetorial é um conjunto de contadores, um para cada nodo. Um marcador vetorial para três nodos (azul, verde e preto) seria similar a `[azul: 43, verde: 54, preto: 12]`. Cada vez que um nodo sofre uma atualização interna, ele atualiza o seu próprio contador, de modo que uma atualização no nodo verde alteraria o vetor para `[azul: 43, verde: 55, preto: 12]`. Sempre que dois nodos se comunicam, sincronizam seus marcadores vetoriais. Há diversas variações de como a sincronização é feita exatamente. Estamos estabelecendo o termo "marcador vetorial" como um termo geral neste livro; você também irá se deparar com **relógios vetoriais** e **vetores de versões**, que são formas específicas de marcadores vetoriais, os quais diferem no modo pelo qual se sincronizam.

Utilizando esse esquema, é possível descobrir se um marcador de versão é mais novo do que outro, pois o marcador mais novo terá seus contadores maiores ou iguais aos do mais velho. Assim, `[azul: 1, verde: 2, preto: 5]` é mais novo do que `[azul: 1, verde: 1, preto: 5]`, pois um de seus contadores é maior. Se ambos os marcadores tiverem um contador maior do que o outro, como, por exemplo, `[azul: 1, verde: 2, preto: 5]` e `[azul: 2, verde: 1, preto: 5]`, então tem-se um conflito de gravação.

Pode haver valores faltando no vetor e, nesses casos, costumamos tratar o valor que falta como zero. Assim `[azul: 6, preto: 2]` seria tratado como `[azul: 6, verde: 0, preto:2]`. Isso permite que novos nodos sejam adicionados facilmente, sem invalidar os marcadores existentes.

Marcadores vetoriais são uma ferramenta valiosa para a localização de inconsistências, mas não as resolvem. Qualquer resolução de conflito dependerá do domínio no qual você estiver trabalhando. Essa é a parte do equilíbrio entre consistência e latência. Você tem de conviver com o fato de que partições de rede podem tornar seu sistema indisponível. Caso contrário, você tem de detectar e lidar com inconsistências.

6.3 Pontos chave

- Marcadores de versões ajudam a detectar conflitos de concorrência. Quando você lê os dados e depois os atualiza, pode verificar o marcador de versão para assegurar-se de que ninguém atualizou os dados entre sua leitura e sua gravação.
- Marcadores de versões podem ser implementados por meio de contadores, GUIDs, hashes de conteúdo, timestamps ou uma combinação deles.
- Com sistemas distribuídos, um marcador vetorial permite detectar quando nodos diferentes têm atualizações conflitantes.

CAPÍTULO 7

Map-Reduce (Mapear-Reduzir)

A ascensão dos bancos de dados orientados a agregados deve-se, em grande parte, ao crescimento dos clusters. Executar em um cluster implica equilibrar o armazenamento de dados de maneira diferente daquela executada em uma única máquina. Clusters não mudam apenas as regras de armazenamento de dados – eles também mudam as regras da computação. Se você armazena muitos dados em um cluster, para processá-los de maneira eficiente é necessário um outro modo para organizar seu processamento.

Com um banco de dados centralizado, geralmente há duas maneiras para executar a lógica de processamento sobre ele: ou no próprio servidor de banco de dados, ou em uma máquina cliente. Executá-lo em uma máquina cliente lhe dará mais flexibilidade na escolha de um ambiente de programação, o que geralmente contribui para tornar os programas mais fáceis de criar e ampliar. Isso acarreta a necessidade de trazer muitos dados do servidor de banco de dados. Se você precisa acessar muitos dados, então faz sentido executar o processamento no servidor, pagando o preço em conveniência de programação e aumentando a carga no servidor de banco de dados.

Quando você tem um cluster, as boas novas são imediatas – você tem muitas máquinas para diluir a computação. Entretanto, você ainda precisa tentar reduzir a quantidade de dados que deve ser transferida pela rede, processando tanto quanto for possível no nodo no qual estiverem os dados de que precisa.

O padrão map-reduce (uma forma de *Scatter-Gather* [Hohpe e Woolf]) é uma forma de organizar o processamento de maneira a aproveitar as múltiplas máquinas de um cluster. Ao mesmo tempo, mantém-se o quanto for possível do processamento e dos dados de que ele precisa na mesma máquina. Esse padrão destacou-se pela primeira vez com o framework MapReduce do Google [Dean e Ghemawat]. Uma implementação open-source amplamente utilizada faz parte do projeto Hadoop, embora diversos bancos de dados incluam suas próprias implementações. Assim como na maioria dos padrões, há diferenças nos detalhes entre essas implementações. Então nos concentraremos no conceito geral. O nome "map-reduce" revela sua inspiração a partir das operações de mapeamento e redução nas coleções em linguagens de programação funcional.

7.1 Map-reduce básico

Para explicar a ideia básica, começaremos a partir de um exemplo que já vimos bastante – o dos clientes e pedidos. Suponha que tenhamos escolhido os pedidos como nosso agregado, com cada pedido contendo itens. Cada item possui um ID do produto, a quantidade e o preço. Esse agregado faz bastante sentido, uma vez que, normalmente, as pessoas querem ver o pedido inteiro em um acesso. Temos muitos pedidos, de forma que fragmentamos o conjunto de dados em muitas máquinas.

Todavia, o pessoal de análise de vendas quer ver um produto e sua receita total nos últimos sete dias. Essa necessidade não se adapta à estrutura do agregado que temos, o

que é uma desvantagem do uso de agregados. Para obter o relatório de receita dos produtos, você terá de examinar vários registros em cada máquina do cluster.

Esse é exatamente o tipo de situação que demanda o map-reduce. A primeira etapa do map-reduce é o "map" (mapeamento), que é uma função cuja entrada é um único agregado e cuja saída são alguns pares de chave-valor. Nesse caso, a entrada seria um pedido e a saída seria pares de chave-valor correspondendo aos itens. Cada par teria o ID do produto como chave e um mapa embutido com a quantidade e o preço como valores (Figura 7.1).

Cada aplicação da função map é independente de todas as outras. Isso permite que elas sejam paralelizáveis com segurança, de forma que um framework map-reduce consegue criar tarefas de mapeamento eficientes em cada nodo e alocar, livremente, cada pedido a uma tarefa de mapeamento, o que produz bastante paralelismo e localidade no acesso aos dados. Para esse exemplo, estamos apenas selecionando um valor de registro, mas não há um motivo para não executarmos alguma função arbitrariamente complexa como parte do mapeamento; fornecê-la depende, apenas, do valor do dado de um agregado.

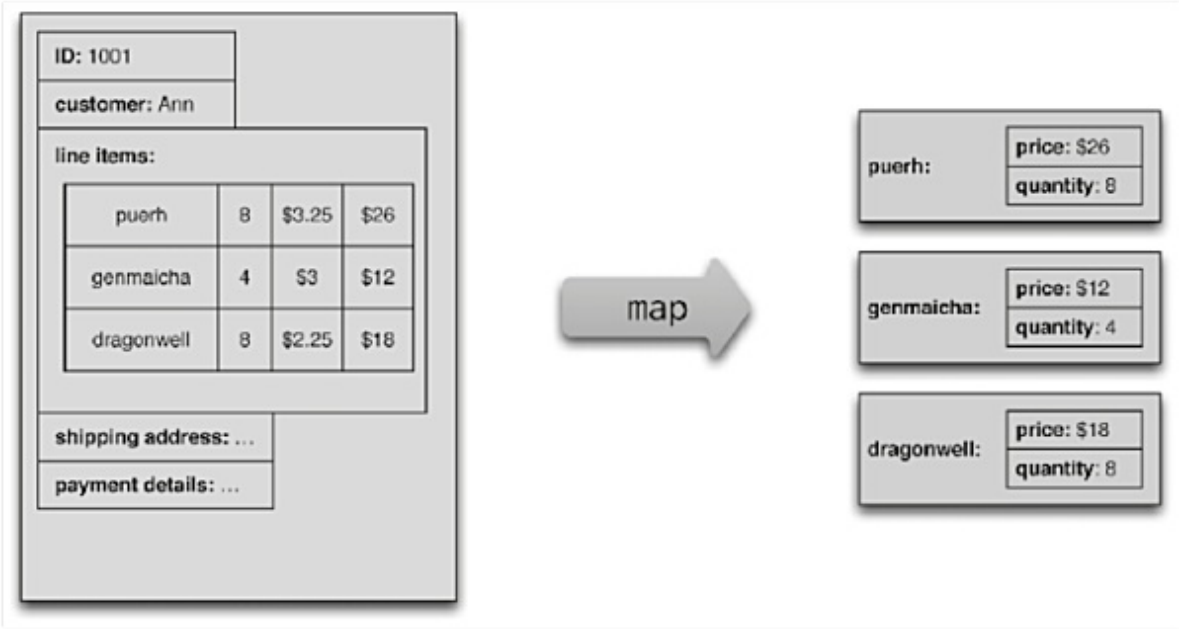

Figura 7.1 – A função map lê registros do banco de dados e emite pares de chave-valor.

Uma operação map depende apenas de um único registro; a função reduce (redução) recebe múltiplos resultados do mapeamento com a mesma chave e combina seus valores. Assim, uma função map poderia produzir 1.000 itens de pedidos para o produto "Database Refactoring"; a função reduce reduziria essas 1.000 linhas a apenas uma, com os totais para quantidade e receita. Embora a função map esteja limitada a trabalhar apenas nos dados de um único agregado, a função reduce pode utilizar todos os valores enviados para uma única chave (Figura 7.2).

Figura 7.2 – Uma função reduce recebe diversos pares de chave-valor com a mesma chave e os agrega em um.

O framework map-reduce organiza as tarefas de mapeamento para que sejam executadas nos nodos corretos, de forma que todos os documentos sejam processados e, também, de modo que os dados sejam passados para a função reduce. Para facilitar a escrita dessa função, o framework coleta todos os valores de um único par e aplica a função reduce uma vez com a chave e o conjunto de todos os valores desta chave. Assim, para executar uma função map-reduce, você apenas precisa escrever essas duas funções.

7.2 Particionando e combinando

De forma mais simples, pensamos em uma função map-reduce como tendo uma única função de redução. As saídas de todas as funções map que são executadas nos diversos nodos serão concatenadas e enviadas para a função reduce. Embora isso funcione, podemos fazer mais para aprimorar o paralelismo e reduzir a transferência de dados (Figura 7.3).

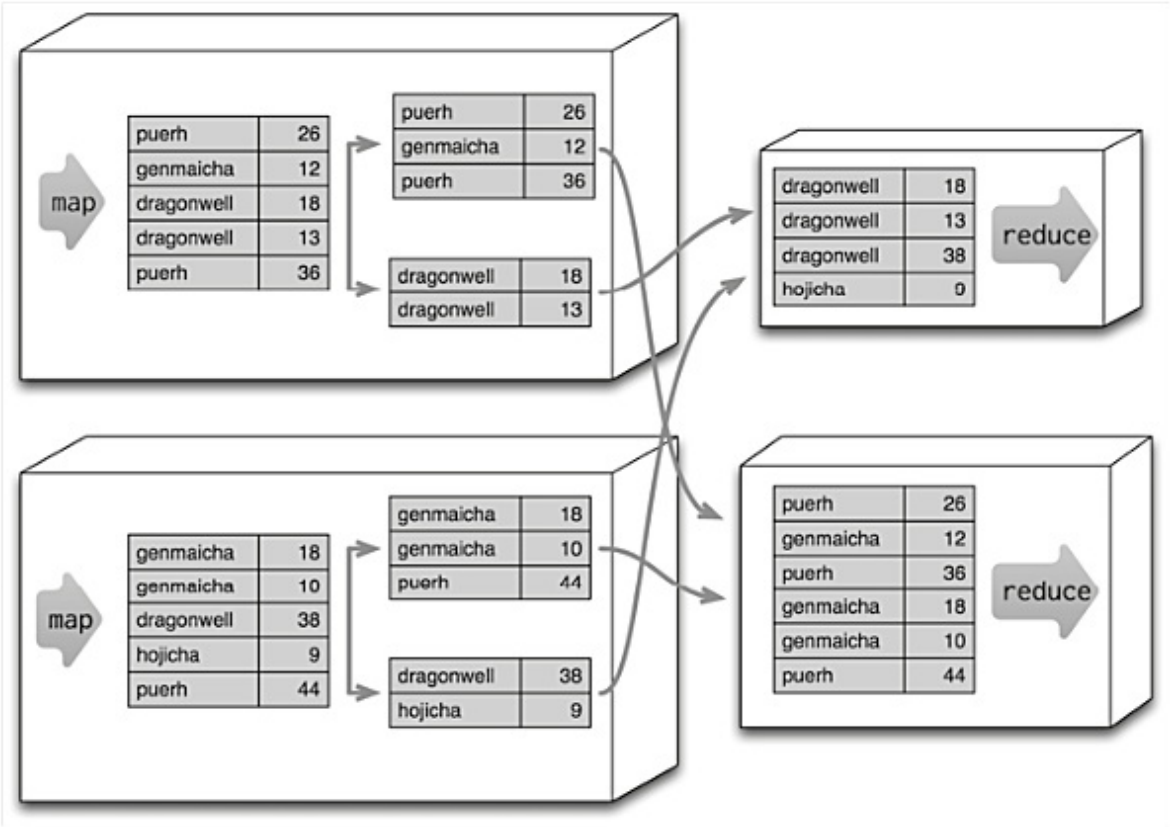

Figura 7.3 – Particionar permite que funções reduce sejam executadas em paralelo sobre diferentes chaves.

O primeiro passo a seguir é aumentar o paralelismo, particionando a saída dos mapeadores. Cada função de redução opera sobre os resultados de uma única chave. Essa é uma limitação – o que significa que você não pode fazer nada na função reduce que trabalhe com mais de uma chave –, mas é, também, um benefício, pois permite que você execute diversos redutores em paralelo. Para tirar proveito dessa situação, os resultados do mapeador são divididos com base na chave em cada nodo processador. Geralmente, múltiplas chaves são agrupadas em partições. Assim, o framework pega os dados de todos os nodos de uma partição, concentra-os em um único grupo para essa partição e os envia para um redutor. Múltiplos redutores podem, então, trabalhar nas partições em paralelo, com os resultados finais sendo agrupados. Esse passo também é chamado de shuffling (embaralhamento) e as partições, às vezes, são chamadas de buckets (regiões).

O próximo problema com o qual podemos lidar é a quantidade de dados que são transferidos de um nodo para outro entre as etapas de mapeamento e redução. Uma grande parte desses dados é repetitiva, consistindo de múltiplos pares de chave-valor para a mesma chave. Uma função combinadora diminui essa quantidade, agrupando todos os dados para a mesma chave dentro de único valor (Figura 7.4). Uma função combinadora é, em sua essência, um redutor, pois, em muitos casos, a mesma função pode ser utilizada para combinar e para a redução final. A função de redução precisa de um formato especial para funcionar: a sua saída deve corresponder à sua entrada. Chamamos tal função de **redutora combinável**.

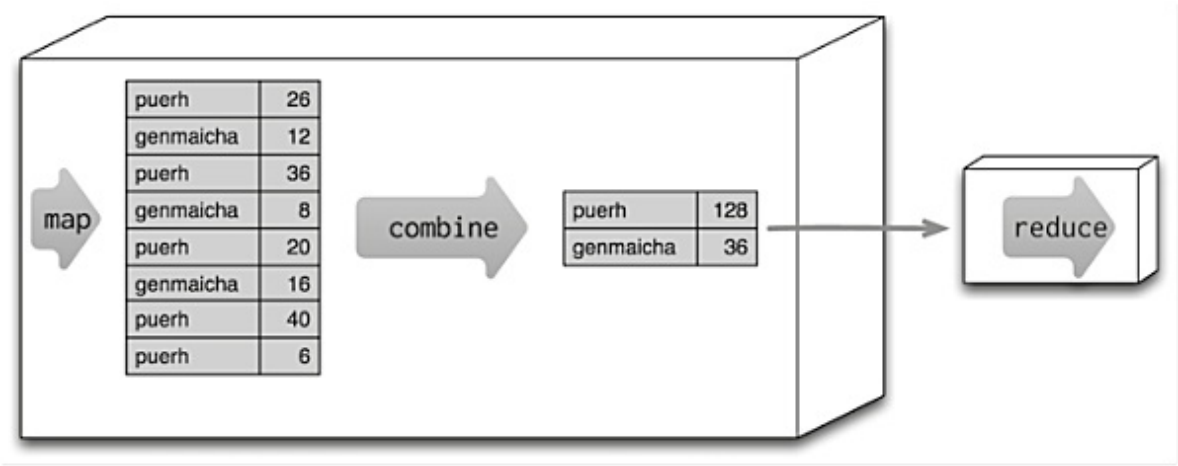

Figura 7.4 – Combinar reduz os dados antes de enviá-los pela rede.

Nem todas as funções de redução são combináveis. Considere uma função que conte o número de clientes exclusivos de um determinado produto. Para tal operação, a função map precisaria enviar o produto e o cliente. O redutor pode, então, combiná-los e contar quantas vezes cada cliente aparece para um determinado produto, enviando o produto e o contador (Figura 7.5). Esse resultado do cliente, porém, é diferente da entrada, de modo que não pode ser utilizado como um combinador. Você ainda pode executar uma função combinadora aqui: uma que elimine pares duplicados de produto-cliente, mas que será diferente da redutora final.

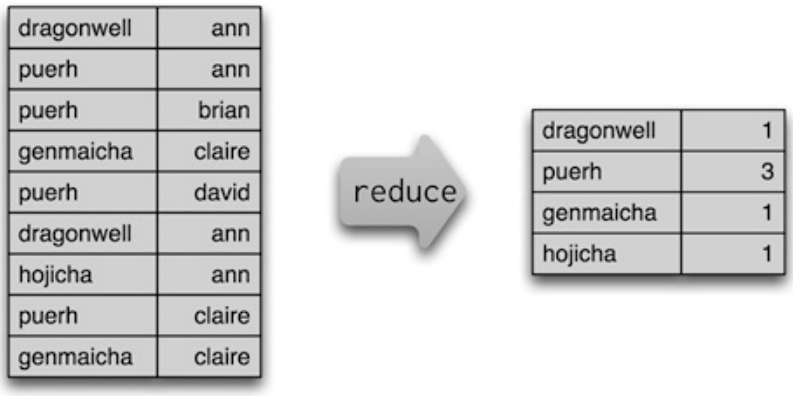

Figura 7.5 – Essa função redutora, que conta quantos clientes exclusivos encomendam um determinado chá, não é combinável.

Quando você tiver redutoras combinadas, o framework map-reduce poderá ser executado com segurança não apenas em paralelo (para reduzir diferentes partições), mas também em séries, a fim de reduzir a mesma partição em períodos e lugares diferentes. Além de permitir que a combinação ocorra em um nodo antes da transmissão dos dados, você também pode iniciar a combinação antes que os mapeadores tenham terminado. Isso fornece uma quantidade extra de flexibilidade ao processamento de map-reduce. Alguns frameworks map-reduce necessitam de que todas as redutoras sejam combinadas, fato que maximiza essa flexibilidade. Se você precisa executar um redutor não combinado com um desses frameworks, terá de separar o processamento em etapas map-reduce com pipelines (encadeamentos).

7.3 Criando cálculos map-reduce

A abordagem map-reduce é uma forma de pensar no processamento concorrente que equilibra a flexibilidade na forma pela qual você estrutura sua computação por um modelo relativamente direto de paralelização da computação em um cluster. Por ser um equilíbrio, há restrições quanto ao que você pode fazer em seus cálculos. Dentro de uma tarefa de mapeamento, você somente pode trabalhar em um único agregado. Dentro de uma tarefa de redução, você somente pode trabalhar em uma única chave. Isso significa que você tem de mudar a maneira como estrutura seus programas, de forma a fazê-los funcionarem bem dentro dessas restrições.

Uma limitação simples seria você ter de estruturar seus cálculos em torno de operações que se adaptem bem à noção de uma operação de redução. Um bom exemplo é o cálculo de médias. Vamos considerar o tipo de pedidos que temos examinado até então; suponha que queiramos saber a quantidade média encomendada de cada produto. Uma propriedade importante de médias é que elas não são combináveis, ou seja, se eu tiver dois grupos de pedidos, não posso combinar apenas as suas médias. Em vez disso, preciso obter a quantidade total e o contador dos pedidos de cada grupo, combiná-los e, então, calcular a média a partir do contador e da soma combinados (Figura 7.6).

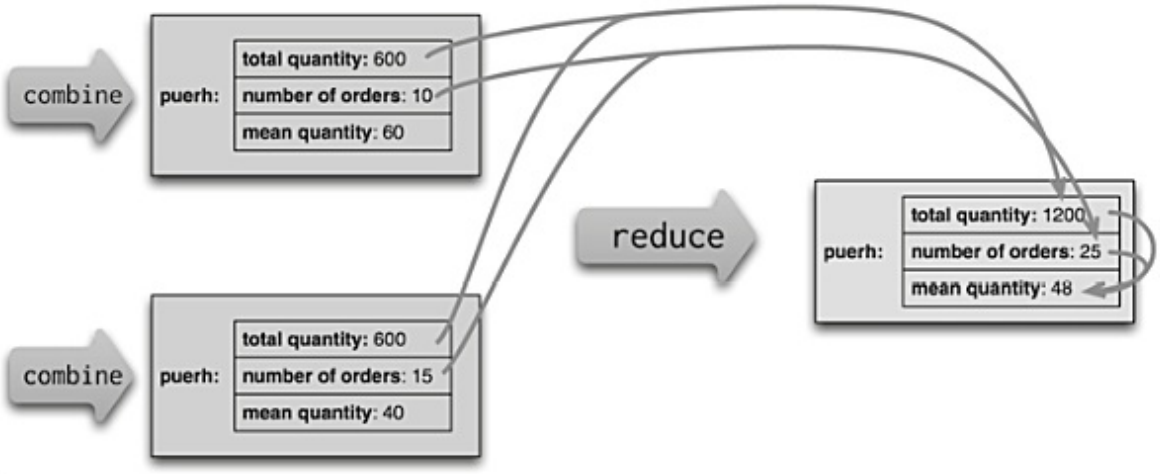

Figura 7.6 – Ao calcular médias, a soma e o contador podem ser combinados na função reduce, mas a média deve ser calculada a partir da soma e do contador combinados.

Essa noção de procurar cálculos que reduzam perfeitamente também afeta a maneira com a qual executamos as contagens. Para fazer um contador, a função de mapeamento produzirá campos de contador com o valor igual a 1, o que pode ser resumido para se obter uma contagem total (Figura 7.7).

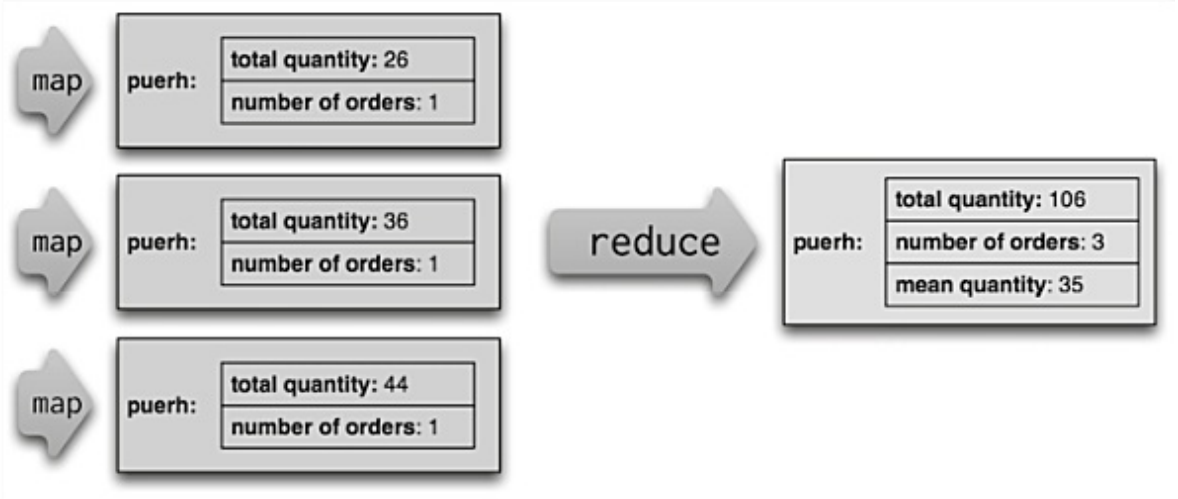

Figura 7.7 – Ao fazer uma conta, cada mapa gera 1, o que pode ser somado para se obter um total.

7.3.1 Um exemplo de map-reduce em duas etapas

À medida que os cálculos de map-reduce ficam mais complexos, é útil dividi-los em etapas utilizando a abordagem de pipes e filtros. Dessa maneira, a saída de uma etapa servirá como entrada da próxima, assim como os pipelines do Unix.

Considere o seguinte exemplo: queremos comparar as vendas de produtos em cada mês de 2011 com as dos meses do ano anterior. Para fazer isso, dividiremos os cálculos em duas etapas. A primeira produzirá registros mostrando valores do agregado para um único produto em um único mês do ano. A segunda etapa utiliza-os, então, como uma entrada e produz o resultado de um único produto, comparando os resultados de um mês com os do mesmo mês do ano anterior (Figura 7.8).

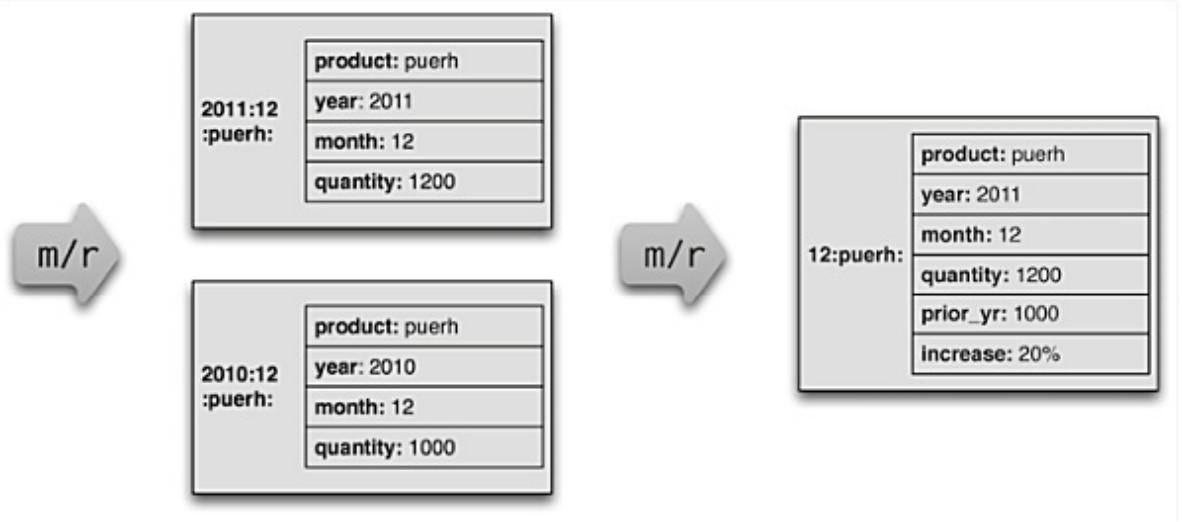

Figura 7.8 – Um cálculo dividido em etapas de map-reduce, que serão expandidas nas próximas três figuras.

Uma primeira etapa (Figura 7.9) leria os registros dos pedidos originais e produziria uma série de pares de chave-valor para as vendas de cada produto por mês.

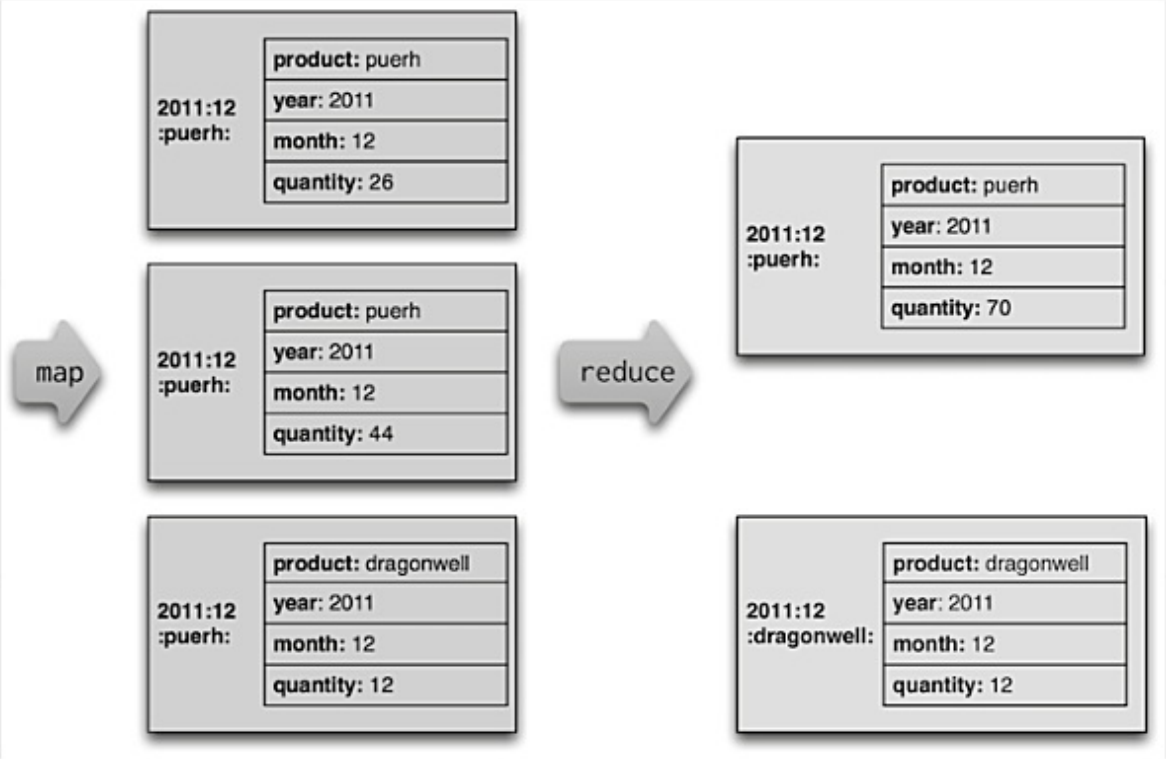

Figura 7.9 – Criando registros para as vendas mensais por produto.

Essa etapa é semelhante aos exemplos de map-reduce que vimos até aqui. A única característica diferente é o uso de uma chave composta, de modo que possamos reduzir os registros baseados nos valores de múltiplos campos.

Os mapeadores da segunda etapa (Figura 7.10) processam esse resultado dependendo do ano. Um registro de 2011 é utilizado para aumentar a quantidade desse ano, enquanto um registro de 2010 aumenta a quantidade do ano anterior. Registros de anos anteriores (como 2009) não produzem resultados mapeados.

A redução, nesse caso (Figura 7.11), é uma fusão de registros, em que a combinação de valores pelo somatório permite que os resultados de dois anos sejam reduzidos a um único valor (com um cálculo baseado nos valores reduzidos, adicionados para uma boa medição).

Decompor esse relatório em múltiplas etapas de map-reduce facilita a escrita. Do mesmo modo, isso acontece com muitos exemplos de transformação. Uma vez que você tenha encontrado um framework de transformação que facilite a composição de etapas, torna-se mais fácil compor muitas etapas pequenas do que tentar colocar muita lógica em uma única etapa.

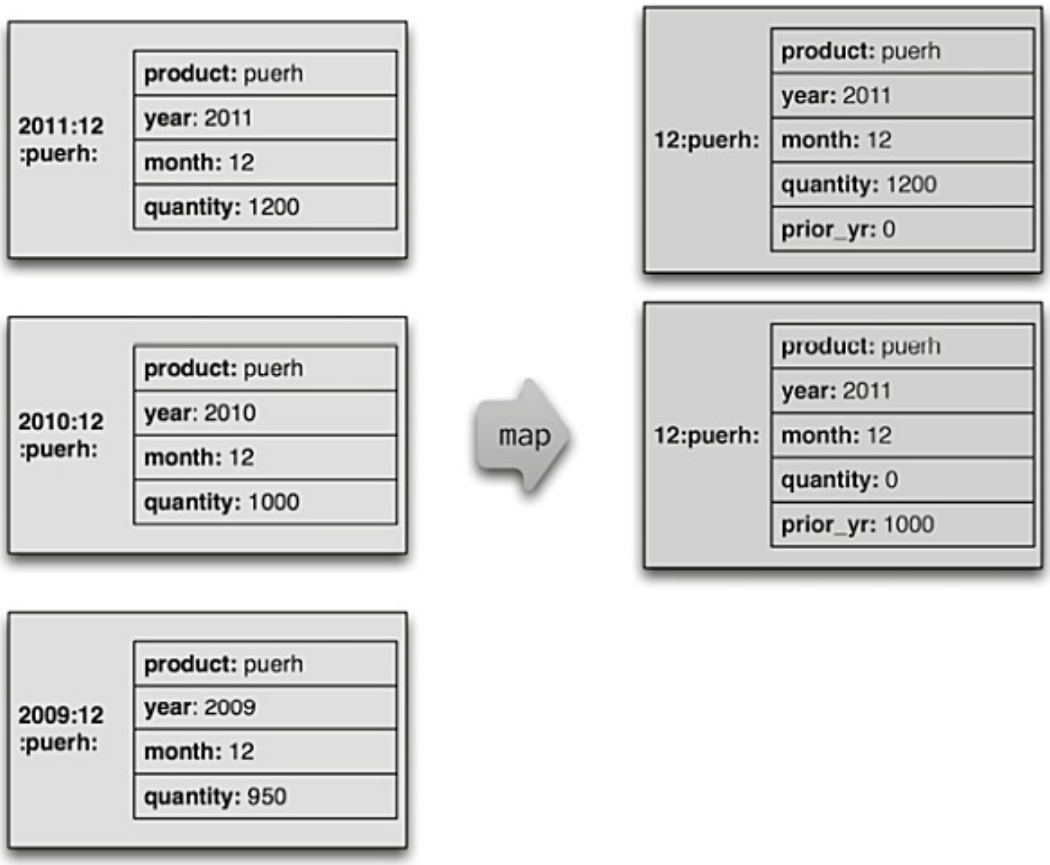

Figura 7.10 – A segunda etapa cria registros básicos para comparações ano a ano.

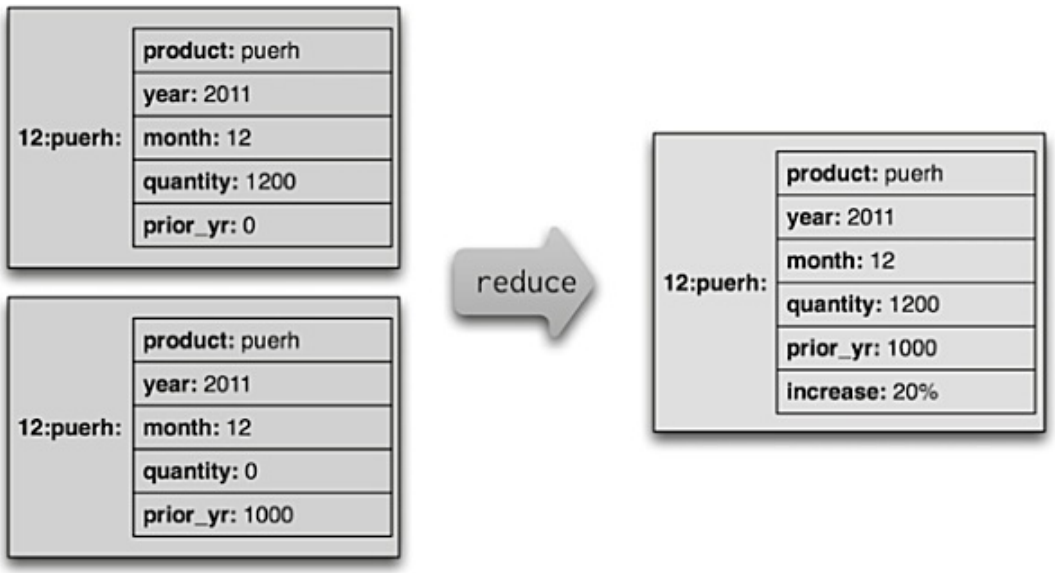

Figura 7.11 – A etapa de redução é uma fusão de registros incompletos.

Outra vantagem é que a saída intermediária pode ser útil, também, para diferentes saídas, de modo que você possa obter alguma reutilização, que é importante, uma vez que economiza tempo tanto na programação quanto na execução. Os registros intermediários podem ser armazenados, formando uma visão materializada (“Visões materializadas”, p. 64). As etapas iniciais de operações map-reduce são, particularmente, valiosas para salvar, uma vez que, muitas vezes, representam a maior quantidade de acesso aos dados. Por esse

motivo, criá-las de forma que possam ser reutilizadas poupa muito trabalho. Todavia, da mesma forma que qualquer atividade relacionada à reutilização, é importante criá-las a partir da experiência com consultas reais, uma vez que a reutilização especulativa raramente tem o desempenho esperado. Assim, é importante examinar os formatos das diversas consultas à medida que forem criadas e colocar as partes comuns dos cálculos em visões materializadas.

O map-reduce é um padrão que pode ser implementado em qualquer linguagem de programação. Entretanto, as restrições do estilo tornam-no apropriado para linguagens projetadas especificamente para computações map-reduce. Apache Pig [Pig], um ramo do projeto Hadoop [Hadoop], é uma linguagem criada especificamente para facilitar a escrita de programas map-reduce. Ela certamente facilita mais o trabalho com o Hadoop do que com as bibliotecas Java subjacentes. De um modo semelhante, se você quiser especificar programas map-reduce utilizando uma sintaxe semelhante a SQL, existe a Hive [Hive], outra ramificação do Hadoop.

É importante conhecer o padrão map-reduce mesmo fora do contexto dos bancos de dados NoSQL. O sistema map-reduce original do Google trabalhava em arquivos armazenados em um sistema de arquivos distribuído – uma abordagem utilizada pelo projeto open-source Hadoop. Embora seja necessário um pouco de reflexão para se acostumar com as restrições para estruturar computações nas etapas de map-reduce, o resultado é um cálculo inerentemente apropriado para ser executado em um cluster. Ao lidar com grandes volumes de dados, você precisa utilizar uma abordagem orientada a clusters. Bancos de dados orientados a agregados são apropriados a esse estilo de cálculo. Acreditamos que, nos próximos anos, muitas outras organizações processarão os volumes de dados que demandam uma solução orientada a clusters – e o padrão map-reduce será cada vez mais utilizado.

7.3.2 Map-reduce incremental

Os exemplos que discutimos até então são computações map-reduce completas, em que começamos com entradas brutas e criamos um resultado final. Muitas computações map-reduce demoram um pouco para serem executadas (mesmo com hardware clusterizado) e novos dados continuam chegando, o que significa que precisamos executar novamente a computação para manter o resultado atualizado. Começar sempre do zero demora demais, de forma que, muitas vezes, torna-se útil estruturar uma computação map-reduce para permitir atualizações incrementais, permitindo, assim, que apenas a computação mínima seja executada.

As etapas de mapeamento de um map-reduce são fáceis de lidar de forma incremental – o mapeador somente precisará ser executado novamente se os dados da entrada mudarem. Uma vez que mapas ficam isolados uns dos outros, atualizações incrementais tornam-se simples.

O caso mais complexo é o da redução, uma vez que ela junta os resultados de muitos mapeamentos e qualquer alteração nos resultados deles poderia disparar uma nova redução. Essa nova execução da computação pode ser diminuída dependendo do quão paralela for a etapa da redução. Se estivermos particionando os dados para redução, então qualquer partição que não tiver sido alterada não precisa ser reduzida novamente. De modo semelhante, se houver uma etapa combinadora e se os dados-fonte não tiverem sido

alterados, não haverá necessidade de que se execute novamente.

Se o nosso redutor puder ser combinado, há mais algumas oportunidades para evitar computações. Se as alterações forem aditivas, ou seja, se somente estivermos adicionando registros novos, mas não alterando ou excluindo algum registro antigo, então podemos apenas fazer a redução com os resultados existentes e as novas adições. Se houver mudanças destrutivas, sejam atualizações ou dados deletados, podemos evitar alguma recomputação ao separar operações reduzidas em etapas e apenas recalcular aquelas etapas em que houve alteração na entrada utilizando, essencialmente, um Dependency Network [Fowler DSL] para organizar a computação.

O framework map-reduce controla muito isso, de forma que você tem de entender como um framework específico suporta operações incrementais.

7.4 Leituras complementares

Se você for utilizar cálculos map-reduce, seu ponto de partida será a documentação do banco de dados específico que estiver utilizando. Cada banco de dados tem sua própria abordagem, vocabulário e particularidades e é com isso que você precisa estar familiarizado. Além disso, há uma necessidade de obter informações mais gerais sobre como estruturar trabalhos de map-reduce, de forma a maximizar a capacidade de manutenção e desempenho. Não temos livros específicos para indicar ainda, mas supomos que uma fonte boa, porém facilmente negligenciada, sejam os livros sobre o Hadoop. Embora o Hadoop não seja um banco de dados, ele é uma ferramenta que utiliza bastante map-reduce, então, escrever uma tarefa map-reduce efetiva com o Hadoop provavelmente seja útil em outros contextos (poderá haver alterações nos detalhes entre o Hadoop e quaisquer sistemas que você estiver utilizando).

7.5 Pontos chave

- Map-reduce é um padrão que permite que computações sejam paralelizadas em um cluster.
- A tarefa de mapeamento lê dados de um agregado e os agrupa em pares de chave-valor relevantes. Mapeamentos somente leem um único registro de cada vez e podem, assim, ser paralelizados e executados no nodo que armazena o registro.
- A tarefa redução recebe muitos valores de uma única chave de saída, a partir da tarefa mapeamento, e os resume em uma única saída. Cada redutora trabalha sobre o resultado de uma única chave, de modo que podem ser paralelizados por chave.
- Redutoras que tenham o mesmo formato de entrada e saída podem ser combinadas em pipelines. Isso melhora o paralelismo e reduz a quantidade de dados a serem transferidos.
- Operações de map-reduce podem ser compostas em pipelines, nas quais a saída de uma redução é a entrada do mapeamento de outra operação.
- Se o resultado de uma computação map-reduce for amplamente utilizado, pode ser armazenado como uma visão materializada.
- Visões materializadas podem ser atualizadas por meio de operações map-reduce que apenas computem alterações na visão, em vez de computar novamente tudo desde o

início.

PARTE II

Implementar

CAPÍTULO 8

Bancos de dados de chave-valor

Um depósito de chave-valor é uma tabela hash simples, utilizada principalmente quando todo o acesso ao banco de dados é feito por meio da chave primária. Pense em uma tabela em um SGBDR tradicional com duas colunas, tais quais `ID` e `NAME`, sendo a coluna `ID` a chave e a coluna `NAME`, armazenando o valor. Em um SGBDR, a coluna `NAME` restringe-se a armazenar dados do tipo `String`. O aplicativo pode fornecer um `ID` e um `VALUE` (*VALOR*) e persistir o par; se o `ID` já existir, o valor atual é sobrescrito. Caso contrário, uma nova entrada é criada. Veja como a terminologia se compara no Oracle e no Riak.

Oracle	Riak
Instância de banco de dados	Cluster Riak
Tabela	Bucket
Linha	chave-valor
Rowid	Chave

8.1 Depósitos de chave-valor

Depósitos de chave-valor são os depósitos de dados NoSQL mais simples de utilizar a partir da perspectiva de uma API. O cliente pode obter o valor para uma determinada chave, inserir um valor para uma determinada chave ou apagar uma chave do depósito de dados. O valor é um blob que o depósito de dados apenas armazena, sem se preocupar ou saber o que há dentro dele; é responsabilidade do aplicativo entender o que foi armazenado. Já que depósitos de chave-valor sempre fazem o acesso pela chave primária, eles têm, geralmente, um ótimo desempenho e podem ser escaláveis facilmente.

Alguns dos bancos de dados de chave-valor mais populares são o Riak [Riak], Redis (muitas vezes chamado de servidor Data Structure) [Redis], Memcached DB e suas variedades [Memcached], Berkeley DB [Berkeley DB], HamsterDB (especialmente apropriado para uso interno) [HamsterDB], Amazon DynamoDB [Amazon's Dynamo] (que não é open-source) e Project Voldemort [Project Voldemort] (uma implementação open-source do Amazon DynamoDB).

Em alguns armazenamentos de chave-valor, como o Redis, o agregado armazenado não tem de ser um objeto do domínio, ou seja, ele pode ser qualquer estrutura de dados. O Redis suporta o armazenamento de listas, conjuntos, hashes e pode fazer operações de intervalos, diferença, união e intersecção. Esses recursos permitem que o Redis seja utilizado de formas mais diferenciadas do que um depósito padrão de chave-valor.

Há muitos outros bancos de dados de chave-valor e muitos novos estão sendo criados neste momento. Para manter uma discussão mais simples neste livro, focalizaremos o Riak. Ele nos permite armazenar chaves em buckets, o que é apenas uma forma de segmentar as chaves – pense em buckets como namespaces para as chaves.

Se quiséssemos armazenar dados de sessão do usuário, informações de carrinhos de compras e preferências dos usuários no Riak, poderíamos armazenar tudo isso no mesmo bucket com uma única chave e um único valor para todos esses objetos. Nesse cenário, teríamos um único objeto que armazena todos os dados e é colocado em um único bucket (Figura 8.1).

A desvantagem de armazenar todos os objetos diferentes (agregados) em um único bucket é que somente ele armazenaria diferentes tipos de agregados, aumentando a chance de conflitos de chaves. Uma abordagem alternativa seria inserir o nome do objeto na chave, como `288790b8a421_userProfile`, de forma que possamos chegar a objetos individuais conforme eles sejam necessários (Figura 8.2).

Também poderíamos criar buckets para armazenar dados específicos. No Riak, eles são conhecidos como **buckets de domínio**, permitindo que a serialização e a desserialização sejam controladas pelo driver do cliente.

```
Bucket bucket = client.fetchBucket(bucketName).execute();
DomainBucket<UserProfile> profileBucket =
   DomainBucket.builder(bucket, UserProfile.class).build();
```

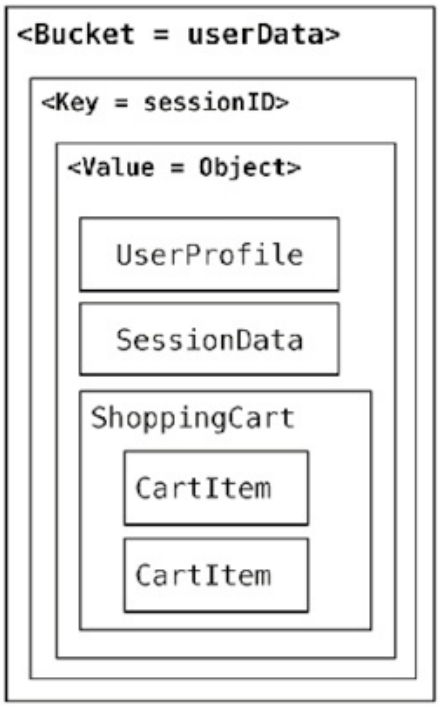

Figura 8.1 – Armazenando todos os dados em um único bucket.

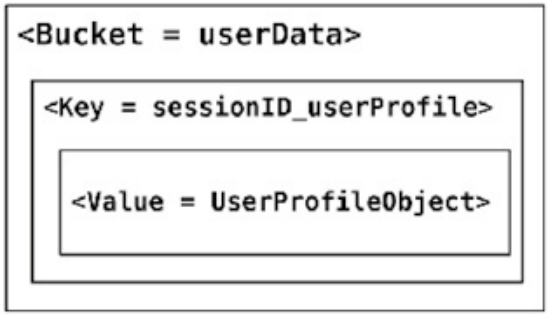

Figura 8.2 – Altere o modelo da chave para segmentar os dados de um único bucket.

Utilizar buckets de domínio ou buckets diferentes para diferentes objetos (como

`UserProfile` e `ShoppingCart`) segmenta os dados em diferentes buckets, permitindo que você leia apenas o objeto de que precisa, sem ter de alterar o projeto da chave.

Depósitos de chave-valor, como o Redis, também suportam estruturas de dados aleatórias, as quais podem ser conjuntos, hashes, strings e assim por diante. Esse recurso pode ser utilizado para armazenar listas variadas, como `states` (*listas de estados*) ou `AddressTypes` (*tiposDeEndereco*), ou, ainda, um array com as visitas do usuário.

8.2 Recursos dos depósitos de chave-valor

Ao utilizar quaisquer depósitos de dados NoSQL, há uma necessidade inevitável de entender como os seus recursos se comparam com os depósitos de dados padrão de SGBDR com os quais estamos acostumados. O principal motivo é entender quais recursos estão faltando e como a arquitetura do aplicativo precisa mudar para utilizar melhor os recursos de um armazenamento de dados de chave-valor. Alguns dos recursos que discutiremos para todos os armazenamentos de dados NoSQL são consistência, transações, recursos de consultas, estrutura dos dados e escalabilidade.

8.2.1 Consistência

A consistência é aplicável apenas às operações em uma única chave, já que essas operações são a obtenção, gravação ou exclusão em uma única chave. Gravações otimistas podem ser executadas, mas são muito caras para implementar, pois uma alteração no valor não pode ser determinada pelo armazenamento de dados.

Em implementações distribuídas de armazenamentos de chave-valor, como o Riak, o modelo de *consistência eventual* é implementado (p. 88). Uma vez que o valor pode já ter sido replicado em outros nodos, o Riak tem duas maneiras de resolver conflitos de atualização: ou a gravação mais recente prevalece sobre as mais antigas, ou ambos (todos) os valores são retornados, permitindo ao cliente resolver o conflito.

No Riak, essas opções podem ser configuradas durante a criação do bucket. Buckets são apenas uma forma de colocar as chaves em namespaces, o que reduz colisões de chaves – por exemplo, todas as chaves de clientes podem ficar no bucket customer. Ao criar um bucket, valores padrões para consistência podem ser fornecidos. Por exemplo, uma gravação é considerada boa apenas quando os dados forem consistentes por todos os nodos em que estiverem armazenados.

```
Bucket bucket = connection
    .createBucket(bucketName)
    .withRetrier(attempts(3))
    .allowSiblings(siblingsAllowed)
    .nVal(numberOfReplicasOfTheData)
    .w(numberOfNodesToRespondToWrite)
    .r(numberOfNodesToRespondToRead)
    .execute();
```

Se precisarmos que os dados sejam consistentes em todos os nodos, podemos aumentar a configuração de `numberOfNodesToRespondToWrite`, definida pelo `w`, para ser o mesmo valor de `nVal`. É claro que fazer isso piorará o desempenho de gravação do cluster. Para melhorar os conflitos de gravação ou leitura, podemos alterar a flag `allowSiblings` durante a criação do bucket: se for configurada como falsa, permitiremos que a última gravação prevaleça e não

criaremos cópias.

8.2.2 Transações

Diferentes produtos no armazenamento de chave-valor têm diferentes especificações de transação. De modo geral, não há garantias nas gravações. Muitos armazenamentos de dados implementam transações de diferentes formas. O Riak utiliza o conceito de quórum (“Quóruns”, p. 97), implementado usando o valor `W` – quórum de gravação – durante a chamada à API de gravação.

Suponha que tenhamos um cluster Riak com um fator de replicação igual a 5 e forneçamos o valor `W` igual a 3. Essa gravação será informada como bem-sucedida, durante a gravação, apenas quando for realizada e informada como bem-sucedida em pelo menos três dos nodos. Isso permite ao Riak ser tolerante nas gravações; em nosso exemplo, com `N` igual a 5 e `W` igual a 3, o cluster pode tolerar `N - W = 2` nodos falhando para operações de gravação, embora ainda teríamos perdido alguns dados nesses nodos para leitura.

8.2.3 Recursos de consultas

Todos os armazenamentos de chave-valor podem ser consultados pela chave – e isso é tudo. Se você necessitar fazer uma consulta utilizando algum atributo da coluna de valores, não é possível utilizar o banco de dados: seu aplicativo precisa ler o valor para descobrir se ele satisfaz às condições.

Realizar consultas por chave também causa um efeito colateral interessante. E se não conhecermos a chave, especialmente durante a consulta ad-hoc na depuração? A maioria dos depósitos de dados não lhe fornecerá uma lista de todas as chaves primárias; mesmo se o fizesse, recuperar listas de chaves e depois pesquisar o valor seria muito complicado. Alguns bancos de dados de chave-valor contornam isso fornecendo a capacidade de pesquisar dentro do valor, assim como o **Riak Search**, que permite que você consulte os dados da mesma forma que o faria utilizando índices Lucene.

Ao fazer uso de depósitos de chave-valor, deve-se dar muita importância ao projeto da chave. A chave pode ser gerada utilizando algum algoritmo? A chave pode ser fornecida pelo usuário (ID do usuário, e-mail etc.)? Ou derivada de timestamps ou outros dados que possam ser derivados fora do banco de dados?

Essas características de consulta tornam os armazenamentos de chave-valor prováveis candidatos para armazenar dados de sessão (com o ID da sessão como chave), dados de carrinhos de compras, perfis de usuário e assim por diante. A propriedade `expiry_secs` pode ser utilizada para expirar as chaves após um determinado intervalo de tempo, especialmente para objetos de carrinhos de compras/sessão.

```
Bucket bucket = getBucket(bucketName);
IRiakObject riakObject = bucket.store(key, value).execute();
```

Ao gravar no bucket do Riak utilizando a sua API de armazenamento (`store`), o objeto é armazenado na chave fornecida. De modo semelhante, podemos obter o valor armazenado para a chave utilizando a API de recuperação (`fetch`).

```
Bucket bucket = getBucket(bucketName);
IRiakObject riakObject = bucket.fetch(key).execute();
byte[] bytes = riakObject.getValue();
String value = new String(bytes);
```

O Riak fornece uma interface baseada em HTTP, de modo que todas as operações podem ser executadas a partir do navegador web ou na linha de comando utilizando o `curl`. Vamos gravar esses dados no Riak:

```
{
   "lastVisit":1324669989288,
   "user":{
      "customerId":"91cfdf5bcb7c",
      "name":"buyer",
      "countryCode":"US",
      "tzOffset":0
   }
}
```

Utilize o comando `curl` para `POST` (postar) os dados, armazenando-os no bucket `session` com a chave `a7e618d9db25` (temos de fornecer essa chave):

```
curl -v -X POST -d '
{
   "lastVisit":1324669989288,
   "user":{
      "customerId":"91cfdf5bcb7c",
      "name":"buyer",
      "countryCode":"US",
      "tzOffset":0
   }
} '
-H "Content-Type: application/json"
http://localhost:8098/buckets/session/keys/a7e618d9db25
```

Os dados da chave `a7e618d9db25` podem ser recuperados utilizando o comando `curl`:

```
curl -i http://localhost:8098/buckets/session/keys/a7e618d9db25
```

8.2.4 Estrutura de dados

Bancos de dados de chave-valor não se importam com o que é armazenado na parte do valor do par chave-valor. O valor pode ser um blob, texto, JSON, XML e assim por diante. No Riak, podemos utilizar o `Content-Type` (tipo de conteúdo) na solicitação `POST` para especificar o tipo dos dados.

8.2.5 Escalabilidade

Muitos armazenamentos de chave-valor podem ser escalados utilizando a fragmentação (“Fragmentação”, p. 74). Com ela, o nome da chave determina em qual nodo a chave deve ser armazenada. Suponhamos que estejamos fragmentando pelo primeiro caractere da chave; se a chave for `f4b19d79587d`, que começa com um `f`, será enviada para um nodo diferente do da chave `ad9c7a396542`. Esse tipo de configuração de fragmentação pode melhorar o desempenho na medida em que mais nodos são acrescentados ao cluster.

A fragmentação também traz alguns problemas. Se o nodo utilizado para armazenar `f` falhar, os dados armazenados nesse nodo ficam indisponíveis e novos dados não podem ser gravados com chaves que comecem com `f`.

Bancos de dados como o Riak permitem que você controle os aspectos do teorema CAP (p. 91): `N` (o número de nodos para armazenar as réplicas chave-valor), `R` (o número de

nodos que devem ter os dados pesquisados antes da leitura ser considerada como bem-sucedida) e `W` (o número de nodos nos quais a gravação deve ser executada antes de ser considerada bem-sucedida).

Suponhamos que temos um cluster Riak com cinco nodos. Configurar `N` como 3 significa que todos os dados serão replicados em pelo menos três nodos. Configurar `R` como 2 significa que dois nodos quaisquer devem responder a uma solicitação `GET` para que esta seja considerada bem-sucedida. Por fim, configurar `W` como 2 garante que a solicitação `PUT` seja gravada em dois nodos antes de a gravação ser considerada bem-sucedida.

Essas configurações permitem-nos fazer um ajuste detalhado nas falhas dos nodos em operações de leitura e de gravação. Baseados em nossas necessidades, podemos alterar esses valores para obter mais disponibilidade de leitura ou de gravação. De modo geral, selecione um valor para `W` que satisfaça às suas necessidades de consistência; esses valores podem ser configurados como padrões durante a criação dos buckets.

8.3 Casos apropriados para uso

Discutiremos alguns dos problemas para os quais os armazenamentos de chave-valor são uma solução apropriada.

8.3.1 Armazenando informações de sessão

Geralmente, toda sessão web é única e recebe um valor único de `sessionid`. Aplicativos que armazenam o `sessionid` em disco ou em um SGBDR serão muito beneficiados com a mudança para um armazenamento de chave-valor, já que tudo a respeito da sessão pode ser armazenado por uma única solicitação `PUT` ou recuperado utilizando `GET`. Essa operação de solicitação única torna esse processo muito rápido, pois tudo o que está relacionado à sessão é armazenado em um único objeto. Soluções como Memcached são utilizadas por muitos aplicativos web, e o Riak pode ser utilizado quando a disponibilidade for importante.

8.3.2 Perfis de usuários, preferências

Quase todos os usuários têm um único `userId` (ID do usuário), um único `username` (nome do usuário), ou algum outro atributo único, além de preferências, como idioma, cor, fuso horário, a quais produtos o usuário têm acesso, e assim por diante. Tudo isso pode ser colocado em um objeto, de modo que as preferências de um usuário sejam obtidas por meio de uma única operação `GET`. De modo semelhante, perfis de produtos também podem ser armazenados.

8.3.3 Dados de carrinhos de compras

Websites de comércio eletrônico possuem carrinhos de compras associados ao usuário. Como queremos que os carrinhos de compras estejam disponíveis todo o tempo, em vários navegadores, máquinas e sessões, todas as informações de compras podem ser colocadas no valor onde a chave é `userid`. Um cluster Riak seria mais apropriado para esses tipos de aplicativos.

8.4 Quando não utilizar

Há situações nas quais os depósitos de chave-valor não são a melhor opção.

8.4.1 Relacionamentos entre dados

Se você precisar de relacionamentos entre diferentes conjuntos de dados, ou então correlacionar os dados entre diferentes conjuntos de chaves, armazenamentos de chave-valor não são a melhor opção, ainda que alguns deles forneçam recursos de navegação entre conexões (link-walking).

8.4.2 Transações com múltiplas operações

Se você estiver salvando múltiplas chaves e ocorrer uma falha na gravação de alguma delas, e você quiser reverter ou desfazer o restante das operações, armazenamentos de chave-valor não são a melhor opção.

8.4.3 Consulta por dados

Se você precisa pesquisar as chaves baseando-se em algo que esteja na parte do valor dos pares chave-valor, então depósitos de chave-valor não poderão ajudar muito. Não há como inspecionar o valor pelo banco de dados, com a exceção de alguns produtos, como o Riak Search, ou os mecanismos de indexação, como o Lucene [Lucene] ou o Solr [Solr].

8.4.4 Operações por conjuntos

Já que as operações são limitadas a uma chave por vez, não há como trabalhar em múltiplas chaves ao mesmo tempo. Se você precisar trabalhar sobre múltiplas chaves, terá de lidar com isso a partir do lado cliente.

CAPÍTULO 9

Bancos de dados de documentos

Documentos são o conceito principal em bancos de dados de documentos. O banco de dados armazena e recupera documentos, os quais podem ser XML, JSON, BSON, entre outros. Esses documentos são estruturas de dados na forma de árvores hierárquicas e autodescritivas, constituídas de mapas, coleções e valores escalares. Os documentos armazenados são semelhantes entre si, mas não têm de ser exatamente os mesmos. Bancos de dados de documentos armazenam documentos na parte do valor do armazenamento de chave-valor; pense neles como depósitos de chave-valor, em que o valor pode ser examinado. Veja como a terminologia se compara no Oracle e no MongoDB.

Oracle	MongoDB
Instância de banco de dados	Instância MongoDB
Esquema	Banco de dados
Tabela	Coleção
Linha	Documento
Rowid	_id
Junção	DBRef

O campo `_id` é um campo especial encontrado em todos os documentos no Mongo, assim como o `ROWID` no Oracle. No MongoDB, o `_id` pode ser atribuído pelo usuário, desde que seja único.

9.1 O que é um banco de dados de documentos?

```
{ "firstname": "Martin",
   "likes": [ "Biking", "Photography" ],
   "lastcity": "Boston"
}
```

O documento anterior pode ser considerado uma linha em um SGBDR tradicional. Vamos examinar outro documento:

```
{
   "firstname": "Pramod",
   "citiesvisited": [ "Chicago", "London", "Pune", "Bangalore" ],
   "addresses": [
   { "state": "AK",
      "city": "DILLINGHAM",
      "type": "R"
   },
   { "state": "MH",
      "city": "PUNE",
      "type": "R" }
```

```
    ],
    "lastcity": "Chicago"
}
```

Examinando esses documentos, podemos ver que são semelhantes, mas têm diferenças nos nomes dos atributos. Isso é permitido em bancos de dados de documentos. O esquema dos dados pode diferir entre os documentos, mas estes ainda podem pertencer à mesma coleção – ao contrário de um SGBDR, no qual todas as linhas de uma tabela devem seguir o mesmo esquema. Representamos uma lista de cidades visitadas (`citiesvisited`) com um array. Representamos uma lista de endereços (`addresses`) com uma lista de documentos dentro do documento principal. Documentos filhos internos, como subobjetos dentro de documentos, fornecem um acesso fácil e um melhor desempenho.

Se você examinar os documentos, verá que alguns dos atributos são semelhantes, tais como `firstname` (primeiro nome) ou `city` (cidade). Ao mesmo tempo, há atributos no segundo documento que não existem no primeiro, como `addresses` (endereços), enquanto `likes` (gosta) aparece no primeiro, mas não no segundo.

Essa diferente representação dos dados não é a mesma que se utiliza em um SGBDR, em que todas as colunas devem ser definidas e, se não contiverem dados, são marcadas como vazias ou nulas (`null`). Em documentos, não há atributos vazios; se um determinando atributo não for encontrado, supomos que não estava configurado ou não era relevante para o documento. Os documentos permitem que novos atributos sejam criados sem a necessidade de definição prévia ou de alteração nos documentos existentes.

Alguns dos bancos de dados de documentos populares que temos visto são MongoDB [MongoDB], CouchDB [CouchDB], Terrastore [Terrastore], OrientDB [OrientDB], RavenDB [RavenDB] e, é claro, o bem conhecido e muitas vezes criticado Lotus Notes [Notes Storage Facility], que utiliza armazenamento de documentos.

9.2 Características

Embora existam muitos bancos de dados especializados para documentos, utilizaremos o MongoDB como um representante do conjunto de características. Tenha em mente que cada produto possui algumas características que podem não ser encontradas em outros bancos de dados de documentos.

Vamos parar um pouco para entender como o MongoDB funciona. Cada instância do MongoDB possui múltiplos bancos de dados e cada banco de dados pode ter múltiplas coleções. Quando comparamos isso com os SGBDRs, uma instância de SGBDR é igual a uma instância em MongoDB, os esquemas de SGBDRs são semelhantes aos bancos de dados MongoDB e as tabelas de SGBDRs são coleções em MongoDB. Quando armazenamos um documento, temos de escolher em qual banco de dados e em qual coleção ele ficará – por exemplo, `database.collection.insert(document)`, que, geralmente, é representado como `db.coll.insert(document)`.

9.2.1 Consistência

A consistência em bancos de dados MongoDB é configurada utilizando os conjuntos de réplicas e optando por esperar que as gravações sejam replicadas em todos os escravos ou em um determinado número de escravos. Cada gravação pode especificar o número de escravos na qual deve ser propagada antes de ser considerada bem-sucedida.

Um comando como `db.runCommand({ getlasterror : 1 , w : "majority" })` informa ao banco de dados o quão alta é a consistência que você quer. Por exemplo: se você tiver apenas um servidor e especificar `w` como `majority` (maioria), a gravação retornará imediatamente, uma vez que há apenas um nodo. Se você tiver três nodos no conjunto de réplicas e especificar `w` como `majority`, a gravação terá de se completar em pelo menos dois nodos antes de ser informada como bem-sucedida. Você pode aumentar o valor de `w` para obter uma consistência mais alta, mas terá perda no desempenho, uma vez que, agora, as gravações têm de ser concluídas em mais nodos. Conjuntos de réplicas também possibilitam que você aumente o desempenho da leitura, permitindo a leitura a partir de escravos ao configurar `slaveOk`; esse parâmetro pode ser configurado na conexão, no banco de dados, na coleção ou, individualmente, para cada operação.

```
Mongo mongo = new Mongo("localhost:27017");
mongo.slaveOk();
```

Aqui estamos configurando `slaveOk` por operação. Dessa forma, podemos decidir quais operações podem trabalhar com dados do nodo escravo.

```
DBCollection collection = getOrderCollection();
BasicDBObject query = new BasicDBObject();
query.put("name", "Martin");
DBCursor cursor = collection.find(query).slaveOk();
```

Semelhante às diversas opções disponíveis para leitura, você pode alterar as configurações para obter uma consistência alta de gravação, se desejar. Por padrão, uma gravação é informada como bem-sucedida assim que o banco de dados a receber; você pode alterar isso para que espere até as gravações serem sincronizadas no disco ou propagadas para dois ou mais escravos, o que é conhecido como `WriteConcern`: você garante que determinadas gravações sejam feitas no mestre e em alguns escravos, configurando `WriteConcern` como `REPLICAS_SAFE`. A seguir, está o código onde estamos configurando `WriteConcern` para todas as gravações em uma coleção:

```
DBCollection shopping = database.getCollection("shopping");
shopping.setWriteConcern(REPLICAS_SAFE);
```

O `WriteConcern` também pode ser configurado por operação especificando-o no comando de gravação:

```
WriteResult result = shopping.insert(order, REPLICAS_SAFE);
```

Existe um equilíbrio que precisa ser cuidadosamente pensado com base nas necessidades do seu aplicativo e em seus requisitos de negócio. Dessa forma, pode-se descobrir quais configurações fazem sentido para `slaveOk` durante a leitura ou qual nível de segurança se deseja durante a gravação com `WriteConcern`.

9.2.2 Transações

Transações, no sentido tradicional de SGBDRs, significam que você pode começar a modificar o banco de dados com os comandos `insert`, `update` e `delete` em diferentes tabelas e, depois, decidir se quer manter as mudanças ou desfazê-las, usando os comandos `commit` ou `rollback`. Essas construções, geralmente, não estão disponíveis em soluções NoSQL – uma gravação é bem-sucedida ou falha. Transações no nível de um único documento são conhecidas como transações atômicas. Transações envolvendo mais de uma operação não são possíveis, embora existam produtos como o RavenDB, que suportam transações com

múltiplas operações.

Por padrão, todas as gravações são reportadas como bem-sucedidas. Um controle mais minucioso sobre a gravação pode ser obtido utilizando-se o parâmetro `WriteConcern`. Garantimos que o pedido (`order`) seja gravado em mais de um nodo antes que seja informado como tendo sido bem-sucedido utilizando `WriteConcern.REPLICAS_SAFE`. Níveis diferentes de `WriteConcern` permitem que você escolha o nível de segurança durante as gravações: por exemplo, ao escrever entradas em log, você pode utilizar o menor nível de segurança, `WriteConcern.NONE`.

```
final Mongo mongo = new Mongo(mongoURI);
mongo.setWriteConcern(REPLICAS_SAFE);
DBCollection shopping = mongo.getDB(orderDatabase)
                             .getCollection(shoppingCollection);
try {
   WriteResult result = shopping.insert(order, REPLICAS_SAFE);
   // As gravações chegaram ao primário e pelo menos ao secundário
} catch (MongoException writeException) {
   // As gravações não chegaram a um mínimo de dois nodos, incluindo o primário
   dealWithWriteFailure(order, writeException);
}
```

9.2.3 Disponibilidade

Segundo o teorema CAP (p. 91), podemos escolher apenas duas destas opções: Consistência, Disponibilidade e Tolerância a partições. Bancos de dados de documentos tentam melhorar a disponibilidade replicando os dados, utilizando a configuração mestre-escravo. Os mesmos dados ficam disponíveis em múltiplos nodos, os quais os clientes podem acessar mesmo quando o nodo primário estiver indisponível. Geralmente, o código do aplicativo não tem de determinar se o nodo primário está disponível ou não. O MongoDB implementa a replicação, fornecendo alta disponibilidade ao utilizar os **conjuntos de réplicas**.

Em um conjunto de réplicas, há dois ou mais nodos participando de uma replicação assíncrona mestre-escravo. Os nodos do conjunto elegem, entre eles, o mestre (também chamado de primário). Supondo que todos os nodos tenham direitos iguais de votação, alguns nodos podem ser favorecidos por estarem mais próximos dos outros servidores, por terem mais RAM e assim por diante; os usuários podem alterar isso atribuindo uma prioridade – um número entre 0 e 1.000 – para um nodo.

Todas as solicitações vão para o nodo mestre e os dados são replicados nos nodos escravos. Se o nodo mestre falhar, os nodos restantes no conjunto de réplicas votam entre si para eleger um novo mestre; todas as futuras solicitações são direcionadas para o novo mestre e os nodos escravos começam a obter os dados do novo mestre. Quando o nodo que falhou ficar online novamente, ele volta a participar como um escravo e se equipara aos outros nodos ao trazer os dados de que precisa para se atualizar.

A figura 9.1 é uma configuração de exemplo de conjuntos de réplicas. Temos dois nodos, o **mongo A** e o **mongo B**, executando o banco de dados MongoDB no datacenter principal, e um nodo, o mongo C, no datacenter secundário. Se quisermos que os nodos do datacenter principal sejam eleitos como primários, podemos atribuir a eles uma prioridade maior do que a dos outros nodos. Mais nodos podem ser adicionados aos conjuntos de

réplicas sem ter de colocá-los em modo offline.

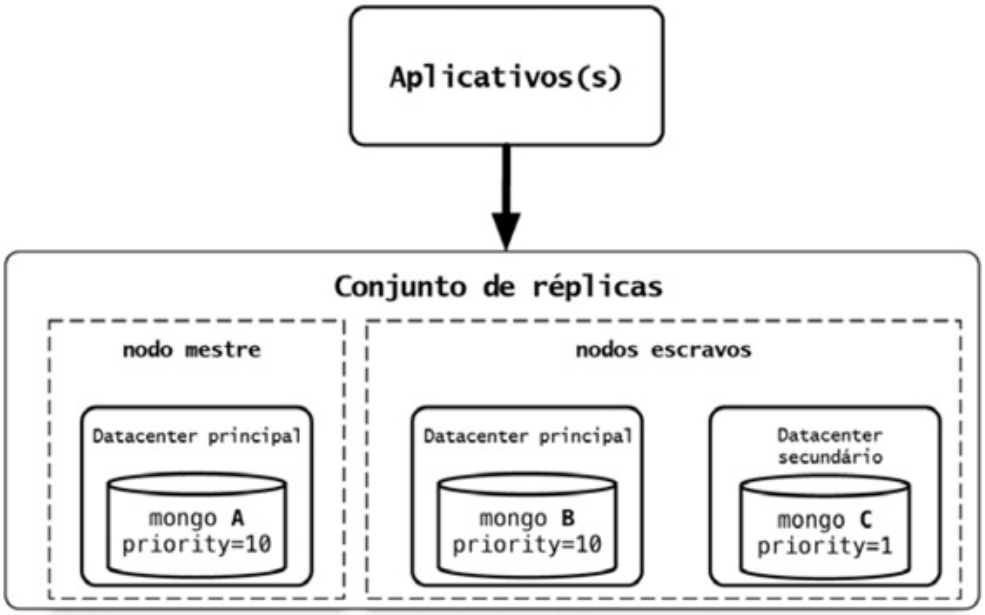

Figura 9.1 – Configuração de conjunto de réplicas com prioridade maior atribuída a nodos no mesmo datacenter.

O aplicativo grava ou lê a partir do nodo primário (mestre). Quando a conexão é estabelecida, o aplicativo somente precisa se conectar a um nodo (primário ou não, não importa) no conjunto de réplicas e o restante dos nodos são descobertos automaticamente. Quando o nodo primário falha, o driver comunica-se com o novo primário eleito pelo conjunto de réplicas. O aplicativo não tem de gerenciar falhas na comunicação ou critérios de seleção de nodos. Utilizar conjuntos de réplicas lhe confere a possibilidade de ter um armazenamento de dados de documentos altamente disponível.

Conjuntos de réplicas geralmente são utilizados para redundância de dados, recuperação automática de falhas, ampliação na capacidade de leitura, manutenção de servidor sem tirar o aplicativo do ar e recuperação após desastres. Configurações de disponibilidade semelhantes podem ser obtidas com CouchDB, RavenDB, Terrastore e outros produtos.

9.2.4 Recursos de consulta

Bancos de dados de documentos fornecem diferentes recursos de consulta. O CouchDB permite que você consulte por meio de visões – consultas complexas em documentos, as quais podem ser materializadas (“Visões materializadas”, p. 64) ou dinâmicas (pense nelas como visões em SGBDRs, que são materializadas ou não). Com o CouchDB, se você precisar agregar o número de resenhas de um produto, bem como a avaliação média, pode adicionar uma visão implementada por meio de map-reduce (“Map-reduce básico”, p. 108) para obter retorno do contador de revisões e das médias de suas avaliações.

Quando houver muitas solicitações, você não optará por calcular o contador e a média para cada uma; em vez disso, poderá adicionar uma visão materializada que calcule previamente os valores e armazene os resultados no banco de dados. Essas visões materializadas são atualizadas quando consultadas, caso algum dado tiver sido alterado desde a última atualização.

Um dos recursos interessantes dos bancos de dados de documentos, comparados com os

armazenamentos de chave-valor, é que podemos consultar os dados dentro do documento sem ter de recuperar o documento inteiro por sua chave e depois examiná-lo. Esse recurso aproxima esses bancos de dados ao modelo de consulta SGBDR.

O MongoDB possui uma linguagem de consulta que é expressa via JSON e possui estruturas como `$query` para a cláusula `where` (onde), `$orderby` para ordenar os dados, ou `$explain` para mostrar o plano de execução da consulta. Há mais estruturas como essas que podem ser combinadas para criar uma consulta em MongoDB.

Vamos examinar determinadas consultas que podemos fazer no MongoDB. Suponha que queiramos retornar todos os documentos em uma coleção de pedidos (todas as linhas da tabela de pedidos). O SQL para isso seria:

```
SELECT * FROM order
```

O equivalente no Mongo shell seria:

```
db.order.find()
```

Selecionar os pedidos de um único `customerId` igual a `883c2c5b4e5b` seria:

```
SELECT * FROM order WHERE customerId = "883c2c5b4e5b"
```

A consulta equivalente em Mongo para obter todos os pedidos de um único `customerId` igual a `883c2c5b4e5b`:

```
db.order.find({"customerId":"883c2c5b4e5b"})
```

De forma semelhante, selecionar `orderId` e `orderDate` para um cliente em SQL seria:

```
SELECT orderId,orderDate FROM order WHERE customerId = "883c2c5b4e5b"
```

e o equivalente em Mongo seria:

```
db.order.find({customerId:"883c2c5b4e5b"},{orderId:1,orderDate:1})
```

De forma semelhante, estão disponíveis consultas para contar, somar e assim por diante. Uma vez que os documentos são objetos agregados, é muito fácil consultar documentos que tenham de ser comparados utilizando os campos com objetos filhos. Digamos que queiramos consultar todos os pedidos em que um dos itens comprados tenha o termo "`Refactoring`" em seu nome. O SQL para essa requisição seria:

```
SELECT * FROM customerOrder, orderItem, product
WHERE
customerOrder.orderId = orderItem.customerOrderId
AND orderItem.productId = product.productId
AND product.name LIKE '%Refactoring%'
```

e o equivalente em Mongo seria:

```
db.orders.find({"items.product.name":/Refactoring/})
```

A consulta para MongoDB é mais simples, pois os objetos estão inseridos em um único documento e você pode consultar baseando-se nos documentos filhos internos.

9.2.5 Escalabilidade

A ideia de escalar consiste em adicionar nodos ou alterar o armazenamento de dados sem simplesmente migrar o banco de dados para uma máquina mais potente. Não estamos falando em fazer alterações no aplicativo para lidar com mais carga; em vez disso, estamos interessados em quais características o banco de dados possui, de modo que ele possa lidar com mais carga.

A escalabilidade para lidar com grandes quantidades de leitura pode ser alcançada adicionando-se mais escravos de leitura, de forma que todas as leituras possam ser direcionadas para os escravos. Dado um aplicativo com muita leitura, com o nosso cluster de conjunto de réplicas de três nodos, podemos adicionar mais capacidade de leitura ao cluster à medida que a carga de leitura aumenta. Isso tudo simplesmente acrescentando mais nodos escravos ao conjunto de réplicas para executar leituras com a flag `slaveOk` (Figura 9.2). Essa é a escalabilidade horizontal para leituras.

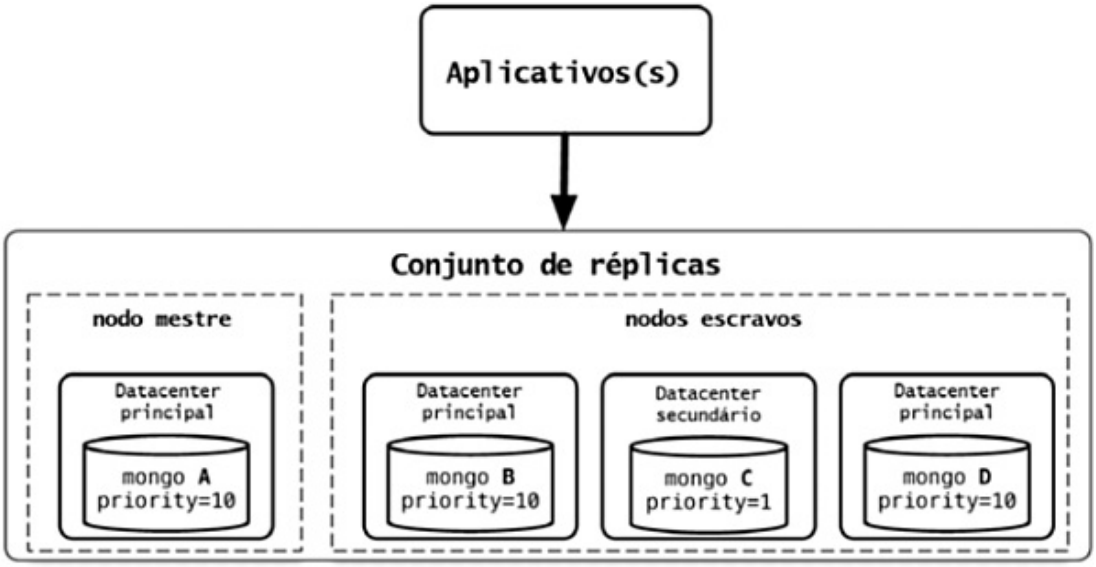

Figura 9.2 – Adicionando um novo nodo, mongo D, a um cluster de conjunto de réplicas existente.

Assim que o novo nodo, mongo D, for iniciado, precisa ser adicionado ao conjunto de réplicas.

```
rs.add("mongod:27017");
```

Quando um nodo novo é adicionado, ele entrará em sincronia com os existentes, tornando-se parte do conjunto de réplicas como nodo secundário, e começará a servir às solicitações de leitura. Uma vantagem dessa configuração é que não temos de reiniciar qualquer outro nodo e o aplicativo não fica tempo algum fora do ar.

Quando queremos escalar para a gravação, podemos iniciar a fragmentação dos dados (“Fragmentação”, p. 74). A fragmentação é semelhante a partições em SGBDRs, em que dividimos os dados por valor em uma determinada coluna, como, por exemplo, estado ou ano. Com SGBDRs, as partições geralmente ficam no mesmo nodo, de forma que o aplicativo cliente não tenha de consultar uma partição específica, mas, sim, continuar consultando a tabela base; o SGBDR fica responsável por encontrar a partição correta para a consulta e devolver os dados.

Na fragmentação, os dados também são divididos por determinado campo, mas nesse caso, são movidos para nodos diferentes no Mongo. Os dados são movidos dinamicamente entre os nodos para assegurar-se de que os fragmentos sempre estejam balanceados. Podemos adicionar mais nodos ao cluster e aumentar o número de nodos graváveis, permitindo a escalabilidade horizontal das gravações.

```
db.runCommand( { shardcollection : "ecommerce.customer",
                 key : {firstname : 1} } )
```

Dividir os dados no primeiro nome do cliente garante que eles fiquem balanceados pelos fragmentos para um desempenho de escrita ideal; além disso, cada fragmento pode ser um conjunto de réplicas, garantindo melhor desempenho de leitura dentro do fragmento (Figura 9.3). Quando adicionamos um novo fragmento a esse cluster fragmentado já existente, os dados serão balanceados entre quatro fragmentos, em vez de entre os três. Enquanto esses dados se movem e a infraestrutura é refatorada, o aplicativo não ficará indisponível, embora o cluster possa não ter desempenho ideal quando grandes quantidades de dados estiverem sendo movimentados para rebalancear os fragmentos.

A chave de fragmentação tem um papel importante. Você pode querer colocar os fragmentos do seu banco de dados MongoDB mais próximo aos usuários, de forma que a fragmentação baseada na localização do usuário pode ser uma boa ideia. Ao fragmentar pela localização do cliente, todos os dados dos usuários da Costa Leste dos EUA ficam nos fragmentos que estão em datacenters localizados na Costa Leste e todos os dados dos usuários da Costa Oeste ficam nos fragmentos que estão localizados na Costa Oeste.

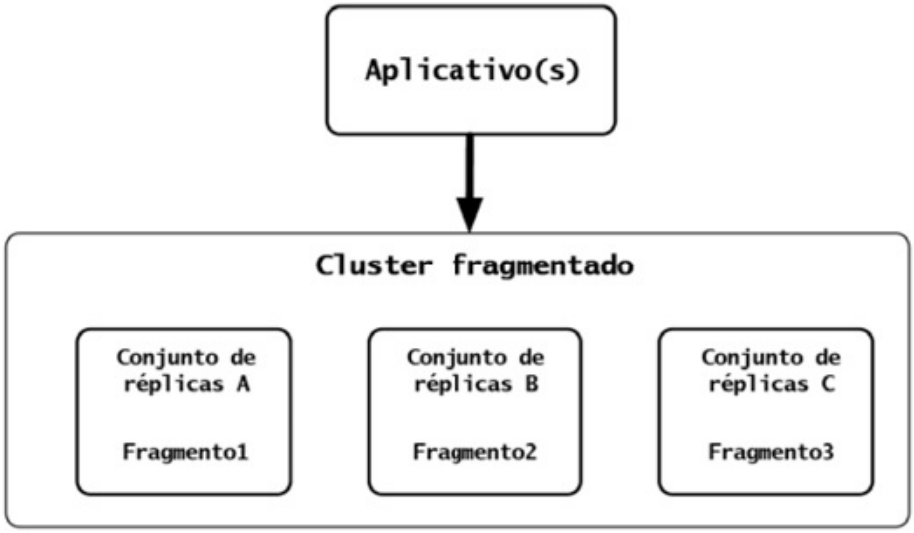

Figura 9.3 – Configuração fragmentada do MongoDB, em que cada fragmento é um conjunto de réplicas.

9.3 Casos de uso apropriados

9.3.1 Registro de eventos (log)

Os aplicativos têm diferentes necessidades de registro de eventos; dentro de empresas, há muitos aplicativos diferentes que querem registrar eventos. Bancos de dados de documentos podem armazenar todos esses diferentes tipos de eventos e atuar como um depósito central de dados para o registro desses eventos. Isso acontece, principalmente, quando o tipo de dados capturados pelos eventos estiver em constante mudança. Os eventos podem ser divididos pelo nome do aplicativo no qual o evento originou-se ou pelo tipo do evento, como `order_processed` ou `customer_logged` (`pedido_processado` ou `cliente_logado`).

9.3.2 Sistema de Gerenciamento de Conteúdo (CMS), plataformas de blog

Uma vez que os bancos de dados de documentos não têm esquemas predefinidos e

geralmente entendem documentos JSON, eles funcionam bem em sistemas de gerenciamento de conteúdo ou aplicativos para publicação de websites, gerenciando os comentários dos usuários, seus registros, perfis e documentos visualizados pela web.

9.3.3 Análises web ou em tempo real (analytics)

Bancos de dados de documentos podem armazenar dados para análises em tempo real; uma vez que partes do documento podem ser atualizadas, é muito fácil armazenar visualizações de páginas ou visitantes únicos. Além disso, novas métricas podem ser adicionadas com facilidade, sem alterações no esquema.

9.3.4 Aplicativos de comércio eletrônico

Aplicativos de comércio eletrônico, muitas vezes, precisam ter esquemas flexíveis para produtos e pedidos, assim como precisam ter a capacidade de alterar seus modelos de dados sem refatorações custosas de bancos de dados ou migração de dados ("Alterações no esquema em um armazenamento de dados NoSQL", p. 183).

9.4 Quando não utilizar

Existem situações nas quais os bancos de dados de documentos não são a melhor opção.

9.4.1 Transações complexas que abranjam diferentes operações

Se você precisar de operações atômicas em múltiplos documentos, os bancos de dados de documentos podem não ser o ideal. Entretanto, há alguns bancos de dados de documentos que suportam esses tipos de operações, como o RavenDB.

9.4.2 Consultas em estruturas agregadas variáveis

Um esquema flexível significa que o banco de dados não lhe impõe restrições. Os dados são gravados na forma de entidades do aplicativo. Se você precisa consultar essas entidades no local (ad hoc), suas consultas mudarão (em termos de SGBDRs, isso significa que, quando se adiciona critérios entre tabelas, elas, ao serem agrupadas, continuam mudando). Uma vez que os dados são gravados na forma de agregados, se o formato do agregado é constantemente alterado, você precisa gravá-los no nível mais baixo de granularidade – basicamente, você precisa normalizar os dados. Nesse cenário, os bancos de dados de documentos podem não funcionar.

CAPÍTULO 10

Armazenamento em famílias de colunas

Armazenamentos em famílias de colunas, como fazem o Cassandra [Cassandra], HBase [Hbase], Hypertable [Hypertable] e Amazon SimpleDB [Amazon SimpleDB], permitem que você armazene dados com chaves mapeadas para valores, e os valores são agrupados em múltiplas famílias de colunas, cada uma dessas famílias de colunas funcionando como um mapa de dados.

SGBDR	Cassandra
Instância de bancos de dados	Cluster
Banco de dados	Keyspace
Tabela	Família de colunas
Linha	Linha
Coluna (a mesma para todas as linhas)	Coluna (podem ser diferentes por linha)

10.1 O que é um depósito de dados de família de colunas?

Há muitos bancos de dados de família de colunas. Neste capítulo, falaremos sobre o Cassandra, mas também referenciaremos outros, a fim de discutir características que podem ser interessantes em cenários específicos.

Bancos de dados de família de colunas armazenam dados em famílias de colunas como linhas que tenham muitas colunas associadas, fazendo uso de uma chave de linha (Figura 10.1). Famílias de colunas são grupos de dados relacionados que, frequentemente, são acessados juntos. Por exemplo, muitas vezes acessamos as informações de perfil de um cliente ao mesmo tempo, mas não seus pedidos.

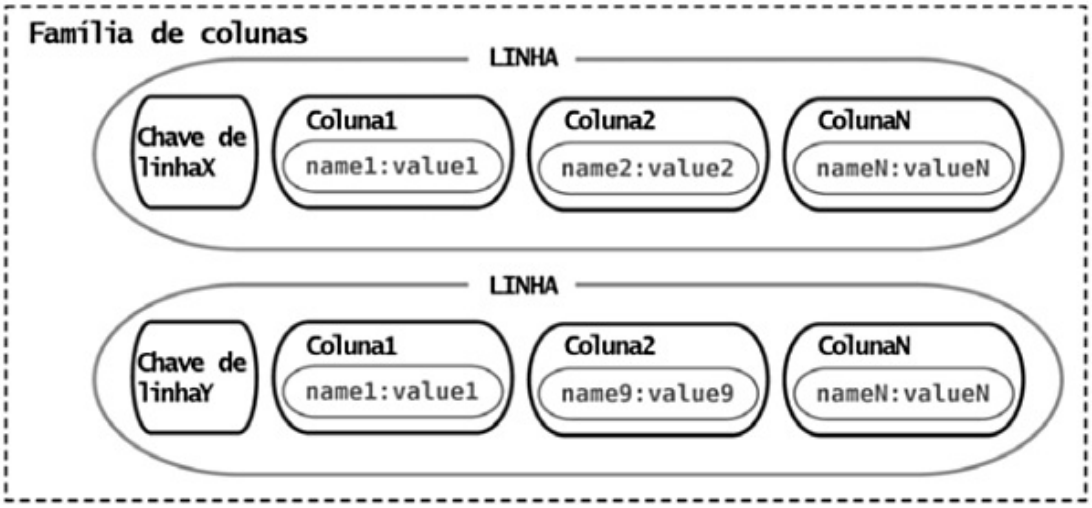

Figura 10.1 – Modelo de dados do Cassandra com famílias de colunas.

O Cassandra é um dos bancos de dados de família de colunas mais popular, embora

existam outros, como HBase, Hypertable e Amazon DynamoDB [Amazon DynamoDB]. O Cassandra pode ser descrito como rápido e de fácil crescimento escalar, com operações de gravação distribuídas pelo cluster, que não possui um nodo mestre, de forma que tanto leitura quanto gravação podem ser feitas por qualquer um de seus nodos.

10.2 Características

Começaremos examinando a maneira como os dados são estruturados no Cassandra. A unidade básica de armazenamento do Cassandra é uma coluna. Uma coluna no Cassandra consiste em um par de nome-valor em que o nome também se comporta como a chave. Cada um desses pares de chave-valor é uma única coluna e é sempre armazenada com um valor de timestamp. O timestamp é utilizado para expirar os dados, resolver conflitos de gravação, lidar com dados desatualizados e outras ações. Assim que os dados da coluna não tiverem mais uso, o espaço pode ser recuperado, posteriormente, durante a fase de compactação.

```
{
   name: "firstName",
   value: "Martin",
   timestamp: 12345667890
}
```

A coluna possui uma chave `firstName` e o valor `Martin`, além de possuir um timestamp associado. Uma linha é uma coleção de colunas anexadas ou associadas a uma chave; uma coleção de linhas semelhantes constitui uma família de colunas. Quando as colunas de uma família de colunas são simples, ela é conhecida como **família de colunas padrão**.

```
// família de colunas
{
   // linha
   "pramod-sadalage" : {
      firstName: "Pramod",
      lastName: "Sadalage",
      lastVisit: "2012/12/12"
   }
   // linha
   "martin-fowler" : {
      firstName: "Martin",
      lastName: "Fowler",
      location: "Boston"
   }
}
```

Cada família de colunas pode ser comparada a um contêiner de linhas em uma tabela de SGBDR, onde a chave identifica a linha, que é constituída de múltiplas colunas. A diferença é que as linhas não têm de ter as mesmas colunas, e novas colunas podem ser adicionadas a qualquer linha, a qualquer momento, sem a obrigatoriedade de ter de adicioná-las às outras linhas também. Temos a linha `pramod-sadalage` e a linha `martin-fowler`, com colunas diferentes; ambas são parte da mesma família de colunas.

Quando uma coluna corresponde a um mapa de colunas, então temos uma **supercoluna,** que consiste em um nome e um valor, sendo que o valor é um mapa de colunas. Pense em uma supercoluna como um contêiner de colunas.

```
{
   name: "book:978-0767905923",
   value: {
      author: "Mitch Albon",
      title: "Tuesdays with Morrie",
      isbn: "978-0767905923"
   }
}
```

Quando utilizamos supercolunas para criar uma **família de colunas**, obtemos uma família de supercolunas.

```
// família de supercolunas
{
// linha
name: "billing:martin-fowler",
value: {
   address: {
      name: "address:default",
      value: {
         fullName: "Martin Fowler",
         street:"100 N. Main Street",
         zip: "20145"
      }
   },
   billing: {
      name: "billing:default",
      value: {
         creditcard: "8888-8888-8888-8888",
         expDate: "12/2016"
      }
   }
}
// linha
name: "billing:pramod-sadalage",
value: {
   address: {
      name: "address:default",
      value: {
         fullName: "Pramod Sadalage",
         street:"100 E. State Parkway",
         zip: "54130"
      }
   },
   billing: {
      name: "billing:default",
      value: {
         creditcard: "9999-8888-7777-4444",
         expDate: "01/2016"
         }
      }
   }
}
```

Famílias de supercolunas são eficazes para manter dados relacionados agrupados. Porém, mesmo quando algumas das colunas não forem necessárias durante a maior parte do tempo, estas colunas ainda assim são lidas e desserializadas pelo Cassandra – o que pode não ser o ideal.

O Cassandra coloca as famílias de colunas padrão e supercolunas em **keyspaces**. Um keyspace é semelhante a um banco de dados em SGBDRs, em que todas as famílias de colunas relacionadas ao aplicativo ficam armazenadas. Keyspaces têm de ser criados de modo que as famílias de colunas possam ser atribuídas a eles:

```
create keyspace ecommerce
```

10.2.1 Consistência

Quando uma gravação é recebida pelo Cassandra, os dados são gravados primeiro em um commit log (registro de operações), depois em uma estrutura na memória conhecida como **memtable**. Uma operação de gravação é considerada bem-sucedida quando for gravada no commit log e na memtable. As gravações são colocadas em lotes na memória e gravadas periodicamente em estruturas conhecidas como **SSTable**. As SSTables não são gravadas novamente após terem sido carregadas; se houver alterações nos dados, uma nova SSTable é gravada. SSTables não utilizadas são recuperadas pela **compactação**.

Vamos examinar a operação de leitura para ver como as configurações de consistência as afetam. Se tivermos uma configuração de consistência igual a `ONE` (um) como o padrão para todas as operações de leitura, então, quando for feita uma solicitação de leitura, o Cassandra retorna os dados da primeira réplica, mesmo se os dados forem antigos. Se esse for o caso, leituras subsequentes obterão os dados mais recentes; esse processo é conhecido como **reparação de leitura**. O nível baixo de consistência é bom para ser utilizado quando você não se importar se obtiver dados antigos e/ou se tiver requisitos de desempenho de leitura altos.

De forma semelhante, se você estiver realizando gravações, o Cassandra grava no commit log de um nodo e retorna uma resposta para o cliente. A consistência igual a `ONE` é eficaz se você tiver requisitos de desempenho muito altos e, além disso, não se importar se algumas das gravações forem perdidas – o que pode acontecer se o nodo ficar indisponível antes de a gravação ser replicada nos outros nodos.

```
quorum = new ConfigurableConsistencyLevel();
quorum.setDefaultReadConsistencyLevel(HConsistencyLevel.QUORUM);
quorum.setDefaultWriteConsistencyLevel(HConsistencyLevel.QUORUM);
```

Utilizar a configuração de consistência `QUORUM` para operações de leitura e de gravação assegura que a maioria dos nodos responderá à leitura, e a coluna com o timestamp mais recente será retornada para o cliente, enquanto as réplicas que não possuírem os dados mais recentes são reparadas via operações de reparo de leitura. Durante as operações de gravação, a configuração de consistência `QUORUM` significa que a gravação tem de ser propagada para a maioria dos nodos antes de ser considerada bem-sucedida e antes de o cliente ser notificado.

Utilizar um nível de consistência `ALL` significa que todos os nodos terão de responder a leituras ou gravações, o que tornará o cluster intolerante a falhas – mesmo quando um único nodo estiver indisponível, a gravação ou a leitura são bloqueadas e informadas como falha. Depende, portanto, dos projetistas do sistema ajustarem os níveis de consistência na

medida em que os requisitos do aplicativo mudarem. Dentro do mesmo aplicativo pode haver diferentes necessidades de consistência; eles também podem mudar baseados em cada operação. Por exemplo, há diferentes necessidades de consistência entre mostrar as resenhas de um produto e fazer a leitura do status do último pedido feito pelo cliente.

Durante a criação do **keyspace**, podemos configurar o número de réplicas dos dados que precisamos armazenar. Esse número determina o fator de replicação dos dados. Se você tiver um fator de replicação igual a três, os dados serão copiados para três nodos. Ao gravar e ler dados com o Cassandra, se você especificar o valor de consistência igual a 2, obterá esse `R + W` maior do que o fator de replicação (`2 + 2 > 3`), o que dá melhor consistência durante as gravações e leituras.

Podemos executar o comando de reparo do nodo para o keyspace e fazer o Cassandra comparar cada chave pela qual é responsável com o resto das réplicas. Como essa operação é custosa, também podemos reparar apenas uma família de colunas específica ou uma lista de famílias de colunas:

```
repair ecommerce
repair ecommerce customerInfo
```

Enquanto um nodo estiver indisponível, os dados que devem ser armazenados por ele são passados para outros nodos. Quando o nodo ficar online novamente, as alterações feitas nos dados são retornadas para ele. Essa técnica é conhecida como **hinted handoff**. Ela permite uma recuperação mais rápida de nodos que tenham tido problemas.

10.2.2 Transações

O Cassandra não tem transações no sentido tradicional – onde poderíamos iniciar múltiplas gravações e depois decidir se confirmaríamos as alterações ou não. No Cassandra, uma gravação é atômica em relação a linha, o que significa que a inserção ou atualização em colunas para uma determinada chave de linha será tratada como uma única gravação, podendo ser bem-sucedida ou falha. As gravações são gravadas primeiramente no commit log e em memtables e somente são consideradas válidas quando tanto a gravação no commit log quanto a gravação na memtable tiverem sido bem-sucedidas. Se um nodo ficar indisponível, o commit log é utilizado para aplicar as alterações no nodo, da mesma forma que o **redo log** no Oracle.

Você pode utilizar bibliotecas externas de transações, como a ZooKeeper [ZooKeeper], para sincronizar suas gravações e leituras. Também há bibliotecas, como a Cages [Cages], que permitem que você envolva suas transações no ZooKeeper.

10.2.3 Disponibilidade

O Cassandra é, desde sua concepção original, altamente disponível, uma vez que não há um mestre no cluster e todos os nodos têm, neste, o mesmo status. A disponibilidade de um cluster pode ser aumentada ao se reduzir o nível de consistência das solicitações. A disponibilidade é coordenada pela fórmula `(R + W) > N` (“Quóruns”, p. 97), em que `W` é o número mínimo de nodos nos quais a gravação deve ser realizada com sucesso, `R` é o número mínimo de nodos que devem responder com sucesso a uma leitura e `N` é o número de nodos que participam da replicação dos dados. Você pode ajustar a disponibilidade alterando os valores de `R` e `W` para um valor fixo de `N`.

Em um cluster de 10 nodos do Cassandra, com um fator de replicação para o keyspace

configurado como 3 (`N = 3`), se configurarmos `R = 2` e `W = 2`, então teremos `(2 + 2) > 3`. Neste cenário, quando um nodo fica indisponível, a disponibilidade não é muito afetada, uma vez que os dados podem ser recuperados a partir dos outros dois nodos. Se `W = 2` e `R = 1`, quando dois nodos estiverem indisponíveis, o cluster não estará disponível para gravação, mas ainda poderá ser lido. De modo semelhante, se `R = 2` e `W = 1`, podemos gravar, mas o cluster não estará disponível para leitura. Com a equação `R + W > N`, você tomará decisões conscientes sobre o balanceamento da consistência.

Você deve configurar seus keyspaces e operações de gravação/leitura baseando-se em suas necessidades – disponibilidade maior para gravação ou disponibilidade maior para leitura.

10.2.4 Recursos de consulta

Ao projetar o modelo de dados no Cassandra, é aconselhado que sejam otimizadas as colunas e as famílias de colunas para a leitura dos dados, uma vez que ele não tem uma linguagem de consulta muito poderosa; à medida que dados são inseridos nas famílias de colunas, os dados de cada linha são ordenados por nomes de coluna. Se tivermos uma coluna que é recuperada com muito mais frequência do que as outras, é melhor para o desempenho que utilizemos esse valor para a chave da linha.

10.2.4.1 Consultas básicas

Consultas básicas que possam ser executadas utilizando um cliente do Cassandra incluem `GET`, `SET` e `DEL`. Antes de iniciar a consulta de dados, temos de executar o comando `use ecommerce;` para assegurar que todas as nossas consultas sejam executadas no keyspace `ecommerce`, onde colocamos nossos dados. Antes de iniciar o uso de família de colunas no keyspace, temos de definir essa família.

```
CREATE COLUMN FAMILY Customer
WITH comparator = UTF8Type
AND key_validation_class=UTF8Type
AND column_metadata = [
   {column_name: city, validation_class: UTF8Type}
   {column_name: name, validation_class: UTF8Type}
   {column_name: web, validation_class: UTF8Type}
];
```

Temos uma família de colunas chamada `Customer` (cliente), com as colunas `name` (nome), `city` (cidade) e `web`, e estamos inserindo dados na família de colunas com um cliente do Cassandra.

```
SET Customer['mfowler']['city']='Boston';
SET Customer['mfowler']['name']='Martin Fowler';
SET Customer['mfowler']['web']='www.martinfowler.com';
```

Utilizando o cliente Hector [Hector], escrito em Java, podemos inserir os mesmos dados na família de colunas.

```
ColumnFamilyTemplate<String, String> template =
        cassandra.getColumnFamilyTemplate();
ColumnFamilyUpdater<String, String> updater =
        template.createUpdater(key);
for (String name : values.keySet()) {
```

```
    updater.setString(name, values.get(name));
}
try {
    template.update(updater);
} catch (HectorException e) {
    handleException(e);
}
```

Podemos ler os dados de volta utilizando o comando `GET`. Há múltiplas maneiras para obter-se os dados; podemos obter a família de colunas inteira.

```
GET Customer['mfowler'];
```

Podemos, até, obter apenas a coluna na qual estivermos interessados na família de colunas.

```
GET Customer['mfowler']['web'];
```

Obter a coluna específica de que precisamos é mais eficiente, já que apenas os dados que nos interessam são retornados – o que economiza muita movimentação de dados, especialmente quando a família de colunas possui um grande número de colunas. Para atualizar os dados, utilize o comando `SET` na coluna que precisar receber o novo valor. Utilizando o comando `DEL`, podemos excluir uma coluna ou a família inteira de colunas.

```
DEL Customer['mfowler']['city'];
DEL Customer['mfowler'];
```

10.2.4.2 Consultas avançadas e indexação

O Cassandra permite que você indexe outras colunas além das chaves da família de colunas. Podemos definir um índice para a coluna `city`.

```
UPDATE COLUMN FAMILY Customer
WITH comparator = UTF8Type
AND column_metadata = [{column_name: city,
                        validation_class: UTF8Type,
                        index_type: KEYS}];
```

Agora podemos consultar diretamente a coluna indexada.

```
GET Customer WHERE city = 'Boston';
```

Esses índices são implementados na forma de índices bit-mapped (mapeados em bits) e têm bom desempenho com valores em colunas de baixa cardinalidade.

10.2.4.3 A linguagem de consulta do Cassandra (CQL – Cassandra Query Language)

O Cassandra possui uma linguagem de consulta que suporta comandos do tipo SQL, conhecida como CQL (do nome original em inglês Cassandra Query Language). Podemos utilizar os comandos CQL para criar uma família de colunas.

```
CREATE COLUMNFAMILY Customer (
    KEY varchar PRIMARY KEY,
    name varchar,
    city varchar,
    web varchar);
```

Inserimos os mesmos dados utilizando o CQL.

```
INSERT INTO Customer (KEY,name,city,web)
    VALUES ('mfowler',
```

```
            'Martin Fowler',
            'Boston',
            'www.martinfowler.com');
```

Podemos ler dados utilizando o comando `SELECT`. Aqui lemos todas as colunas:

```
SELECT * FROM Customer
```

Ou podemos selecionar apenas as colunas de que precisamos.

```
SELECT name,web FROM Customer
```

Colunas indexadas são criadas a partir do comando `CREATE INDEX` e depois podem ser utilizadas para consultar os dados.

```
SELECT name,web FROM Customer WHERE city='Boston'
```

O CQL tem muito mais recursos para consultar dados, mas não tem todos os recursos que o SQL possui. O CQL não permite junções ou subconsultas e suas cláusulas `WHERE` geralmente são simples.

10.2.5 Escalabilidade

Para fazer um cluster existente crescer em escala no Cassandra, basta adicionar mais nodos. Como nenhum nodo é mestre, quando adicionamos nodos ao cluster, estamos melhorando sua capacidade de suportar mais gravações e leituras. Esse tipo de escalabilidade horizontal permite que você obtenha o máximo de uptime, uma vez que o cluster continua a servir as solicitações dos clientes enquanto novos nodos são adicionados a ele.

10.3 Casos de uso apropriados

Discutiremos alguns dos problemas para os quais os bancos de dados de família de colunas são apropriados.

10.3.1 Registro de eventos (log)

Bancos de dados de família de colunas, com sua capacidade de armazenar quaisquer estruturas de dados, são uma ótima escolha para armazenar informações sobre eventos, como, por exemplo, os estados do aplicativo ou os erros encontrados por ele. Dentro de uma empresa, todos os aplicativos podem gravar seus eventos no Cassandra com suas próprias colunas e a rowkey (chave de linha) na forma `nomeaplicativo:timestamp`. Como é possível escalar as gravações, o Cassandra funcionaria de maneira ideal para um sistema de registro de eventos (Figura 10.2).

Figura 10.2 – Registro de eventos com o Cassandra.

10.3.2 Sistemas de gerenciamento de conteúdo (CMS), plataformas de blog

Utilizando famílias de colunas, você pode armazenar entradas de blogs com tags, categorias, links e trackbacks em diferentes colunas. Comentários podem ser armazenados

na mesma linha ou movidos para um keyspace diferente; de forma semelhante, os usuários dos blogs e os próprios blogs podem ser colocados em diferentes famílias de colunas.

10.3.3 Contadores

Muitas vezes, em aplicativos web, você precisa contar e categorizar visitantes de uma página para fazer análises. Você pode utilizar `CounterColumnType` durante a criação de uma família de colunas.

```
CREATE COLUMN FAMILY visit_counter
WITH default_validation_class=CounterColumnType
AND key_validation_class=UTF8Type AND comparator=UTF8Type;
```

Uma vez que uma família de colunas é criada, você pode ter colunas arbitrárias para cada página visitada e por cada usuário no aplicativo web.

```
INCR visit_counter['mfowler'][home] BY 1;
INCR visit_counter['mfowler'][products] BY 1;
INCR visit_counter['mfowler'][contactus] BY 1;
```

Incrementando contadores utilizando CQL:

```
UPDATE visit_counter SET home = home + 1 WHERE KEY='mfowler'
```

10.3.4 Expirando o uso

Você pode fornecer acesso demonstrativo aos usuários ou talvez queira mostrar banners de propaganda em um website durante um tempo específico utilizando colunas que expirem: o Cassandra permite que você tenha colunas que, após um determinado período de tempo, são excluídas automaticamente. Esse período é conhecido como TTL (do termo original em inglês Time To Live – tempo para viver) e é definido em segundos. A coluna é excluída após o fim do tempo TTL; quando a coluna não existir, o acesso pode ser revogado ou o banner pode ser removido.

```
SET Customer['mfowler']['demo_access'] = 'allowed' WITH ttl=2592000;
```

10.4 Quando não utilizar

Há problemas para os quais os bancos de dados de famílias de colunas não são a melhor opção. Um exemplo são os sistemas que requerem transações ACID para gravações e leituras. Se você precisa que o banco de dados agregue os dados utilizando consultas (como `SUM` ou `AVG`), terá de fazer isso no lado cliente, utilizando dados recuperados por ele a partir de todas as linhas.

O Cassandra não é apropriado para os primeiros protótipos ou para adoções iniciais de tecnologia: durante as primeiras etapas, não temos certeza sobre como os padrões de consulta poderão mudar e, quando mudarem, teremos de alterar o formato das famílias de colunas. Isso gera atrito na equipe de inovação de produtos e atrasa a produtividade do desenvolvedor. SGBDRs impõem um alto custo na mudança de esquemas, o que é equilibrado por um custo baixo nas mudanças em consultas; no Cassandra, o custo pode ser maior na mudança em consultas do que na mudança de esquema.

CAPÍTULO 11

Bancos de dados de grafos

Bancos de dados de grafos permitem que você armazene entidades e também relacionamentos entre essas entidades. Entidades também são conhecidas como nodos, os quais possuem propriedades. Pense em um nodo como uma instância de um objeto do aplicativo. Os relacionamentos são conhecidos como arestas que podem ter propriedades. As arestas têm significância direcional; nodos são organizados por relacionamentos, os quais permitem que você encontre padrões interessantes entre eles. A organização do grafo permite que os dados sejam armazenados uma vez e depois interpretados de formas diferentes baseadas em relacionamentos.

11.1 O que é um banco de dados de grafos?

No grafo de exemplo da figura 11.1, vemos vários nodos relacionados entre si. Nodos são entidades que têm propriedades, como, por exemplo, nomes. O nodo chamado Martin, na verdade, é um **nodo** com a **propriedade** `name` configurada como `Martin`.

Também vemos que as arestas têm tipos, como `likes` (curte) e `author` (autor), entre outros. Essas propriedades permitem-nos organizar os nodos; por exemplo, os nodos `Martin` e `Pramod` têm uma aresta que os conecta com um relacionamento do tipo `friend` (amigo). As arestas podem ter múltiplas propriedades. Podemos atribuir uma propriedade `since` (desde) no tipo de relacionamento `friend`, entre `Martin` e `Pramod`. Tipos de relacionamentos têm significância direcional; o tipo de relacionamento `friend` é bidirecional, mas o `likes`, não. Quando a `Dawn` curte o livro `NoSQL Essencial`, isso não significa automaticamente que o livro curta a `Dawn`.

Assim que tivermos criado um grafo desses nodos e arestas, podemos consultar o grafo de muitas formas, tal como "obtenha todos os nodos que são funcionários da `BigCo` e curtiram o `NoSQL Essencial`". Uma consulta no grafo também é conhecida como **travessia** do grafo. Uma vantagem dos bancos de dados de grafos é que podemos alterar os requisitos de travessia sem ter de alterar os nodos ou arestas. Se quisermos "obter todos os nodos que curtiram o `NoSQL Essencial`", podemos fazê-lo sem ter de alterar os dados existentes ou o modelo do banco de dados, porque podemos percorrer o grafo da forma que quisermos.

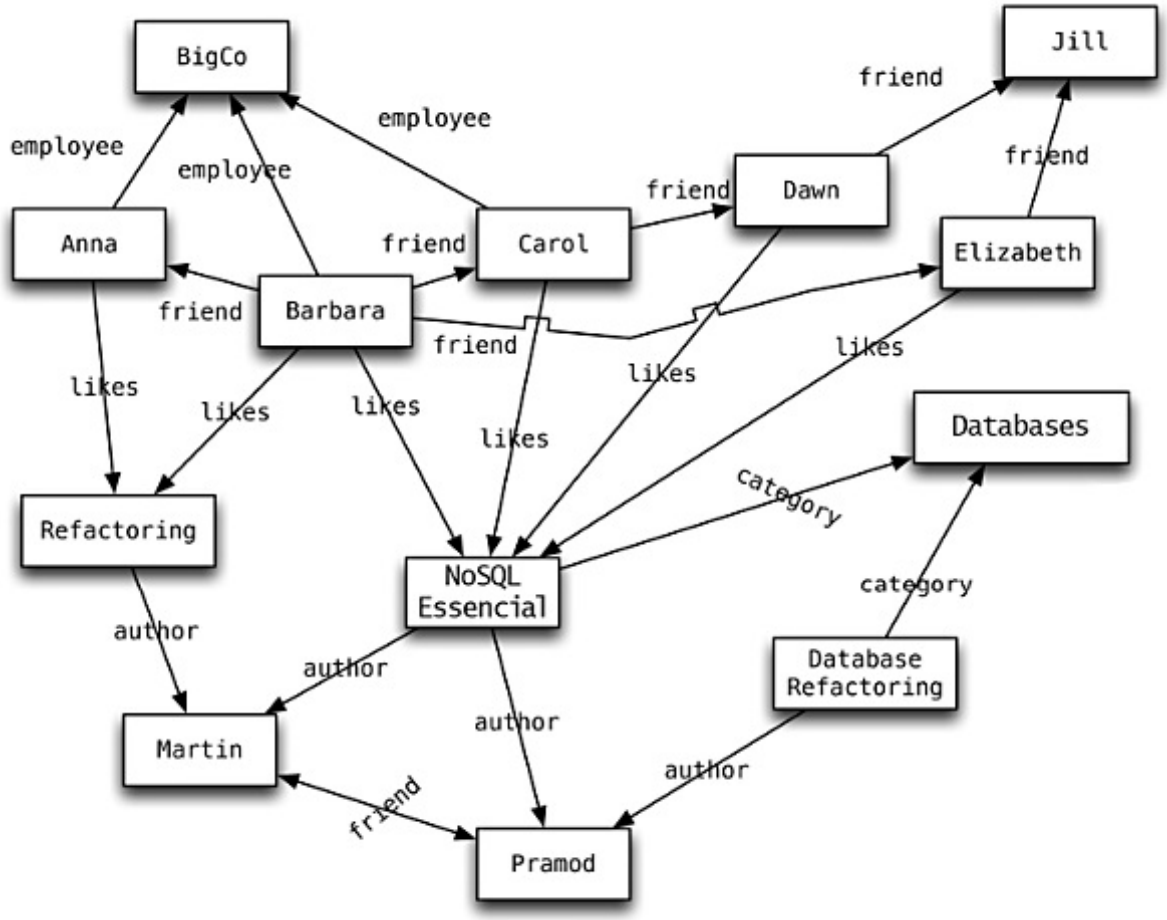

Figura 11.1 – Um exemplo de estrutura de grafo.

Geralmente, quando armazenamos uma estrutura como um grafo em um SGBDR, armazenamos para um único tipo de relacionamento ("quem é o meu gerente", é um exemplo comum). Adicionar outro relacionamento, geralmente, implica muitas alterações no esquema e uma movimentação de dados, o que não é o caso quando estamos utilizando bancos de dados de grafos. De forma semelhante, em bancos de dados relacionais, modelamos o grafo antecipadamente, com base na travessia que quisermos; se ela mudar, os dados terão de mudar também.

Em bancos de dados de grafos, percorrer as junções ou relacionamentos é muito rápido. O relacionamento entre nodos não é calculado no momento da consulta, mas, sim, persistido na forma de um relacionamento. Percorrer relacionamentos persistidos é mais rápido do que calculá-los a cada consulta.

Os nodos podem ter tipos diferentes de relacionamentos entre si, permitindo que você represente relacionamentos entre as entidades do domínio e tenha relacionamentos secundários para categorias, caminhos, árvores cronológicas, *quadtrees* para indexação espacial ou listas encadeadas para acesso ordenado. Uma vez que não há limite para o número e para o tipo de relacionamento que um nodo pode ter, todos podem ser representados no mesmo banco de dados de grafos.

11.2 Recursos

Há muitos bancos de dados de grafos disponíveis, tais como o Neo4J [Neo4J], o Infinite

Graph [Infinite Graph], o OrientDB [OrientDB] ou o FlockDB [FlockDB] (que é um caso especial: um banco de dados de grafos que suporta apenas relacionamentos em uma única profundidade ou listas de adjacência, em que você não pode percorrer mais de um nível de profundidade para relacionamentos). Pegaremos o Neo4J como representante das soluções para banco de dados de grafos, para discutir como elas funcionam e como podem ser utilizadas para resolver problemas de aplicativos.

Com o Neo4J, criar um grafo é simples: basta criar dois nos nodos e depois um relacionamento. Criaremos dois nodos, `Martin` e `Pramod`:

```
Node martin = graphDb.createNode();
martin.setProperty("name", "Martin");

Node pramod = graphDb.createNode();
pramod.setProperty("name", "Pramod");
```

Atribuímos à propriedade `name` dos dois nodos os valores `Martin` e `Pramod`. Assim que tivermos mais de um nodo, poderemos criar um relacionamento:

```
martin.createRelationshipTo(pramod, FRIEND);
pramod.createRelationshipTo(martin, FRIEND);
```

Temos de criar relacionamentos entre os nodos em ambas as direções, pois a direção do relacionamento é importante: por exemplo, um nodo de produto pode ser curtido por um usuário, mas o produto não pode curtir o usuário. Essa direcionalidade ajuda no projeto de um modelo de domínio rico (Figura 11.2). Os nodos conhecem os relacionamentos de entrada (`INCOMING`) e de saída (`OUTGOING`) que podem ser percorridos em ambas as direções.

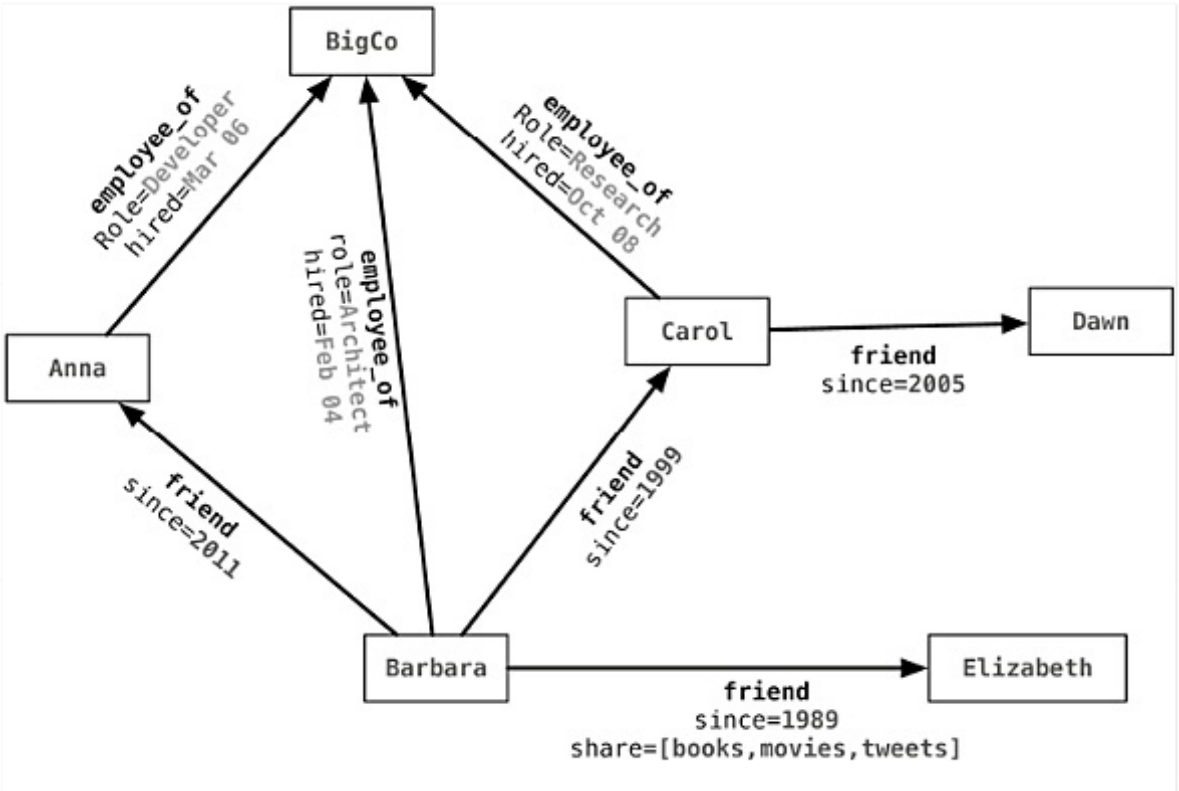

Figura 11.2 – Relacionamentos com propriedades.

Relacionamentos são cidadãos de primeira classe em bancos de dados de grafos. A maior parte do valor dos bancos de dados de grafos é derivada dos relacionamentos, que não têm

apenas um tipo, um nodo inicial e um nodo final, mas também propriedades exclusivas. Utilizando essas propriedades nos relacionamentos, podemos adicionar inteligência a eles – por exemplo, desde quando são amigos, qual a distância entre os nodos ou quais aspectos são compartilhados entre eles. Essas propriedades nos relacionamentos podem ser utilizadas para consultar o grafo.

Como a maior parte do poder de bancos de dados em grafos vem dos relacionamentos e de suas propriedades, é necessário muito trabalho de planejamento e projeto para modelar os relacionamentos no domínio em que estamos tentando trabalhar. Adicionar novos tipos de relacionamento é fácil; alterar nodos existentes e seus relacionamentos é semelhante à migração de dados (“Migrações em bancos de dados de grafos”, p. 187), pois essas alterações terão de ser feitas em cada nodo e em cada relacionamento nos dados existentes.

11.2.1 Consistência

Já que bancos de dados de grafos operam em nodos conectados, a maioria das soluções deste tipo de banco de dados, geralmente, não suporta a distribuição de nodos em servidores diferentes. Há algumas soluções, entretanto, que suportam a distribuição de nodos em um cluster de servidores, como o Infinite Graph. Dentro de um único servidor, os dados são sempre consistentes, especialmente no Neo4J, que é totalmente compatível com as propriedades ACID. Ao executar o Neo4J em um cluster, uma gravação no mestre acaba sendo sincronizada com os escravos, enquanto que os escravos estão sempre disponíveis para leitura. Gravações nos escravos são permitidas e imediatamente sincronizadas com o mestre; outros escravos, porém, não serão sincronizados imediatamente – eles terão de esperar que os dados se propaguem a partir do mestre.

Bancos de dados de grafos garantem consistência por meio de transações. Eles não permitem relacionamentos pendentes: o nodo inicial e o nodo final sempre têm de existir e nodos somente podem ser excluídos se não tiverem relacionamentos associados a eles.

11.2.2 Transações

O Neo4J é compatível com as propriedades ACID. Antes de alterar quaisquer nodos ou adicionar algum relacionamento a nodos já existentes, temos de iniciar uma transação. Se não envolvermos as operações em transações, obteremos uma exceção `NotInTransactionException`. Operações de leitura podem ser executadas sem iniciar uma transação.

```
Transaction transaction = database.beginTx();
try {
    Node node = database.createNode();
    node.setProperty("name", "NoSQL Essencial");
    node.setProperty("published", "2012");
    transaction.success();
} finally {
    transaction.finish();
}
```

No código anterior, iniciamos uma transação no banco de dados, depois criamos um nodo e configuramos as propriedades dele. Marcamos a transação como bem-sucedida (`success`) e finalmente a completamos com um `finish`. Uma transação precisa ser marcada

como bem-sucedida, senão o Neo4J supõe que ela falhou e a desfaz quando o comando `finish` for executado. Da mesma maneira, utilizar o `success`, mas omitir o `finish` também não confirmará os dados no banco de dados. Essa maneira de gerenciar transações deve ser lembrada durante o desenvolvimento, uma vez que difere do modo padrão de executar transações em um SGBDR.

11.2.3 Disponibilidade

O Neo4J, a partir da versão 1.8, obtém alta disponibilidade ao permitir escravos replicados. Esses escravos também podem lidar com gravações: quando algo é gravado neles, eles sincronizam a gravação com o mestre atual e essa gravação é confirmada, primeiro no mestre e depois no escravo. Outros escravos receberão a atualização eventualmente. Outros bancos de dados de grafos, como o Infinite Graph e o FlockDB, fornecem armazenamento distribuído para os nodos.

O Neo4J utiliza o Apache ZooKeeper [ZooKeeper] para registrar os IDs da última transação persistida em cada nodo escravo e em cada nodo mestre atual. Assim que um servidor é iniciado, ele se comunica com o ZooKeeper e descobre qual servidor é o mestre. Se o servidor é o primeiro a entrar no cluster, torna-se o mestre; quando um servidor falha, o cluster elege um mestre a partir dos nodos disponíveis, fornecendo, assim, alta disponibilidade.

11.2.4 Recursos de consulta

Bancos de dados de grafos são suportados por linguagens de consulta, como a Gremlin [Gremlin]. Gremlin é uma linguagem específica de domínio para percorrer grafos; ela pode percorrer todos os bancos de dados de grafos que implementem grafos de propriedades no padrão Blueprints [Blueprints]. O Neo4J também possui a linguagem de consulta Cypher [Cypher] para pesquisar o grafo. Além destas linguagens de consulta, o Neo4J permite que você consulte propriedades dos nodos do grafo, ou percorra os relacionamentos dos nodos utilizando bindings da linguagem.

As propriedades de um nodo podem ser indexadas por meio do serviço de indexação. De forma semelhante, as propriedades de relacionamentos ou arestas podem ser indexadas, de modo que um nodo ou aresta possa ser encontrado por seus valores. Os índices devem ser pesquisados para encontrar o nodo inicial e, assim, começar a travessia. Examinaremos a pesquisa do nodo utilizando a indexação de nodos.

Se tivermos o grafo mostrado na figura 11.1, podemos indexar os nodos à medida que são adicionados ao banco de dados, ou podemos indexar todos os nodos posteriormente, percorrendo-os. Precisamos primeiro criar um índice para os nodos utilizando o **IndexManager**.

```
Index<Node> nodeIndex = graphDb.index().forNodes("nodes");
```

Estamos indexando os nodos pela propriedade `name`. O Neo4J utiliza o Lucene [Lucene] como seu serviço de indexação. Veremos, posteriormente, que também podemos utilizar a capacidade de pesquisa de texto integral do Lucene. Quando novos nodos são criados, eles podem ser adicionados ao índice.

```
Transaction transaction = graphDb.beginTx();
try {
    Index<Node> nodeIndex = graphDb.index().forNodes("nodes");
```

```
    nodeIndex.add(martin, "name", martin.getProperty("name"));
    nodeIndex.add(pramod, "name", pramod.getProperty("name"));
    transaction.success();
} finally {
    transaction.finish();
}
```

A adição de nodos ao índice é feita dentro do contexto de uma transação. Assim que os nodos são indexados, podemos pesquisá-los utilizando a propriedade indexada. Se pesquisarmos o nodo com o nome de `Barbara`, podemos consultar o índice da propriedade `name` que tenha um valor igual a `Barbara`.

```
Node node = nodeIndex.get("name", "Barbara").getSingle();
```

Obtemos o nodo cujo nome é `Martin`. Dado o nodo, podemos obter todos os seus relacionamentos.

```
Node martin = nodeIndex.get("name", "Martin").getSingle();
allRelationships = martin.getRelationships();
```

Podemos obter os relacionamentos `INCOMING` ou `OUTGOING`.

```
incomingRelations = martin.getRelationships(Direction.INCOMING);
```

Ao pesquisar um relacionamento, também podemos aplicar filtros direcionais nas consultas. Com o grafo da figura 11.1, se quisermos encontrar todas as pessoas que curtiram o `NoSQL Essencial`, podemos localizar o nodo `NoSQL Essencial` e obter seus relacionamentos com `Direction.INCOMING`. Nesse momento, também podemos adicionar o tipo do relacionamento ao filtro de pesquisa, uma vez que estamos procurando apenas nodos que curtiram (`LIKES`) o `NoSQL Essencial`.

```
Node nosqlEssencial = nodeIndex.get("name", "NoSQL Essencial").getSingle();
relationships = nosqlEssencial.getRelationships(Direction.INCOMING, LIKES);
for (Relationship relationship : relationships) {
    likesNoSQLEssencial.add(relationship.getStartNode());
}
```

Encontrar nodos e suas relações imediatas é fácil, mas isso também pode ser feito em bancos de dados relacionais. Bancos de dados de grafos são realmente poderosos quando você quer percorrer os grafos em qualquer profundidade e especificar um nodo inicial para a travessia. Isto é útil, principalmente, quando você estiver tentando encontrar nodos que estejam relacionados ao nodo inicial em mais de um nível de profundidade. À medida que a profundidade do grafo aumenta, faz mais sentido percorrer os relacionamentos utilizando um `Traverser` em que você possa especificar que está procurando tipos de relacionamento `INCOMING`, `OUTGOING` ou `BOTH` (ambos). Você também pode fazer a travessia de cima para baixo ou lateralmente no grafo utilizando `Order` igual a `BREADTH_FIRST` ou `DEPTH_FIRST`. A travessia tem de iniciar em algum nodo – neste exemplo, tentamos encontrar todos os nodos, em qualquer profundidade, que estejam relacionados como `FRIEND` à `Barbara`:

```
Node barbara = nodeIndex.get("name", "Barbara").getSingle();

Traverser friendsTraverser = barbara.traverse(
    Order.BREADTH_FIRST,
    StopEvaluator.END_OF_GRAPH,
    ReturnableEvaluator.ALL_BUT_START_NODE,
    EdgeType.FRIEND,
    Direction.OUTGOING);
```

O `friendsTraverser` nos fornece uma forma de encontrar todos os nodos que estejam relacionados a `Barbara` e nos quais o tipo de relacionamento seja `FRIEND`. Os nodos podem ter qualquer profundidade – amigo de um amigo em qualquer nível –, permitindo que você explore estruturas de árvore.

Um dos recursos interessantes dos bancos de dados de grafos é encontrar caminhos entre dois nodos – determinar se há múltiplos caminhos, encontrando todos os caminhos ou o mais curto. No grafo da figura 11.1, sabemos que `Barbara` está conectada a `Jill` por dois caminhos distintos; para encontrar todos esses caminhos e também a distância entre `Barbara` e `Jill` por meio deles, podemos utilizar

```
Node barbara = nodeIndex.get("name", "Barbara").getSingle();
Node jill = nodeIndex.get("name", "Jill").getSingle();
PathFinder<Path> finder = GraphAlgoFactory.allPaths(
    Traversal.expanderForTypes(FRIEND, Direction.OUTGOING),
    MAX_DEPTH);
Iterable<Path> paths = finder.findAllPaths(barbara, jill);
```

Esse recurso é utilizado em redes sociais para mostrar relacionamentos entre dois nodos quaisquer. Para encontrar todos os caminhos e a distância entre os nodos em cada caminho, primeiro obtemos uma lista de caminhos distintos entre os dois nodos. O tamanho de cada caminho é o **número de pulos**, no grafo, necessários para alcançar o nodo de destino a partir do nodo inicial. Muitas vezes, você precisa obter o caminho mais curto entre dois nodos; dos dois caminhos de `Barbara` até `Jill`, o mais curto pode ser encontrado utilizando

```
PathFinder<Path> finder = GraphAlgoFactory.shortestPath(
    Traversal.expanderForTypes(FRIEND, Direction.OUTGOING),
    MAX_DEPTH);
Iterable<Path> paths = finder.findAllPaths(barbara, jill);
```

Muitos outros algoritmos de grafos podem ser aplicados a esse grafo, como o algoritmo de Dijkstra [Dijkstra's] para encontrar o caminho mais rápido ou menos custoso entre dois nodos.

```
START beginingNode = (especifique o nodo inicial)
MATCH (relacionamento, padrão de pesquisa)
WHERE (condição de filtragem: em dados nos nodos e relacionamentos)
RETURN (o que retornar: nodos, relacionamentos, propriedades)
ORDER BY (propriedades para ordenar)
SKIP (nodos para ignorar no topo)
LIMIT (limitar os resultados)
```

O Neo4J também fornece a linguagem de consulta **Cypher** para pesquisar no grafo. O Cypher precisa de um nodo `START` para iniciar (`START`) a consulta. O nodo inicial pode ser identificado pelo seu ID de nodo, uma lista de IDs de nodos, ou buscas em índices. O Cypher utiliza a palavra-chave `MATCH` para buscar padrões em relacionamentos; a palavra-chave `WHERE` filtra as propriedades em um nodo ou relacionamento. A palavra-chave `RETURN` especifica o que é retornado pela consulta – nodos, relacionamentos ou campos nos nodos ou relacionamentos.

O Cypher também fornece métodos para `ORDER`, `AGGREGATE`, `SKIP` e `LIMIT` (ordenar, agregar, pular e limitar) os dados. Na figura 11.2, encontramos todos os nodos conectados a `Barbara`, sejam os de entrada ou de saída, utilizando o `--`.

```
START barbara = node:nodeIndex(name = "Barbara")
MATCH (barbara)--(connected_node)
RETURN connected_node
```

Quando estivermos interessados em significância direcional, poderemos utilizar

```
MATCH (barbara)<--(connected_node)
```

para relacionamentos que chegam, ou

```
MATCH (barbara)-->(connected_node)
```

para relacionamentos que saem. A busca também pode ser feita em relacionamentos específicos utilizando a convenção `:RELATIONSHIP_TYPE` e retornando os campos ou nodos requeridos.

```
START barbara = node:nodeIndex(name = "Barbara")
MATCH (barbara)-[:FRIEND]->(friend_node)
RETURN friend_node.name,friend_node.location
```

Começamos com `Barbara`, encontramos todos os relacionamentos de saída com o tipo `FRIEND` e retornamos com os nomes dos amigos. A consulta por tipo de relacionamento somente funciona na profundidade de um nível; podemos fazê-la funcionar em profundidades maiores e descobrir a profundidade de cada um dos nodos resultantes.

```
START barbara=node:nodeIndex(name = "Barbara")
MATCH path = barbara-[:FRIEND*1..3]->end_node
RETURN barbara.name,end_node.name, length(path)
```

De modo semelhante, podemos consultar relacionamentos nos quais exista uma propriedade específica de relacionamento. Também podemos filtrar propriedades de relacionamentos e consultar se uma propriedade existe ou não.

```
START barbara = node:nodeIndex(name = "Barbara")
MATCH (barbara)-[relation]->(related_node)
WHERE type(relation) = 'FRIEND' AND relation.share
RETURN related_node.name, relation.since
```

Há muitos outros recursos de consulta na linguagem Cypher que podem ser utilizados para consultar grafos em bancos de dados.

11.2.5 Escalabilidade

Em bancos de dados NoSQL, uma das técnicas de crescimento em escala comumente utilizada é a fragmentação, na qual os dados são divididos e distribuídos em diferentes servidores. Com bancos de dados de grafos, a fragmentação é difícil, uma vez que esse tipo de banco de dados não é orientado a agregados, mas, sim, a relacionamentos. Como todo nodo pode ser relacionado a um outro qualquer, armazenar nodos relacionados no mesmo servidor é melhor para a travessia do grafo. Percorrer um grafo quando os nodos estão em máquinas diferentes não é bom para o desempenho. Conhecendo essa limitação dos bancos de dados, ainda assim podemos ampliá-los utilizando algumas técnicas comuns descritas por Jim Webber [Webber Neo4J Scaling].

De modo geral, há três maneiras de fazer os bancos de dados em grafos crescerem em escala. Já que as máquinas atuais possuem bastante memória RAM, podemos adicionar tal memória ao servidor, de modo que o conjunto de nodos e relacionamentos sendo trabalhados esteja inteiramente na memória. Esta técnica somente é útil se o conjunto de dados no qual estivermos trabalhando couber em uma quantidade realista de RAM.

Podemos melhorar o desempenho na leitura do banco de dados adicionando aos dados mais escravos com acesso apenas de leitura, com todas as gravações sendo enviadas para o mestre. Esse padrão de muitos servidores, de gravar uma vez e ler, é uma técnica comprovada em clusters MySQL e é realmente útil quando o conjunto de dados for suficientemente grande para não caber na memória RAM de uma única máquina, mas suficientemente pequeno para ser replicado em múltiplas máquinas. Os escravos também podem contribuir com a disponibilidade e a melhora na leitura, uma vez que podem ser configurados para que nunca se tornem um mestre, permanecendo sempre com a função de fazer apenas a leitura.

Quando o tamanho do conjunto de dados torna impraticável a replicação, podemos fragmentar ("Fragmentação", p. 74) os dados a partir do lado do aplicativo utilizando o conhecimento específico do domínio. Por exemplo, nodos que se relacionam com a América do Norte podem ser criados em um servidor, enquanto os que se relacionam com a Ásia são criados em outro. Essa fragmentação em nível de aplicativo precisa compreender que nodos são armazenados em bancos de dados fisicamente diferentes (Figura 11.3).

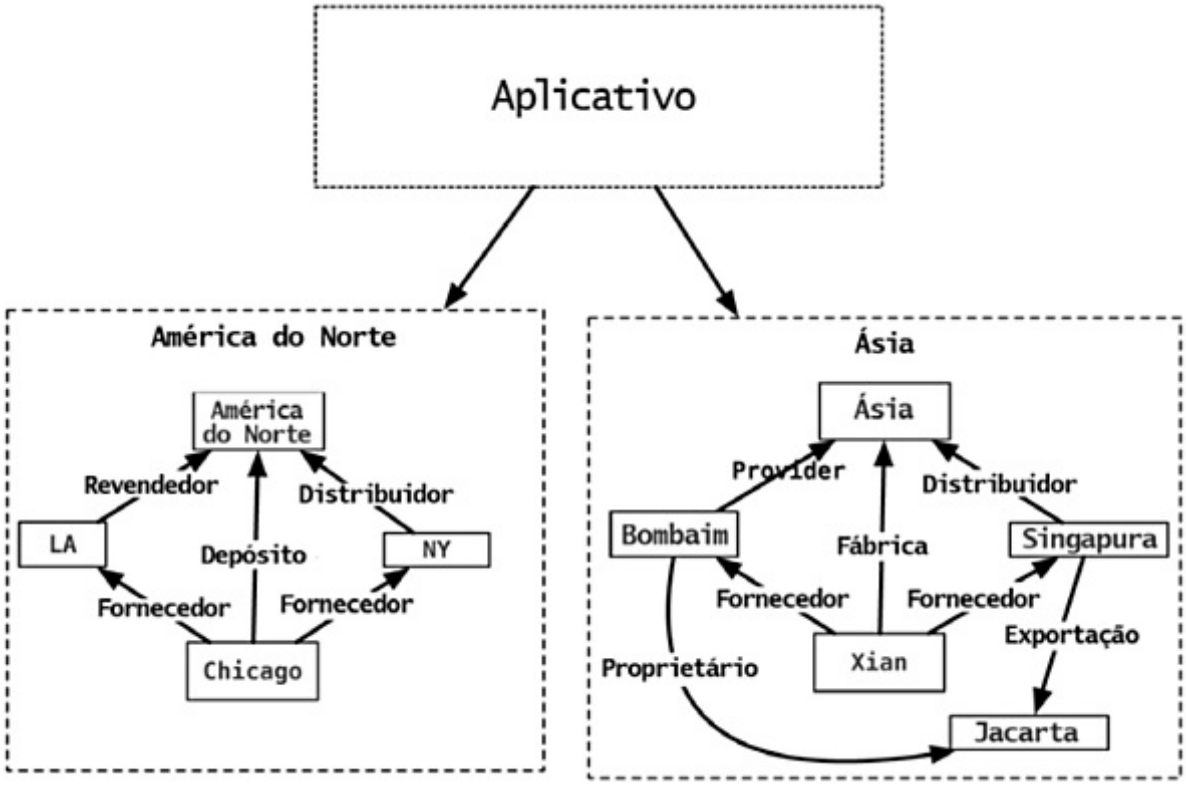

Figura 11.3 – Fragmentação de nodos em nível de aplicativo.

11.3 Casos de uso apropriados

Examinaremos alguns casos de uso apropriados para bancos de dados de grafos.

11.3.1 Dados conectados

É nas redes sociais que os bancos de dados de grafos podem ser instalados e utilizados de maneira muito eficaz. Esses grafos sociais não têm de ser apenas do tipo que relaciona amigos; eles podem, por exemplo, representar funcionários, seu conhecimento e onde

trabalharam com outros funcionários em diferentes projetos. Qualquer domínio rico em links é apropriado para bancos de dados de grafos.

Se você tiver relacionamentos entre entidades de diferentes domínios (como social, espacial, comercial) em um único banco de dados, pode tornar esses relacionamentos mais valiosos, fornecendo a eles a capacidade de percorrer domínios.

11.3.2 Roteamento, envio e serviços baseados em localização

Todo local ou endereço que possui uma entrega é considerado um nodo, e todos os nodos onde a entrega tiver de ser efetuada pelo entregador pode ser modelado na forma de um grafo de nodos. Os relacionamentos entre os nodos podem ter a propriedade de distância, permitindo, assim, que você entregue as encomendas de um modo eficiente. As propriedades distância e localização também podem ser utilizadas em grafos de lugares de interesse, de modo que seu aplicativo possa fornecer recomendações de bons restaurantes ou opções de divertimento próximas. Você também pode criar nodos para seus pontos de venda, como livrarias ou restaurantes, e notificar os usuários quando estiverem perto desses nodos, a fim de fornecer serviços baseados em localização.

11.3.3 Mecanismos de recomendação

À medida que os nodos e os relacionamentos são criados no sistema, eles podem ser utilizados para fazer recomendações do tipo "seus amigos também compraram este produto" ou "ao faturar este item, estes outros geralmente também o são". Além disso, podem ser utilizados para fazer recomendações a viajantes, mencionando, por exemplo, que, quando outros visitantes vêm a Barcelona, geralmente visitam as criações de Antonio Gaudí.

Um efeito colateral muito interessante, proveniente do uso de bancos de dados de grafos para recomendações, é que, à medida que o tamanho dos dados aumenta, o número de nodos e relacionamentos disponíveis para fazer recomendações também aumenta, rapidamente. Os mesmos dados também podem ser utilizados para encontrar informações – por exemplo, quais produtos são sempre comprados juntos ou quais itens são sempre faturados juntos. Alertas podem ser disparados quando essas condições não forem satisfeitas. Assim como outros mecanismos de recomendação, os bancos de dados de grafos podem ser utilizados para pesquisar padrões em relacionamentos a fim de detectar fraudes em transações.

11.4 Quando não utilizar

Em algumas situações, os bancos de dados de grafos podem não ser apropriados. Quando você quiser atualizar todas as entidades ou um subconjunto delas – por exemplo, em um aplicativo de análise no qual todas as entidades talvez precisem ser atualizadas com uma propriedade alterada –, bancos de dados de grafos podem não ser ideais, uma vez que alterar uma propriedade em todos os nodos não é uma operação direta. Mesmo se o modelo de dados funcionar para o domínio do problema, alguns bancos de dados podem não conseguir lidar com muitos dados, especialmente em operações globais de grafos (aquelas envolvendo o grafo inteiro).

CAPÍTULO 12

Migrações de esquema

12.1 Alterações no esquema

Uma tendência recente na discussão de bancos de dados NoSQL consiste em destacar sua natureza livre de esquemas – esse é um recurso popular que permite aos desenvolvedores concentrarem-se no projeto do domínio sem a preocupação com alterações no esquema. Isso torna-se particularmente verdadeiro com a ascensão dos métodos ágeis [Agile Methods], em que é importante atender às mudanças nos requisitos.

Discussões, iterações e feedbacks entre os especialistas no domínio e entre os proprietários dos produtos são importantes para que se obtenha a compreensão correta dos dados; essas discussões não devem ser prejudicadas pela complexidade do esquema de um banco de dados. Armazenando dados no NoSQL, alterações no esquema podem ser feitas mais facilmente, melhorando a produtividade do desenvolvedor ("Surgimento do NoSQL", p. 34). Vimos que desenvolver e realizar a manutenção de um aplicativo nesse novo mundo de bancos de dados sem esquema requer bastante atenção quanto à migração do esquema.

12.2 Alterações de esquema em SGBDRs

Trabalhando com tecnologias SGBDR padrão, desenvolvemos objetos, suas tabelas correspondentes e seus relacionamentos. Considere um modelo de objetos e um modelo de dados simples que tenha `Customer`, `Order` e `OrderItems`. O modelo ER se pareceria com a figura 12.1.

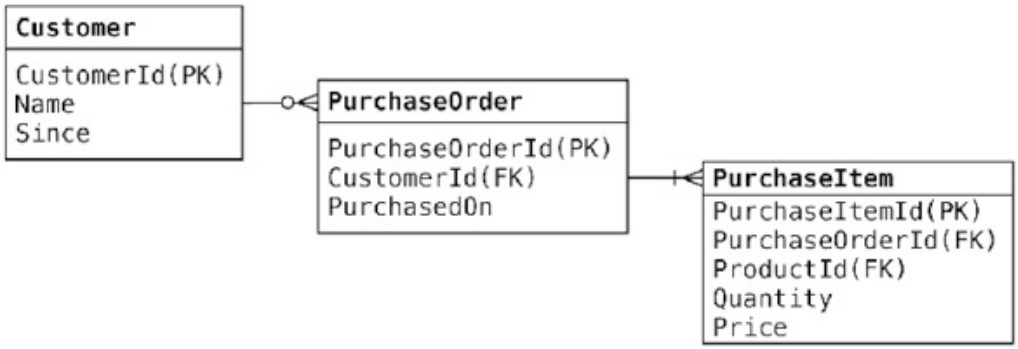

Figura 12.1 – Modelo de dados de um sistema de comércio eletrônico.

Enquanto esse modelo de dados suportar o modelo de objetos atual, tudo estará bem. Na primeira vez em que houver uma mudança no modelo de objetos, como a introdução de `preferredShippingType` no objeto `Customer`, teremos de alterar o objeto e a tabela do banco de dados porque, sem alterar a tabela, o aplicativo ficará fora de sincronia com o banco de dados. Quando obtivermos erros como `ORA-00942: table or view does not exist` ou `ORA-00904: "PREFERRED_SHIPPING_TYPE": invalid identifier`, saberemos que temos esse problema.

Geralmente, uma migração de esquema de banco de dados já é um projeto por si só. Para

instalar as alterações no esquema, scripts de alteração do banco de dados são desenvolvidos utilizando técnicas de diferenciação para todas as alterações no banco de dados de desenvolvimento. Essa abordagem que envolve a criação de scripts de migração durante o desenvolvimento/lançamento é propensa a erros e não suporta métodos de desenvolvimento ágil.

12.2.1 Migrações para projetos sem restrições anteriores

É melhor fazer os scripts das alterações de esquemas de banco de dados durante o desenvolvimento, uma vez que podemos armazenar essas alterações no esquema acompanhado dos scripts de migração de dados em um mesmo arquivo de script. Esses arquivos de script devem receber nomes sequenciais crescentes que reflitam as versões do banco de dados; por exemplo, a primeira alteração no banco de dados poderia ter um arquivo de script chamado `001_Descricao_da_mudanca.sql`. Colocar as alterações em scripts permite que as migrações do banco de dados sejam executadas preservando-se a ordem das alterações. Na figura 12.2, podemos ver todas as alterações realizadas em um banco de dados até o momento.

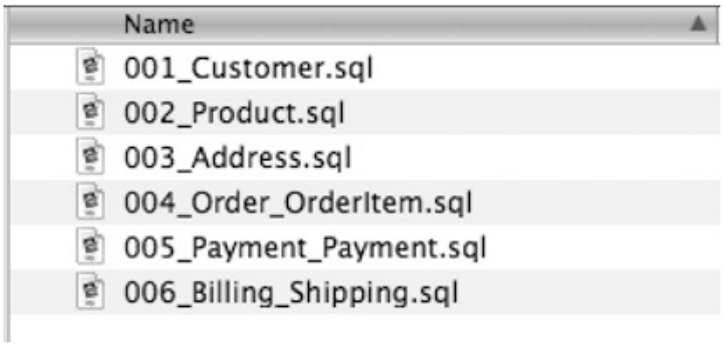

Figura 12.2 – Sequência de migrações aplicadas a um banco de dados.

Suponha agora que precisemos alterar a tabela `OrderItem` para armazenar o `DiscountedPrice` e o `FullPrice` do item. Isso demandará uma alteração na tabela `OrderItem` e será a alteração número `007` na nossa sequência de alterações, conforme mostrado na figura 12.3.

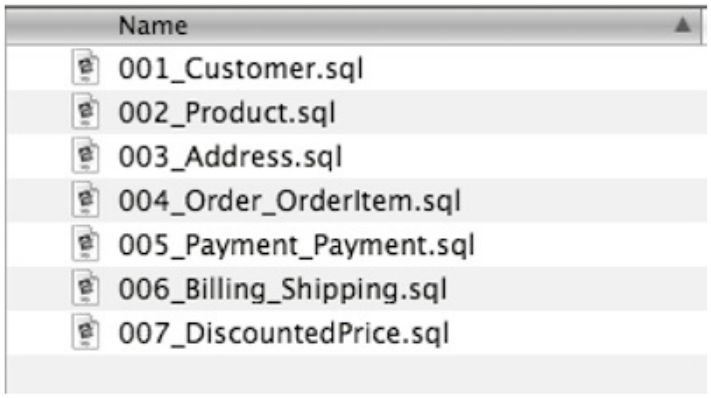

Figura 12.3 – A nova alteração 007_DiscountedPrice.sql aplicada ao banco de dados.

Aplicamos uma nova alteração ao banco de dados. O script dessa alteração possui um código para adicionar uma nova coluna, renomear a coluna existente e migrar os dados necessários para fazer o novo recurso funcionar. A seguir, podemos ver o script da

alteração `007_DiscountedPrice.sql`:

```
ALTER TABLE orderitem ADD discountedprice NUMBER(18,2) NULL;
UPDATE orderitem SET discountedprice = price;
ALTER TABLE orderitem MODIFY discountedprice NOT NULL;
ALTER TABLE orderitem RENAME COLUMN price TO fullprice;
--//@UNDO
ALTER TABLE orderitem RENAME fullprice TO price;
ALTER TABLE orderitem DROP COLUMN discountedprice;
```

O script de alteração mostra as alterações do esquema no banco de dados, assim como as migrações de dados que precisam ser feitas. No exemplo apresentado, estamos utilizando DBDeploy [DBDeploy] como o framework para gerenciar as alterações no banco de dados. O DBDeploy mantém uma tabela no banco de dados, chamada `ChangeLog`, na qual são armazenadas todas as alterações realizadas no banco de dados. Nessa tabela, `Change_Number` informa a todos quais alterações foram aplicadas ao banco de dados. Esse `Change_Number`, que é a versão para o banco de dados, é utilizado, então, para encontrar o script correspondente enumerado na pasta e aplicar as alterações que ainda não foram aplicadas. Quando escrevemos um script com a alteração número `007` e o aplicamos ao banco de dados utilizando o DBDeploy, ele verificará o `ChangeLog` e, então, pegará todos os scripts da pasta que ainda não tiverem sido aplicados. A figura 12.4 é uma cópia da tela do DBDeploy aplicando a alteração no banco de dados.

```
project $>ant dbupgrade
Buildfile: /project/build.xml

init:

dbupgrade:
 [dbdeploy] dbdeploy 3.0M3
 [dbdeploy] Reading change scripts from directory /project/db/migrations...
 [dbdeploy] Changes currently applied to database:
 [dbdeploy]   1..6
 [dbdeploy] Scripts available:
 [dbdeploy]   1..7
 [dbdeploy] To be applied:
 [dbdeploy]   7
 [dbdeploy] Applying #7: 007_DiscountedPrice.sql...
 [dbdeploy]  -> statement 1 of 4...
 [dbdeploy]  -> statement 2 of 4...
 [dbdeploy]  -> statement 3 of 4...
 [dbdeploy]  -> statement 4 of 4...

BUILD SUCCESSFUL
Total time: 0 seconds
project $>
```

Figura 12.4 – O DBDeploy melhorando o banco de dados com a alteração número 007.

A melhor forma de integrar-se com o resto dos desenvolvedores é utilizando o repositório de controle de versões do seu projeto para armazenar todos esses scripts de alteração, de modo que você possa registrar a versão do software e o banco de dados em um mesmo lugar, eliminando possíveis incompatibilidades entre o banco de dados e o aplicativo. Há muitas outras ferramentas para realizar tais atualizações, incluindo o Liquibase [Liquibase], o MyBatis Migrator [MyBatis Migrator] e o DBMaintain [DBMaintain].

12.2.2 Migrações em projetos legados

Nem todo projeto está livre de restrições: como implementar migrações quando um aplicativo existente estiver em produção? Descobrimos que pegar um banco de dados e extrair sua estrutura em scripts, junto com todo o código do banco de dados e quaisquer dados de referência, funciona como uma base para o projeto. Essa base não deve conter dados transacionais. Assim que estiver pronta, alterações posteriores podem ser efetuadas utilizando as técnicas de migração descritas anteriormente (Figura 12.5).

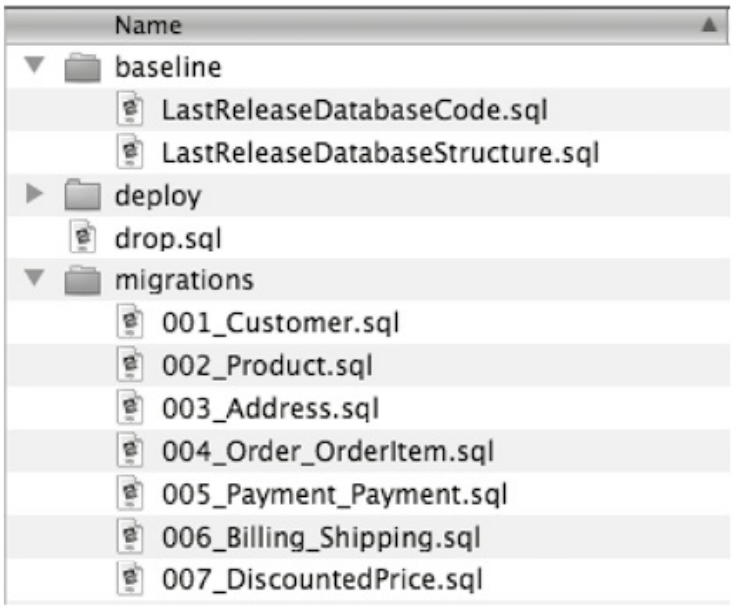

Figura 12.5 – O uso de scripts com um banco de dados legado.

Um dos principais aspectos da migração deve ser a manutenção da compatibilidade retroativa do esquema do banco de dados. Em muitas empresas, há vários aplicativos utilizando o banco de dados; quando alteramos o banco de dados para um aplicativo, isso não deve prejudicar o funcionamento dos outros aplicativos. Podemos obter uma compatibilidade retroativa implementando uma fase de transição para a alteração, conforme descrito em Refactoring Databases (Refatorando Bancos de Dados) [Ambler and Sadalage].

Durante a fase de transição, os esquemas antigo e novo são mantidos em paralelo e ficam disponíveis para todos os aplicativos que utilizam o banco de dados. Para isso, temos de introduzir códigos de apoio, como gatilhos (triggers), visões e colunas virtuais, de modo a garantir que os outros aplicativos possam acessar o esquema do banco de dados de que necessitam sem alterações no código.

```
ALTER TABLE customer ADD fullname VARCHAR2(60);
UPDATE customer SET fullname = fname;

CREATE OR REPLACE TRIGGER SyncCustomerFullName
BEFORE INSERT OR UPDATE
ON customer
REFERENCING OLD AS OLD NEW AS NEW
FOR EACH ROW
BEGIN
  IF :NEW.fname IS NULL THEN
    :NEW.fname := :NEW.fullname;
```

```
    END IF;
    IF :NEW.fullname IS NULL THEN
      :NEW.fullname := :NEW.fname
    END IF;
  END;
  /
  --Eliminar o trigger e a coluna fname
  --quando todos os aplicativos começarem a usar customer.fullname
```

Nesse exemplo, estamos tentando renomear a coluna `customer.fname` para `customer.fullname`, pois queremos evitar ambiguidades de `fname` significando `fullname` ou `firstname`. Uma renomeação direta da coluna `fname` e a alteração do código do aplicativo pelo qual somos responsáveis pode funcionar bem para o nosso aplicativo, mas não para os outros aplicativos que estiverem acessando o mesmo banco de dados na empresa.

Utilizando a técnica da fase de transição, introduzimos a nova coluna `fullname` e copiamos os dados para ela. Porém, mantemos a coluna antiga `fname`. Introduzimos, também, um gatilho `BEFORE UPDATE` para sincronizar os dados entre as colunas antes que sejam confirmados no banco de dados.

Agora, quando os aplicativos lerem os dados da tabela, lerão `fname` ou `fullname`, mas sempre obterão os dados corretos. Podemos eliminar o gatilho e a coluna `fname` assim que todos os aplicativos começarem a utilizar a nova coluna `fullname`.

É muito difícil executar migrações de esquema em conjuntos grandes de dados em SGBDRs, especialmente se tivermos de manter o banco de dados disponível para os aplicativos, pois movimentações de grandes volumes de dados e alterações estruturais criam, geralmente, bloqueios nas tabelas do banco de dados.

12.3 Alterações no esquema em um armazenamento de dados NoSQL

Um banco de dados relacional tem de ser alterado antes do aplicativo. Isso é o que a abordagem sem esquemas (schema-free ou schemaless) tenta evitar, visando à flexibilidade nas alterações de esquema por entidade. Alterações frequentes no esquema são necessárias para atender às alterações frequentes no mercado, bem como a inovações de produtos.

Ao desenvolver com bancos de dados NoSQL, em alguns casos, o esquema não precisa ser planejado de antemão. Ainda temos de projetar e pensar em outros aspectos, como os tipos de relacionamentos (com bancos de dados de grafos) ou como os nomes das famílias de colunas, linhas, colunas, ordem das colunas (com bancos de dados de colunas) ou, ainda, como as chaves estão atribuídas e qual é a estrutura dos dados dentro do objeto de valores (com depósitos de chave-valor). Mesmo que não planejássemos isso logo no início, ou mesmo que quiséssemos alterar nossas decisões, também seria fácil fazê-lo.

A alegação de que os bancos de dados NoSQL são inteiramente desprovidos de esquema é enganosa; embora armazenem os dados sem considerar o esquema ao qual estão associados, esse esquema tem de ser definido pelo aplicativo, pois o fluxo de dados tem de ser analisado por ele ao fazer a leitura dos dados do banco de dados. Além disso, o aplicativo tem de criar os dados que seriam gravados no banco de dados. Se ele não puder analisar os dados do banco de dados, temos uma incompatibilidade de esquema mesmo que, ao invés de o banco de dados relacional gerar um erro, esse erro é, agora, encontrado pelo aplicativo. Assim, mesmo nos bancos de dados sem esquema, o esquema dos dados

tem de ser levado em consideração ao refatorar o aplicativo.

Alterações no esquema são especialmente importantes quando há um aplicativo instalado e quando existir dados de produção. Para manter a simplicidade, suponha que estamos utilizando um armazenamento de dados de documentos, como o MongoDB [MongoDB], e que temos o mesmo modelo de dados de antes: `customer`, `order` e `orderItems`.

```
{
    "_id": "4BD8AE97C47016442AF4A580",
    "customerid": 99999,
    "name": "Foo Sushi Inc",
    "since": "12/12/2012",
    "order": {
        "orderid": "4821-UXWE-122012",
        "orderdate": "12/12/2001",
        "orderItems": [{"product": "Fortune Cookies",
                        "price": 19.99}]
    }
}
```

Código do aplicativo para gravar a estrutura desse documento no MongoDB:

```
BasicDBObject orderItem = new BasicDBObject();
orderItem.put("product", productName);
orderItem.put("price", price);
orderItems.add(orderItem);
```

Código para ler o documento novamente a partir do banco de dados:

```
BasicDBObject item = (BasicDBObject) orderItem;
String productName = item.getString("product");
Double price = item.getDouble("price");
```

Alterar os objetos para adicionar `preferredShippingType` não requer qualquer alteração no banco de dados, uma vez que este não se importa que documentos diferentes não sigam o mesmo esquema. Isso permite um desenvolvimento mais rápido e instalações fáceis. Basta que se instale o aplicativo – nenhuma alteração é necessária no lado do banco de dados. O código tem de assegurar que os documentos que não tenham o atributo `preferredShippingType` ainda possam ser analisados – e isso é tudo.

É claro que, aqui, estamos simplificando a situação da alteração de esquema. Vamos examinar a alteração de esquema que fizemos anteriormente: introduzir `discountedPrice` e renomear `price` para `fullPrice`. Para fazer essa alteração, renomeamos o atributo `price` para `fullPrice` e adicionamos o atributo `discountedPrice`. O documento alterado fica

```
{
    "_id": "5BD8AE97C47016442AF4A580",
    "customerid": 66778,
    "name": "India House",
    "since": "12/12/2012",
    "order": {
        "orderid": "4821-UXWE-222012",
        "orderdate": "12/12/2001",
        "orderItems": [{"product": "Chair Covers",
                        "fullPrice": 29.99,
                        "discountedPrice":26.99}]
    }
```

```
}
```

Assim que instalarmos essa alteração, novos clientes e seus pedidos podem ser gravados e lidos sem problemas, mas, para os pedidos já existentes, o preço do produto não pode ser lido, pois agora o código está procurando `fullPrice` e o documento somente possui `price`.

12.3.1 Migração incremental

Incompatibilidades de esquema atrapalham os iniciantes no mundo de NoSQL. Quando o esquema é alterado no aplicativo, temos de garantir que todos os dados existentes sejam convertidos para o novo esquema (dependendo do tamanho dos dados, essa pode ser uma operação custosa). Outra opção seria se assegurar de que os dados, antes de o esquema ser alterado, ainda possam ser analisados pelo novo código e, quando forem gravados, de que se sejam gravados de volta no novo esquema. Essa técnica, conhecida como migração incremental, migrará os dados no decorrer do tempo; alguns dados talvez nunca sejam migrados, porque nunca foram acessados. Estamos lendo `price` e `fullPrice` do documento:

```
BasicDBObject item = (BasicDBObject) orderItem;
String productName = item.getString("product");
Double fullPrice = item.getDouble("price");
if (fullPrice == null) {
    fullPrice = item.getDouble("fullPrice");
}
Double discountedPrice = item.getDouble("discountedPrice");
```

Ao gravar o documento de volta, o atributo `price` antigo não é gravado:

```
BasicDBObject orderItem = new BasicDBObject();
orderItem.put("product", productName);
orderItem.put("fullPrice", price);
orderItem.put("discountedPrice", discountedPrice);
orderItems.add(orderItem);
```

Ao utilizar a migração incremental, haverá muitas versões do objeto no lado do aplicativo que podem traduzir o esquema do antigo para o novo; ao se gravar o objeto de volta, utiliza-se o novo objeto. Essa migração gradual dos dados ajuda o aplicativo a desenvolver-se mais rapidamente.

A técnica da migração incremental complicará o projeto do objeto, especialmente quando novas alterações forem introduzidas, mas as antigas ainda não tiverem sido retiradas. Esse período entre a instalação da alteração e o último objeto do banco de dados migrando para o novo esquema é conhecido como período de transição (Figura 12.6). Mantenha-o tão curto quanto for possível e enfoque-o no menor escopo possível – isso lhe ajudará a manter seus objetos claros.

A técnica da migração incremental também pode ser implementada com um campo `schema_version` (versão de esquema) nos dados, utilizado pelo aplicativo para selecionar o código correto para manipular os dados dos objetos. Ao fazer a gravação, os dados são migrados para a versão mais recente. O `schema_version` é atualizado para refletir isso.

Ter uma camada de tradução apropriada entre seu domínio e seu banco de dados é importante, de modo que, quando o esquema mudar, o gerenciamento de versões múltiplas do esquema se restrinja à camada de tradução e não se espalhe pelo aplicativo inteiro.

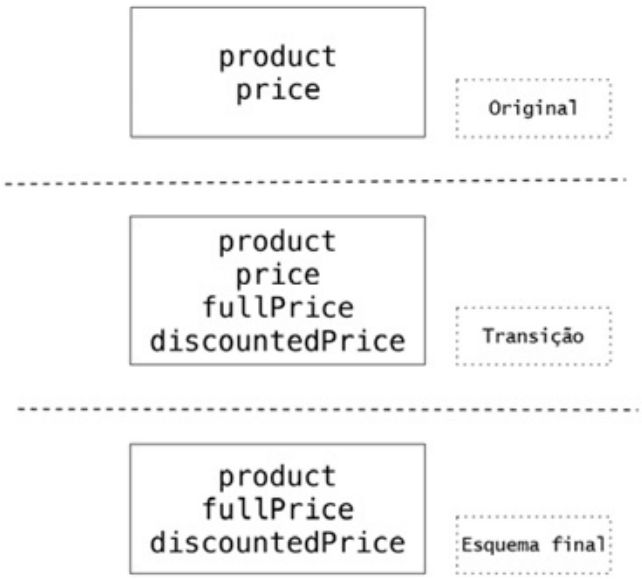

Figura 12.6 – Período de transição de alterações no esquema.

Aplicativos móveis criam necessidades especiais. Uma vez que não podemos impor as atualizações mais recentes do aplicativo, ele deve ser capaz de lidar com quase todas as versões do esquema.

12.3.2 Migrações em bancos de dados de grafos

Bancos de dados de grafos têm arestas que possuem tipos e propriedades. Se você alterar o tipo dessas arestas no código do aplicativo, não poderá mais percorrer o banco de dados, tornando-o inutilizável. Para contornar isso, você pode percorrer todas as arestas e alterar o tipo de cada uma. Essa operação pode ser custosa e requer que você escreva códigos para migrar todas as arestas do banco de dados.

Se precisarmos manter compatibilidade retroativa ou não quisermos alterar o grafo inteiro de uma só vez, podemos simplesmente criar novas arestas entre os nodos; posteriormente, quando estivermos seguros em relação à alteração, as arestas antigas podem ser eliminadas. Podemos utilizar códigos com vários tipos de arestas para percorrer o grafo utilizando os tipos novos e antigos de arestas. Essa técnica pode ajudar bastante com bancos de dados grandes, especialmente se quisermos manter a alta disponibilidade.

Se tivermos de alterar as propriedades em todos os nodos ou arestas, temos de buscar todos os nodos e alterar todas as propriedades que precisarem ser alteradas. Um exemplo seria adicionar `NodeCreatedBy` e `NodeCreatedOn` a todos os nodos existentes para registrar as alterações realizadas em cada nodo.

```
for (Node node : database.getAllNodes()) {
    node.setProperty("NodeCreatedBy", getSystemUser());
    node.setProperty("NodeCreatedOn", getSystemTimeStamp());
}
```

Talvez tenhamos de alterar os dados nos nodos. Novos dados podem ser derivados dos dados do nodo existente, ou poderiam ser importados de alguma outra fonte. A migração pode ser realizada buscando-se todos os nodos a partir de um índice fornecido pela origem dos dados e gravando dados relevantes em cada nodo.

12.3.3 Alterando a estrutura do agregado

Às vezes, você precisa alterar o projeto do esquema, por exemplo, dividindo objetos grandes em objetos menores para que sejam armazenados independentemente. Suponha que você tenha um agregado de cliente que contenha todos os pedidos de clientes e queira separar o cliente e cada um de seus pedidos em unidades agregadas diferentes.

Você tem, então, de assegurar-se de que o código possa funcionar em ambas as versões dos agregados. Se ele não encontrar os objetos antigos, procurará os novos agregados.

O código que é executado em segundo plano pode ler um agregado de cada vez, fazer a alteração necessária e gravar os dados de volta nos agregados diferentes. A vantagem de trabalhar em um agregado de cada vez é que, dessa forma, você não afeta a disponibilidade de dados para o aplicativo.

12.4 Leituras complementares

Para saber mais sobre migrações com bancos de dados relacionais, veja [Ambler e Sadalage]. Embora muito desse conteúdo seja específico ao trabalho relacional, os princípios gerais sobre migração também se aplicam a outros bancos de dados.

12.5 Pontos chave

- Bancos de dados com esquemas fortes, como os bancos de dados relacionais, podem ser migrados gravando cada alteração do esquema, mais a sua migração de dados, em uma sequência controlada por versões.
- Bancos de dados sem esquema ainda precisam de migração cuidadosa devido ao esquema implícito nos códigos que acessam os dados.
- Bancos de dados sem esquema podem utilizar as mesmas técnicas de migração dos bancos de dados com esquemas fortes.
- Bancos de dados sem esquema também podem ler dados de forma tolerante às alterações no esquema implícito de dados e, além disso, podem utilizar migração incremental para atualizá-los.

CAPÍTULO 13

Persistência poliglota

Diferentes bancos de dados são projetados para resolver diferentes problemas. Geralmente, utilizar um único mecanismo de banco de dados para todas as necessidades resulta em soluções de baixo desempenho; armazenar dados transacionais, guardar em cache as informações de sessão, percorrer grafos de clientes e os produtos que seus amigos compraram são problemas essencialmente diferentes. Mesmo na área dos SGBDRs, as necessidades de um sistema OLAP e OLTP são muito diferentes, no entanto, muitas vezes, são forçados para o mesmo esquema.

Vamos pensar em relacionamentos de dados. Soluções SGBDRs são eficientes em reforçarem que relacionamentos existam. Se quisermos descobrir relacionamentos ou tivermos de descobrir dados de diferentes tabelas e que pertençam ao mesmo objeto, então, o uso de SGBDRs começa a ficar difícil.

Mecanismos de bancos de dados são projetados para executar muito bem certas operações em determinadas estruturas de dados e em determinadas quantidades de dados, tais como operar em conjuntos ou armazenamentos de dados e recuperar chaves e seus valores de modo muito rápido ou, ainda, armazenar documentos ricos ou grafos com informações complexas.

13.1 Necessidades diferentes de armazenamento de dados

Muitas empresas tendem a utilizar o mesmo mecanismo de banco de dados para armazenar transações de negócio, dados de gerenciamento de sessão e para outras necessidades de armazenamento, tais como relatórios, BI (Business Intelligence – Inteligência Empresarial), data warehouse ou informações de registros (logs) (Figura 13.1).

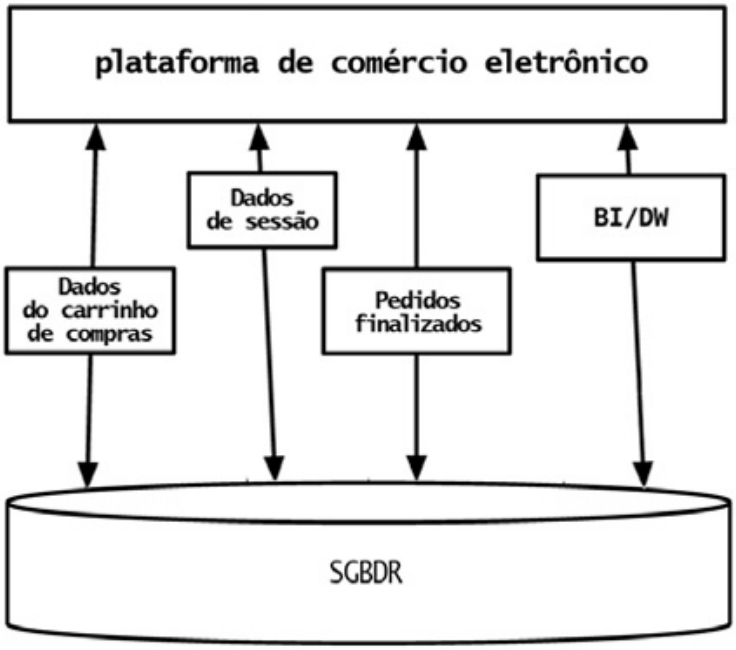

Figura 13.1 – Uso de SGBDRs para cada aspecto de armazenamento do aplicativo.

Os dados da sessão, do carrinho de compras ou do pedido não precisam das mesmas propriedades de disponibilidade, consistência ou necessidades de cópias de segurança. O armazenamento do gerenciamento da sessão precisa da mesma estratégia rigorosa de cópias de segurança/recuperação do que os dados de pedidos de comércio eletrônico? O armazenamento do gerenciamento de sessão precisa de mais disponibilidade do que uma instância do mecanismo de banco de dados para gravar/ler dados de sessão?

Em 2006, Neal Ford cunhou o termo **programação poliglota** para expressar a ideia de que os aplicativos devem ser escritos em uma mistura de linguagens, de modo a aproveitar o fato de que diferentes linguagens são apropriadas para lidar com diferentes problemas. Aplicações complexas combinam diferentes tipos de problemas, de modo que escolher a linguagem certa para cada trabalho possa ser mais produtivo do que a tentativa de ajustar todos os aspectos a uma única linguagem.

De maneira semelhante, ao trabalhar em um problema de comércio eletrônico, é importante utilizar um armazenamento de dados para o carrinho de compras que tenha alta disponibilidade e possa ser ampliado em escala, mas o mesmo armazenamento de dados não pode auxiliar a encontrar produtos comprados pelos amigos do cliente – o que é uma questão totalmente diferente. Utilizamos o termo **persistência poliglota** para definir essa abordagem híbrida para persistência.

13.2 Uso de armazenamentos de dados poliglotas

Vamos utilizar nosso exemplo de comércio eletrônico e a abordagem da persistência poliglota para ver como alguns desses armazenamentos de dados podem ser aplicados (Figura 13.2). Um armazenamento de dados de chave-valor pode ser utilizado para armazenar os dados do carrinho de compras antes de o pedido ser confirmado pelo cliente e também para armazenar os dados da sessão, de modo que o SGBDR não seja utilizado para esses dados transientes. Depósitos de chave-valor são aplicáveis aqui, já que o carrinho de compras geralmente é acessado pelo ID do usuário e, uma vez confirmado e

pago pelo cliente, pode ser gravado no SGBDR. De forma semelhante, dados de sessão são acessados pelo ID da sessão.

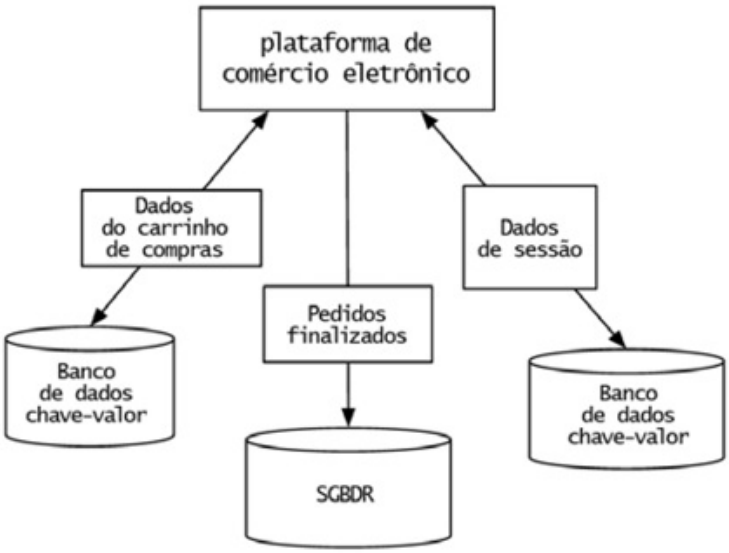

Figura 13.2 – Uso de depósitos de chave-valor para guardar os dados do carrinho de compras e da sessão.

Se precisarmos recomendar produtos para clientes quando eles estiverem colocando-os em seus carrinhos de compras – por exemplo, "seus amigos também compraram esses produtos" ou "seus amigos compraram esses acessórios para esse produto" –, então, a introdução de um armazenamento de dados de grafos ao conjunto é importante (Figura 13.3).

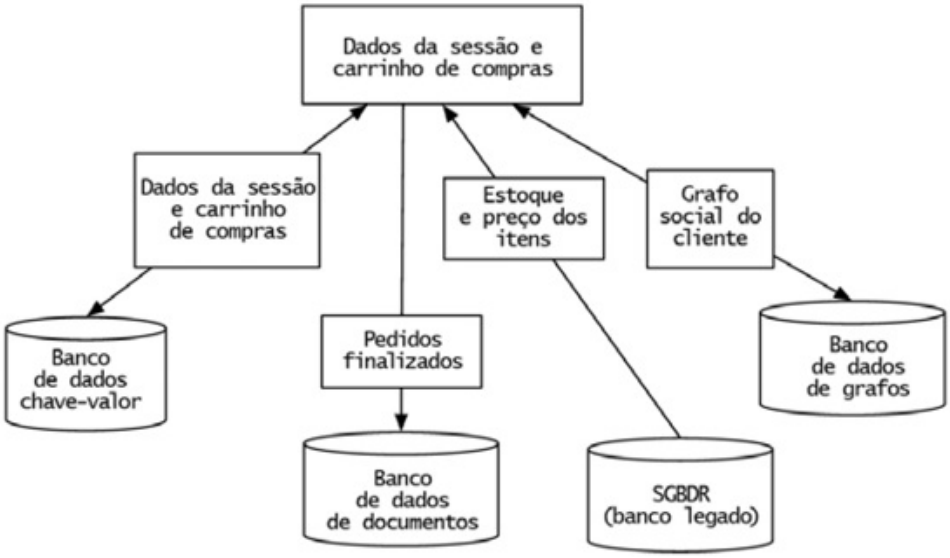

Figura 13.3 – Exemplo de implementação de persistência poliglota.

Não é necessário que o aplicativo utilize um único armazenamento de dados para todas as suas necessidades, uma vez que diferentes bancos de dados são criados para diferentes propósitos e nem todos os problemas podem ser resolvidos de forma eficaz por um único banco de dados.

Nem mesmo o uso de bancos de dados relacionais especializados para propósitos diferentes, como componentes de análise ou de data warehouse dentro do mesmo

aplicativo, pode ser visto como persistência poliglota.

13.3 Uso de serviços em vez do acesso direto ao depósitos de dados

À medida que implementamos múltiplos bancos de dados no aplicativo, outros aplicativos na empresa poderão se beneficiar do uso desses bancos ou dos dados neles armazenados. Utilizando nosso exemplo, o banco de dados de grafos pode fornecer dados para outros aplicativos que precisam entender, por exemplo, quais produtos estão sendo comprados por um determinado segmento da base de clientes.

Em vez de cada aplicativo se comunicar diretamente de forma independente com o banco de dados de grafos, podemos encapsular esse banco em um serviço, de forma que todos os relacionamentos entre os nodos possam ser gravados em um lugar e consultados por todos os aplicativos (Figura 13.4). A propriedade dos dados e as APIs fornecidas pelo serviço são mais úteis do que um único aplicativo comunicando-se com vários bancos de dados.

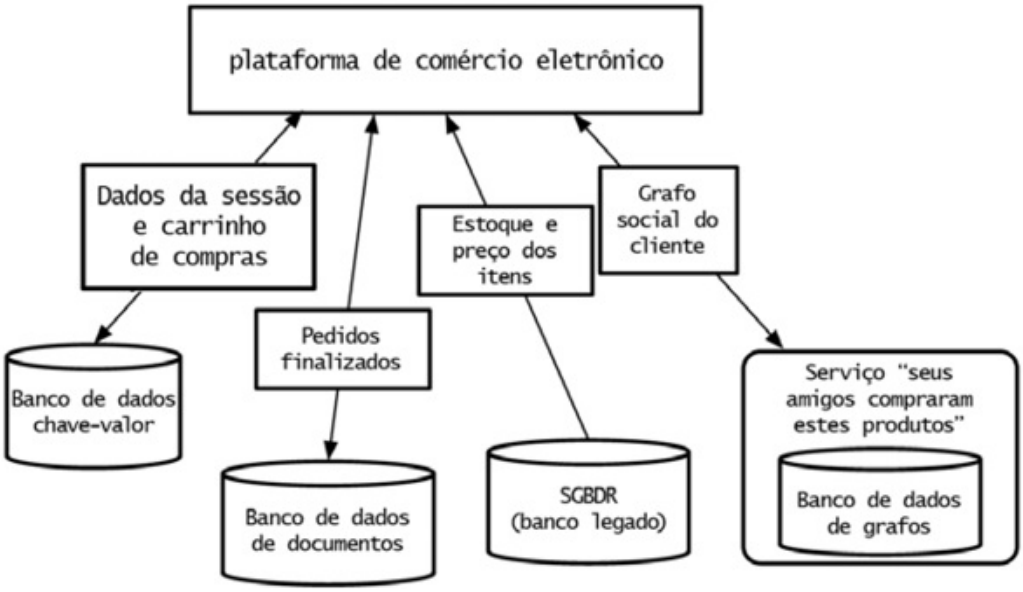

Figura 13.4 – Exemplo de implementação de encapsulamento de bancos de dados em serviços.

A filosofia do encapsulamento em serviços pode ser levada mais além: você poderia encapsular todos os bancos de dados em serviços, permitindo que a aplicação apenas se comunique com alguns serviços (Figura 13.5). Isso permite que os bancos de dados dentro dos serviços desenvolvam-se sem que você tenha de alterar os aplicativos dependentes.

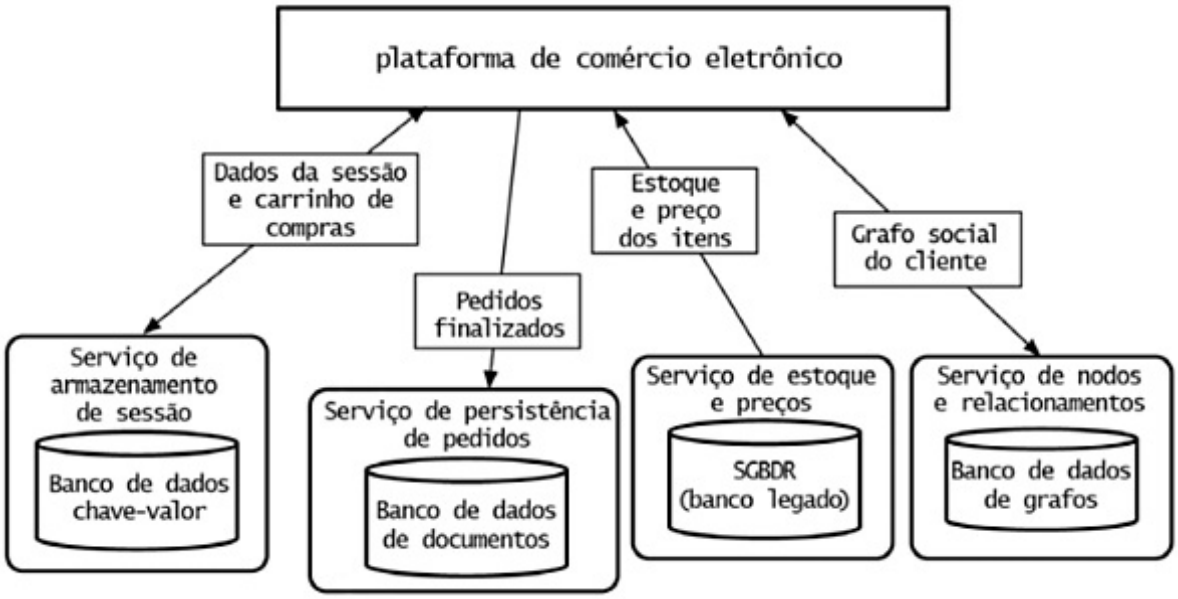

Figura 13.5 – Utilizando serviços em vez de comunicar-se com bancos de dados.

Muitos produtos de armazenamento de dados NoSQL, como o Riak [Riak] e o Neo4J [Neo4J], fornecem, de fato, APIs REST prontas para o uso.

13.4 Expandindo para melhorar a funcionalidade

Muitas vezes, não podemos alterar o banco de dados para um uso específico devido à existência de aplicativos legados e sua dependência ao banco de dados existente. Podemos, entretanto, adicionar funcionalidades, como um cache para obter melhor desempenho, ou utilizar mecanismos de indexação, como o Solr [Solr], de modo que a pesquisa seja mais eficiente (Figura 13.6). Quando tecnologias como essas são introduzidas, temos de nos assegurar de que os dados estejam sincronizados entre o armazenamento de dados do aplicativo e o cache ou o mecanismo de indexação.

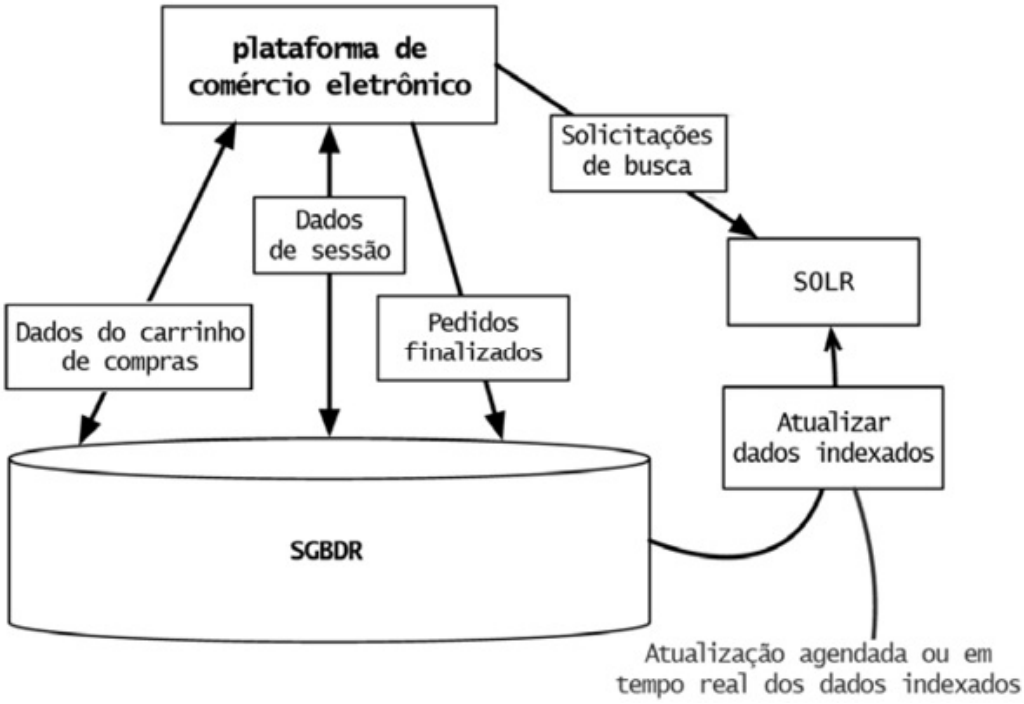

Figura 13.6 – Utilizando um armazenamento suplementar para melhorar o depósito legado.

Ao fazer isso, precisamos atualizar os dados indexados conforme os dados do banco de dados do aplicativo forem alterados. O processo de atualização dos dados pode ser realizado em tempo real ou agendado, desde que nos asseguremos de que o aplicativo possa lidar com dados antigos no mecanismo de índice/pesquisa. O padrão event sourcing (“Event sourcing”, p. 202) pode ser utilizado para atualizar o índice.

13.5 Escolhendo a tecnologia certa

Há uma ampla gama de soluções de armazenamento de dados. Inicialmente, o pêndulo havia mudado de lado, de bancos de dados especializados para banco de dados relacional único, o qual permite que todos os tipos de modelos de dados sejam armazenados, embora com alguma abstração. A tendência, agora, está voltando a ser o uso de armazenamentos de dados que suportem nativamente a implementação de soluções.

Se quisermos recomendar produtos para clientes baseando-nos no que há em seus carrinhos de compras e em outros produtos comprados por clientes que adquiriram esses mesmos produtos, podemos implementar isso em qualquer um dos armazenamentos de dados, persistindo os dados com os atributos corretos para responder às nossas questões. O truque é utilizar a tecnologia certa, de modo que, quando as questões mudarem, elas ainda possam ser realizadas ao mesmo banco de dados, sem perder os dados já existentes ou alterá-los para um novo formato.

Vamos voltar à nossa nova funcionalidade. Podemos utilizar SGBDRs para resolver isso por meio de uma consulta hierárquica e modelando as tabelas de acordo. Quando precisarmos alterar a travessia, teremos de refatorar o banco de dados, migrar os dados e começar a persistir os novos dados. Em vez disso, se tivéssemos utilizado um armazenamento de dados que registrasse relacionamentos entre nodos, poderíamos simplesmente ter programado as novas relações e continuar utilizando o mesmo banco de dados com mudanças mínimas.

13.6 Preocupações empresariais com a persistência poliglota

A introdução de tecnologias de armazenamento de dados NoSQL obrigará os DBAs das empresas a pensarem em como utilizar o novo banco. A empresa está acostumada a ter ambientes SGBDRs uniformes; seja qual for o banco de dados que uma empresa utilize inicialmente, é provável que, no decorrer dos anos, todos os seus aplicativos sejam criados em torno do mesmo banco de dados. Nesse novo mundo da persistência poliglota, os grupos de DBAs terão de ter múltiplas habilidades, para aprender como algumas dessas tecnologias NoSQL funcionam, monitorar esses sistemas, suportá-los e mover dados para dentro e para fora desses sistemas.

Assim que as empresas decidem utilizar alguma tecnologia NoSQL, surgem questões como licenças, suporte, ferramentas, atualizações, drivers, auditoria e segurança. Muitas tecnologias NoSQL são open-source e têm uma comunidade ativa de apoiadores. Além disso, há empresas que fornecem suporte comercial. Não é um ecossistema pleno de ferramentas, mas os vendedores de ferramentas e a comunidade open-source estão avançando, criando ferramentas, como o MongoDB Monitoring Service [Monitoring], o Datastax Ops Center [OpsCenter]e o navegador Rekon para o Riak [Rekon].

Uma outra área com a qual as empresas estão preocupadas é a segurança dos dados – a capacidade de criar usuários e atribuir direitos de ver ou não os dados no nível do banco de dados. A maioria dos bancos de dados NoSQL não tem recursos muito robustos de segurança, mas isso porque é projetada para operar de modo diferente. Em SGBDRs tradicionais, os dados são servidos pelo banco de dados e podemos interagir com estes utilizando qualquer ferramenta de consulta. Com os bancos de dados NoSQL, há ferramentas de consulta também, mas a ideia é que o aplicativo tenha domínio dos dados e que os sirva utilizando serviços. Com essa abordagem, a responsabilidade pela segurança fica com o aplicativo. Dito isso, há tecnologias NoSQL que introduzem recursos de segurança.

Muitas vezes, as empresas têm sistemas de data warehouse, BI e sistemas de análise que podem precisar de dados das fontes de dados poliglotas. As empresas terão de assegurar-se de que as ferramentas ETL ou qualquer outro mecanismo que estejam utilizando para mover os dados dos sistemas de origem para o data warehouse possam ler dados do armazenamento NoSQL. Os vendedores de ferramentas ETL estão trazendo a capacidade de comunicação com bancos de dados NoSQL; por exemplo, o Pentaho [Pentaho] pode se comunicar com o MongoDB e com o Cassandra.

Toda empresa faz análises de algum tipo. À medida que o volume de dados brutos que precisam ser obtidos aumenta, as empresas lutam para ampliar seus sistemas SGBDRs de modo que possam gravar todos esses dados nos bancos de dados. Um número muito grande de gravações, além da necessidade de um crescimento em escala para gravações, são um ótimo motivo para a utilização de bancos de dados NoSQL, que permitem que você grave grandes volumes de dados.

13.7 Complexidade de instalação

Assim que começamos a percorrer a jornada do uso de persistência poliglota no aplicativo, a **complexidade de instalação** precisa de uma análise cuidadosa. Agora, o aplicativo precisa de todos os bancos de dados em produção ao mesmo tempo. Você precisará ter esses bancos de dados em seus ambientes de UAT, QA e Dev. Como a maioria dos produtos NoSQL é open-source, há poucas ramificações nos custos das licenças. Eles também suportam a automação de instalação e configuração. Por exemplo, para instalar um banco de dados, tudo o que precisa ser feito é baixar e descompactar o arquivo, o que pode ser automatizado utilizando-se os comandos `curl` e `unzip`. Esses produtos também têm valores default apropriados e podem ser iniciados com um mínimo de configuração.

13.8 Pontos chave

- A persistência poliglota está relacionada com o uso de diferentes tecnologias de armazenamento de dados, de forma que possa lidar com necessidades variáveis de armazenamento de dados.
- A persistência poliglota pode ser aplicada em uma empresa ou dentro de um único aplicativo.
- Encapsular o acesso aos dados em serviços reduz o impacto de decisões sobre armazenamento de dados em outras partes de um sistema.
- Adicionar mais tecnologias aumenta a complexidade na programação e nas operações,

de modo que as vantagens de um armazenamento de dados apropriado precisam ser comparadas com essa complexidade.

CAPÍTULO 14

Além do NoSQL

O surgimento de bancos de dados NoSQL realmente movimentou e abriu o mundo dos bancos de dados, mas acreditamos que o tipo de bancos de dados NoSQL que discutimos aqui é apenas parte do cenário da persistência poliglota. Assim, faz sentido passar mais tempo discutindo soluções que não se adequam facilmente ao espaço NoSQL.

14.1 Sistemas de arquivos

Bancos de dados são muito comuns, mas sistemas de arquivos são quase onipresentes. Nas últimas décadas, eles têm sido utilizados amplamente para documentos de produtividade pessoal, mas não para aplicativos empresariais. Eles não fazem referência a qualquer estrutura interna, de modo que são mais similares a armazenamentos de chave-valor com uma chave hierárquica. Eles também fornecem pouco controle sobre a concorrência, além do simples bloqueio de arquivos – o que, por si só, é semelhante à forma pela qual o NoSQL fornece apenas bloqueio dentro de um único agregado.

Sistemas de arquivos têm a vantagem de serem simples e amplamente implementados. Eles lidam bem com arquivos muito grandes, como vídeos e áudios. Muitas vezes, bancos de dados são utilizados para indexar mídia armazenada em arquivos. Os arquivos também funcionam bem para o acesso sequencial, como o streaming, o que pode ser útil para dados que recebam apenas inserções (append-only).

Os ambientes clusterizados viram um aumento nos sistemas de arquivos distribuídos. Tecnologias como o Google File System e o Hadoop [Hadoop] fornecem suporte para a replicação de arquivos. Muito da discussão acerca de map-reduce se relaciona com a manipulação de arquivos grandes em sistemas clusterizados, com ferramentas para a divisão automática de arquivos grandes em segmentos, de maneira que possam ser processados em múltiplos nodos. De fato, um caminho de entrada comum para NoSQL é a partir de organizações que utilizam o Hadoop.

Sistemas de arquivos funcionam melhor com um número relativamente pequeno de arquivos grandes, que possam ser processados em grandes fatias, de preferência na forma de streaming. Lidar com um grande número de arquivos pequenos geralmente resulta em um desempenho pior – é aí que o armazenamento de dados torna-se mais eficiente. Os arquivos também não fornecem suporte a consultas sem ferramentas de indexação adicionais, como o Solr [Solr].

14.2 Event sourcing

Event sourcing é uma abordagem à persistência que se concentra em persistir todas as alterações em um estado persistente em vez de persistir o próprio estado corrente do aplicativo. É um padrão de arquitetura que funciona muito bem com a maioria das tecnologias de persistência, incluindo bancos de dados relacionais. Nós a mencionamos

aqui pois ela também serve de base para algumas das formas mais incomuns de se pensar sobre a persistência.

Considere como exemplo um sistema que mantenha um registro da localização de navios (Figura 14.1). Ele tem um registro simples de navios, o qual armazena o nome do navio e sua localização atual. Normalmente, quando ouvimos que o navio King Roy chegou a São Francisco, alteramos o valor do campo `location` (localização) do King Roy para São Francisco. Posteriormente, ficamos sabendo que ele partiu e, então, alteramos o campo para "navegando" e o alteramos novamente assim que soubermos que chegou em Hong Kong.

Com um sistema event-sourced, a primeira etapa é criar um objeto que capture as informações sobre a alteração (Figura 14.2). Esse objeto de evento é armazenado em um registro de evento durável. Finalmente, processamos o evento para atualizar o estado do aplicativo.

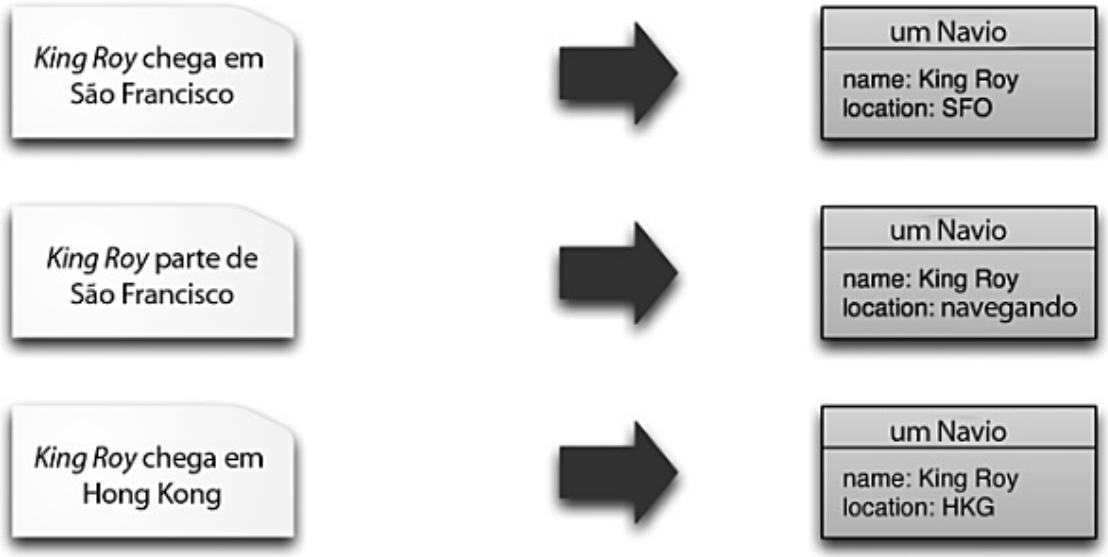

Figura 14.1 – Em um sistema típico, a notícia de uma mudança causa uma alteração no estado do aplicativo.

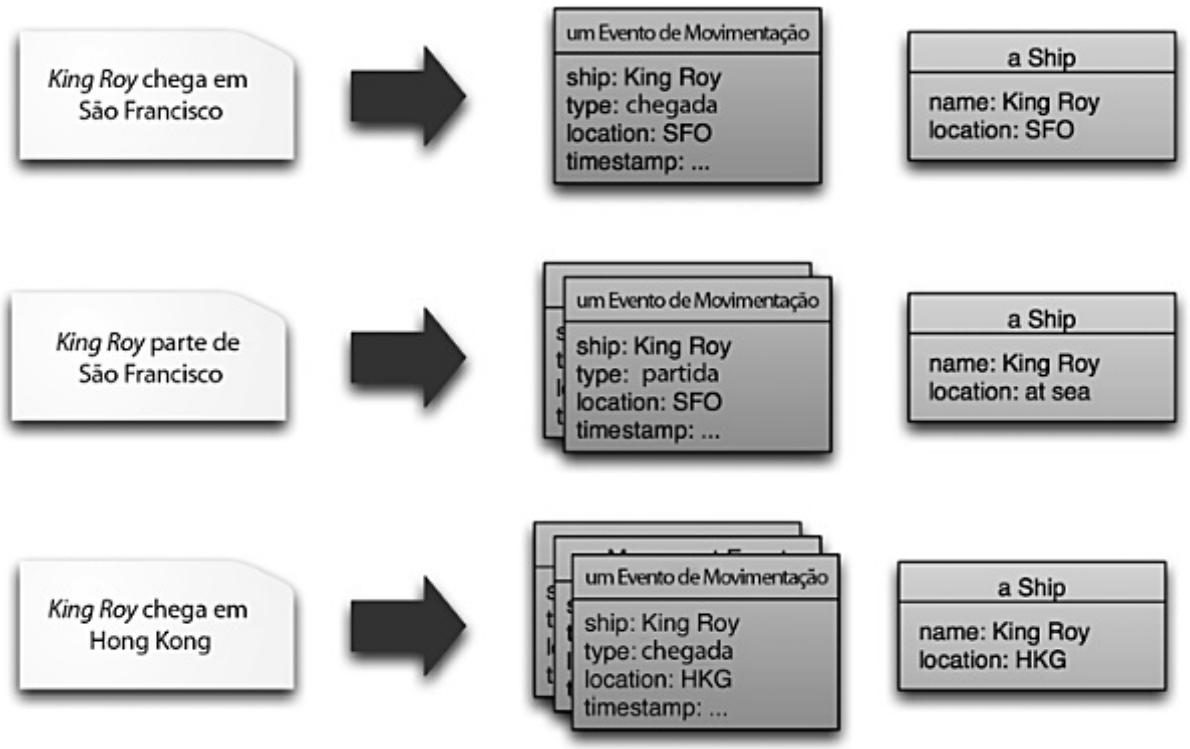

Figura 14.2 – Com event source, o sistema armazena cada evento, juntamente ao estado derivado do aplicativo.

Como consequência, em um sistema event-sourced, armazenamos todos os eventos que tenham causado uma alteração de estado do sistema no registro de eventos e o estado do aplicativo é inteiramente derivável desse registro de eventos. A qualquer momento, podemos nos desvencilhar do estado do aplicativo e recriá-lo a partir do registro de eventos.

Em teoria, registros de eventos são tudo de que você precisa, porque sempre que for necessário poderá recriar o estado do aplicativo, executando novamente o registro de eventos. Na prática, isso pode ficar lento demais. Consequentemente, em geral, é melhor permitir o armazenamento e a recriação do estado do aplicativo em um *snapshot*. Um snapshot é projetado para persistir a imagem da memória otimizada para recuperação rápida do estado. É uma ajuda na otimização, de modo que nunca deve ter precedência sobre o registro de eventos quanto à sua autoridade sobre os dados.

A frequência com a qual você tira snapshots depende do tempo de atividade. O snapshot não precisa estar completamente atualizado, já que você pode recriar a memória carregando o snapshot mais recente e depois executando novamente todos os eventos processados desde que esse snapshot foi tirado. Um exemplo seria tirar um snapshot todas as noites; se o sistema saísse do ar durante o dia, você poderia recarregar o snapshot da noite anterior seguido pelos eventos de hoje. Se você conseguir fazer isso de modo suficientemente rápido, tudo ficará bem.

Para obter um registro inteiro de cada alteração no estado do seu aplicativo, você precisa continuar voltando no registro de eventos até o início de seu aplicativo. Em muitos casos, porém, não é necessário ter um registro tão antigo, de modo que você pode guardar eventos mais antigos em um snapshot e utilizar o registro de eventos somente depois da data dele.

Utilizar event sourcing tem diversas vantagens. Você pode transmitir eventos para múltiplos sistemas, cada um podendo criar um estado diferente do aplicativo para diferentes propósitos (Figura 14.3). Para sistemas com muita atividade de leitura, você pode fornecer múltiplos nodos de leitura, com esquemas potencialmente diferentes, ao mesmo tempo em que concentra as gravações em um sistema diferente de processamento (uma abordagem amplamente conhecida como CQRS [CQRS]).

Event sourcing também é uma plataforma efetiva para analisar informações de históricos, já que você pode replicar qualquer estado passado no registro de eventos. Você também pode investigar facilmente cenários alternativos, introduzindo eventos hipotéticos em um processador de análises.

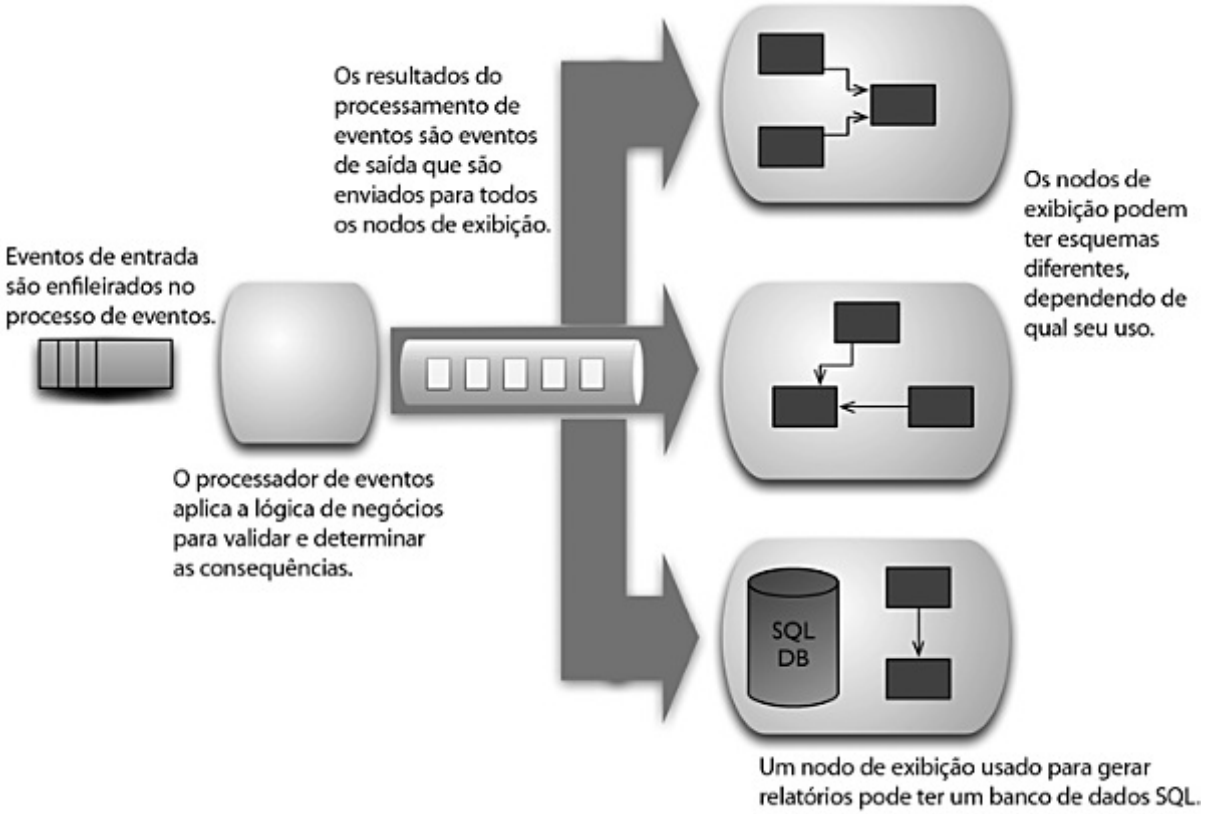

Figura 14.3 – Os eventos podem ser transmitidos para múltiplos sistemas de exibição.

O event sourcing é um pouco mais complexo – notavelmente, você tem de se assegurar de que todas as mudanças de estado sejam capturadas e armazenadas como eventos. Algumas estruturas e ferramentas podem tornar isso inconveniente. Qualquer colaboração com sistemas externos precisa levar o event sourcing em consideração; por isso, você precisará ser cauteloso com os efeitos colaterais externos ao executar novamente os eventos para recriar um estado da aplicação.

14.3 Imagem de memória

Uma das consequências do event sourcing é que o registro de eventos torna-se o registro persistente definitivo, mas não é necessário que o estado do aplicativo seja persistente. Isso abre a possibilidade de manter o estado do aplicativo em memória, utilizando apenas estruturas de dados na memória. Manter todos os seus dados de trabalho em memória fornece uma vantagem no desempenho, já que não há I/O em disco para se lidar quando

um evento é processado. Também simplifica a programação, já que não há necessidade de executar mapeamentos entre estruturas de dados em disco e na memória.

A limitação óbvia aqui é que você deve ser capaz de armazenar todos os dados que precisará acessar na memória. Essa é uma opção cada vez mais viável – lembrando que os tamanhos de disco eram consideravelmente menores do que os tamanhos atuais de memória. Você também deve certificar-se de que conseguirá recuperar o sistema de modo suficientemente rápido depois de uma falha – recarregando eventos a partir do registro de eventos ou executando uma réplica do sistema e transferindo para ela as operações.

Você precisará de algum mecanismo explícito para lidar com a concorrência. Um caminho seria um sistema de memória transacional, como o que vem com a linguagem Clojure. Outro caminho seria executar todo o processamento de entrada em uma única thread. Projetado com cuidado, um processador de eventos de thread única pode obter um rendimento impressionante com uma latência baixa [Fowler lmax].

Quebrar a separação entre dados persistentes e dados na memória também afeta o modo pelo qual você lida com os erros. Uma abordagem comum consiste em atualizar um modelo e desfazer quaisquer alterações caso houver alguma falha. Com uma imagem de memória, você geralmente não terá um dispositivo de rollback automatizado; você terá de escrever o seu próprio (o que é bastante complicado) ou garantir que tenha feito uma validação minuciosa antes de começar a aplicar quaisquer alterações.

14.4 Controle de versões

Para a maioria dos desenvolvedores de software, a sua experiência mais comum com o uso de sistemas event-sourced é o sistema de controle de versões. Esses sistemas permitem que muitas pessoas em uma equipe coordenem suas modificações em um sistema complexo interconectado, com a capacidade de explorar estados passados desse sistema e realidades alternativas por meio de ramificações (branches).

Quando pensamos em armazenamento de dados, tendemos a pensar com uma visão global em um único momento do tempo, o que é muito limitado se comparado com a complexidade suportada por um sistema de controle de versões. É, portanto, surpreendente que ferramentas de armazenamento de dados não tenham pegado emprestado algumas das ideias vindas dos sistemas de controle de versão. Afinal, muitas situações requerem consultas em históricos e suporte para múltiplas visões do mundo.

Sistemas de controle de versões são criados sobre sistemas de arquivos e, assim, têm muitas das limitações de armazenamento de dados que um sistema de arquivos. Eles não são projetados para o armazenamento de dados de aplicativos, de forma que é difícil utilizá-los nesse contexto. Entretanto, vale a pena considerá-los em cenários em que suas capacidades de linha de tempo sejam úteis.

14.5 Bancos de dados XML

Pela virada do milênio, as pessoas pareciam querer utilizar o XML para tudo e havia uma agitação de interesse em bancos de dados especificamente projetados para armazenar e consultar documentos XML. Embora essa agitação tenha tido pouco impacto sobre o domínio relacional, tal como rompantes anteriores, bancos de dados XML ainda existem.

Pensamos em bancos de dados XML como bancos de dados de documentos, no qual os documentos são armazenados em um modelo de dados compatível com XML e em que diversas tecnologias XML são utilizadas para manipular o documento. Você pode utilizar diversas formas de definição de esquemas XML (DTDs, XML Schema, RelaxNG) para verificar formatos de documentos, executar consultas com XPath e XQuery e executar transformações com XSLT.

Bancos de dados relacionais misturaram essas capacidades XML com as de banco de dados relacionais, geralmente inserindo documentos XML como um tipo de coluna e possibilitando alguma forma de misturar as linguagens de consulta SQL e XML.

É claro que não há motivo para que você não possa utilizar o XML como um mecanismo de estruturação dentro de um depósito de chave-valor. O XML é menos apropriado, atualmente, do que o JSON, mas é igualmente capaz de armazenar agregados complexos e as capacidades de consulta e esquema do XML são maiores do que as que você geralmente obtém com JSON. Utilizar um banco de dados XML implica que o próprio banco de dados seja capaz de aproveitar a estrutura XML e não apenas tratar o valor como um blob, mas essa vantagem precisa ser comparada com as outras características do banco de dados.

14.6 Bancos de dados de objetos

Quando a programação orientada a objetos começou a ficar popular, houve uma onda de interesse em bancos de dados orientados a objetos. O foco, aqui, era a complexidade do mapeamento de estruturas de dados na memória para tabelas relacionais. A ideia de um banco de dados orientado a objetos consiste em que você evite essa complexidade – o banco de dados gerenciaria automaticamente o armazenamento das estruturas da memória para o disco. Você poderia pensar nisso como um sistema de memória virtual persistente, permitindo que programasse com persistência sem se preocupar com o banco de dados.

Os bancos de dados orientados a objetos não deslancharam. Um motivo para isso foi que o benefício da integração próxima com o aplicativo significava que você não podia acessar os dados com facilidade de outra forma que não fosse pelo próprio aplicativo. Uma mudança nos bancos de dados de integração para os de aplicativos poderia ter tornado os bancos de dados orientados a objetos mais viáveis no futuro.

Uma questão importante relacionada aos bancos de dados orientados a objetos consiste em como lidar com a migração quando as estruturas de dados mudarem. Aqui, a conexão íntima entre o armazenamento persistente e as estruturas na memória pode se tornar um problema. Alguns bancos de dados orientados a objetos incluem a capacidade de adicionar funções de migração às definições de objetos.

14.7 Ponto chave

O NoSQL é apenas um conjunto de tecnologias de armazenamento de dados. Como aumentam a facilidade com a persistência poliglota, devemos analisar outras tecnologias de armazenamento de dados, possuindo ou não o rótulo NoSQL.

CAPÍTULO 15

Escolhendo o seu banco de dados

A esta altura do livro, já examinamos muitas questões gerais sobre as quais você precisa ter ciência para tomar decisões no novo mundo da persistência poliglota. Agora é o momento de falarmos sobre a escolha de seus bancos de dados para futuros trabalhos de desenvolvimento. Naturalmente, não conhecemos suas circunstâncias específicas, de modo que não podemos dar-lhe uma resposta nem podemos reduzi-la a um simples conjunto de regras a seguir. Além disso, ainda estamos nos primórdios do uso em produção dos sistemas NoSQL, de forma que falta maturidade a esse conhecimento – em alguns anos, talvez pensemos de modo diferente.

Vemos dois grandes motivos para considerar um banco de dados NoSQL: a produtividade do programador e o desempenho no acesso aos dados. Em diferentes cenários, essas forças podem ser complementares ou contraditórias. Ambas são difíceis de avaliar no início de um projeto, o que é complexo, já que sua escolha por um modelo de armazenamento de dados é tão difícil de abstrair quanto de permitir que você mude de ideia posteriormente.

15.1 Produtividade do programador

Converse com qualquer desenvolvedor de um aplicativo corporativo e você sentirá a frustração de trabalhar com bancos de dados relacionais. As informações geralmente são coletadas e exibidas em termos de agregados, mas têm de ser transformadas em relações para que sejam persistidas. Essa tarefa é mais fácil do que costumava ser; durante a década de 1990, muitos projetos se ressentiam do esforço de criar camadas de mapeamento objeto-relacional. Na década de 2000, vimos frameworks ORM populares, tais como o Hibernate, o iBATIS e o Rails Active Record, que reduzem muito desse fardo. Mas isso não eliminou o problema. ORMs são uma abstração imperfeita, sempre há alguns casos que precisam de mais atenção, especialmente para obter um desempenho aceitável.

Nessa situação, bancos de dados orientados a agregados podem oferecer um negócio tentador. Podemos remover o ORM e persistir os agregados de forma natural, conforme os utilizamos. Nos deparamos com diversos projetos que alegam benefícios palpáveis por terem passado para uma solução orientada a agregados.

Bancos de dados de grafos oferecem uma simplificação diferente. Bancos de dados relacionais não fazem um bom trabalho com dados que possuem muitos relacionamentos. Um banco de dados de grafos oferece uma API de armazenamento mais natural para esse tipo de dado e uma capacidade de consulta projetada em torno desses tipos de estruturas.

Todos os tipos de sistemas NoSQL são mais apropriados para dados não uniformes. Se você tiver dificuldades em obter um esquema forte para suportar campos ad-hoc, então os bancos de dados NoSQl sem esquema podem oferecer uma ajuda considerável.

Esses são os principais motivos pelos quais o modelo de programação de bancos de

dados NoSQL pode melhorar a produtividade de sua equipe de desenvolvimento. O primeiro passo nessa avaliação para o seu contexto é examinar o que seu software precisará fazer. Observe os recursos atuais e veja como o uso dos dados é apropriado. Enquanto fizer isso, talvez comece a ver que um determinado modelo de dados parece apropriado. Essa proximidade de compatibilidade sugere que o uso desse modelo levará a uma programação mais fácil.

Quando fizer isso, lembre-se de que a persistência poliglota se relaciona ao uso de múltiplas soluções de armazenamento de dados. Talvez você veja diferentes modelos de dados ajustarem-se a diferentes partes dos dados. Isso sugeriria o uso de diferentes bancos de dados para diferentes aspectos de seus dados. Utilizar múltiplos bancos de dados é inerentemente mais complexo do que utilizar um único armazenamento, mas pode haver mais vantagens de uma boa adaptação em cada caso, de forma geral.

Quando você examinar a adaptação de modelos de dados, preste bastante atenção nos casos em que exista um problema. Talvez você veja que a maioria de seus recursos funcionará bem com um agregado, mas alguns poucos, não. Ter alguns recursos que não se adéquam bem ao modelo não é razão para evitá-lo – as complicações da não adaptação podem não superar as vantagens de uma boa adaptação – mas é útil localizar e destacar os casos de má adaptação.

Examinar o que precisa ser feito e avaliar suas necessidades de dados deve levar a uma ou mais alternativas de como lidar com as necessidades de seu banco de dados. Isso lhe dará um ponto de partida, mas o próximo passo é experimentar criando um software de fato. Escolhaalguns recursos iniciais e os desenvolva, ao mesmo tempo em que presta atenção se é realmente fácil utilizar a tecnologia que estiver considerando. Nessa situação, pode valer a pena criar os mesmos recursos com alguns bancos de dados diferentes, para ver qual funciona melhor. Muitas vezes, as pessoas relutam em fazer isso – ninguém gosta de criar software que será descartado. Mesmo assim, essa é uma forma essencial de julgar o quão efetivo é um determinado framework.

Infelizmente, não há como medir apropriadamente o quão produtivos diferentes projetos são. Não temos como medir apropriadamente o resultado. Mesmo que você crie exatamente os mesmos recursos, não podemos comparar integralmente a produtividade, porque o conhecimento adquirido ao desenvolver um primeiro projeto torna mais fácil a realização de um segundo e você não pode desenvolvê-los simultaneamente com equipes idênticas. O que você pode fazer é assegurar-se de que as pessoas que executaram o trabalho possam dar uma opinião. A maioria dos desenvolvedores pode sentir quando são mais produtivos em um ambiente do que em outro. Embora esse seja um julgamento subjetivo e possam existir opiniões diferentes entre os membros da equipe, é o melhor julgamento que conseguirá. No final, acreditamos que a equipe que realiza o trabalho é que deverá decidir.

Ao experimentar um banco de dados para julgar sua produtividade, também é importante experimentar os casos de má adaptação que mencionamos anteriormente. Dessa forma, a equipe pode ter uma ideia do caminho fácil e do caminho difícil para obter uma impressão geral.

Essa abordagem tem suas falhas. Muitas vezes, você não consegue uma avaliação integral de uma tecnologia sem passar muitos meses utilizando-a – e realizar uma avaliação muito longa dificilmente é efetiva quanto ao custo. Mas como tudo na vida, precisamos fazer a

melhor avaliação que pudermos, conhecer suas falhas e seguir em frente com ela. O essencial, aqui, é basear a decisão em tanta programação real quanto possível. Mesmo uma mera semana de trabalho com a tecnologia pode lhe dar informações que você nunca obteria em uma centena de apresentações do vendedor.

15.2 Desempenho no acesso aos dados

A preocupação que levou à expansão dos bancos de dados NoSQL foi o acesso rápido a grandes quantidades de dados. À medida que surgiam websites grandes, esses queriam se expandir horizontalmente e ser executados em clusters grandes. Eles desenvolveram os primeiros bancos de dados NoSQL, de modo que lhes auxiliassem a ser executados com eficiência em tais estruturas. À medida que outros usuários seguiram esse caminho, novamente o foco foi o acesso rápido aos dados, muitas vezes com grandes volumes envolvidos.

Há muitos fatores que podem determinar um desempenho melhor de um banco de dados do que o padrão relacional em várias circunstâncias. Um banco de dados orientado a agregados pode ser muito rápido para ler e recuperar agregados, se comparado a um banco de dados relacional, em que os dados ficam espalhados por muitas tabelas. A replicação e a segmentação facilitada em clusters possibilita uma ampliação horizontal. Um banco de dados de grafos pode recuperar dados altamente conectados com mais rapidez do que utilizando junções relacionais.

Se você estiver estudando bancos de dados NoSQL, baseando-se em desempenho, o mais importante a ser feito é testar seu desempenho em cenários que sejam importantes para você. Argumentar sobre o desempenho de um banco de dados pode lhe ajudar a criar uma lista breve, mas a única forma pela qual você pode avaliar o desempenho apropriadamente é criando algo, executando e medindo-o.

Ao executar uma avaliação de desempenho, o mais difícil, muitas vezes, é obter um conjunto realista de testes de desempenho. Você não pode desenvolver o seu sistema real, então precisa criar um subconjunto representativo. É importante, entretanto, que esse subconjunto seja um representante tão fiel quanto for possível. Não adianta pegar um banco de dados que deve servir centenas de usuários concorrentes e avaliar seu desempenho com um único usuário. Você precisará criar cargas e volumes de dados representativos.

Especialmente se você estiver criando um website público, pode ser difícil criar um ambiente de testes com carga alta. Aqui, um bom argumento pode ser obtido utilizando recursos de computação em nuvem (cloud computing) tanto para gerar uma carga quanto para criar um cluster de teste. A natureza elástica da provisão da nuvem é muito útil para o trabalho curto de avaliação de desempenho.

Você não poderá testar todas as formas como o seu aplicativo será utilizado, então precisará criar um subconjunto representativo. Selecione cenários que sejam os mais comuns, os mais dependentes de desempenho e os que não pareçam se adaptar bem ao seu modelo de banco de dados. Este último pode lhe alertar para quaisquer riscos fora de seus principais cenários de uso.

Criar volumes para testes pode ser complicado, especialmente no início de um projeto, quando não estiver claro quais serão os volumes que você provavelmente terá. Você deverá

criar algo em que basear seu raciocínio, então assegure-se de explicitá-lo e comunicá-lo a todos os interessados. Explicitá-lo reduz a chance de que diferentes pessoas tenham ideias diferentes sobre o que é uma "carga de leitura alta". Também permite que você localize problemas mais facilmente, caso suas descobertas posteriores estiverem longe de suas suposições originais. Sem explicitar suas suposições, é mais fácil abandoná-las sem perceber suas necessidades de refazer seu ambiente de trabalho à medida que obtém novas informações.

15.3 Permanecendo com o padrão

Acreditamos, naturalmente, que o NoSQL é uma opção viável em muitas circunstâncias – caso contrário, não teríamos investido vários meses escrevendo este livro. Mas também percebemos que há muitas situações, de fato a maioria dos casos, nas quais você ficará melhor se permanecer com a opção padrão de um banco de dados relacional.

Bancos de dados relacionais são bem conhecidos; você pode encontrar facilmente pessoas com experiência em seu uso. Eles são maduros, de modo que você corre menos risco de se deparar com partes pouco provadas de uma tecnologia nova. Há muitas ferramentas criadas na tecnologia relacional que você pode aproveitar. Você também não tem de lidar com as questões políticas de fazer uma escolha incomum – escolher uma tecnologia nova sempre implicará o risco de problemas em caso de dificuldades.

Assim, de modo geral, tendemos a pensar que, para escolher um banco de dados NoSQL, você precisa mostrar uma vantagem real em relação aos bancos de dados relacionais para a sua situação. Não há problema algum em fazer as avaliações para capacidade de programação e desempenho, não encontrar uma vantagem clara e permanecer com a opção relacional. Acreditamos que há muitos casos em que é vantajoso utilizar bancos de dados NoSQL, mas "muitos" não significa "todos", ou mesmo, "a maioria".

15.4 Restringindo suas apostas

Uma das maiores dificuldades que temos ao aconselhar na escolha de uma opção de armazenamento de dados é que não temos uma quantidade muito grande de dados para analisar. Conforme escrevíamos este livro, vimos os primeiros usuários discutindo suas experiências com o uso dessas tecnologias, de modo que não temos uma visão clara dos verdadeiros prós e contras.

Com a situação neste grau de incerteza, há mais de um argumento para encapsular sua escolha de banco de dados – manter todo o seu código de banco de dados em uma área de seu código base que seja relativamente fácil de substituir caso você decida fazer outra escolha de banco de dados posteriormente. A forma clássica de fazer isso é por meio de uma camada de armazenamento de dados explícita em seu aplicativo – utilizando padrões como o Data Mapper e o Repository [Fowler PoEAA]. Tal camada de encapsulamento traz um custo, especialmente quando você não tem certeza sobre o uso de modelos tão diferentes, como modelos de dados de grafos e de chave-valor. O pior é que ainda não temos experiência com o encapsulamento de camadas de dados entre esses tipos muito diferentes de armazenamento de dados.

De modo geral, nosso conselho é encapsular como uma estratégia padrão, mas preste

atenção ao custo da camada de isolamento. Se estiver se tornando um fardo, dificultando, por exemplo, o uso de alguns recursos úteis do banco de dados, então é um bom argumento para utilizar o banco de dados que tenha esses recursos. Essas informações podem ser o que você precisa para tomar uma decisão sobre o banco de dados e, assim, eliminar o encapsulamento.

Esse é outro argumento para decompor a camada de banco de dados em serviços que encapsulem o armazenamento de dados (“Uso de serviços em vez do acesso direto ao depósitos de dados”, p. 194). Assim como reduzir o acoplamento entre diversos serviços, isso tem a vantagem adicional de facilitar a substituição de um banco de dados se algo não funcionar no futuro. Essa é uma abordagem plausível, mesmo se você utilizar o mesmo banco de dados em todos os lugares – se algo não funcionar, você pode trocá-lo gradualmente, enfocando primeiro os serviços mais problemáticos.

Esse conselho de projeto aplica-se igualmente se você preferir continuar com uma opção relacional. Encapsulando segmentos do seu banco de dados em serviços, você pode substituir partes de seu armazenamento de dados por uma tecnologia NoSQL na medida em que ela amadurece e as vantagens tornam-se mais claras.

15.5 Pontos chave

- Os dois principais motivos para utilizar a tecnologia NoSQL são:
  - melhorar a produtividade do programador utilizando um banco de dados que se adapte melhor às necessidades de um aplicativo;
  - melhorar o desempenho no acesso aos dados por meio de alguma combinação na manipulação de volumes maiores de dados, reduzindo a latência e melhorando o rendimento.
- É essencial testar suas expectativas sobre a produtividade do programador e/ou desempenho antes de decidir utilizar uma tecnologia NoSQL.
- O encapsulamento de serviços facilita as alterações nas tecnologias de armazenamento de dados quando as necessidades e tecnologias evoluírem. Separar partes de aplicativos em serviços também permite a você introduzir o NoSQL em um aplicativo já existente.
- A maioria dos aplicativos, especialmente os não estratégicos, deve continuar com a tecnologia relacional – pelo menos até que o ecossistema NoSQL amadureça mais.

15.6 Considerações finais

Esperamos que você tenha achado este livro esclarecedor. Quando começamos a escrevê-lo, estávamos frustrados pela falta de algo que nos desse uma boa visão do mundo NoSQL. Ao escrever este livro, tivemos de fazer nós mesmos a pesquisa e foi uma jornada agradável. Esperamos que sua jornada por este material seja consideravelmente mais rápida, mas não menos agradável.

A esta altura, você talvez esteja considerando o uso de uma tecnologia NoSQL. Se esse for o caso, este livro é apenas um passo inicial na construção de seu conhecimento. Nós lhe incentivamos a baixar alguns bancos de dados e trabalhar com eles, porque temos a firme convicção de que você somente conseguirá entender uma tecnologia apropriadamente trabalhando com ela – descobrindo seus pontos fortes e suas inevitáveis

fraquezas que nunca chegam à documentação.

Esperamos que a maioria das pessoas, incluindo a maioria dos leitores deste livro, não utilize o NoSQL ainda por algum tempo. É uma tecnologia nova e ainda estamos no início do processo de compreensão sobre quando e como utilizá-la bem. Mas, assim como tudo no mundo do software, há uma mudança mais rápida do que ousaríamos prever, de modo que você deve ficar atento ao que está acontecendo nessa área.

Esperamos que você também encontre outros livros e artigos que lhe ajudem. Acreditamos que o melhor material sobre NoSQL será escrito após este livro estar pronto, de modo que não podemos lhe indicar algo específico neste momento. Temos uma presença ativa na web, então, para conhecer nossas opiniões mais recentes sobre o mundo NoSQL, veja `http://www.sadalage.com` e `http://martinfowler.com/nosql.html`.

Bibliografia

[Agile Methods] `www.agilealliance.org`.

[Amazon's Dynamo] `www.allthingsdistributed.com/2007/10/amazons_dynamo.html`.

[Amazon DynamoDB] `http://aws.amazon.com/dynamodb`.

[Amazon SimpleDB] `http://aws.amazon.com/simpledb`.

[Ambler and Sadalage] Ambler, Scott and Pramodkumar Sadalage. *Refactoring Databases: Evolutionary Database Design*. Addison-Wesley. 2006. ISBN 978-0321293534.

[Berkeley DB] `www.oracle.com/us/products/database/berkeley-db`.

[Blueprints] `https://github.com/tinkerpop/blueprints/wiki`.

[Brewer] Brewer, Eric. *Towards Robust Distributed Systems*. `www.cs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf`.

[Cages] `http://code.google.com/p/cages`.

[Cassandra] `http://cassandra.apache.org`.

[Chang etc.] Chang, Fay, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. *Bigtable: A Distributed Storage System for Structured Data*. `http://research.google.com/archive/bigtable-osdi06.pdf`.

[CouchDB] `http://couchdb.apache.org`.

[CQL] `www.slideshare.net/jericevans/cql-sql-in-cassandra`.

[CQRS] `http://martinfowler.com/bliki/CQRS.html`.

[C-Store] Stonebraker, Mike, Daniel Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O'Neil, Pat O'Neil, Alex Rasin, Nga Tran, and Stan Zdonik. *C-Store: A Columnoriented DBMS*. `http://db.csail.mit.edu/projects/cstore/vldb.pdf`.

[Cypher] `http://docs.neo4j.org/chunked/1.6.1/cypher-query-lang.html`.

[Daigneau] Daigneau, Robert. *Service Design Patterns*. Addison-Wesley. 2012. ISBN 032154420X.

[DBDeploy] http://dbdeploy.com.

[DBMaintain] www.dbmaintain.org.

[Dean and Ghemawat] Dean, Jeffrey and Sanjay Ghemawat. *MapReduce: Simplified Data Processing on Large Clusters*. http://static.usenix.org/event/osdi04/tech/full_papers/dean/dean.pdf.

[Dijkstra's] http://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.

[Evans] Evans, Eric. *Domain-Driven Design*. Addison-Wesley. 2004. ISBN 0321125215.

[FlockDB] https://github.com/twitter/flockdb.

[Fowler DSL] Fowler, Martin. *Domain-Specific Languages*. Addison-Wesley. 2010. ISBN 0321712943.

[Fowler lmax] Fowler, Martin. *The LMAX Architecture*. http://martinfowler.com/articles/lmax.html.

[Fowler PoEAA] Fowler, Martin. *Patterns of Enterprise Application Architecture.* Addison-Wesley. 2003. ISBN 0321127420.

[Fowler UML] Fowler, Martin. *UML Distilled*. Addison-Wesley. 2003. ISBN 0321193687.

[Gremlin] https://github.com/tinkerpop/gremlin/wiki.

[Hadoop] http://hadoop.apache.org/mapreduce.

[HamsterDB] http://hamsterdb.com.

[Hbase] http://hbase.apache.org.

[Hector] https://github.com/rantav/hector.

[Hive] http://hive.apache.org.

[Hohpe and Woolf] Hohpe, Gregor and Bobby Woolf. *Enterprise Integration Patterns*. Addison-Wesley. 2003. ISBN 0321200683.

[HTTP] Fielding, R., J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee. *Hypertext Transfer Protocol—HTTP/1.1*. www.w3.org/Protocols/rfc2616/rfc2616.html.

[Hypertable] http://hypertable.org.

[Infinite Graph] www.infinitegraph.com.

[JSON] http://json.org.

[LevelDB] http://code.google.com/p/leveldb.

[Liquibase] www.liquibase.org.

[Lucene] http://lucene.apache.org.

[Lynch and Gilbert] Lynch, Nancy and Seth Gilbert. *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services*. http://lpd.epfl.ch/sgilbert/pubs/BrewersConjecture-SigAct.pdf.

[Memcached] http://memcached.org.

[MongoDB] www.mongodb.org.

[Monitoring] www.mongodb.org/display/DOCS/MongoDB+Monitoring+Service.

[MyBatis Migrator] http://mybatis.org.

[Neo4J] http://neo4j.org.

[NoSQL Debrief] http://blog.oskarsson.nu/post/22996140866/nosql-debrief.

[NoSQL Meetup] http://nosql.eventbrite.com.

[Notes Storage Facility] http://en.wikipedia.org/wiki/IBM_Lotus_Domino.

[OpsCenter] www.datastax.com/products/opscenter.

[OrientDB] www.orientdb.org.

[Oskarsson] *Private Correspondence*.

[Pentaho] www.pentaho.com.

[Pig] http://pig.apache.org.

[Pritchett] www.infoq.com/interviews/dan-pritchett-ebay-architecture.

[Project Voldemort] http://project-voldemort.com.

[RavenDB] http://ravendb.net.

[Redis] http://redis.io.

[Rekon] https://github.com/basho/rekon.

[Riak] http://wiki.basho.com/Riak.html.

[Solr] http://lucene.apache.org/solr.

[Strozzi NoSQL] www.strozzi.it/cgi-bin/CSA/tw7/I/en_US/NoSQL.

[Tanenbaum and Van Steen] Tanenbaum, Andrew and Maarten Van Steen. Distributed Systems. Prentice-Hall. 2007. ISBN 0132392275.

[Terrastore] http://code.google.com/p/terrastore.

[Vogels] Vogels, Werner. *Eventually Consistent—Revisited*. www.allthingsdistributed.com/2008/12/eventually_consistent.html.

[Webber Neo4J Scaling] http://jim.webber.name/2011/03/22/ef4748c3-6459-40b6-bcfa-818960150e0f.aspx.

[ZooKeeper] http://zookeeper.apache.org.